JACS: JSON Agent Communication Standard

JACS is a cryptographic provenance layer for agent systems. Use it when an output, tool call, or agent handoff crosses a trust boundary and logs alone are not enough.

Start With The Deployment

Most teams adopt JACS in one of four ways:

- **LangChain / LangGraph / CrewAI / FastAPI**: add signing at tool or API boundaries without changing the rest of the app
- **MCP**: secure a local tool server or expose JACS itself as an MCP tool suite
- **A2A**: publish an Agent Card, exchange signed artifacts, and apply trust policy across organizations
- **Core signing**: sign JSON, files, or agreements directly from Rust, Python, Node.js, or Go

The book now focuses on those supported workflows first. Older roadmap-style integration chapters have been reduced or removed from navigation.

What JACS Gives You

- **Signed JSON and file envelopes** with tamper detection
- **Persistent agent identity** with encrypted private keys
- **Trust bootstrap primitives** such as `share_public_key`, `share_agent`, and `trust_agent_with_key`
- **A2A artifact signing and trust policies** (`open` , `verified` , `strict`)
- **MCP integration paths** for ready-made servers, transport security, or tool registration
- **Framework adapters** for Python and Node.js ecosystems
- **Multi-party agreements** with quorum, timeout, and algorithm constraints
- **Cross-language compatibility** across Rust, Python, Node.js, and Go

Best Entry Points

If you are choosing where to start:

1. Which Integration?
2. Use Cases
3. MCP Overview
4. A2A Interoperability

5. Python Framework Adapters
6. Node.js LangChain.js

Implementations

Rust

- Deepest feature surface
- CLI plus library APIs
- Best fit when you want a ready-made MCP server via `jacs mcp`

Python (`jacs`)

- Best fit for LangChain, LangGraph, CrewAI, FastAPI, and local MCP/A2A helpers
- Strong adapter story for adding provenance inside an existing app

Node.js (`@hai.ai/jacs`)

- Best fit for Express, Koa, Vercel AI SDK, LangChain.js, and MCP transport/tool integration
- Also exposes A2A helpers and Express discovery middleware

Go (`jacsgo`)

- Good fit for services that need signing and verification without framework adapters

Quick Start

Rust CLI

```
cargo install jacs-cli
jacs quickstart --name my-agent --domain my-agent.example.com
```

Python

```
pip install jacs
```

Node.js

```
npm install @hai.ai/jacs
```

Go

```
go get github.com/HumanAssisted/JACS/jacsgo
```

Rust, Python, and Node quickstart flows create or load a persistent agent and return agent metadata including config and key paths.

What This Book Does Not Claim

- It does **not** treat MCP and A2A as the same thing. MCP is for model-to-tool calls inside an application boundary; A2A is for agent discovery and exchange across boundaries.
- It does **not** assume every aspirational integration is first-class. If a chapter describes a feature that is not fully supported today, it has been moved out of the main path and tracked separately.
- It does **not** require a registry or blockchain to work. JACS identity is key-based and can be used entirely locally.

Community

- GitHub Repository
- Issue Tracker

What is JACS?

JACS (JSON Agent Communication Standard) is a comprehensive framework designed to solve a critical problem in AI systems: **How do agents communicate and collaborate securely with verifiable trust?**

The Problem JACS Solves

As AI systems become more sophisticated, we're moving toward multi-agent architectures where different AI agents need to:

- **Exchange tasks** and delegate work to each other
- **Create agreements** and verify their completion
- **Share data** with guaranteed authenticity
- **Maintain audit trails** of decisions and actions
- **Establish trust** with flexible key resolution (local trust stores, DNS, optional key services)

Traditional approaches fall short because they lack:

- Cryptographic integrity for agent communications
- Standardized formats for agent interactions
- Built-in versioning and audit trails
- Support for multi-agent agreements and workflows

JACS Core Philosophy

🎯 Agent-First Design

JACS is built specifically for AI agent communication patterns, while still being usable as a general-purpose signed JSON provenance layer. It understands concepts like:

- **Agents** with identities and capabilities
- **Tasks** that can be delegated and tracked
- **Agreements** between multiple parties
- **Versioning** for iterative improvements

🔐 Trust Through Cryptography

Every JACS document includes:

- **Digital signatures** proving authenticity
- **Hash verification** ensuring integrity
- **Public key cryptography** for identity verification
- **Timestamps** for chronological ordering

📋 Standards-Based

JACS builds on proven standards:

- **JSON** for universal compatibility
- **JSON Schema** for structure validation
- **RFC 3339** timestamps for consistency
- **Standard cryptographic algorithms** (RSA, Ed25519, post-quantum)

Key Concepts

Agents

An **Agent** is an autonomous entity with:

- A unique identity (UUID)
- Cryptographic keys for signing
- Capabilities defined in services
- The ability to create and verify documents

Documents

A **Document** is any JSON object that includes:

- JACS header fields (ID, version, creator, etc.)
- A cryptographic signature
- A hash for integrity verification
- Business logic specific to the document type

Tasks

A **Task** is a special document type representing:

- Work to be performed
- Success/failure criteria
- Input/output specifications

- Delegation and completion tracking

Agreements

An **Agreement** is a mechanism for:

- Multiple agents to consent to terms
- Tracking signatures from all required parties
- Ensuring all participants have signed before proceeding
- Creating binding commitments between agents

How JACS Works

```
graph TD
    A[Agent A] -->|Creates Task| T[JACS Task Document]
    T -->|Contains| S[Digital Signature]
    T -->|Contains| H[SHA256 Hash]
    T -->|Contains| M[Metadata]

    A -->|Sends to| B[Agent B]
    B -->|Verifies| T
    B -->|Signs Agreement| AG[Agreement Document]
    AG -->|Returns to| A
```

1. **Agent A** creates a task document with their requirements
2. The document is **signed** with Agent A's private key
3. A **hash** is calculated for integrity verification
4. **Agent B** receives and verifies the signature and hash
5. Agent B can **create an agreement** to accept the task
6. Both agents have a **verifiable record** of the interaction

Real-World Examples

🤖 AI Content Pipeline

```
Content Agent → Research Agent → Review Agent → Publishing Agent
```

Each handoff includes signed task documents with clear requirements and deliverables.

📊 Data Processing Workflow

```
Data Ingestion Agent → Processing Agent → Validation Agent → Storage Agent
```

Each step is tracked with verifiable completion certificates and quality metrics.

🔍 Multi-Agent Analysis

```
Query Agent → Research Agent → Analysis Agent → Reporting Agent
```

Complex analysis tasks are broken down with clear accountability for each step.

Benefits Over Alternatives

Feature	JACS	Traditional APIs	General Signing
Agent Identity	✅ Built-in	❌ Custom implementation	❌ Not agent-focused
Task Management	⚠️ Schema-native (lifecycle via integrations)	❌ Application-specific	❌ Not applicable
Multi-Party Agreements	✅ Core feature	❌ Complex to implement	⚠️ Possible but difficult
Audit Trails	✅ Automatic	❌ Manual logging	⚠️ Basic signing only
Schema Validation	✅ JSON Schema	❌ Custom validation	❌ No structure
Versioning	✅ Built-in	❌ Manual versioning	❌ Not supported
Cross-Platform	✅ JSON everywhere	⚠️ Protocol dependent	⚠️ Format dependent

---

JACS provides signed artifacts, schemas, trust primitives, and auditability. Real-time transport and task orchestration are handled by integrations (e.g., A2A, MCP, HTTP server layers).

When to Use JACS

✅ **Perfect for:**

- Multi-agent AI systems
- Task delegation and tracking
- Audit trail requirements
- Cross-organization AI collaboration
- Compliance-critical AI applications
- Research environments with multiple AI models

⚠️ **Consider alternatives for:**

- Simple single-agent systems
- Real-time streaming data
- High-frequency micro-transactions
- Systems where trust is not a concern

Next Steps

Ready to dive deeper? Continue with:

- **Core Concepts** - Learn about agents, documents, and agreements
- **Quick Start** - Get hands-on experience
- **Implementation guides** for Rust, Node.js, or Python

Which JACS Path Should I Use?

Choose the smallest supported integration that matches your deployment.

Start Here

If you need...	Start here	Why
Signed tool outputs inside **LangChain / LangGraph** on Python	Python Framework Adapters	Smallest path: sign tool results without adding MCP
Signed tool outputs inside **LangChain.js / LangGraph** on Node	Node.js LangChain.js	Same idea for TypeScript
A ready-made **local MCP server** for Claude, Codex, or another MCP client	MCP Overview and `jacs-mcp`	Fastest full server path
To secure your **existing MCP server/client code**	Python MCP or Node.js MCP	Use wrappers or transport proxies around code you already have
Cross-organization agent discovery and signed artifact exchange	A2A Interoperability	MCP is not enough for this boundary
Signed HTTP APIs without adopting MCP	Python Framework Adapters, Express, Koa	Sign requests or responses at the web layer
Multi-party approval or quorum workflows	Multi-Agent Agreements	Agreements are the right primitive, not just one-off signatures
Direct signing from scripts, jobs, or services	Quick Start, Python Basic Usage, Node Basic Usage, Go Installation	Start from sign/verify before adding framework layers

When You Probably Do Not Need JACS

- Everything stays inside one service you control and your own logs are enough
- You only need integrity, not signer identity or third-party verification
- A plain checksum or database audit log already satisfies the requirement

Recommended Adoption Order

1. **Prototype** with quickstart and simple sign/verify calls.
2. **Attach provenance** at the boundary that already exists in your system: LangChain tool, FastAPI response, MCP call, or A2A artifact.
3. **Add trust policy** only when other agents or organizations enter the picture.
4. **Add agreements, DNS, or attestations** only if your deployment actually needs them.

The mistake to avoid is starting with the broadest story. Start with the boundary you need to secure now.

Use Cases

This chapter stays close to current product use, not roadmap integrations.

1. Secure A Local MCP Tool Server

Use this when: Claude Desktop, Codex, or another MCP client is calling tools that should not run on blind trust.

Recommended JACS path:

- Use `jacs-mcp` if you want a full server immediately
- Use Python MCP Integration or Node.js MCP Integration if you already have server code

What JACS adds:

- Signed JSON-RPC messages
- Fail-closed verification by default
- Agent identity and auditability for tool calls

2. Add Provenance To LangChain Or LangGraph

Use this when: your model already runs inside LangChain or LangGraph and you want signed tool outputs without introducing MCP.

Recommended JACS path:

- Python Framework Adapters
- Node.js LangChain.js

What JACS adds:

- Signed tool results
- Optional strict mode at the adapter boundary
- Minimal changes to existing framework code

3. Exchange Signed Artifacts Across Organizations

Use this when: one agent produces work that another organization, service, or team must verify before acting on it.

Recommended JACS path:

- A2A Interoperability
- A2A Quickstart

What JACS adds:

- Agent Cards with JACS provenance metadata
- Signed A2A artifacts
- Trust policies for admission control

4. Sign HTTP Or API Boundaries Without MCP

Use this when: the boundary is an API route, not an MCP transport.

Recommended JACS path:

- Python Framework Adapters for FastAPI
- Express Middleware
- Koa Middleware

What JACS adds:

- Signed JSON responses
- Verified inbound requests
- A clean upgrade path to A2A discovery on the same app boundary

5. Run Multi-Agent Approval Workflows

Use this when: multiple agents must sign off on the same document, deployment, or decision.

Recommended JACS path:

- Multi-Agent Agreements
- Rust Agreements

What JACS adds:

- M-of-N quorum
- Timeout and algorithm constraints
- Verifiable signature chain across signers

6. Keep Signed Files Or JSON As Durable Artifacts

Use this when: you need an artifact to stay verifiable after it leaves the process that created it.

Recommended JACS path:

- Verifying Signed Documents
- Working with Documents
- Python Basic Usage
- Node.js Basic Usage

What JACS adds:

- Self-contained signed envelopes
- Re-verification at read time
- Cross-language interoperability

7. Publish Public Identity Without A Central Auth Service

Use this when: external systems need to verify your agent identity but you do not want a shared auth server in the middle.

Recommended JACS path:

- DNS-Based Verification
- DNS Trust Anchoring

What JACS adds:

- Public key fingerprint anchoring
- DNS-based verification flows
- Local private-key custody

Core Concepts

Understanding JACS requires familiarity with several key concepts that work together to create a secure, verifiable communication framework for AI agents.

Agents

An **Agent** is the fundamental entity in JACS - an autonomous participant that can create, sign, and verify documents.

Agent Identity

```
{
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsVersion": "123e4567-e89b-12d3-a456-426614174000",
  "jacsType": "agent",
  "name": "Content Creation Agent",
  "description": "Specialized in creating marketing content"
}
```

Key Properties:

- **jacsId**: Permanent UUID identifying the agent
- **jacsVersion**: UUID that changes with each update
- **Cryptographic Keys**: Ed25519, RSA, or post-quantum key pairs
- **Services**: Capabilities the agent offers
- **Contacts**: How to reach the agent

Agent Lifecycle

1. **Creation**: Generate keys and initial agent document
2. **Registration**: Store public keys for verification
3. **Operation**: Create and sign documents
4. **Updates**: Version changes while maintaining identity
5. **Verification**: Other agents validate signatures

Verification: load() vs verify_standalone()

When **consuming** signed documents, you can verify in two ways:

- **With a loaded agent** (`load(config)` first): Call `verify(signedDocument)` . The loaded agent uses its config (trust store, key resolution) to resolve the signer's public key and verify the signature. Use this when your process already has a JACS config (e.g. it also signs) or when you want to use a specific key directory and resolution order.
- **Without loading an agent** (one-off verification): Call `verify_standalone(signedDocument, options)` (or the language equivalent: `verifyStandalone` , `VerifyStandalone`). This verifies the document using only the options you pass (e.g. `keyResolution` , `keyDirectory`). No config file or persistent agent state is required. Use this in lightweight services that only need to verify incoming documents.

Documents

A **Document** is any JSON object that follows JACS conventions for identity, versioning, and cryptographic integrity.

Document Structure

```
{
  "jacsId": "doc-uuid-here",
  "jacsVersion": "version-uuid-here",
  "jacsType": "task",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsPreviousVersion": "previous-version-uuid",

  "title": "Analyze Q4 Sales Data",
  "description": "Generate insights from sales data",

  "jacsSha256": "hash-of-document-content",
  "jacsSignature": {
    "agentID": "agent-uuid",
    "agentVersion": "agent-version-uuid",
    "signature": "base64-signature",
    "signingAlgorithm": "ring-Ed25519",
    "publicKeyHash": "hash-of-public-key",
    "date": "2024-01-15T10:30:00Z",
    "fields": ["jacsId", "title", "description"]
  }
}
```

Required JACS Fields

Field	Purpose	Example
`$schema`	JSON Schema reference	URL to schema
`jacsId`	Permanent document identifier	UUID v4

Field	Purpose	Example
`jacsVersion`	Version identifier (changes on update)	UUID v4
`jacsType`	Document type	"agent", "task", "message"
`jacsVersionDate`	When this version was created	RFC 3339 timestamp
`jacsOriginalVersion`	Original version UUID	UUID v4
`jacsOriginalDate`	Original creation timestamp	RFC 3339 timestamp
`jacsLevel`	Data level/intent	"raw", "config", "artifact", "derived"
`jacsPreviousVersion`	Previous version UUID (optional)	UUID v4 or null
`jacsSha256`	Hash of document content	SHA-256 hex string
`jacsSignature`	Cryptographic signature	Signature object

Document Types

Agent Documents

- Define agent identity and capabilities
- Contain service definitions and contact information
- Self-signed by the agent

Task Documents

- Describe work to be performed
- Include success/failure criteria
- Can be delegated between agents

Message Documents

- General communication between agents
- Can include attachments and metadata
- Support threaded conversations

Agreement Documents

- Multi-party consent mechanisms
- Track required and actual signatures
- Enforce completion before proceeding

Tasks

Tasks represent work that can be delegated, tracked, and verified between agents.

Task Structure

```
{
  "jacsType": "task",
  "title": "Generate Marketing Copy",
  "description": "Create compelling copy for product launch",

  "actions": [
    {
      "id": "research",
      "name": "Research competitors",
      "description": "Analyze competitor messaging",
      "success": "Complete competitive analysis report",
      "failure": "Unable to access competitor data"
    }
  ],

  "jacsTaskCustomer": {
    "agentID": "customer-agent-uuid",
    "signature": "customer-signature"
  }
}
```

Task Lifecycle

1. **Creation**: Customer agent creates task with requirements
2. **Delegation**: Task sent to service provider agent
3. **Agreement**: Provider signs agreement to accept task
4. **Execution**: Provider performs the work
5. **Completion**: Provider creates completion document
6. **Verification**: Customer verifies and accepts results

Task Components

Actions: Individual steps within a task

- **id**: Unique identifier within the task
- **name**: Human-readable action name
- **description**: Detailed requirements
- **success**: Definition of successful completion
- **failure**: What constitutes failure

Services: Required capabilities

- **type**: Service category
- **requirements**: Specific needs
- **constraints**: Limitations or restrictions

Agreements

Agreements enable multi-party consent and coordination between agents.

Agreement Structure

```
{
  "jacsType": "agreement",
  "title": "Task Acceptance Agreement",
  "question": "Do you agree to complete the marketing copy task?",
  "context": "Task ID: abc123, Deadline: 2024-01-20",

  "agents": [
    "agent-1-uuid",
    "agent-2-uuid",
    "agent-3-uuid"
  ],

  "jacsAgreement": {
    "agent-1-uuid": {
      "agentID": "agent-1-uuid",
      "signature": "base64-signature",
      "date": "2024-01-15T10:30:00Z"
    },
    "agent-2-uuid": {
      "agentID": "agent-2-uuid",
      "signature": "base64-signature",
      "date": "2024-01-15T11:15:00Z"
    }
    // agent-3-uuid has not signed yet
  },

  "jacsAgreementHash": "hash-of-agreement-content"
}
```

Agreement Process

1. **Creation**: Initial agent creates agreement with required participants
2. **Distribution**: Agreement sent to all required agents
3. **Review**: Each agent reviews terms and conditions
4. **Signing**: Agents add their signatures if they consent

5. **Completion**: Agreement becomes binding when all parties have signed
6. **Verification**: Any party can verify all signatures

Agreement Types

Task Agreements: Consent to perform specific work **Service Agreements**: Long-term service provision contracts
Data Sharing Agreements: Permission to access or use data **Update Agreements**: Consent to system or process changes

Cryptographic Security

JACS uses industry-standard cryptographic primitives for security.

Supported Algorithms

Current Standards

- **ring-Ed25519**: Fast elliptic curve signatures using the ring library (recommended)
- **RSA-PSS**: Traditional RSA with probabilistic signature scheme

Post-Quantum

- **pq-dilithium**: NIST-standardized post-quantum signatures

Signature Process

1. **Content Extraction**: Specific fields are extracted for signing
2. **Canonicalization**: Fields are sorted and formatted consistently
3. **Hashing**: SHA-256 hash of the canonical content
4. **Signing**: Private key signs the hash
5. **Verification**: Public key verifies the signature

Key Management

- **Agent Keys**: Each agent has a unique key pair
- **Public Key Distribution**: Public keys shared through secure channels
- **Key Rotation**: Agents can update keys while maintaining identity
- **Key Verification**: Public key hashes ensure integrity

Versioning and Audit Trails

JACS provides comprehensive versioning for tracking document evolution.

Version Management

- **Immutable IDs**: `jacsId` never changes for a document
- **Version IDs**: `jacsVersion` changes with each update
- **Previous Versions**: `jacsPreviousVersion` creates a chain
- **Timestamps**: `jacsVersionDate` provides chronological order

Audit Trail Benefits

- **Complete History**: Track all changes to any document
- **Attribution**: Know exactly who made each change
- **Verification**: Cryptographic proof of authenticity
- **Compliance**: Meet regulatory audit requirements

Storage and Transport

JACS documents are designed to be storage and transport agnostic.

Storage Options

- **File System**: Simple JSON files
- **Databases**: Store as JSON/JSONB fields
- **Object Storage**: S3, Azure Blob, Google Cloud Storage
- **Version Control**: Git repositories for change tracking

Transport Mechanisms

- **HTTP APIs**: RESTful or GraphQL endpoints
- **Message Queues**: RabbitMQ, Kafka, SQS
- **Email**: Documents as attachments
- **Direct Transfer**: USB drives, file sharing

Format Compatibility

- **JSON**: Universal compatibility across all systems
- **Schema Validation**: Ensures consistent structure
- **Self-Contained**: All necessary information in the document
- **Human Readable**: Can be inspected and debugged easily

Next Steps

Now that you understand the core concepts:

1. **Quick Start** - Try JACS hands-on
2. **Choose Implementation**:
    - Rust CLI for command-line usage
    - Node.js for web applications
    - Python for AI/ML workflows
3. **Examples** - See real-world usage patterns

Quick Start Guide

Get signing and verifying in under a minute. No manual setup needed.

Zero-Config Quick Start

`quickstart(name, domain, ...)` creates a persistent agent with keys on disk (default algorithm: `pq2025`). If `./jacs.config.json` already exists, it loads it; otherwise it creates a new agent. Returned `AgentInfo` includes config/key paths (`config_path` / `public_key_path` / `private_key_path` in Python, `configPath` / `publicKeyPath` / `privateKeyPath` in Node) plus key directory metadata so you can locate key material immediately. Rust/CLI quickstart requires an explicit password source (`JACS_PRIVATE_KEY_PASSWORD` , or `JACS_PASSWORD_FILE` in CLI). Python/Node can auto-generate a password if needed; set `JACS_SAVE_PASSWORD_FILE=true` if you want that generated password persisted to `./jacs_keys/.jacs_password` .

Password bootstrap

Rust CLI quickstart requires exactly one explicit password source:

```
# Recommended
export JACS_PRIVATE_KEY_PASSWORD='use-a-strong-password'

# CLI convenience (file contains only the password)
export JACS_PASSWORD_FILE=/secure/path/jacs-password.txt
```

If both `JACS_PRIVATE_KEY_PASSWORD` and `JACS_PASSWORD_FILE` are set, CLI fails fast to avoid ambiguity. Python/Node quickstart can auto-generate a secure password if `JACS_PRIVATE_KEY_PASSWORD` is unset. Set `JACS_SAVE_PASSWORD_FILE=true` if you want the generated password persisted to `./jacs_keys/.jacs_password` . In production, set `JACS_PRIVATE_KEY_PASSWORD` explicitly. One call and you're signing.

Python | Node.js

```
pip install jacs
```

```python
import jacs.simple as jacs

info = jacs.quickstart(name="my-agent", domain="my-agent.example.com")
print(info.config_path, info.public_key_path, info.private_key_path)
signed = jacs.sign_message({"action": "approve", "amount": 100})
result = jacs.verify(signed.raw)
print(f"Valid: {result.valid}, Signer: {result.signer_id}")
```

Rust CLI

Pass `algorithm="ring-Ed25519"` (or `{ algorithm: 'ring-Ed25519' }` in JS, `--algorithm ring-Ed25519` in CLI) to override the default (`pq2025`).

> **That's it -- you're signing.** For most use cases, the quick start above is all you need. Jump to Which integration should I use? to find the right framework adapter, or read on for manual agent setup.

macOS Homebrew install (Rust CLI)

```
brew tap HumanAssisted/homebrew-jacs
brew install jacs
```

MCP server (Rust CLI)

The MCP server is built into the `jacs` binary. No separate install step needed.

```
# Start the MCP server (stdio transport)
jacs mcp
```

Advanced: Explicit Agent Setup

For full control over agent creation, you can set up an agent manually with a config file and `JACS_PRIVATE_KEY_PASSWORD` environment variable. This is optional since `quickstart(...)` already creates a persistent agent.

Rust CLI	Node.js	Python

Install

```
cargo install jacs-cli
```

Initialize

```
# Create configuration and agent in one step
jacs init

# Or step by step:
# 1. Create config
jacs config create
# 2. Create agent with keys
jacs agent create --create-keys true
# 3. Verify
jacs agent verify
```

Sign a document

```
jacs document create -f mydata.json
```

Programmatic Agent Creation (v0.6.0+)

For scripts, CI/CD, and server environments where you need agents created programmatically with explicit parameters (without interactive prompts), use `create()`. For most cases, `quickstart(...)` above is simpler and also creates a persistent agent.

Python	Node.js

```python
import jacs.simple as jacs

agent = jacs.create(
    name="my-agent",
    password="Str0ng-P@ssw0rd!",  # or set JACS_PRIVATE_KEY_PASSWORD
    algorithm="pq2025",
)
print(f"Agent: {agent.agent_id}")
```

Go	Rust

Password requirements: At least 8 characters, with uppercase, lowercase, a digit, and a special character.

Algorithm note: `pq-dilithium` is deprecated in v0.6.0. Use `pq2025` (ML-DSA-87, FIPS-204) instead.

Understanding What Happened

When you completed the quick start, several important things occurred:

1. Agent Creation

- A unique identity (UUID) was generated for your agent
- Cryptographic key pair was created for signing
- Agent document was created and self-signed
- Public key was stored for verification

2. Configuration Setup

- Storage directories were configured
- Cryptographic algorithm was selected
- Agent identity was linked to configuration

3. Task Creation

- Task document was structured according to JACS schema
- Document was signed with your agent's private key
- SHA-256 hash was calculated for integrity
- Signature metadata was embedded in the document

Verify Everything Works

Let's verify that the documents are properly signed and can be validated:

🦀 Rust | 🟢 Node.js | 🐍 Python

```
# Verify agent signature
jacs agent verify

# Verify a specific document
jacs document verify -f ./jacs_data/[document-id].json

# Sign a document
jacs document sign -f ./jacs_data/[document-id].json
```

Next Steps: Multi-Agent Workflow

Now let's create a second agent and demonstrate inter-agent communication:

🦀 Rust

```
# Create a second agent configuration
cp jacs.config.json reviewer.config.json
# Edit reviewer.config.json to set jacs_agent_id_and_version to null

# Create reviewer agent (uses JACS_CONFIG_PATH environment variable)
JACS_CONFIG_PATH=./reviewer.config.json jacs agent create --create-keys true

# Create an agreement on a document
jacs agreement create -f ./document.json \
  --agents [agent-1-id],[agent-2-id] \
  --question "Do you agree to collaborate on this content task?"

# Sign the agreement as first agent
jacs agreement sign -f ./document.json

# Sign as second agent (using reviewer config)
JACS_CONFIG_PATH=./reviewer.config.json jacs agreement sign -f ./document.json

# Verify agreement is complete
jacs agreement check -f ./document.json
```

🟢 Node.js	🐍 Python

What You've Accomplished

Congratulations! You've successfully:

✅ **Created JACS agents** with cryptographic identities ✅ **Generated and signed documents** with verifiable integrity
✅ **Established multi-agent agreements** with cryptographic consent ✅ **Verified signatures** and document authenticity ✅ **Created an audit trail** of all interactions

Key Takeaways

- **Everything is verifiable**: All documents have cryptographic signatures
- **Agents are autonomous**: Each has its own identity and keys
- **Agreements enable trust**: Multi-party consent before proceeding
- **Audit trails are automatic**: Complete history of all interactions
- **JSON is universal**: Documents work everywhere

Where to Go Next

Now that you have the basics working:

1. **Verify Signed Documents** - Verify any document from CLI, Python, or Node.js -- no agent required
2. **A2A Quickstart** - Make your agent discoverable by other A2A agents in minutes
3. **Framework Adapters** - Add auto-signing to LangChain, FastAPI, CrewAI, or Anthropic SDK in 1-3 lines
4. **Multi-Agent Agreements** - Cross-trust-boundary verification with quorum and timeout
5. **Rust Deep Dive** - Learn the full Rust API
6. **Node.js Integration** - Add MCP support
7. **Python MCP** - Build authenticated MCP servers
8. **Production Security** - Harden runtime settings and key management
9. **Real Examples** - See production patterns

Troubleshooting

Agent creation fails: Check that the data and key directories exist and are writable **Signature verification fails**: Ensure public keys are properly stored and accessible **Agreement signing fails**: Verify all agent IDs are correct and agents exist **Documents not found**: Check the data directory configuration

Need help? Check the GitHub issues or review the detailed implementation guides.

Multi-Agent Agreements

Three agents from different organizations sign an agreement with 2-of-3 quorum.

Imagine three departments -- Finance, Compliance, and Legal -- must approve a production deployment. Requiring all three creates bottlenecks. With JACS quorum agreements, any two of three is sufficient: cryptographically signed, independently verifiable, with a full audit trail.

No central authority. No shared database. Every signature is independently verifiable.

The Lifecycle

```
Create Agreement --> Agent A Signs --> Agent B Signs --> Quorum Met (2/3) --> Verified
```

Python

```python
from jacs.client import JacsClient

# Step 1: Create three agents (one per organization)
finance = JacsClient.quickstart(
    name="finance",
    domain="finance.example.com",
    algorithm="ring-Ed25519",
    config_path="./finance.config.json",
)
compliance = JacsClient.quickstart(
    name="compliance",
    domain="compliance.example.com",
    algorithm="ring-Ed25519",
    config_path="./compliance.config.json",
)
legal = JacsClient.quickstart(
    name="legal",
    domain="legal.example.com",
    algorithm="ring-Ed25519",
    config_path="./legal.config.json",
)

# Step 2: Finance proposes an agreement with quorum
from datetime import datetime, timedelta, timezone

proposal = {
    "action": "Deploy model v2 to production",
    "conditions": ["passes safety audit", "approved by 2 of 3 signers"],
}
deadline = (datetime.now(timezone.utc) + timedelta(hours=1)).isoformat()

agreement = finance.create_agreement(
    document=proposal,
    agent_ids=[finance.agent_id, compliance.agent_id, legal.agent_id],
    question="Do you approve deployment of model v2?",
    context="Production rollout pending safety audit sign-off.",
    quorum=2,           # only 2 of 3 need to sign
    timeout=deadline,
)

# Step 3: Finance signs
agreement = finance.sign_agreement(agreement)

# Step 4: Compliance co-signs -- quorum is now met
agreement = compliance.sign_agreement(agreement)

# Step 5: Verify -- any party can confirm independently
status = finance.check_agreement(agreement)
print(f"Complete: {status.complete}")  # True -- 2 of 3 signed

for s in status.signers:
    label = "signed" if s.signed else "pending"
    print(f"  {s.agent_id[:12]}... {label}")
```

Node.js / TypeScript

```typescript
import { JacsClient } from "@hai.ai/jacs/client";

async function main() {
  // Step 1: Create three agents
  const finance    = await JacsClient.ephemeral("ring-Ed25519");
  const compliance = await JacsClient.ephemeral("ring-Ed25519");
  const legal      = await JacsClient.ephemeral("ring-Ed25519");

  // Step 2: Finance proposes an agreement with quorum
  const proposal = {
    action: "Deploy model v2 to production",
    conditions: ["passes safety audit", "approved by 2 of 3 signers"],
  };
  const deadline = new Date(Date.now() + 60 * 60 * 1000).toISOString();
  const agentIds = [finance.agentId, compliance.agentId, legal.agentId];

  let agreement = await finance.createAgreement(proposal, agentIds, {
    question: "Do you approve deployment of model v2?",
    context: "Production rollout pending safety audit sign-off.",
    quorum: 2,
    timeout: deadline,
  });

  // Step 3: Finance signs
  agreement = await finance.signAgreement(agreement);

  // Step 4: Compliance co-signs -- quorum is now met
  agreement = await compliance.signAgreement(agreement);

  // Step 5: Verify
  const doc = JSON.parse(agreement.raw);
  const ag = doc.jacsAgreement;
  const sigCount = ag.signatures?.length ?? 0;
  console.log(`Signatures: ${sigCount} of ${agentIds.length}`);
  console.log(`Quorum met: ${sigCount >= (ag.quorum ?? agentIds.length)}`);
}

main().catch(console.error);
```

What Just Happened?

1. **Three independent agents** were created, each with their own keys -- no shared secrets.
2. **Finance proposed** an agreement requiring 2-of-3 quorum with a one-hour deadline.
3. **Finance and Compliance signed.** Legal never needed to act -- quorum was met.
4. **Any party can verify** the agreement independently. The cryptographic proof chain is self-contained.

Every signature includes: the signer's agent ID, the signing algorithm, a timestamp, and a hash of the agreement content. If anyone tampers with the document after signing, verification fails.

Next Steps

- Agreements API Reference -- timeout, algorithm constraints, and more
- Python Framework Adapters -- use agreements inside LangChain, FastAPI, CrewAI
- Security Model -- how the cryptographic proof chain works

Verifying Signed Documents

Verify a JACS-signed document in under 2 minutes. Verification confirms two things: the document was signed by the claimed agent, and the content has not been modified since signing.

Verification does NOT require creating an agent. You only need the signed document (and optionally access to the signer's public key).

CLI: `jacs verify`

The fastest way to verify a document from the command line. No config file, no agent setup.

```
# Verify a local file
jacs verify signed-document.json

# Verify with JSON output (for scripting)
jacs verify signed-document.json --json

# Verify a remote document by URL
jacs verify --remote https://example.com/signed-doc.json

# Specify a directory containing public keys
jacs verify signed-document.json --key-dir ./trusted-keys/
```

Output on success:

```
Status:    VALID
Signer:    550e8400-e29b-41d4-a716-446655440000
Signed at: 2026-02-10T12:00:00Z
```

JSON output (`--json`):

```
{
  "valid": true,
  "signerId": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-02-10T12:00:00Z"
}
```

The exit code is `0` for valid, `1` for invalid or error. Use this in CI/CD pipelines:

```
if jacs verify artifact.json --json; then
  echo "Artifact is authentic"
else
  echo "Verification failed" >&2
  exit 1
fi
```

If a `jacs.config.json` and agent keys exist in the current directory, the CLI uses them automatically. Otherwise, it creates a temporary ephemeral verifier internally.

Python

With an agent loaded

```python
import jacs.simple as jacs

jacs.load("./jacs.config.json")

result = jacs.verify(signed_json)
if result.valid:
    print(f"Signed by: {result.signer_id}")
else:
    print(f"Errors: {result.errors}")
```

Without an agent (standalone)

```python
import jacs.simple as jacs

result = jacs.verify_standalone(
    signed_json,
    key_resolution="local",
    key_directory="./trusted-keys/"
)
print(f"Valid: {result.valid}, Signer: {result.signer_id}")
```

`verify_standalone` does not use a global agent. Pass the key resolution strategy and directories explicitly.

Verify by document ID

If the document is in local storage and you know its ID:

```python
result = jacs.verify_by_id("550e8400-e29b-41d4:1")
```

Node.js

With an agent loaded

```
import * as jacs from '@hai.ai/jacs/simple';

await jacs.load('./jacs.config.json');

const result = await jacs.verify(signedJson);
console.log(`Valid: ${result.valid}, Signer: ${result.signerId}`);
```

Without an agent (standalone)

```
import { verifyStandalone } from '@hai.ai/jacs/simple';

const result = verifyStandalone(signedJson, {
  keyResolution: 'local',
  keyDirectory: './trusted-keys/',
});
console.log(`Valid: ${result.valid}, Signer: ${result.signerId}`);
```

Verify by document ID

```
const result = await jacs.verifyById('550e8400-e29b-41d4:1');
```

Verification Links

Generate a URL that lets anyone verify a signed document through a web verifier (e.g., hai.ai):

Python:

```
url = jacs.generate_verify_link(signed_doc.raw_json)
# https://hai.ai/jacs/verify?s=<base64url-encoded-document>
```

Node.js:

```
const url = jacs.generateVerifyLink(signed.raw);
```

The document is base64url-encoded into the URL query parameter. Documents must be under ~1.5 KB to fit within the 2048-character URL limit. For larger documents, share the file directly and verify with the CLI or SDK.

DNS Verification

DNS verification checks that an agent's public key hash matches a DNS TXT record published at `_v1.agent.jacs.<domain>`. This provides a decentralized trust anchor: anyone can look up the agent's expected key fingerprint via DNS without contacting a central server.

Publishing a DNS record

```
jacs agent dns --domain example.com --provider plain
```

This outputs the TXT record to add to your DNS zone. Provider options: `plain`, `aws`, `azure`, `cloudflare`.

Looking up an agent by domain

```
jacs agent lookup example.com
```

This fetches the agent's public key from `https://example.com/.well-known/jacs-pubkey.json` and checks the DNS TXT record at `_v1.agent.jacs.example.com`.

CLI verification with DNS

```
# Require DNS validation (fail if no DNS record)
jacs agent verify --require-dns

# Require strict DNSSEC validation
jacs agent verify --require-strict-dns
```

For full DNS setup instructions, see DNS-Based Verification and DNS Trust Anchoring.

Cross-Language Verification

JACS signatures are language-agnostic. A document signed by a Rust agent verifies identically in Python and Node.js, and vice versa. This holds for both Ed25519 and post-quantum (ML-DSA-87/pq2025) algorithms.

This is tested on every commit: Rust generates signed fixtures, then Python calls `verify_standalone()` and Node.js calls `verifyStandalone()` to verify them. Each binding also countersigns the fixture with a different algorithm, proving round-trip interoperability.

Test sources:

- Rust fixture generator: `jacs/tests/cross_language/mod.rs`
- Python consumer: `jacspy/tests/test_cross_language.py`
- Node.js consumer: `jacsnpm/test/cross-language.test.js`

Key Resolution Order

When verifying a document, JACS resolves the signer's public key in a configurable order. Set `JACS_KEY_RESOLUTION` to control this:

Value	Source
`local`	Local trust store (added via `trust_agent`)
`dns`	DNS TXT record lookup
`hai`	HAI key distribution service

Default: `local,hai`. Example: `JACS_KEY_RESOLUTION=local,dns,hai`.

What Is an Attestation?

> **Signing says WHO. Attestation says WHO plus WHY.**

A JACS attestation is a cryptographically signed document that goes beyond proving *who* signed something. It records *why* a piece of data should be trusted -- the evidence, the claims, and the reasoning behind that trust.

Signing vs. Attestation

	`sign_message()`	`create_attestation()`
Proves	Who signed it	Who signed it + why it's trustworthy
Contains	Signature + hash	Signature + hash + subject + claims + evidence
Use case	Data integrity	Trust decisions, audit trails, compliance
Verification	Was it tampered with?	Was it tampered with? Are the claims valid? Is the evidence fresh?

Key Concepts

Subject

What is being attested. Every attestation targets a specific *subject* -- an artifact, agent, workflow, or identity. The subject is identified by type, ID, and cryptographic digests.

Claims

What you assert about the subject. Claims are structured statements with a name, value, optional confidence score (0.0-1.0), and assurance level (`self-asserted` , `verified` , or `independently-attested`).

Evidence

What supports the claims. Evidence references link to external proofs (A2A messages, email headers, JWT tokens, TLS notary sessions) with their own digests and timestamps.

Derivation Chain

How the attestation was produced. When one attestation builds on another -- for example, a review attestation that references an earlier scan attestation -- the derivation chain captures the full transformation history.

Architecture Layers

```
Layer 2: Adapters (A2A, email, JWT, TLSNotary)
           |
Layer 1: Attestation Engine (create, verify, lift, DSSE export)
           |
Layer 0: JACS Core (sign, verify, agreements, storage)
```

Attestations build *on top of* existing JACS signing. Every attestation is also a valid signed JACS document. You can verify an attestation with `verify()` for signature checks, or use `verify_attestation()` for the full trust evaluation.

Quick Example

```python
from jacs.client import JacsClient

client = JacsClient.ephemeral(algorithm="ed25519")

# Sign a document (Layer 0)
signed = client.sign_message({"action": "approve", "amount": 100})

# Attest WHY it's trustworthy (Layer 1)
att = client.create_attestation(
    subject={"type": "artifact", "id": signed.document_id,
             "digests": {"sha256": "..."}},
    claims=[{"name": "reviewed", "value": True, "confidence": 0.95}],
)

# Verify the full trust chain
result = client.verify_attestation(att.raw_json, full=True)
print(f"Valid: {result['valid']}")
```

Attestation vs. A2A Trust Policy

Attestation (Layer C) provides trust context: claims, evidence, and derivation chains. It answers *"why should this data be trusted?"* A2A trust policy (Layer B) handles agent admission: *"is this agent allowed to communicate?"*

For transport trust decisions, see A2A Interoperability. For how attestation and A2A compose, see A2A + Attestation Composition. For the full three-layer model, see Trust Layers.

When to Use Attestations

Use attestations when you need to answer questions like:

- **Why should I trust this data?** (claims + evidence)
- **Who reviewed it and when?** (issuer, timestamps, assurance level)
- **How was it produced?** (derivation chain)
- **Can I independently verify the trust chain?** (DSSE export, evidence verification)

If you only need to prove *who* signed something and that it hasn't been tampered with, `sign_message()` is sufficient. See Sign vs. Attest for a detailed decision guide.

JACS Trust Layers

JACS organizes trust into three distinct layers. Each layer has a clear scope and its own vocabulary. Understanding which layer you need prevents confusion between identity, transport policy, and evidentiary trust.

The Three Layers

Layer A: Identity + Integrity (JACS Core)

Scope: Who signed what, and has it been tampered with?

APIs: `sign_message()`, `verify()`, `verify_standalone()`

This is the foundation. Every JACS document carries a cryptographic signature that proves which agent created it and that the content hasn't changed. Layer A answers: *"Is this signature valid?"*

Crypto status values: `Verified` · `SelfSigned` · `Unverified` · `Invalid`

- **Verified**: Signature is valid and signer's key was resolved from a trusted source.
- **SelfSigned**: Signature is valid but signer is the same as verifier (no third-party trust).
- **Unverified**: Signature could not be checked because the signer's key was not available.
- **Invalid**: Signature check failed — the content was tampered with or the wrong key was used.

Layer B: Exchange + Discovery (A2A Integration)

Scope: Is this agent allowed to communicate with me?

APIs: `sign_artifact()`, `verify_wrapped_artifact()`, `assess_remote_agent()`, `discover_agent()`

Layer B handles cross-boundary exchange between agents using the A2A protocol. It adds trust *policy* on top of Layer A's cryptographic status. Layer B answers: *"Should I accept artifacts from this agent?"*

Policy status values: `allowed` · `blocked` · `not_assessed`

Trust policies (`open`, `verified`, `strict`) control admission:

Policy	Requirement
`open`	Accept all agents
`verified`	Agent must have the `urn:jacs:provenance-v1` extension

Policy	Requirement
`strict`	Agent must be in the local trust store

See A2A Interoperability for full details.

Layer C: Trust Context (Attestation)

Scope: Why should this data be trusted?

APIs: `create_attestation()`, `verify_attestation()`, `lift_to_attestation()`, `export_attestation_dsse()`

Layer C records the *reasoning* behind trust: claims, evidence, derivation chains, and assurance levels. Layer C answers: *"What evidence supports this data?"*

Attestation status values: `local_valid` · `full_valid`

- **local_valid**: Signature and hash are correct; claims are structurally valid.
- **full_valid**: All of the above, plus evidence digests verified and derivation chain intact.

See What Is an Attestation? for full details.

Terminology Glossary

Term	Layer	Meaning
Crypto status	A	Outcome of signature verification: `Verified`, `SelfSigned`, `Unverified`, `Invalid`
Policy status	B	Outcome of trust policy check: `allowed`, `blocked`, `not_assessed`
Attestation status	C	Outcome of attestation verification: `local_valid`, `full_valid`
Verified	A	Signature is valid and signer key was resolved
SelfSigned	A	Signature is valid but signer is the verifier
Unverified	A	Key not available — cannot check signature
Invalid	A	Signature check failed
Allowed	B	Agent passes the configured trust policy
Blocked	B	Agent does not pass the trust policy
Not assessed	B	No agent card provided — trust not evaluated

Quick Decision Flow

"Which layer do I need?"

1. **I just need to prove data hasn't been tampered with** → Layer A. Use `sign_message()` and `verify()`.
2. **I need to exchange signed data with other agents** → Layer B. Use `sign_artifact()` and A2A discovery. See the A2A Quickstart.
3. **I need to record WHY data should be trusted** → Layer C. Use `create_attestation()`. See the Attestation Tutorial.
4. **I need both exchange AND trust evidence** → Layer B + C. See A2A + Attestation Composition.

Common Misconceptions

- **"Unverified" does not mean "Invalid."** Unverified means the signer's key wasn't available. Invalid means the signature check actively failed. These have very different security implications.
- **A2A trust policy is not attestation verification.** A2A policy (Layer B) answers "should I talk to this agent?" Attestation (Layer C) answers "why should I trust this data?" They compose but are not interchangeable.
- **"Trusted" is not the same as "Verified."** In JACS, "trusted" refers to trust store membership (Layer B). "Verified" refers to cryptographic signature validation (Layer A).

Deployment Compatibility

JACS includes native bindings (Rust compiled to platform-specific libraries), so deployment depends on pre-built binary availability for your target platform.

Supported Platforms

Platform	Language	Notes
Linux (x86_64, aarch64)	All	Primary target
macOS (Apple Silicon, Intel)	All	Full support
Windows (x86_64)	Rust, Node.js	Python wheels may need manual build
AWS Lambda	Python, Node.js	Use Lambda layers for native deps
Docker / Kubernetes	All	Standard containerization
Vercel (Node.js runtime)	Node.js	Via serverless functions

Not Yet Supported

Platform	Why	Workaround
Cloudflare Workers	No native module support (WASM-only)	Use a proxy service
Deno Deploy	No native Node.js addons	Use Deno with `--allow-ffi` locally
Bun	Native builds may fail	Use Node.js runtime instead
Browser / WASM	Post-quantum crypto not available in WASM	Planned for a future release

Version Requirements

Language	Minimum Version
Rust	1.93+ (edition 2024)
Python	3.10+
Node.js	18+ (LTS recommended)

Docker Example

```
FROM python:3.12-slim
RUN pip install jacs
COPY . /app
WORKDIR /app
RUN python -c "import jacs.simple as j; j.quickstart(name='docker-agent',
domain='docker.local')"
CMD ["python", "main.py"]
```

Lambda Deployment

For AWS Lambda, include the JACS native library in a Lambda layer or bundle it in your deployment package. Set `JACS_PRIVATE_KEY_PASSWORD` as a Lambda environment variable (use AWS Secrets Manager for production).

Building from Source

If no pre-built binary exists for your platform:

```
# Python
pip install maturin
cd jacspy && maturin develop --release

# Node.js
cd jacsnpm && npm run build
```

Requires Rust 1.93+ toolchain installed via rustup.

Troubleshooting

Common issues and solutions when installing or using JACS.

Installation Issues

pip install fails

Check your Python version (3.10+ required). If no pre-built wheel exists for your platform, install the Rust toolchain and build from source:

```
pip install maturin
cd jacspy && maturin develop --release
```

npm install fails

Pre-built binaries are available for Linux/macOS/Windows x64 and ARM64 macOS. If no pre-built binary matches your platform, you need the Rust toolchain installed so the native addon can compile during `npm install`.

Alpine Linux / musl libc

The default wheels and binaries target glibc. On Alpine or other musl-based systems, build from source with the Rust toolchain, or use a Debian-based container image instead.

Configuration Issues

Config not found

Run `jacs quickstart --name my-agent --domain my-agent.example.com` to auto-create a config, or copy the example:

```
cp jacs.config.example.json jacs.config.json
```

Private key decryption failed

Wrong or missing password. Check `JACS_PRIVATE_KEY_PASSWORD`. For CLI, you may also set `JACS_PASSWORD_FILE` to a file that contains only the password. Set exactly one explicit source; if both are set, CLI fails by design.

Algorithm detection failed

Set the `signingAlgorithm` field in your config, or pass it explicitly to `quickstart(...)` / `create(...)`. Valid values: `pq2025`, `ring-Ed25519`, `RSA-PSS`.

Runtime Issues

Agent creation fails

Ensure the data and key directories exist and are writable. By default these are `./jacs_data` and `./jacs_keys`.

Signature verification fails

Ensure the signer's public key is accessible. If verifying a document from another agent, you may need to import their public key or use the trust store.

Documents not found

Check the `jacs_data_directory` path in your config. Documents are stored as JSON files in that directory.

Building from Source

```
git clone https://github.com/HumanAssisted/JACS.git
cd JACS

# Rust core + CLI
cargo build --release
cargo install --path jacs --features cli

# Python binding
cd jacspy && maturin develop --release

# Node.js binding
cd jacsnpm && npm run build
```

Requires Rust 1.93+ (install via rustup).

Getting Help

- GitHub Issues -- report bugs and feature requests
- Quick Start Guide -- step-by-step setup

Installation

This guide covers installing the JACS Rust CLI and library.

Requirements

- **Rust**: Version 1.93 or later (Edition 2024)
- **Cargo**: Included with Rust installation

Verify Rust Version

```
rustc --version
# Should show rustc 1.93.0 or later
```

If you need to update Rust:

```
rustup update stable
```

Installing the CLI

From crates.io (Recommended)

```
cargo install jacs-cli
```

From Homebrew (macOS)

```
brew tap HumanAssisted/homebrew-jacs
brew install jacs
```

From Source

```
git clone https://github.com/HumanAssisted/JACS
cd JACS
cargo install --path jacs-cli
```

Verify Installation

```
jacs --help
```

MCP Server

The MCP server is built into the `jacs` binary. No separate install step needed.

```
# Start the MCP server (stdio transport)
jacs mcp
```

Using as a Library

Add JACS to your `Cargo.toml`:

```
[dependencies]
jacs = "0.3"
```

With Optional Features

JACS supports several optional features for observability and integrations:

```
[dependencies]
# Basic library usage
jacs = "0.3"

# With OpenTelemetry logging
jacs = { version = "0.3", features = ["otlp-logs"] }

# With OpenTelemetry metrics
jacs = { version = "0.3", features = ["otlp-metrics"] }

# With OpenTelemetry tracing
jacs = { version = "0.3", features = ["otlp-tracing"] }

# With all observability features
jacs = { version = "0.3", features = ["otlp-logs", "otlp-metrics", "otlp-tracing"] }
```

Available Features

Feature	Description
`cli`	*(Deprecated -- use `cargo install jacs-cli` instead)*

Feature	Description
`otlp-logs`	OpenTelemetry Protocol logging backend
`otlp-metrics`	OpenTelemetry Protocol metrics backend
`otlp-tracing`	OpenTelemetry Protocol distributed tracing
`sqlite`	Lightweight sync SQLite backend (default)
`sqlx-sqlite`	Async SQLite backend via sqlx (requires tokio)
`agreements`	Agreement lifecycle support
`a2a`	Agent-to-Agent protocol support
`attestation`	Attestation support

Platform Support

JACS supports the following platforms:

Platform	Architecture	Support
Linux	x86_64, aarch64	Full support
macOS	x86_64, aarch64	Full support
Windows	x86_64	Full support
WebAssembly	wasm32	Partial (no post-quantum crypto, limited storage)

WebAssembly Notes

When targeting WebAssembly, some features are unavailable:

- Post-quantum cryptographic algorithms (`pq2025` , legacy `pq-dilithium`)
- File system storage backend
- HTTP-based remote operations

Configuration

After installation, initialize JACS:

```
# Create configuration and agent in one step
jacs init
```

This creates:

- `./jacs.config.json` - Configuration file

- Cryptographic keys for your agent
- Initial agent document

Manual Configuration

Alternatively, create configuration and agent separately:

```
# Create configuration only
jacs config create

# Create agent with keys
jacs agent create --create-keys true
```

Environment Variables

JACS respects the following environment variables:

Variable	Description	Default
`JACS_CONFIG_PATH`	Path to configuration file	`./jacs.config.json`
`JACS_USE_SECURITY`	Enable/disable security features	`true`
`JACS_DATA_DIRECTORY`	Directory for document storage	`./jacs_data`
`JACS_KEY_DIRECTORY`	Directory for cryptographic keys	`./jacs_keys`
`JACS_DEFAULT_STORAGE`	Storage backend (`fs` , `memory`)	`fs`
`JACS_AGENT_KEY_ALGORITHM`	Key algorithm (`ring-Ed25519` , `RSA-PSS` , `pq2025` , legacy `pq-dilithium`)	`ring-Ed25519`

Troubleshooting

Build Errors

"edition 2024 is required" Update Rust to version 1.93 or later:

```
rustup update stable
```

Missing dependencies on Linux Install build essentials:

```
# Debian/Ubuntu
sudo apt-get install build-essential pkg-config libssl-dev

# Fedora
sudo dnf install gcc openssl-devel
```

Runtime Errors

"Configuration file not found" Run `jacs init` or set `JACS_CONFIG_PATH` environment variable.

"Key directory does not exist" Create the key directory or run `jacs init`:

```
mkdir -p ./jacs_keys
```

"Permission denied" Ensure you have write permissions to the data and key directories.

Next Steps

- CLI Usage - Learn CLI commands
- Creating an Agent - Create your first agent
- Rust Library API - Use JACS as a library

CLI Tutorial

This page walks through common CLI workflows. For a complete command reference, see the CLI Command Reference. For practical scripting examples, see CLI Examples.

The JACS CLI provides a command-line interface for managing agents, documents, tasks, and agreements.

Getting Help

```
# General help
jacs --help

# Command-specific help
jacs agent --help
jacs document --help
jacs task --help
```

Commands Overview

Command	Description
`jacs init`	Initialize JACS (create config and agent with keys)
`jacs version`	Print version information
`jacs config`	Manage configuration
`jacs agent`	Manage agents
`jacs document`	Manage documents
`jacs task`	Manage tasks
`jacs mcp`	Start the built-in MCP server (stdio transport)

Initialization

Quick Start

```
# Initialize everything in one step
jacs init
```

This command:

1. Creates a configuration file (`jacs.config.json`)
2. Generates cryptographic keys
3. Creates an initial agent document

MCP Server

The MCP server is built into the `jacs` binary. No separate install step needed.

```
# Start the MCP server (stdio transport)
jacs mcp
```

Configuration Commands

Create Configuration

```
jacs config create
```

Creates a new `jacs.config.json` file in the current directory with default settings.

Read Configuration

```
jacs config read
```

Displays the current configuration, including values from both the config file and environment variables.

Agent Commands

Create Agent

```
jacs agent create --create-keys true

# With a custom agent definition file
jacs agent create --create-keys true -f my-agent.json

# Without creating new keys (use existing)
jacs agent create --create-keys false -f my-agent.json
```

Options:

Option	Short	Required	Description
`--create-keys`		Yes	Whether to create new cryptographic keys
`-f`		No	Path to JSON file with agent definition

Verify Agent

```
# Verify agent from config
jacs agent verify

# Verify specific agent file
jacs agent verify -a ./path/to/agent.json

# With DNS validation options
jacs agent verify --require-dns
jacs agent verify --require-strict-dns
jacs agent verify --no-dns
jacs agent verify --ignore-dns
```

Options:

Option	Short	Description
`-a`	`--agent-file`	Path to agent file (optional)
`--no-dns`		Disable DNS validation
`--require-dns`		Require DNS validation (not strict)
`--require-strict-dns`		Require DNSSEC validation
`--ignore-dns`		Ignore DNS validation entirely

DNS Commands

```
# Generate DNS TXT record commands for agent publishing
jacs agent dns --domain example.com --agent-id [uuid]

# With different output formats
jacs agent dns --domain example.com --encoding hex
jacs agent dns --domain example.com --provider aws

# With custom TTL
jacs agent dns --domain example.com --ttl 7200
```

Options:

Option	Default	Description
`--domain`		Domain for DNS record
`--agent-id`		Agent UUID (optional, uses config if not provided)
`--ttl`	3600	Time-to-live in seconds
`--encoding`	base64	Encoding format (base64, hex)
`--provider`	plain	Output format (plain, aws, azure, cloudflare)

Lookup Agent

```
# Look up another agent's public key from their domain
jacs agent lookup agent.example.com

# With strict DNSSEC validation
jacs agent lookup agent.example.com --strict

# Skip DNS lookup
jacs agent lookup agent.example.com --no-dns
```

Task Commands

Create Task

```
jacs task create -n "Task Name" -d "Task description"

# With optional agent file
jacs task create -n "Task Name" -d "Description" -a ./agent.json

# With input file
jacs task create -n "Task Name" -d "Description" -f ./task-details.json
```

Options:

Option	Short	Required	Description
`-n`	`--name`	Yes	Name of the task
`-d`	`--description`	Yes	Description of the task
`-a`	`--agent-file`	No	Path to agent file
`-f`	`--filename`	No	Path to JSON file with additional task data

Document Commands

Create Document

```
# Create from a JSON file
jacs document create -f ./document.json

# Create from a directory of files
jacs document create -d ./documents/

# With custom schema
jacs document create -f ./document.json -s ./custom-schema.json

# With file attachments
jacs document create -f ./document.json --attach ./attachment.pdf

# Embed attachments in document
jacs document create -f ./document.json --attach ./files/ --embed true

# Output to specific file
jacs document create -f ./document.json -o ./output.json

# Print to stdout instead of saving
jacs document create -f ./document.json --no-save
```

Options:

Option	Short	Description
`-f`	`--filename`	Path to input JSON file
`-d`	`--directory`	Path to directory of JSON files
`-o`	`--output`	Output filename
`-s`	`--schema`	Path to custom JSON schema
`--attach`		Path to file/directory for attachments
`--embed`	`-e`	Embed documents (true/false)
`--no-save`	`-n`	Print to stdout instead of saving

Option	Short	Description
`-v`	`--verbose`	Enable verbose output
`-a`	`--agent-file`	Path to agent file

Update Document

```
# Update an existing document with new content
jacs document update -f ./original.json -n ./updated.json

# With output file
jacs document update -f ./original.json -n ./updated.json -o ./result.json

# With file attachments
jacs document update -f ./original.json -n ./updated.json --attach ./new-file.pdf
```

Options:

Option	Short	Required	Description
`-f`	`--filename`	Yes	Path to original document
`-n`	`--new`	Yes	Path to new version
`-o`	`--output`	No	Output filename
`--attach`		No	Path to file attachments
`--embed`	`-e`	No	Embed documents (true/false)

Verify Document

```
# Verify a document
jacs document verify -f ./document.json

# Verify all documents in a directory
jacs document verify -d ./documents/

# With custom schema
jacs document verify -f ./document.json -s ./schema.json

# Verbose output
jacs document verify -f ./document.json -v
```

Options:

Option	Short	Description
`-f`	`--filename`	Path to document file
`-d`	`--directory`	Path to directory of documents
`-s`	`--schema`	Path to JSON schema for validation

Option	Short	Description
`-v`	`--verbose`	Enable verbose output
`-a`	`--agent-file`	Path to agent file

Extract Embedded Content

```
# Extract embedded content from a document
jacs document extract -f ./document.json

# Extract from all documents in directory
jacs document extract -d ./documents/
```

Agreement Commands

```
# Create an agreement requiring signatures from specified agents
jacs document create-agreement -f ./document.json -i agent1-uuid,agent2-uuid

# Check agreement status
jacs document check-agreement -f ./document.json

# Sign an agreement
jacs document sign-agreement -f ./document.json
```

Create Agreement Options:

Option	Short	Required	Description
`-f`	`--filename`	Yes	Path to document
`-i`	`--agentids`	Yes	Comma-separated list of agent UUIDs
`-o`	`--output`	No	Output filename
`--no-save`	`-n`	No	Print to stdout

Environment Variables

The CLI respects the following environment variables:

```
# Use a specific configuration file
JACS_CONFIG_PATH=./custom-config.json jacs agent verify

# Override settings
JACS_DATA_DIRECTORY=./data jacs document create -f ./doc.json
JACS_KEY_DIRECTORY=./keys jacs agent create --create-keys true
```

Common Workflows

Create and Sign a Document

```
# 1. Initialize (if not done)
jacs init

# 2. Create document
jacs document create -f ./my-document.json

# 3. Verify the signed document
jacs document verify -f ./jacs_data/[document-id].json
```

Multi-Agent Agreement

```
# 1. Create agreement on a document
jacs document create-agreement -f ./document.json -i agent1-id,agent2-id

# 2. First agent signs
jacs document sign-agreement -f ./document.json

# 3. Second agent signs (using their config)
JACS_CONFIG_PATH=./agent2.config.json jacs document sign-agreement -f ./document.json

# 4. Check agreement is complete
jacs document check-agreement -f ./document.json
```

Verify Another Agent

```
# Look up agent by domain
jacs agent lookup other-agent.example.com

# Verify with strict DNS
jacs agent verify -a ./other-agent.json --require-strict-dns
```

Exit Codes

Code	Meaning
0	Success
1	General error
2	Invalid arguments
3	File not found

Code	Meaning
4	Verification failed
5	Signature invalid

Next Steps

- Creating an Agent - Detailed agent creation guide
- Working with Documents - Document operations in depth
- Agreements - Multi-agent agreements

Creating an Agent

An agent is the fundamental identity in JACS - an autonomous entity that can create, sign, and verify documents. This guide covers creating and managing agents.

What is an Agent?

A JACS agent is:

- A unique identity with a UUID that never changes
- A holder of cryptographic keys for signing
- A provider of services defined in the agent document
- Self-signed to prove authenticity

Creating Your First Agent

Quick Method (Recommended)

```
# Initialize JACS (creates config and agent)
jacs init
```

This creates:

- Configuration file
- Cryptographic key pair
- Initial agent document

Manual Method

```
# 1. Create configuration
jacs config create

# 2. Create agent with new keys
jacs agent create --create-keys true
```

With Custom Agent Definition

Create an agent definition file (`my-agent.json`):

```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsAgentType": "ai",
  "jacsAgentDomain": "myagent.example.com",
  "name": "Content Creation Agent",
  "description": "AI agent specialized in content creation",
  "jacsServices": [
    {
      "name": "content-generation",
      "serviceDescription": "Generate high-quality content",
      "successDescription": "Engaging, accurate content delivered",
      "failureDescription": "Unable to generate requested content"
    }
  ]
}
```

Then create the agent:

```
jacs agent create --create-keys true -f my-agent.json
```

Agent Types

JACS supports four agent types:

Type	Description	Contacts Required
`ai`	Fully artificial intelligence	No
`human`	Individual person	Yes
`human-org`	Group of people (organization)	Yes
`hybrid`	Human-AI combination	Yes

AI Agent Example

```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsAgentType": "ai",
  "name": "DataBot",
  "description": "Data processing agent",
  "jacsServices": [
    {
      "name": "data-processing",
      "serviceDescription": "Process and transform data",
      "successDescription": "Data transformed successfully",
      "failureDescription": "Input data could not be processed"
    }
  ]
}
```

Human Agent Example

```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsAgentType": "human",
  "name": "John Smith",
  "description": "Software engineer",
  "jacsContacts": [
    {
      "firstName": "John",
      "lastName": "Smith",
      "email": "john@example.com",
      "isPrimary": true
    }
  ],
  "jacsServices": [
    {
      "name": "code-review",
      "serviceDescription": "Review code for quality and security",
      "successDescription": "Actionable review delivered",
      "failureDescription": "Could not complete review"
    }
  ]
}
```

Agent Services

Services define what an agent can do. Each service has:

```
{
  "name": "service-identifier",
  "serviceDescription": "What the service does",
  "successDescription": "Definition of successful completion",
  "failureDescription": "What constitutes failure"
}
```

Detailed Service Example

```
{
  "name": "document-processing",
  "serviceDescription": "Process and analyze documents",
  "successDescription": "Documents processed accurately",
  "failureDescription": "Unable to process one or more documents",
  "costDescription": "Usage-based pricing",
  "privacyPolicy": "https://example.com/privacy",
  "termsOfService": "https://example.com/terms"
}
```

Agent Contacts

For human and hybrid agents, contacts are required:

```
{
  "jacsContacts": [
    {
      "firstName": "Example",
      "lastName": "Agent",
      "email": "agent@example.com",
      "phone": "+1-555-0123",
      "isPrimary": true
    }
  ]
}
```

Cryptographic Keys

Key Algorithms

JACS supports multiple cryptographic algorithms:

Algorithm	Description	Recommended For
`ring-Ed25519`	Fast elliptic curve signatures	General use (default)
`RSA-PSS`	Traditional RSA signatures	Legacy compatibility
`pq2025`	Post-quantum ML-DSA-87 signatures	Future-proof security
`pq-dilithium`	Legacy post-quantum signatures	Backward compatibility only (deprecated)

Configure Key Algorithm

In `jacs.config.json`:

```
{
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

Or via environment variable:

```
JACS_AGENT_KEY_ALGORITHM=ring-Ed25519 jacs agent create --create-keys true
```

Key Storage

Keys are stored in the key directory (default: `./jacs_keys`):

```
jacs_keys/
├── private_key.pem    # Private key (keep secure!)
└── public_key.pem     # Public key (can be shared)
```

Verifying Agents

Verify Your Own Agent

```
jacs agent verify
```

Verify a Specific Agent File

```
jacs agent verify -a ./path/to/agent.json
```

With DNS Verification

```
# Require DNS validation
jacs agent verify --require-dns

# Require strict DNSSEC
jacs agent verify --require-strict-dns
```

Updating Agents

Agent updates create a new version while maintaining the same `jacsId`:

1. Modify the agent document
2. Re-sign with the agent's keys

The `jacsVersion` changes but `jacsId` remains constant.

Agent Document Structure

A complete agent document looks like:

```json
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsVersion": "123e4567-e89b-12d3-a456-426614174000",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsOriginalVersion": "123e4567-e89b-12d3-a456-426614174000",
  "jacsOriginalDate": "2024-01-15T10:30:00Z",
  "jacsType": "agent",
  "jacsLevel": "config",

  "jacsAgentType": "ai",
  "jacsAgentDomain": "myagent.example.com",
  "name": "Content Creation Agent",
  "description": "AI agent for content generation",

  "jacsServices": [
    {
      "name": "content-generation",
      "serviceDescription": "Generate high-quality content",
      "successDescription": "High-quality content generated",
      "failureDescription": "Unable to generate requested content"
    }
  ],

  "jacsSha256": "hash-of-document",
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "agentVersion": "123e4567-e89b-12d3-a456-426614174000",
    "signature": "base64-encoded-signature",
    "signingAlgorithm": "ring-Ed25519",
    "publicKeyHash": "hash-of-public-key",
    "date": "2024-01-15T10:30:00Z",
    "fields": ["jacsId", "jacsVersion", "jacsAgentType", "name", "jacsServices"]
  }
}
```

Best Practices

Security

1. **Protect private keys**: Never share or commit private keys
2. **Use strong algorithms**: Prefer Ed25519 or post-quantum
3. **Enable DNS verification**: For production agents
4. **Regular key rotation**: Update keys periodically

Agent Design

1. **Clear service definitions**: Be specific about capabilities

2. **Meaningful names**: Use descriptive agent names
3. **Contact information**: Include for human agents
4. **Version control**: Track agent document changes

Operations

1. **Backup keys**: Keep secure backups of private keys
2. **Monitor signatures**: Watch for unauthorized signing
3. **Document services**: Keep service definitions current

Next Steps

- Working with Documents - Create signed documents
- Agreements - Multi-agent coordination
- DNS Verification - Publish agent identity

Working with Documents

Documents are the core data structure in JACS. Any JSON object can become a JACS document by adding the required header fields and a cryptographic signature.

What is a JACS Document?

A JACS document is a JSON object that includes:

- **Identity**: Unique ID and version tracking
- **Metadata**: Type, timestamps, and origin information
- **Signature**: Cryptographic proof of authenticity
- **Hash**: Integrity verification

Creating Documents

From a JSON File

Create a simple JSON document (`my-document.json`):

```
{
  "title": "Project Proposal",
  "description": "Q1 development plan",
  "budget": 50000,
  "deadline": "2024-03-31"
}
```

Sign it with JACS:

```
jacs document create -f my-document.json
```

This adds JACS headers and signature, producing a signed document.

From a Directory

Process multiple documents at once:

```
jacs document create -d ./documents/
```

With Custom Schema

Validate against a custom JSON schema:

```
jacs document create -f my-document.json -s ./schemas/proposal.schema.json
```

Output Options

```
# Save to specific file
jacs document create -f my-document.json -o ./output/signed-doc.json

# Print to stdout instead of saving
jacs document create -f my-document.json --no-save

# Verbose output
jacs document create -f my-document.json -v
```

Document Structure

After signing, a document looks like:

```
{
  "$schema": "https://hai.ai/schemas/header/v1/header.schema.json",
  "jacsId": "doc-uuid-here",
  "jacsVersion": "version-uuid-here",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsOriginalVersion": "version-uuid-here",
  "jacsOriginalDate": "2024-01-15T10:30:00Z",
  "jacsType": "document",
  "jacsLevel": "artifact",

  "title": "Project Proposal",
  "description": "Q1 development plan",
  "budget": 50000,
  "deadline": "2024-03-31",

  "jacsSha256": "a1b2c3d4...",
  "jacsSignature": {
    "agentID": "agent-uuid",
    "agentVersion": "agent-version-uuid",
    "signature": "base64-signature",
    "signingAlgorithm": "ring-Ed25519",
    "publicKeyHash": "hash-of-public-key",
    "date": "2024-01-15T10:30:00Z",
    "fields": ["jacsId", "title", "description", "budget", "deadline"]
  }
}
```

Required Header Fields

Field	Description	Auto-generated
`$schema`	JSON Schema reference	Yes
`jacsId`	Permanent document UUID	Yes
`jacsVersion`	Version UUID (changes on update)	Yes
`jacsVersionDate`	When this version was created	Yes
`jacsOriginalVersion`	First version UUID	Yes
`jacsOriginalDate`	Original creation timestamp	Yes
`jacsType`	Document type	Yes
`jacsLevel`	Data level (raw, config, artifact, derived)	Yes

Document Levels

The `jacsLevel` field indicates the document's purpose:

Level	Description	Use Case
`raw`	Original data, should not change	Source documents
`config`	Configuration, meant to be updated	Agent definitions, settings
`artifact`	Generated output	Reports, summaries
`derived`	Computed from other documents	Analysis results

File Attachments

Attach Files

```
# Attach a single file
jacs document create -f my-document.json --attach ./report.pdf

# Attach a directory of files
jacs document create -f my-document.json --attach ./attachments/
```

Embed vs. Reference

```
# Embed files directly in the document (larger document, self-contained)
jacs document create -f my-document.json --attach ./files/ --embed true

# Reference files (smaller document, files stored separately)
jacs document create -f my-document.json --attach ./files/ --embed false
```

Attachment Structure

Embedded attachments appear in the `jacsFiles` field:

```
{
  "jacsFiles": [
    {
      "jacsFileName": "report.pdf",
      "jacsFileMimeType": "application/pdf",
      "jacsFileSha256": "file-hash",
      "jacsFileContent": "base64-encoded-content"
    }
  ]
}
```

Verifying Documents

Basic Verification

```
jacs document verify -f ./signed-document.json
```

Verification checks:

1. Hash integrity (document hasn't been modified)
2. Signature validity (signature matches content)
3. Schema compliance (if schema specified)

Verify with Schema

```
jacs document verify -f ./document.json -s ./schema.json
```

Verify Directory

```
jacs document verify -d ./documents/
```

Verbose Output

```
jacs document verify -f ./document.json -v
```

Updating Documents

Updates create a new version while maintaining the same `jacsId`:

```
jacs document update -f ./original.json -n ./modified.json
```

The update process:

1. Reads the original document
2. Applies changes from the modified file
3. Increments `jacsVersion`
4. Links to previous version via `jacsPreviousVersion`
5. Re-signs with agent's key

Update with Attachments

```
jacs document update -f ./original.json -n ./modified.json --attach ./new-file.pdf
```

Extracting Embedded Content

Extract attachments from a document:

```
jacs document extract -f ./document-with-attachments.json
```

Extract from multiple documents:

```
jacs document extract -d ./documents/
```

Document Types

Task Documents

Tasks are specialized documents for work tracking:

```
jacs task create -n "Code Review" -d "Review PR #123"
```

See Task Schema for details.

Message Documents

Messages for agent communication:

```
{
  "$schema": "https://hai.ai/schemas/message/v1/message.schema.json",
  "jacsType": "message",
  "jacsMessageContent": "Hello, I've completed the task.",
  "jacsMessageReplyTo": "previous-message-uuid"
}
```

Custom Documents

Any JSON can be a JACS document. Create custom schemas:

```
{
  "$schema": "https://example.com/schemas/invoice.schema.json",
  "jacsType": "invoice",
  "invoiceNumber": "INV-001",
  "amount": 1000,
  "currency": "USD"
}
```

Version History

JACS tracks document history through version chains:

```
Version 1 (jacsOriginalVersion)
    ↓
Version 2 (jacsPreviousVersion → Version 1)
    ↓
Version 3 (jacsPreviousVersion → Version 2)
    ↓
Current Version
```

Each version is a complete document that can be independently verified.

Working with Multiple Agents

Different Agent Signs Document

```
# Use a specific agent's keys
jacs document create -f ./document.json -a ./other-agent.json
```

Verify Document from Unknown Agent

```
# Verify with strict DNS requirement
jacs document verify -f ./document.json --require-strict-dns
```

Best Practices

Document Design

1. **Use appropriate levels**: Match `jacsLevel` to document purpose
2. **Include context**: Add descriptive fields for human readability
3. **Version control**: Keep source files in git alongside JACS documents

Security

1. **Verify before trusting**: Always verify signatures
2. **Check agent identity**: Verify the signing agent
3. **Validate schemas**: Use custom schemas for strict validation

Performance

1. **External attachments**: Use `--embed false` for large files
2. **Batch processing**: Use directory mode for multiple documents
3. **Selective verification**: Verify only when needed

Common Workflows

Create and Share Document

```
# 1. Create document
jacs document create -f ./proposal.json -o ./signed-proposal.json

# 2. Share the signed document
# The recipient can verify it:
jacs document verify -f ./signed-proposal.json
```

Track Document Changes

```
# 1. Create initial version
jacs document create -f ./contract-v1.json

# 2. Make changes and update
jacs document update -f ./contract-v1.json -n ./contract-v2.json

# 3. Continue updating
jacs document update -f ./contract-v2.json -n ./contract-v3.json
```

Process Multiple Documents

```
# Create all documents in a directory
jacs document create -d ./input-docs/

# Verify all documents
jacs document verify -d ./signed-docs/
```

Next Steps

- Agreements - Multi-agent consent
- Task Schema - Task document structure

- Custom Schemas - Create your own schemas

Creating and Using Agreements

Agreements enable multi-party consent in JACS. They allow multiple agents to cryptographically sign a document, creating binding commitments between parties.

What is an Agreement?

An agreement is a mechanism for:

- **Collecting signatures** from multiple agents
- **Tracking consent** from required parties
- **Enforcing completion** before proceeding
- **Creating audit trails** of who agreed and when

Agreement Lifecycle

```
1. Create Agreement → 2. Distribute → 3. Agents Sign → 4. Verify Complete
```

1. **Create**: Initial agent creates agreement with required participants
2. **Distribute**: Agreement document shared with all parties
3. **Sign**: Each agent reviews and adds their signature
4. **Verify**: Check that all required parties have signed

Creating Agreements

Basic Agreement

```
# Create agreement requiring signatures from two agents
jacs document create-agreement \
  -f ./document.json \
  -i agent1-uuid,agent2-uuid
```

With Context

Include a question and context for clarity:

```
{
  "jacsAgreement": {
    "jacsAgreementQuestion": "Do you agree to the terms of this contract?",
    "jacsAgreementContext": "Service agreement for Q1 2024",
    "jacsAgreementAgents": ["agent1-uuid", "agent2-uuid"]
  }
}
```

Signing Agreements

Sign as Current Agent

```
jacs document sign-agreement -f ./document-with-agreement.json
```

Sign as Different Agent

```
# Use a different configuration/agent
JACS_CONFIG_PATH=./agent2.config.json jacs document sign-agreement -f ./document.json
```

Sign with Response

When signing, agents can include a response:

```
{
  "jacsAgreement": {
    "signatures": {
      "agent1-uuid": {
        "agentID": "agent1-uuid",
        "signature": "base64-signature",
        "date": "2024-01-15T10:30:00Z",
        "response": "Agreed with minor reservation about timeline",
        "responseType": "agree"
      }
    }
  }
}
```

Response types:

- `agree` - Agent consents
- `disagree` - Agent does not consent
- `reject` - Agent considers the question invalid or irrelevant

Checking Agreement Status

Check if Complete

```
jacs document check-agreement -f ./document.json
```

This shows:

- Which agents have signed
- Which agents still need to sign
- Whether the agreement is complete

Agreement Structure

A document with an agreement includes:

```
{
  "jacsId": "doc-uuid",
  "jacsType": "contract",

  "jacsAgreement": {
    "jacsAgreementQuestion": "Do you agree to these terms?",
    "jacsAgreementContext": "Annual service contract",
    "jacsAgreementAgents": [
      "550e8400-e29b-41d4-a716-446655440000",
      "123e4567-e89b-12d3-a456-426614174000"
    ],
    "signatures": {
      "550e8400-e29b-41d4-a716-446655440000": {
        "agentID": "550e8400-e29b-41d4-a716-446655440000",
        "agentVersion": "version-uuid",
        "signature": "base64-signature",
        "signingAlgorithm": "ring-Ed25519",
        "publicKeyHash": "hash",
        "date": "2024-01-15T10:30:00Z",
        "responseType": "agree",
        "fields": ["jacsId", "jacsAgreement"]
      }
    }
  },

  "jacsAgreementHash": "hash-of-agreement-content"
}
```

Task Agreements

Tasks have built-in support for start and end agreements:

```
{
  "jacsType": "task",
  "jacsTaskName": "Code Review",

  "jacsStartAgreement": {
    "jacsAgreementQuestion": "Do you agree to start this task?",
    "jacsAgreementAgents": ["customer-uuid", "provider-uuid"]
  },

  "jacsEndAgreement": {
    "jacsAgreementQuestion": "Do you agree the task is complete?",
    "jacsAgreementAgents": ["customer-uuid", "provider-uuid"]
  }
}
```

Multi-Agent Workflow Example

```
# 1. Agent A creates a task
jacs task create -n "Write Report" -d "Quarterly sales report"

# 2. Agent A adds agreement requiring both agents
jacs document create-agreement \
  -f ./task.json \
  -i agent-a-uuid,agent-b-uuid

# 3. Agent A signs the agreement
jacs document sign-agreement -f ./task.json

# 4. Agent B signs the agreement
JACS_CONFIG_PATH=./agent-b.config.json \
  jacs document sign-agreement -f ./task.json

# 5. Check agreement is complete
jacs document check-agreement -f ./task.json
```

Agreement Hash

The `jacsAgreementHash` ensures all agents agree to the same content:

1. Hash is computed from the agreement content
2. Each signature includes the hash
3. If content changes, hash changes, invalidating existing signatures

This prevents modifications after some parties have signed.

Agreement Options (v0.6.2+)

Timeout

Set a deadline after which the agreement expires:

```
# Python
agreement = client.create_agreement(
    document=proposal,
    agent_ids=[alice.agent_id, bob.agent_id],
    timeout="2025-12-31T23:59:59Z"
)
```

If the deadline passes before all required signatures are collected, `check_agreement()` returns an error.

Quorum (M-of-N Signing)

Require only a subset of agents to sign:

```
# Only 2 of 3 agents need to sign
agreement = client.create_agreement(
    document=proposal,
    agent_ids=[alice.agent_id, bob.agent_id, carol.agent_id],
    quorum=2
)
```

When quorum is met, `check_agreement()` succeeds even if some agents haven't signed.

Algorithm Constraints

Enforce that only specific cryptographic algorithms can be used:

```
# Only post-quantum algorithms allowed
agreement = client.create_agreement(
    document=proposal,
    agent_ids=agent_ids,
    required_algorithms=["pq2025", "pq-dilithium"],
    minimum_strength="post-quantum"
)
```

An agent using RSA-PSS or Ed25519 will be rejected when trying to sign this agreement.

Combined Options

```
agreement = client.create_agreement(
    document={"proposal": "Deploy model v2"},
    agent_ids=[alice.agent_id, bob.agent_id, mediator.agent_id],
    question="Do you approve deployment?",
    timeout="2025-06-30T00:00:00Z",
    quorum=2,
    minimum_strength="post-quantum"
)
```

Using JacsClient (Instance-Based API)

For running multiple agents in one process, use `JacsClient` instead of the module-level API:

Python

```
from jacs.client import JacsClient

alice = JacsClient.ephemeral("ring-Ed25519")  # for testing
bob = JacsClient.ephemeral("ring-Ed25519")

signed = alice.sign_message({"action": "approve"})
# alice.agent_id, bob.agent_id are unique
```

See the full example: examples/multi_agent_agreement.py

Node.js

```
import { JacsClient } from '@hai.ai/jacs/client';

const alice = JacsClient.ephemeral('ring-Ed25519');
const bob = JacsClient.ephemeral('ring-Ed25519');

const signed = alice.signMessage({ action: 'approve' });
```

See the full example: examples/multi_agent_agreement.ts

MCP Tools for Agreements

The JACS MCP server exposes agreement tools that LLMs can use directly:

Tool	Description
`jacs_create_agreement`	Create agreement with quorum, timeout, algorithm constraints
`jacs_sign_agreement`	Co-sign an agreement
`jacs_check_agreement`	Check status: who signed, quorum met, expired

See MCP Integration for setup.

Best Practices

1. **Verify before signing**: Always review documents before signing
2. **Check agent identities**: Verify who you're agreeing with (use DNS)
3. **Include context**: Make the agreement purpose clear
4. **Handle disagreement**: Have a process for when agents disagree
5. **Use quorum for resilience**: Don't require unanimous consent unless necessary
6. **Set timeouts**: Prevent agreements from hanging indefinitely
7. **Enforce post-quantum for sensitive agreements**: Use `minimum_strength: "post-quantum"` for long-term security

Next Steps

- DNS Verification - Verify agent identities
- Task Schema - Task-specific agreements
- Security Model - Agreement security
- Multi-Agent Agreement Example (Python)
- Multi-Agent Agreement Example (Node.js)

DNS-Based Agent Verification

JACS supports DNS-based agent verification using DNS TXT records and DNSSEC. This allows agents to publish their identity in a decentralized, verifiable way that doesn't require a central authority.

Overview

DNS verification in JACS works by:

1. Publishing an agent's public key fingerprint as a DNS TXT record
2. Using DNSSEC to cryptographically verify the DNS response
3. Comparing the fingerprint from DNS with the agent's actual public key

This provides a secure, decentralized way to verify agent identity across the internet.

Why DNS Verification?

- **Decentralized**: No central authority required
- **Existing Infrastructure**: Uses established DNS infrastructure
- **DNSSEC Security**: Cryptographic verification of DNS responses
- **Human-Readable**: Agents can be identified by domain names
- **Widely Supported**: Works with any DNS provider

DNS verification is also a practical bridge for DID-style deployments: you can publish DID metadata for discovery while using DNS TXT + JACS signature verification as the operational trust anchor. No blockchain is required for this model.

Publishing Agent Identity

Generate DNS Commands

```
# Generate DNS TXT record commands for your agent
jacs agent dns --domain myagent.example.com

# Specify agent ID explicitly
jacs agent dns --domain myagent.example.com --agent-id 550e8400-e29b-41d4-a716-
446655440000

# Use hex encoding instead of base64
jacs agent dns --domain myagent.example.com --encoding hex

# Set custom TTL (time-to-live)
jacs agent dns --domain myagent.example.com --ttl 7200
```

Provider-Specific Formats

JACS can generate DNS commands for various providers:

```
# Plain text format (default)
jacs agent dns --domain myagent.example.com --provider plain

# AWS Route 53 format
jacs agent dns --domain myagent.example.com --provider aws

# Azure DNS format
jacs agent dns --domain myagent.example.com --provider azure

# Cloudflare DNS format
jacs agent dns --domain myagent.example.com --provider cloudflare
```

DNS Record Structure

The DNS TXT record follows this format:

```
_v1.agent.jacs.myagent.example.com. 3600 IN TXT "jacs-agent-fingerprint=<fingerprint>"
```

Where:

- `_v1.agent.jacs.` is the JACS-specific subdomain prefix
- `<fingerprint>` is the base64-encoded hash of the agent's public key

Setting Up with Route 53 (AWS)

1. Generate the AWS-formatted command:

```
jacs agent dns --domain myagent.example.com --provider aws
```

2. The output will include an AWS CLI command like:

```
aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch '{
    "Changes": [{
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "_v1.agent.jacs.myagent.example.com",
        "Type": "TXT",
        "TTL": 3600,
        "ResourceRecords": [{"Value": "\"jacs-agent-fingerprint=...\""}]
      }
    }]
  }'
```

3. Replace `YOUR_ZONE_ID` with your actual Route 53 hosted zone ID.

Setting Up with Cloudflare

1. Generate the Cloudflare-formatted command:

```
jacs agent dns --domain myagent.example.com --provider cloudflare
```

2. Or add manually in the Cloudflare dashboard:
   - Type: `TXT`
   - Name: `_v1.agent.jacs`
   - Content: `jacs-agent-fingerprint=<your-fingerprint>`
   - TTL: 3600

Setting Up with Azure DNS

1. Generate the Azure-formatted command:

```
jacs agent dns --domain myagent.example.com --provider azure
```

2. The output will include an Azure CLI command that you can run directly.

Verifying Agents with DNS

Look Up Another Agent

```
# Look up an agent by their domain
jacs agent lookup other-agent.example.com

# With strict DNSSEC validation
jacs agent lookup other-agent.example.com --strict

# Skip DNS verification (not recommended)
jacs agent lookup other-agent.example.com --no-dns
```

Verify Agent with DNS

When verifying an agent, you can specify DNS requirements:

```
# Default: Use DNS if available, but don't require it
jacs agent verify -a ./agent.json

# Require DNS validation (non-strict)
jacs agent verify -a ./agent.json --require-dns

# Require strict DNSSEC validation
jacs agent verify -a ./agent.json --require-strict-dns

# Disable DNS validation entirely
jacs agent verify -a ./agent.json --no-dns

# Ignore DNS (won't fail if DNS unavailable)
jacs agent verify -a ./agent.json --ignore-dns
```

DNS Validation Modes

Mode	Flag	Behavior
Default	(none)	Use DNS if available, fall back to local verification
Require DNS	`--require-dns`	Fail if DNS record not found (DNSSEC not required)
Require Strict	`--require-strict-dns`	Fail if DNSSEC validation fails
No DNS	`--no-dns`	Skip DNS validation entirely
Ignore DNS	`--ignore-dns`	Don't fail on DNS errors, just warn

Agent Domain Configuration

Agents can specify their domain in their agent document:

```
{
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsAgentType": "ai",
  "jacsAgentDomain": "myagent.example.com",
  "jacsServices": [...]
}
```

The `jacsAgentDomain` field is optional but enables DNS-based verification.

DNSSEC Requirements

For maximum security, enable DNSSEC on your domain:

1. **Enable DNSSEC at your registrar**: Most registrars support DNSSEC
2. **Configure your DNS provider**: Ensure your DNS provider signs zones
3. **Use `--require-strict-dns`**: Enforce DNSSEC validation

Checking DNSSEC Status

You can verify DNSSEC is working using standard tools:

```
# Check if DNSSEC is enabled
dig +dnssec _v1.agent.jacs.myagent.example.com TXT

# Verify DNSSEC validation
delv @8.8.8.8 _v1.agent.jacs.myagent.example.com TXT
```

Security Considerations

Trust Model

- **With DNSSEC**: Full cryptographic chain of trust from root DNS servers
- **Without DNSSEC**: Trust depends on DNS infrastructure security
- **Local Only**: Trust is limited to having the correct public key

Best Practices

1. **Always enable DNSSEC** for production agents
2. **Use strict validation** when verifying unknown agents
3. **Rotate keys carefully** - update DNS records before key changes
4. **Monitor DNS records** for unauthorized changes
5. **Use short TTLs during transitions** then increase for stability

Caching

DNS responses are cached based on TTL. Consider:

- **Short TTL (300-600s)**: Better for development or key rotation
- **Long TTL (3600-86400s)**: Better for production stability

Troubleshooting

"DNS record not found"

1. Verify the record exists:

```
dig _v1.agent.jacs.myagent.example.com TXT
```

2. Check DNS propagation (may take up to 48 hours for new records)

3. Verify the domain in the agent document matches

"DNSSEC validation failed"

1. Check DNSSEC is enabled:

```
dig +dnssec myagent.example.com
```

2. Verify DS records at registrar

3. Use `--require-dns` instead of `--require-strict-dns` if DNSSEC isn't available

"Fingerprint mismatch"

1. The public key may have changed - regenerate DNS record:

```
jacs agent dns --domain myagent.example.com
```

2. Update the DNS TXT record with the new fingerprint
3. Wait for DNS propagation

Integration with CI/CD

Automate DNS updates in your deployment pipeline:

```bash
#!/bin/bash
# deploy-agent.sh

# 1. Create new agent keys
jacs agent create --create-keys true

# 2. Generate DNS update command
DNS_CMD=$(jacs agent dns --domain $AGENT_DOMAIN --provider aws)

# 3. Execute DNS update
eval $DNS_CMD

# 4. Wait for propagation
sleep 60

# 5. Verify DNS is working
jacs agent verify --require-dns
```

Next Steps

- Creating an Agent - Set up agents with DNS domains
- Security Model - Deep dive into JACS security
- Agreements - Use DNS-verified agents in agreements

Rust Library API

JACS provides a Rust library for programmatic agent and document management. This chapter covers how to use the JACS library in your Rust applications.

Adding JACS as a Dependency

Add JACS to your `Cargo.toml`:

```toml
[dependencies]
jacs = "0.3"
```

Feature Flags

```toml
[dependencies]
jacs = { version = "0.3", features = ["cli", "observability"] }
```

Feature	Description
`sqlite`	Lightweight sync SQLite backend (default)
`sqlx-sqlite`	Async SQLite backend via sqlx (requires tokio)
`otlp-logs`	OTLP log export support
`otlp-metrics`	OTLP metrics export support
`otlp-tracing`	OTLP distributed tracing support
`agreements`	Agreement lifecycle support
`a2a`	Agent-to-Agent protocol support
`attestation`	Attestation support

Core Types

Agent

The `Agent` struct is the central type in JACS. It holds:

- Schema validators
- Agent identity and keys

- Document storage
- Configuration

```rust
use jacs::{get_empty_agent, load_agent};
use std::error::Error;

fn main() -> Result<(), Box<dyn Error>> {
    // Create a new empty agent
    let agent = get_empty_agent();

    // Or load an existing agent
    let agent = load_agent(Some("path/to/agent.json".to_string()))?;

    Ok(())
}
```

JACSDocument

Documents in JACS are represented by the `JACSDocument` struct:

```rust
pub struct JACSDocument {
    pub id: String,
    pub version: String,
    pub value: serde_json::Value,
    pub jacs_type: String,
}
```

Key methods:

- `getkey()` - Returns `"id:version"` identifier
- `getvalue()` - Returns reference to the JSON value
- `getschema()` - Returns the document's schema URL
- `signing_agent()` - Returns the ID of the signing agent

Creating an Agent

Minimal Agent

```rust
use jacs::{get_empty_agent, create_minimal_blank_agent};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Create agent JSON
    let agent_json = create_minimal_blank_agent(
        "ai".to_string(),                    // agent type
        Some("My service".to_string()),      // service description
        Some("Task completed".to_string()),  // success description
        Some("Task failed".to_string()),     // failure description
    )?;

    // Initialize and load the agent
    let mut agent = get_empty_agent();
    agent.create_agent_and_load(&agent_json, true, None)?;

    // Save the agent
    agent.save()?;

    Ok(())
}
```

Loading by Configuration

```rust
use jacs::get_empty_agent;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut agent = get_empty_agent();

    // Load from config file
    agent.load_by_config("./jacs.config.json".to_string())?;

    // Or load by agent ID
    agent.load_by_id("agent-id:version-id".to_string())?;

    Ok(())
}
```

DNS Strict Mode

```rust
use jacs::load_agent_with_dns_strict;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Load agent with strict DNS verification
    let agent = load_agent_with_dns_strict(
        "path/to/agent.json".to_string(),
        true  // strict mode
    )?;

    Ok(())
}
```

Working with Documents

Creating Documents

The `DocumentTraits` trait provides document operations:

```rust
use jacs::agent::document::DocumentTraits;
use jacs::get_empty_agent;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut agent = get_empty_agent();
    agent.load_by_config("./jacs.config.json".to_string())?;

    // Create a document from JSON
    let json = r#"{"title": "My Document", "content": "Hello, World!"}"#;
    let doc = agent.create_document_and_load(json, None, None)?;

    println!("Document created: {}", doc.getkey());

    Ok(())
}
```

Creating Documents with Attachments

```rust
use jacs::agent::document::DocumentTraits;

// With file attachments
let attachments = Some(vec!["./report.pdf".to_string()]);
let embed = Some(true);  // Embed files in document

let doc = agent.create_document_and_load(
    json,
    attachments,
    embed
)?;
```

Loading Documents

```rust
use jacs::agent::document::DocumentTraits;

// Load a document from JSON string
let doc = agent.load_document(&document_json_string)?;

// Get a stored document by key
let doc = agent.get_document("doc-id:version-id")?;

// List all document keys
let keys = agent.get_document_keys();
```

Updating Documents

```rust
use jacs::agent::document::DocumentTraits;

// Update creates a new version
let updated_doc = agent.update_document(
    "doc-id:version-id",    // original document key
    &modified_json_string,  // new content
    None,                   // optional attachments
    None,                   // embed flag
)?;
```

Verifying Documents

```rust
use jacs::agent::document::DocumentTraits;

// Verify document signature with agent's public key
agent.verify_document_signature(
    "doc-id:version-id",
    None,  // signature key (uses default)
    None,  // fields to verify
    None,  // public key (uses agent's)
    None,  // key encoding type
)?;

// Verify using external public key
agent.verify_external_document_signature("doc-id:version-id")?;
```

Saving Documents

```rust
use jacs::agent::document::DocumentTraits;

// Save document to filesystem
agent.save_document(
    "doc-id:version-id",
    Some("output.json".to_string()),  // output filename
    Some(true),                       // export embedded files
    None,                             // extract only
)?;
```

Creating Tasks

```rust
use jacs::{get_empty_agent, create_task};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut agent = get_empty_agent();
    agent.load_by_config("./jacs.config.json".to_string())?;

    // Create a task
    let task_json = create_task(
        &mut agent,
        "Review Code".to_string(),
        "Review pull request #123".to_string(),
    )?;

    println!("Task created: {}", task_json);

    Ok(())
}
```

Signing and Verification

Signing Documents

The agent's `signing_procedure` method creates cryptographic signatures:

```
use serde_json::json;

let document = json!({
    "title": "Contract",
    "terms": "..."
});

// Sign the document
let signature = agent.signing_procedure(
    &document,
    None,           // fields to sign (None = all)
    "jacsSignature" // placement key
)?;
```

Verification

```
// Verify self-signature (agent document)
agent.verify_self_signature()?;

// Verify hash integrity
agent.verify_hash(&document)?;

// Full signature verification
agent.signature_verification_procedure(
    &document,
    None,                    // fields
    "jacsSignature",         // signature key
    public_key,              // public key bytes
    Some("ring-Ed25519".to_string()),  // algorithm
    None,                    // original public key hash
    None,                    // signature override
)?;
```

Custom Schema Validation

```rust
// Load custom schemas
agent.load_custom_schemas(&[
    "./schemas/invoice.schema.json".to_string(),
    "https://example.com/schemas/contract.schema.json".to_string(),
])?;

// Validate document against custom schema
agent.validate_document_with_custom_schema(
    "./schemas/invoice.schema.json",
    &document_value,
)?;
```

Configuration

Loading Configuration

```rust
use jacs::config::{load_config, find_config, Config};

// Load from specific path
let config = load_config("./jacs.config.json")?;

// Find config in directory
let config = find_config("./".to_string())?;

// Create programmatically
let config = Config::new(
    Some("false".to_string()),           // use_security
    Some("./jacs_data".to_string()),     // data_directory
    Some("./jacs_keys".to_string()),     // key_directory
    Some("private_key.pem".to_string()), // private key filename
    Some("public_key.pem".to_string()),  // public key filename
    Some("ring-Ed25519".to_string()),    // key algorithm
    Some("password".to_string()),        // private key password
    None,                                // agent ID and version
    Some("fs".to_string()),              // storage type
);
```

Accessing Configuration

```rust
// Get key algorithm
let algorithm = config.get_key_algorithm()?;

// Access config fields
let data_dir = config.jacs_data_directory();
let key_dir = config.jacs_key_directory();
let storage_type = config.jacs_default_storage();
```

Observability

Initialize Default Observability

```rust
use jacs::init_default_observability;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Set up file-based logging
    init_default_observability()?;

    // Your application code...

    Ok(())
}
```

Custom Observability Configuration

```rust
use jacs::{
    init_custom_observability,
    ObservabilityConfig,
    LogConfig,
    LogDestination,
    MetricsConfig,
    MetricsDestination,
};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config = ObservabilityConfig {
        logs: LogConfig {
            enabled: true,
            level: "debug".to_string(),
            destination: LogDestination::Otlp {
                endpoint: "http://localhost:4317".to_string(),
                headers: None,
            },
            headers: None,
        },
        metrics: MetricsConfig {
            enabled: true,
            destination: MetricsDestination::Prometheus {
                endpoint: "http://localhost:9090".to_string(),
                headers: None,
            },
            export_interval_seconds: Some(30),
            headers: None,
        },
        tracing: None,
    };

    init_custom_observability(config)?;

    Ok(())
}
```

Storage Backends

JACS supports multiple storage backends:

```rust
use jacs::storage::MultiStorage;

// Filesystem storage (default)
let storage = MultiStorage::new("fs".to_string())?;

// In-memory storage
let storage = MultiStorage::new("memory".to_string())?;

// AWS object storage
let storage = MultiStorage::new("aws".to_string())?;
```

For signed document CRUD/search, prefer the unified `DocumentService` surface:

```rust
use jacs::document::service_from_agent;

let docs = service_from_agent(agent_handle)?;
// `fs` and `rusqlite` currently resolve in JACS core.
```

Error Handling

JACS functions return `Result<T, Box<dyn Error>>`:

```rust
use jacs::get_empty_agent;

fn main() {
    match get_empty_agent().load_by_config("./jacs.config.json".to_string()) {
        Ok(()) => println!("Agent loaded successfully"),
        Err(e) => eprintln!("Failed to load agent: {}", e),
    }
}
```

Thread Safety

The `Agent` struct uses internal mutexes for thread-safe access to:

- Document schemas (`Arc<Mutex<HashMap<String, Validator>>>`)
- Storage operations

For concurrent usage:

```rust
use std::sync::{Arc, Mutex};
use jacs::get_empty_agent;

let agent = Arc::new(Mutex::new(get_empty_agent()));

// Clone Arc for threads
let agent_clone = Arc::clone(&agent);
std::thread::spawn(move || {
    let mut agent = agent_clone.lock().unwrap();
    // Use agent...
});
```

Complete Example

```rust
use jacs::{get_empty_agent, create_task};
use jacs::agent::document::DocumentTraits;
use serde_json::json;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Initialize agent
    let mut agent = get_empty_agent();
    agent.load_by_config("./jacs.config.json".to_string())?;

    // Create a document
    let doc_json = json!({
        "title": "Project Proposal",
        "description": "Q1 development plan",
        "budget": 50000
    });

    let doc = agent.create_document_and_load(
        &doc_json.to_string(),
        None,
        None
    )?;

    println!("Created document: {}", doc.getkey());

    // Verify the document
    agent.verify_document_signature(&doc.getkey(), None, None, None, None)?;
    println!("Document verified successfully");

    // Save to file
    agent.save_document(&doc.getkey(), Some("proposal.json".to_string()), None,
None)?;

    // Create a task
    let task = create_task(
        &mut agent,
        "Review Proposal".to_string(),
        "Review and approve the project proposal".to_string(),
    )?;

    println!("Task created");

    Ok(())
}
```

Next Steps

- Observability - Logging and metrics setup
- Storage Backends - Configure different storage
- Custom Schemas - Define custom document types

Observability (Rust API)

This page covers the Rust-specific observability API: `ObservabilityConfig`, `LogDestination`, `MetricsConfig`, `TracingConfig`, and related types. For a cross-language guide covering structured events, OTEL collector setup, and monitoring backend integration, see the Observability & Monitoring Guide.

JACS provides comprehensive observability features including logging, metrics, and distributed tracing. This chapter covers configuring and using these features in your Rust applications.

Overview

JACS observability is built on the OpenTelemetry standard, providing:

- **Logging**: Structured logging with multiple destinations
- **Metrics**: Counters, gauges, and histograms for monitoring
- **Tracing**: Distributed tracing for request flows

Feature Flags

Enable observability features in your `Cargo.toml`:

```
[dependencies]
jacs = { version = "0.3", features = ["observability"] }
```

Feature	Description
`otlp-logs`	OTLP log export support
`otlp-metrics`	OTLP metrics export support
`otlp-tracing`	OTLP distributed tracing support

Convenience helpers for recording operations are always available (no feature flag needed).

Quick Start

Default Configuration

The simplest way to enable observability:

```rust
use jacs::init_default_observability;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    init_default_observability()?;

    // Your application code...

    Ok(())
}
```

This sets up:

- File-based logging to `./logs/` with daily rotation
- Metrics disabled by default
- Tracing disabled by default

Custom Configuration

For more control, use `init_custom_observability`:

```rust
use jacs::{
    init_custom_observability,
    ObservabilityConfig,
    LogConfig,
    LogDestination,
    MetricsConfig,
    MetricsDestination,
};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config = ObservabilityConfig {
        logs: LogConfig {
            enabled: true,
            level: "info".to_string(),
            destination: LogDestination::Stderr,
            headers: None,
        },
        metrics: MetricsConfig {
            enabled: false,
            destination: MetricsDestination::Stdout,
            export_interval_seconds: None,
            headers: None,
        },
        tracing: None,
    };

    init_custom_observability(config)?;
    Ok(())
}
```

Logging

Log Levels

Supported log levels (from most to least verbose):

- `trace`
- `debug`
- `info`
- `warn`
- `error`

Log Destinations

Stderr (Default)

```
LogDestination::Stderr
```

Logs to standard error. Useful for development and containerized environments.

File

```
LogDestination::File {
    path: "./logs".to_string(),
}
```

Logs to rotating files with daily rotation. Creates files like `app.log.2024-01-15`.

OTLP

```
LogDestination::Otlp {
    endpoint: "http://localhost:4318".to_string(),
    headers: None,
}
```

Exports logs via OpenTelemetry Protocol. Requires `otlp-logs` feature.

Null

```
LogDestination::Null
```

Disables logging completely.

Using Logs

JACS uses the `tracing` crate for logging:

```
use tracing::{info, debug, warn, error};

fn process_document() {
    info!("Processing document");
    debug!("Document details: {:?}", doc);

    if let Err(e) = verify() {
        error!("Verification failed: {}", e);
    }
}
```

Metrics

Enabling Metrics

```
MetricsConfig {
    enabled: true,
    destination: MetricsDestination::Otlp {
        endpoint: "http://localhost:4318".to_string(),
        headers: None,
    },
    export_interval_seconds: Some(30),
    headers: None,
}
```

Metrics Destinations

OTLP

```
MetricsDestination::Otlp {
    endpoint: "http://localhost:4318".to_string(),
    headers: None,
}
```

Exports to an OpenTelemetry collector. Requires `otlp-metrics` feature.

Prometheus (via Collector)

```
MetricsDestination::Prometheus {
    endpoint: "http://localhost:9090".to_string(),
    headers: None,
}
```

Note: Direct Prometheus export requires routing through an OTLP collector.

File

```
MetricsDestination::File {
    path: "./metrics.txt".to_string(),
}
```

Writes metrics to a file.

Stdout

```
MetricsDestination::Stdout
```

Prints metrics to standard output. Useful for testing.

Recording Metrics

JACS provides convenience functions for common metrics:

```
use jacs::observability::metrics::{increment_counter, set_gauge, record_histogram};
use std::collections::HashMap;

// Increment a counter
let mut tags = HashMap::new();
tags.insert("operation".to_string(), "sign".to_string());
increment_counter("jacs_operations_total", 1, Some(tags));

// Set a gauge value
set_gauge("jacs_documents_active", 42.0, None);

// Record a histogram value (e.g., latency)
let mut tags = HashMap::new();
tags.insert("method".to_string(), "verify".to_string());
record_histogram("jacs_operation_duration_ms", 150.0, Some(tags));
```

Built-in Metrics

JACS convenience helpers automatically record:

- `jacs_agent_operations` - Count of agent operations
- `jacs_signature_verifications` - Signature verification results
- `jacs_document_operations` - Document create/update/verify counts

Distributed Tracing

Enabling Tracing

```
use jacs::{TracingConfig, TracingDestination, SamplingConfig, ResourceConfig};
use std::collections::HashMap;

let config = ObservabilityConfig {
    // ... logs and metrics config ...
    tracing: Some(TracingConfig {
        enabled: true,
        sampling: SamplingConfig {
            ratio: 1.0,           // Sample all traces
            parent_based: true,
            rate_limit: None,
        },
        resource: Some(ResourceConfig {
            service_name: "my-jacs-app".to_string(),
            service_version: Some("1.0.0".to_string()),
            environment: Some("production".to_string()),
            attributes: HashMap::new(),
        }),
        destination: Some(TracingDestination::Otlp {
            endpoint: "http://localhost:4318".to_string(),
            headers: None,
        }),
    }),
};
```

Tracing Destinations

OTLP

```
TracingDestination::Otlp {
    endpoint: "http://localhost:4318".to_string(),
    headers: None,
}
```

Jaeger

```
TracingDestination::Jaeger {
    endpoint: "http://localhost:14268/api/traces".to_string(),
    headers: None,
}
```

Sampling Configuration

Control how many traces are captured:

```
SamplingConfig {
    ratio: 0.1,           // Sample 10% of traces
    parent_based: true,   // Inherit parent sampling decision
    rate_limit: Some(100), // Max 100 samples per second
}
```

Using Tracing Spans

```
use tracing::{instrument, info_span};

#[instrument]
fn sign_document(doc: &Document) -> Result<(), Error> {
    // Automatically creates a span named "sign_document"
    // with doc as a field
}

fn manual_span() {
    let span = info_span!("verify_chain", doc_count = 5);
    let _guard = span.enter();

    // Operations within this span
}
```

Configuration File

You can configure observability via `jacs.config.json`:

```json
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "observability": {
    "logs": {
      "enabled": true,
      "level": "info",
      "destination": {
        "file": {
          "path": "./logs"
        }
      }
    },
    "metrics": {
      "enabled": true,
      "destination": {
        "otlp": {
          "endpoint": "http://localhost:4318"
        }
      },
      "export_interval_seconds": 30
    },
    "tracing": {
      "enabled": true,
      "sampling": {
        "ratio": 1.0,
        "parent_based": true
      },
      "resource": {
        "service_name": "jacs-service",
        "service_version": "1.0.0",
        "environment": "production"
      },
      "destination": {
        "otlp": {
          "endpoint": "http://localhost:4318"
        }
      }
    }
  }
}
```

OpenTelemetry Collector Setup

For production use, route telemetry through an OpenTelemetry Collector:

```yaml
# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:

exporters:
  logging:
    loglevel: debug
  prometheus:
    endpoint: "0.0.0.0:9090"
  jaeger:
    endpoint: jaeger:14250

service:
  pipelines:
    logs:
      receivers: [otlp]
      processors: [batch]
      exporters: [logging]
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheus]
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [jaeger]
```

Reset and Cleanup

For testing or reinitialization:

```rust
use jacs::observability::{reset_observability, flush_observability,
force_reset_for_tests};

// Flush pending data
flush_observability();

// Reset configuration
reset_observability();

// Force reset for tests (clears all state)
force_reset_for_tests();
```

Best Practices

Development

```
let config = ObservabilityConfig {
    logs: LogConfig {
        enabled: true,
        level: "debug".to_string(),
        destination: LogDestination::Stderr,
        headers: None,
    },
    metrics: MetricsConfig {
        enabled: false,
        destination: MetricsDestination::Stdout,
        export_interval_seconds: None,
        headers: None,
    },
    tracing: None,
};
```

Production

```
let config = ObservabilityConfig {
    logs: LogConfig {
        enabled: true,
        level: "info".to_string(),
        destination: LogDestination::Otlp {
            endpoint: "http://collector:4318".to_string(),
            headers: Some(auth_headers()),
        },
        headers: None,
    },
    metrics: MetricsConfig {
        enabled: true,
        destination: MetricsDestination::Otlp {
            endpoint: "http://collector:4318".to_string(),
            headers: Some(auth_headers()),
        },
        export_interval_seconds: Some(30),
        headers: None,
    },
    tracing: Some(TracingConfig {
        enabled: true,
        sampling: SamplingConfig {
            ratio: 0.1,  // Sample 10% in production
            parent_based: true,
            rate_limit: Some(1000),
        },
        resource: Some(ResourceConfig {
            service_name: "jacs-production".to_string(),
            service_version: Some(env!("CARGO_PKG_VERSION").to_string()),
            environment: Some("production".to_string()),
            attributes: HashMap::new(),
        }),
        destination: Some(TracingDestination::Otlp {
            endpoint: "http://collector:4318".to_string(),
            headers: Some(auth_headers()),
        }),
    }),
};
```

Troubleshooting

Logs Not Appearing

1. Check that logging is enabled: `logs.enabled: true`
2. Verify log level includes your log statements
3. For file logging, ensure the directory is writable

Metrics Not Exporting

1. Verify `otlp-metrics` feature is enabled
2. Check endpoint connectivity
3. Confirm metrics are enabled: `metrics.enabled: true`

Traces Missing

1. Verify `otlp-tracing` feature is enabled
2. Check sampling ratio isn't filtering all traces
3. Ensure spans are properly instrumented

Next Steps

- Rust Library API - Use observability in your code
- Configuration Reference - Full config options
- Advanced Topics - Security considerations

Node.js Installation

The JACS Node.js package (`@hai.ai/jacs`) provides JavaScript/TypeScript bindings to the JACS Rust library, making it easy to integrate JACS into web applications, servers, and Node.js projects.

Requirements

- **Node.js**: Version 16.0 or higher
- **npm** or **yarn**: For package management
- **Operating System**: macOS, Linux, or Windows with WSL

Installation

Using npm

```
npm install @hai.ai/jacs
```

Using yarn

```
yarn add @hai.ai/jacs
```

Using pnpm

```
pnpm add @hai.ai/jacs
```

Verify Installation

Create a simple test to verify everything is working:

```
// test.js
import { JacsAgent } from '@hai.ai/jacs';

console.log('JACS Node.js bindings loaded successfully!');

// Test basic functionality (async API)
try {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');
  console.log('Agent loaded successfully!');
} catch (error) {
  console.error('Error loading agent:', error);
}
```

Run the test:

```
node test.js
```

Package Structure

The `@hai.ai/jacs` package exposes these entry points:

Core and simple API

```
import { JacsAgent, hashString, createConfig } from '@hai.ai/jacs';
import * as jacs from '@hai.ai/jacs/simple';  // quickstart, load, signMessage,
verify, etc.
```

Instance-based client (recommended for new code)

```
import { JacsClient } from '@hai.ai/jacs/client';
```

MCP (`@hai.ai/jacs/mcp`)

```
import { createJACSTransportProxy, createJACSTransportProxyAsync, registerJacsTools }
from '@hai.ai/jacs/mcp';
```

HTTP / framework adapters

```
import { jacsMiddleware } from '@hai.ai/jacs/express';
import { jacsKoaMiddleware } from '@hai.ai/jacs/koa';
import { JACSExpressMiddleware, JACSKoaMiddleware } from '@hai.ai/jacs/http';  //
legacy
```

TypeScript Support

The package includes full TypeScript definitions:

```
import { JacsAgent, createConfig, hashString } from '@hai.ai/jacs';

// Create an agent instance
const agent: JacsAgent = new JacsAgent();

// Load configuration from file (async)
await agent.load('./jacs.config.json');

// Use utility functions
const hash: string = hashString('some data');

// Create a configuration string
const configJson: string = createConfig(
  undefined,           // jacs_use_security
  './jacs_data',       // jacs_data_directory
  './jacs_keys',       // jacs_key_directory
  undefined,           // jacs_agent_private_key_filename
  undefined,           // jacs_agent_public_key_filename
  'ring-Ed25519',      // jacs_agent_key_algorithm
  undefined,           // jacs_private_key_password
  undefined,           // jacs_agent_id_and_version
  'fs'                 // jacs_default_storage
);
```

Configuration

Basic Configuration

```
const config = {
  // Required fields
  jacs_data_directory: "./jacs_data",      // Where documents are stored
  jacs_key_directory: "./jacs_keys",       // Where keys are stored
  jacs_default_storage: "fs",              // Storage backend
  jacs_agent_key_algorithm: "ring-Ed25519",     // Signing algorithm

  // Optional fields
  jacs_agent_id_and_version: null,         // Existing agent to load
  jacs_agent_private_key_filename: "private.pem",
  jacs_agent_public_key_filename: "public.pem"
};
```

Configuration File

Create a `jacs.config.json` file:

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_default_storage": "fs",
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

Load the configuration:

```
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
agent.load('./jacs.config.json');
```

Environment Variables

JACS reads environment variables that override configuration file settings:

```
export JACS_DATA_DIRECTORY="./production_data"
export JACS_KEY_DIRECTORY="./production_keys"
export JACS_AGENT_KEY_ALGORITHM="ring-Ed25519"
export JACS_DEFAULT_STORAGE="fs"
```

Storage Backends

Configure storage in `jacs.config.json`:

File System (Default)

```
{
  "jacs_default_storage": "fs",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys"
}
```

Local Indexed SQLite

```
{
  "jacs_default_storage": "rusqlite"
}
```

Use `rusqlite` when you want local full-text search and the upgraded `DocumentService` behavior in bindings and MCP.

AWS Storage

```
{
  "jacs_default_storage": "aws"
}
```

AWS credentials are read from standard AWS environment variables.

Memory Storage (Testing)

```
{
  "jacs_default_storage": "memory"
}
```

Cryptographic Algorithms

ring-Ed25519 (Recommended)

```
{
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

Pros: Fast, secure, small signatures **Cons**: Requires elliptic curve support

RSA-PSS

```
{
  "jacs_agent_key_algorithm": "RSA-PSS"
}
```

Pros: Widely supported, proven security **Cons**: Larger signatures, slower

pq-dilithium (Post-Quantum)

```
{
  "jacs_agent_key_algorithm": "pq-dilithium"
}
```

Pros: Quantum-resistant **Cons**: Experimental, large signatures

pq2025 (Post-Quantum Hybrid)

```
{
  "jacs_agent_key_algorithm": "pq2025"
}
```

Pros: Combines ML-DSA-87 with hybrid approach **Cons**: Newest algorithm, largest signatures

Development Setup

Project Structure

```
my-jacs-project/
├── package.json
├── jacs.config.json
├── src/
│   ├── agent.js
│   ├── tasks.js
│   └── agreements.js
├── jacs_data/
│   ├── agents/
│   ├── tasks/
│   └── documents/
└── jacs_keys/
    ├── private.pem
    └── public.pem
```

Package.json Setup

```json
{
  "name": "my-jacs-app",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "@hai.ai/jacs": "^0.6.0",
    "express": "^4.18.0"
  },
  "scripts": {
    "start": "node src/app.js",
    "test": "node test/test.js",
    "dev": "nodemon src/app.js"
  }
}
```

Basic Application

```
// src/app.js
import { JacsAgent } from '@hai.ai/jacs';

async function main() {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  const documentJson = JSON.stringify({
    title: "My First Document",
    content: "Hello from Node.js!"
  });

  const signedDoc = await agent.createDocument(documentJson);
  console.log('Document created:', signedDoc);

  const isValid = await agent.verifyDocument(signedDoc);
  console.log('Document valid:', isValid);
  console.log('JACS agent ready!');
}
main().catch(console.error);
```

Common Issues

Module Not Found

If you get `Module not found` errors:

```
# Check Node.js version
node --version  # Should be 16+

# Clear node_modules and reinstall
rm -rf node_modules package-lock.json
npm install
```

Permission Errors

If you get permission errors accessing files:

```
# Check directory permissions
ls -la jacs_data/ jacs_keys/

# Fix permissions
chmod 755 jacs_data/ jacs_keys/
chmod 600 jacs_keys/*.pem
```

Binary Compatibility

If you get binary compatibility errors:

```
# Rebuild native modules
npm rebuild

# Or reinstall
npm uninstall @hai.ai/jacs
npm install @hai.ai/jacs
```

TypeScript Issues

If TypeScript can't find definitions:

```
// tsconfig.json
{
  "compilerOptions": {
    "moduleResolution": "node",
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true
  }
}
```

Next Steps

Now that you have JACS installed:

1. **Basic Usage** - Learn core JACS operations
2. **MCP Integration** - Add Model Context Protocol support
3. **HTTP Server** - Create JACS HTTP APIs
4. **Express Middleware** - Integrate with Express.js
5. **API Reference** - Complete API documentation

Examples

Check out the complete examples in the examples directory:

- Basic agent creation and task management
- Express.js middleware integration
- MCP server implementation

Simplified API

The simplified API (`@hai.ai/jacs/simple`) provides a streamlined, module-level interface for common JACS operations. It's designed to get you signing and verifying in under 2 minutes.

v0.7.0: Async-First API

All NAPI operations now return Promises by default. Sync variants are available with a `Sync` suffix, following the Node.js convention (like `fs.readFile` vs `fs.readFileSync`).

```
// Async (default, recommended -- does not block the event loop)
const signed = await jacs.signMessage({ action: 'approve' });

// Sync (blocks event loop, use in scripts or CLI tools)
const signed = jacs.signMessageSync({ action: 'approve' });
```

Quick Start

Quickstart -- one call (with required `name` / `domain`) to start signing:

```
const jacs = require('@hai.ai/jacs/simple');

const info = await jacs.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
console.log(info.configPath, info.publicKeyPath, info.privateKeyPath);
const signed = await jacs.signMessage({ action: 'approve', amount: 100 });
const result = await jacs.verify(signed.raw);
console.log(`Valid: ${result.valid}, Signer: ${result.signerId}`);
```

`quickstart(name, domain, ...)` creates a persistent agent with keys on disk (default algorithm: `pq2025`). If `./jacs.config.json` already exists, it loads it; otherwise it creates a new agent. Returned `AgentInfo` includes config/key paths (`config_path` / `public_key_path` / `private_key_path` in Python, `configPath` / `publicKeyPath` / `privateKeyPath` in Node) plus key directory metadata so you can locate key material immediately. Rust/CLI quickstart requires an explicit password source (`JACS_PRIVATE_KEY_PASSWORD` , or `JACS_PASSWORD_FILE` in CLI). Python/Node can auto-generate a password if needed; set `JACS_SAVE_PASSWORD_FILE=true` if you want that generated password persisted to `./jacs_keys/.jacs_password` . Pass `{ algorithm: 'ring-Ed25519' }` to override the default (`pq2025`).

To load an existing agent explicitly, use `load()` instead:

```
const agent = await jacs.load('./jacs.config.json');
const signed = await jacs.signMessage({ action: 'approve', amount: 100 });
```

When to Use the Simplified API

Simplified API	JacsAgent Class
Quick prototyping	Multiple agents in one process
Scripts and CLI tools	Complex multi-document workflows
MCP tool implementations	Fine-grained control
Single-agent applications	Custom error handling

API Reference

Every function that calls into NAPI has both async (default) and sync variants:

Function	Sync Variant	Description
`quickstart(options)`	`quickstartSync(options)`	Create a persistent agent with keys on disk
`create(options)`	`createSync(options)`	Create a new agent programmatically
`load(configPath)`	`loadSync(configPath)`	Load agent from config file
`verifySelf()`	`verifySelfSync()`	Verify agent's own integrity
`updateAgent(data)`	`updateAgentSync(data)`	Update agent document
`updateDocument(id, data)`	`updateDocumentSync(id, data)`	Update existing document
`signMessage(data)`	`signMessageSync(data)`	Sign any JSON data
`signFile(path, embed)`	`signFileSync(path, embed)`	Sign a file
`verify(doc)`	`verifySync(doc)`	Verify signed document
`verifyById(id)`	`verifyByIdSync(id)`	Verify by storage ID
`reencryptKey(old, new)`	`reencryptKeySync(old, new)`	Re-encrypt private key
`createAgreement(doc, ids, ...)`	`createAgreementSync(doc, ids, ...)`	Create multi-party agreement
`signAgreement(doc)`	`signAgreementSync(doc)`	Sign an agreement
`checkAgreement(doc)`	`checkAgreementSync(doc)`	Check agreement status

Function	Sync Variant	Description
`audit(options?)`	`auditSync(options?)`	Run a security audit

Pure sync functions (no NAPI call, no suffix needed):

Function	Description
`verifyStandalone(doc, opts?)`	Verify without loading an agent
`getPublicKey()`	Get public key
`isLoaded()`	Check if agent is loaded
`getDnsRecord(domain, ttl?)`	Get DNS TXT record
`getWellKnownJson()`	Get well-known JSON
`trustAgent(json)`	Add agent to trust store
`listTrustedAgents()`	List trusted agent IDs
`untrustAgent(id)`	Remove from trust store
`isTrusted(id)`	Check if agent is trusted
`getTrustedAgent(id)`	Get trusted agent's JSON
`generateVerifyLink(doc, baseUrl?)`	Generate verification URL

---

quickstart(options)

Create a persistent agent with keys on disk. If `./jacs.config.json` already exists, loads it. Otherwise creates a new agent, saving keys and config to disk. If `JACS_PRIVATE_KEY_PASSWORD` is unset, Node quickstart auto-generates a secure password in-process (`JACS_SAVE_PASSWORD_FILE=true` persists it to `./jacs_keys/.jacs_password`). Call this once before `signMessage()` or `verify()` .

Parameters:

- `options` (object, required fields): `{ name: string, domain: string, description?: string, algorithm?: string, configPath?: string }` . Default algorithm: `"pq2025"` . Also: `"ring-Ed25519"` , `"RSA-PSS"` .

Returns: `Promise<AgentInfo>` (async) or `AgentInfo` (sync)

```
const info = await jacs.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
console.log(`Agent ID: ${info.agentId}`);
console.log(`Config path: ${info.configPath}`);
console.log(`Public key: ${info.publicKeyPath}`);
console.log(`Private key: ${info.privateKeyPath}`);

// Or with a specific algorithm
const info = await jacs.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
  algorithm: 'ring-Ed25519',
});

// Sync variant (blocks event loop)
const info = jacs.quickstartSync({
  name: 'my-agent',
  domain: 'my-agent.example.com',
  algorithm: 'ring-Ed25519',
});
```

---

load(configPath?)

Load a persistent agent from a configuration file. Use this instead of `quickstart(options)` when you want to load a specific config file explicitly.

Parameters:

- `configPath` (string, optional): Path to jacs.config.json (default: "./jacs.config.json")

Returns: `Promise<AgentInfo>` (async) or `AgentInfo` (sync)

```
const info = await jacs.load('./jacs.config.json');
console.log(`Agent ID: ${info.agentId}`);

// Sync variant
const info = jacs.loadSync('./jacs.config.json');
```

---

isLoaded()

Check if an agent is currently loaded.

Returns: boolean

```
if (!jacs.isLoaded()) {
  await jacs.load('./jacs.config.json');
}
```

---

getAgentInfo()

Get information about the currently loaded agent.

Returns: `AgentInfo` or null if no agent is loaded

```
const info = jacs.getAgentInfo();
if (info) {
  console.log(`Agent: ${info.agentId}`);
}
```

---

verifySelf()

Verify the loaded agent's own integrity (signature and hash).

Returns: `Promise<VerificationResult>` (async) or `VerificationResult` (sync)

Throws: Error if no agent is loaded

```
const result = await jacs.verifySelf();
if (result.valid) {
  console.log('Agent integrity verified');
} else {
  console.log('Errors:', result.errors);
}
```

---

signMessage(data)

Sign arbitrary data as a JACS document.

Parameters:

- `data` (any): Object, array, string, or any JSON-serializable value

Returns: `Promise<SignedDocument>` (async) or `SignedDocument` (sync)

Throws: Error if no agent is loaded

```
// Async (recommended)
const signed = await jacs.signMessage({
  action: 'transfer',
  amount: 500,
  recipient: 'agent-123'
});

// Sync
const signed = jacs.signMessageSync({
  action: 'transfer',
  amount: 500,
  recipient: 'agent-123'
});

console.log(`Document ID: ${signed.documentId}`);
console.log(`Signed by: ${signed.agentId}`);
```

---

signFile(filePath, embed?)

Sign a file with optional content embedding.

Parameters:

- `filePath` (string): Path to the file to sign
- `embed` (boolean, optional): If true, embed file content in the document (default: false)

Returns: `Promise<SignedDocument>` (async) or `SignedDocument` (sync)

```
// Reference only (stores hash)
const signed = await jacs.signFile('contract.pdf', false);

// Embed content (creates portable document)
const embedded = await jacs.signFile('contract.pdf', true);
```

---

verify(signedDocument)

Verify a signed document and extract its content.

Parameters:

- `signedDocument` (string): The JSON string of the signed document

Returns: `Promise<VerificationResult>` (async) or `VerificationResult` (sync)

```
const result = await jacs.verify(signedJson);

if (result.valid) {
  console.log(`Signed by: ${result.signerId}`);
  console.log(`Data: ${JSON.stringify(result.data)}`);
} else {
  console.log(`Invalid: ${result.errors.join(', ')}`);
}
```

---

verifyStandalone(signedDocument, options?)

Verify a signed document **without** loading an agent. Use when you only need to verify (e.g. a lightweight API). Does not use the global agent.

Parameters:

- `signedDocument` (string): The signed JACS document JSON
- `options` (object, optional): `{ keyResolution?, dataDirectory?, keyDirectory? }`

Returns: `VerificationResult` (always sync -- no NAPI call)

```
const result = jacs.verifyStandalone(signedJson, { keyResolution: 'local',
keyDirectory: './keys' });
console.log(result.valid, result.signerId);
```

---

audit(options?)

Run a read-only security audit and health checks. Returns an object with `risks`, `health_checks`, `summary`, and `overall_status`. Does not require a loaded agent; does not modify state.

Parameters: `options` (object, optional): `{ configPath?, recentN? }`

Returns: `Promise<object>` (async) or `object` (sync)

See Security Model -- Security Audit for full details and options.

```
const result = await jacs.audit();
console.log(`Risks: ${result.risks.length}, Status: ${result.overall_status}`);
```

---

updateAgent(newAgentData)

Update the agent document with new data and re-sign it.

This function expects a **complete agent document** (not partial updates). Use `exportAgent()` to get the current document, modify it, then pass it here.

Parameters:

- `newAgentData` (object|string): Complete agent document as JSON string or object

Returns: `Promise<string>` (async) or `string` (sync) -- The updated and re-signed agent document

```
const agentDoc = JSON.parse(jacs.exportAgent());
agentDoc.jacsAgentType = 'hybrid';
const updated = await jacs.updateAgent(agentDoc);
```

---

updateDocument(documentId, newDocumentData, attachments?, embed?)

Update an existing document with new data and re-sign it.

Parameters:

- `documentId` (string): The document ID (jacsId) to update
- `newDocumentData` (object|string): Updated document as JSON string or object
- `attachments` (string[], optional): Array of file paths to attach
- `embed` (boolean, optional): If true, embed attachment contents

Returns: `Promise<SignedDocument>` (async) or `SignedDocument` (sync)

```
const original = await jacs.signMessage({ status: 'pending', amount: 100 });
const doc = JSON.parse(original.raw);
doc.content.status = 'approved';
const updated = await jacs.updateDocument(original.documentId, doc);
```

---

exportAgent()

Export the current agent document for sharing or inspection.

Returns: string -- The agent JSON document (pure sync, no suffix needed)

```
const agentDoc = jacs.exportAgent();
const agent = JSON.parse(agentDoc);
console.log(`Agent type: ${agent.jacsAgentType}`);
```

---

getDnsRecord(domain, ttl?)

Return the DNS TXT record line for the loaded agent. Pure sync, no suffix needed.

Parameters: `domain` (string), `ttl` (number, optional, default 3600)

Returns: string

---

getWellKnownJson()

Return the well-known JSON object for the loaded agent. Pure sync, no suffix needed.

Returns: object

---

getPublicKey()

Get the loaded agent's public key in PEM format. Pure sync, no suffix needed.

Returns: string -- PEM-encoded public key

```
const pem = jacs.getPublicKey();
console.log(pem);
```

Type Definitions

AgentInfo

```
interface AgentInfo {
  agentId: string;      // Agent's UUID
  name: string;         // Agent name from config
  publicKeyPath: string; // Path to public key file
  configPath: string;   // Path to loaded config
}
```

SignedDocument

```
interface SignedDocument {
  raw: string;          // Full JSON document with signature
  documentId: string;   // Document's UUID (jacsId)
  agentId: string;      // Signing agent's ID
  timestamp: string;    // ISO 8601 timestamp
}
```

VerificationResult

```
interface VerificationResult {
  valid: boolean;       // True if signature verified
  data?: any;           // Extracted document content
  signerId: string;     // Agent who signed
  timestamp: string;    // When it was signed
  attachments: Attachment[];  // File attachments
  errors: string[];     // Error messages if invalid
}
```

Attachment

```
interface Attachment {
  filename: string;     // Original filename
  mimeType: string;     // MIME type
  hash: string;         // SHA-256 hash
  embedded: boolean;    // True if content is embedded
  content?: Buffer;     // Embedded content (if available)
}
```

Complete Example

```
const jacs = require('@hai.ai/jacs/simple');

// Load agent
const agent = await jacs.load('./jacs.config.json');
console.log(`Loaded agent: ${agent.agentId}`);

// Verify agent integrity
const selfCheck = await jacs.verifySelf();
if (!selfCheck.valid) {
  throw new Error('Agent integrity check failed');
}

// Sign a transaction
const transaction = {
  type: 'payment',
  from: agent.agentId,
  to: 'recipient-agent-uuid',
  amount: 250.00,
  currency: 'USD',
  memo: 'Q1 Service Payment'
};

const signed = await jacs.signMessage(transaction);
console.log(`Transaction signed: ${signed.documentId}`);

// Verify the transaction (simulating recipient)
const verification = await jacs.verify(signed.raw);

if (verification.valid) {
  console.log(`Payment verified from: ${verification.signerId}`);
  console.log(`Amount: ${verification.data.amount} ${verification.data.currency}`);
} else {
  console.log(`Verification failed: ${verification.errors.join(', ')}`);
}

// Sign a file
const contractSigned = await jacs.signFile('./contract.pdf', true);
console.log(`Contract signed: ${contractSigned.documentId}`);

// Update agent metadata
const agentDoc = JSON.parse(jacs.exportAgent());
agentDoc.jacsAgentType = 'ai';
const updatedAgent = await jacs.updateAgent(agentDoc);
console.log('Agent metadata updated');

// Share public key
const publicKey = jacs.getPublicKey();
console.log('Share this public key for verification:');
console.log(publicKey);
```

MCP Integration

The simplified API works well with MCP tool implementations:

```
const { Server } = require('@modelcontextprotocol/sdk/server/index.js');
const jacs = require('@hai.ai/jacs/simple');

// Load agent once at startup
await jacs.load('./jacs.config.json');

// Define a signed tool
server.setRequestHandler('tools/call', async (request) => {
  const { name, arguments: args } = request.params;

  if (name === 'approve_request') {
    const signed = await jacs.signMessage({
      action: 'approve',
      requestId: args.requestId,
      approvedBy: jacs.getAgentInfo().agentId
    });

    return {
      content: [{ type: 'text', text: signed.raw }]
    };
  }
});
```

Error Handling

```
const jacs = require('@hai.ai/jacs/simple');

try {
  await jacs.load('./missing-config.json');
} catch (e) {
  console.error('Config not found:', e.message);
}

try {
  // Will fail if no agent loaded
  await jacs.signMessage({ data: 'test' });
} catch (e) {
  console.error('No agent:', e.message);
}

try {
  await jacs.signFile('/nonexistent/file.pdf');
} catch (e) {
  console.error('File not found:', e.message);
}

// Verification doesn't throw - check result.valid
const result = await jacs.verify('invalid json');
if (!result.valid) {
  console.error('Verification errors:', result.errors);
}
```

See Also

- Basic Usage - JacsAgent class usage
- API Reference - Complete JacsAgent API
- MCP Integration - Model Context Protocol

Basic Usage

This chapter covers fundamental JACS operations in Node.js, including agent initialization, document creation, signing, and verification.

v0.7.0: Async-First API

All NAPI operations now return Promises by default. Sync variants are available with a `Sync` suffix, following the Node.js convention (like `fs.readFile` vs `fs.readFileSync`).

```
// Async (default, recommended)
await agent.load('./jacs.config.json');
const doc = await agent.createDocument(JSON.stringify(content));

// Sync (blocks event loop)
agent.loadSync('./jacs.config.json');
const doc = agent.createDocumentSync(JSON.stringify(content));
```

Initializing an Agent

Create and Load Agent

```
import { JacsAgent } from '@hai.ai/jacs';

// Create a new agent instance
const agent = new JacsAgent();

// Load configuration from file (async)
await agent.load('./jacs.config.json');

// Or use sync variant
agent.loadSync('./jacs.config.json');
```

Configuration File

Create `jacs.config.json`:

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_default_storage": "fs",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "jacs_agent_id_and_version": "agent-uuid:version-uuid"
}
```

Creating Documents

Basic Document Creation

```
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
await agent.load('./jacs.config.json');

// Create a document from JSON
const documentData = {
  title: "Project Proposal",
  content: "Quarterly development plan",
  budget: 50000
};

const signedDocument = await agent.createDocument(JSON.stringify(documentData));
console.log('Signed document:', signedDocument);
```

With Custom Schema

Validate against a custom JSON Schema:

```
const signedDocument = await agent.createDocument(
  JSON.stringify(documentData),
  './schemas/proposal.schema.json'  // custom schema path
);
```

With Output File

```
const signedDocument = await agent.createDocument(
  JSON.stringify(documentData),
  null,                     // no custom schema
  './output/proposal.json' // output filename
);
```

Without Saving

```
const signedDocument = await agent.createDocument(
  JSON.stringify(documentData),
  null,   // no custom schema
  null,   // no output filename
  true    // noSave = true
);
```

With Attachments

```
const signedDocument = await agent.createDocument(
  JSON.stringify(documentData),
  null,                       // no custom schema
  null,                       // no output filename
  false,                      // save the document
  './attachments/report.pdf', // attachment path
  true                        // embed files
);
```

Verifying Documents

Verify Document Signature

```
// Verify a document's signature and hash
const isValid = await agent.verifyDocument(signedDocumentJson);
console.log('Document valid:', isValid);
```

Verify Specific Signature Field

```
// Verify with a custom signature field
const isValid = await agent.verifySignature(
  signedDocumentJson,
  'jacsSignature'  // signature field name
);
```

Updating Documents

Update Existing Document

```
// Original document key format: "id:version"
const documentKey = 'doc-uuid:version-uuid';

// Modified document content
const updatedData = {
  jacsId: 'doc-uuid',
  jacsVersion: 'version-uuid',
  title: "Updated Proposal",
  content: "Revised quarterly plan",
  budget: 75000
};

const updatedDocument = await agent.updateDocument(
  documentKey,
  JSON.stringify(updatedData)
);

console.log('Updated document:', updatedDocument);
```

Update with New Attachments

```
const updatedDocument = await agent.updateDocument(
  documentKey,
  JSON.stringify(updatedData),
  ['./new-report.pdf'],  // new attachments
  true                   // embed files
);
```

Signing and Verification

Sign Arbitrary Data

```
// Sign any string data
const signature = await agent.signString('Important message to sign');
console.log('Signature:', signature);
```

Verify Arbitrary Data

```
// Verify a signature on string data
const isValid = await agent.verifyString(
  'Important message to sign',  // original data
  signatureBase64,              // base64 signature
  publicKeyBuffer,              // public key as Buffer
  'ring-Ed25519'                // algorithm
);
```

Working with Agreements

Create an Agreement

```
// Add agreement requiring multiple agent signatures
const documentWithAgreement = await agent.createAgreement(
  signedDocumentJson,
  ['agent1-uuid', 'agent2-uuid'],           // required signers
  'Do you agree to these terms?',            // question
  'Q1 2024 service contract',                // context
  'jacsAgreement'                            // field name
);
```

Sign an Agreement

```
// Sign the agreement as the current agent
const signedAgreement = await agent.signAgreement(
  documentWithAgreementJson,
  'jacsAgreement'  // agreement field name
);
```

Check Agreement Status

```
// Check which agents have signed
const status = await agent.checkAgreement(
  documentWithAgreementJson,
  'jacsAgreement'
);

console.log('Agreement status:', JSON.parse(status));
```

Agent Operations

Verify Agent

```
// Verify the loaded agent's signature
const isValid = await agent.verifyAgent();
console.log('Agent valid:', isValid);
```

Update Agent

```
// Update agent document
const updatedAgentJson = await agent.updateAgent(JSON.stringify({
  jacsId: 'agent-uuid',
  jacsVersion: 'version-uuid',
  name: 'Updated Agent Name',
  description: 'Updated description'
}));
```

Sign External Agent

```
// Sign another agent's document with registration signature
const signedAgentJson = await agent.signAgent(
  externalAgentJson,
  publicKeyBuffer,
  'ring-Ed25519'
);
```

Request/Response Signing

These methods remain synchronous (V8-thread-only, no `Sync` suffix):

Sign a Request

```
// Sign request parameters as a JACS document
const signedRequest = agent.signRequest({
  method: 'GET',
  path: '/api/resource',
  timestamp: new Date().toISOString(),
  body: { query: 'data' }
});
```

Verify a Response

```
// Verify a signed response
const result = agent.verifyResponse(signedResponseJson);
console.log('Response valid:', result);

// Verify and get signer's agent ID
const resultWithId = agent.verifyResponseWithAgentId(signedResponseJson);
console.log('Signer ID:', resultWithId);
```

Utility Functions

Hash String

```
import { hashString } from '@hai.ai/jacs';

// SHA-256 hash of a string
const hash = hashString('data to hash');
console.log('Hash:', hash);
```

Create Configuration

```
import { createConfig } from '@hai.ai/jacs';

// Programmatically create a config JSON string
const configJson = createConfig(
  undefined,            // jacs_use_security
  './jacs_data',        // jacs_data_directory
  './jacs_keys',        // jacs_key_directory
  undefined,            // private key filename
  undefined,            // public key filename
  'ring-Ed25519',       // key algorithm
  undefined,            // private key password
  undefined,            // agent id and version
  'fs'                  // default storage
);

console.log('Config:', configJson);
```

Error Handling

```
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();

try {
  await agent.load('./jacs.config.json');
} catch (error) {
  console.error('Failed to load agent:', error.message);
}

try {
  const doc = await agent.createDocument(JSON.stringify({ data: 'test' }));
  console.log('Document created');
} catch (error) {
  console.error('Failed to create document:', error.message);
}

try {
  const isValid = await agent.verifyDocument(invalidJson);
} catch (error) {
  console.error('Verification failed:', error.message);
}
```

Complete Example

```
import { JacsAgent, hashString } from '@hai.ai/jacs';

async function main() {
  // Initialize agent
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  // Create a task document
  const task = {
    title: 'Code Review',
    description: 'Review pull request #123',
    assignee: 'developer-uuid',
    deadline: '2024-02-01'
  };

  const signedTask = await agent.createDocument(JSON.stringify(task));
  console.log('Task created');

  // Verify the task
  if (await agent.verifyDocument(signedTask)) {
    console.log('Task signature valid');
  }

  // Create agreement for task acceptance
  const taskWithAgreement = await agent.createAgreement(
    signedTask,
    ['manager-uuid', 'developer-uuid'],
    'Do you accept this task assignment?'
  );

  // Sign the agreement
  const signedAgreement = await agent.signAgreement(taskWithAgreement);
  console.log('Agreement signed');

  // Check agreement status
  const status = await agent.checkAgreement(signedAgreement);
  console.log('Status:', status);

  // Hash some data for reference
  const taskHash = hashString(signedTask);
  console.log('Task hash:', taskHash);
}

main().catch(console.error);
```

Next Steps

- MCP Integration - Model Context Protocol support
- HTTP Server - Create HTTP APIs
- Express Middleware - Express.js integration

- API Reference - Complete API documentation

MCP Integration (Node.js)

Node has two MCP stories:

1. **Wrap an MCP transport** with signing and verification
2. **Register JACS operations as MCP tools** on an existing server

If you want a full out-of-the-box server instead, prefer the Rust `jacs-mcp` binary.

Install

```
npm install @hai.ai/jacs @modelcontextprotocol/sdk
```

1. Wrap A Transport

Use this when you already have an MCP server or client and want signed JSON-RPC messages.

With a loaded client

```
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { JacsClient } from '@hai.ai/jacs/client';
import { createJACSTransportProxy } from '@hai.ai/jacs/mcp';

const client = await JacsClient.quickstart({
  name: 'mcp-agent',
  domain: 'mcp.local',
});

const transport = new StdioServerTransport();
const secureTransport = createJACSTransportProxy(transport, client, 'server');
```

With only a config path

```
import { createJACSTransportProxyAsync } from '@hai.ai/jacs/mcp';

const secureTransport = await createJACSTransportProxyAsync(
  transport,
  './jacs.config.json',
  'server',
);
```

`createJACSTransportProxy()` does **not** take a config path. Use the async factory when the agent is not already loaded.

2. Register JACS Tools On Your MCP Server

Use this when the model should explicitly call JACS operations such as signing, verification, agreement creation, or trust-store inspection.

```
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { JacsClient } from '@hai.ai/jacs/client';
import { registerJacsTools } from '@hai.ai/jacs/mcp';

const server = new Server(
  { name: 'jacs-tools', version: '1.0.0' },
  { capabilities: { tools: {} } },
);

const client = await JacsClient.quickstart({
  name: 'mcp-agent',
  domain: 'mcp.local',
});

registerJacsTools(server, client);
```

The registered tool set includes:

- document signing and verification
- agreement helpers
- audit and agent-info helpers
- trust-store helpers
- setup and registry helper stubs

For lower-level integration, use `getJacsMcpToolDefinitions()` plus `handleJacsMcpToolCall()`.

Failure Behavior

The transport proxy is not permissive by default.

- Signing or verification failures fail closed unless you explicitly pass `allowUnsignedFallback: true`
- `createJACSTransportProxy()` expects a real `JacsClient` or `JacsAgent`, not an unloaded shell

Common Pattern

```
import { McpServer } from '@modelcontextprotocol/sdk/server/mcp.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { JacsClient } from '@hai.ai/jacs/client';
import { createJACSTransportProxy } from '@hai.ai/jacs/mcp';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});

const server = new McpServer({ name: 'my-server', version: '1.0.0' });
const transport = new StdioServerTransport();
const secureTransport = createJACSTransportProxy(transport, client, 'server');

await server.connect(secureTransport);
```

For stdio servers, keep logs on `stderr`, not `stdout`.

Example Paths In This Repo

- `jacsnpm/examples/mcp.stdio.server.js`
- `jacsnpm/examples/mcp.stdio.client.js`
- `jacsnpm/examples/mcp.sse.server.js`
- `jacsnpm/examples/mcp.sse.client.js`

When To Use LangChain Instead

Choose LangChain.js Integration instead when:

- the model and tools already live in the same Node.js process
- you only need signed tool outputs, not an MCP boundary
- you do not need other MCP clients to connect

LangChain.js Integration

Use the LangChain.js adapter when the model already runs inside your Node.js app and you want provenance at the tool boundary.

Choose The Pattern

Give The Agent JACS Tools

Use `createJacsTools()` when the model should explicitly ask to sign, verify, inspect trust, or create agreements.

```
import { JacsClient } from '@hai.ai/jacs/client';
import { createJacsTools } from '@hai.ai/jacs/langchain';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});

const jacsTools = createJacsTools({ client });
const llmWithTools = model.bindTools([...myTools, ...jacsTools]);
```

The tool set includes 14 tools:

- `jacs_sign`
- `jacs_verify`
- `jacs_create_agreement`
- `jacs_sign_agreement`
- `jacs_check_agreement`
- `jacs_verify_self`
- `jacs_trust_agent`
- `jacs_trust_agent_with_key`
- `jacs_list_trusted`
- `jacs_is_trusted`
- `jacs_share_public_key`
- `jacs_share_agent`
- `jacs_audit`
- `jacs_agent_info`

Auto-Sign Existing Tools

Use this when the model should keep using your existing tool set but every result needs a signature.

Wrap one tool:

```
import { signedTool } from '@hai.ai/jacs/langchain';

const signed = signedTool(mySearchTool, { client });
```

Wrap a LangGraph `ToolNode`:

```
import { jacsToolNode } from '@hai.ai/jacs/langchain';

const node = jacsToolNode([tool1, tool2], { client });
```

For custom graph logic:

```
import { jacsWrapToolCall } from '@hai.ai/jacs/langchain';

const wrapToolCall = jacsWrapToolCall({ client });
```

Install

```
npm install @hai.ai/jacs @langchain/core
npm install @langchain/langgraph
```

`@langchain/langgraph` is only required for `jacsToolNode()`.

Strict Mode

Pass `strict: true` when you want wrapper failures to throw instead of returning error-shaped output:

```
const jacsTools = createJacsTools({ client, strict: true });
```

Examples In This Repo

- `jacsnpm/examples/langchain/basic-agent.ts`
- `jacsnpm/examples/langchain/signing-callback.ts`

When To Use MCP Instead

Choose Node.js MCP Integration instead when:

- the model is outside your process and connects over MCP
- you want a shared MCP server usable by multiple clients
- you need transport-level signing in addition to signed tool outputs

Vercel AI SDK

Sign it. Prove it. -- for every AI model output.

The JACS Vercel AI SDK adapter adds cryptographic provenance to AI-generated text and tool results using the `LanguageModelV3Middleware` pattern. Works with `generateText`, `streamText`, and any model provider (OpenAI, Anthropic, etc.).

5-Minute Quickstart

1. Install

```
npm install @hai.ai/jacs ai @ai-sdk/openai
```

2. Create a JACS client

```
import { JacsClient } from '@hai.ai/jacs/client';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
```

3. Sign every model output

```
import { withProvenance } from '@hai.ai/jacs/vercel-ai';
import { openai } from '@ai-sdk/openai';
import { generateText } from 'ai';

const model = withProvenance(openai('gpt-4'), { client });
const { text, providerMetadata } = await generateText({ model, prompt: 'Hello!' });

console.log(providerMetadata?.jacs?.text?.documentId); // JACS document ID
```

Quick Start

```
import { JacsClient } from '@hai.ai/jacs/client';
import { withProvenance } from '@hai.ai/jacs/vercel-ai';
import { openai } from '@ai-sdk/openai';
import { generateText } from 'ai';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
const model = withProvenance(openai('gpt-4'), { client });

const { text, providerMetadata } = await generateText({
  model,
  prompt: 'Summarize the quarterly report.',
});

console.log(text);
console.log(providerMetadata?.jacs?.text?.documentId); // JACS document ID
console.log(providerMetadata?.jacs?.text?.signed);      // true
```

Every model output is signed by your JACS agent. The provenance record is attached to `providerMetadata.jacs`.

Installation

```
npm install @hai.ai/jacs ai @ai-sdk/openai  # or any provider
```

The `ai` package is a peer dependency.

Two Ways to Use

`withProvenance` (convenience)

Wraps a model with the JACS middleware in one call:

```
import { withProvenance } from '@hai.ai/jacs/vercel-ai';

const model = withProvenance(openai('gpt-4'), { client });
```

`jacsProvenance` (composable)

Returns a `LanguageModelV3Middleware` you can compose with other middleware:

```
import { jacsProvenance } from '@hai.ai/jacs/vercel-ai';
import { wrapLanguageModel } from 'ai';

const provenance = jacsProvenance({ client });

const model = wrapLanguageModel({
  model: openai('gpt-4'),
  middleware: provenance,
});
```

Options

```
interface ProvenanceOptions {
  client: JacsClient;                  // Required: initialized JacsClient
  signText?: boolean;                  // Sign generated text (default: true)
  signToolResults?: boolean;           // Sign tool call results (default: true)
  strict?: boolean;                    // Throw on signing failure (default: false)
  metadata?: Record<string, unknown>;  // Extra metadata in provenance records
}
```

Streaming

Streaming works automatically. Text chunks are accumulated and signed when the stream completes:

```
import { streamText } from 'ai';

const result = streamText({
  model: withProvenance(openai('gpt-4'), { client }),
  prompt: 'Write a haiku.',
});

for await (const chunk of result.textStream) {
  process.stdout.write(chunk);
}

// Provenance is available after stream completes
const metadata = await result.providerMetadata;
console.log(metadata?.jacs?.text?.signed); // true
```

Tool Call Signing

When `signToolResults` is true (default), tool results in the prompt are signed:

```
import { generateText, tool } from 'ai';
import { z } from 'zod';

const { text, providerMetadata } = await generateText({
  model: withProvenance(openai('gpt-4'), { client }),
  tools: {
    getWeather: tool({
      parameters: z.object({ city: z.string() }),
      execute: async ({ city }) => `Weather in ${city}: sunny, 72F`,
    }),
  },
  prompt: 'What is the weather in Paris?',
});

// Both text output and tool results are signed
console.log(providerMetadata?.jacs?.text?.signed);
console.log(providerMetadata?.jacs?.toolResults?.signed);
```

Provenance Record

Each signed output produces a `ProvenanceRecord`:

```
interface ProvenanceRecord {
  signed: boolean;       // Whether signing succeeded
  documentId: string;    // JACS document ID
  agentId: string;       // Signing agent's ID
  timestamp: string;     // ISO 8601 timestamp
  error?: string;        // Error message if signing failed
  metadata?: Record<string, unknown>;
}
```

Access records from `providerMetadata.jacs`:

```
const { providerMetadata } = await generateText({ model, prompt: '...' });

const textProvenance = providerMetadata?.jacs?.text;
const toolProvenance = providerMetadata?.jacs?.toolResults;
```

Strict Mode

By default, signing failures are logged but do not throw. Enable `strict` to throw on failure:

```
const model = withProvenance(openai('gpt-4'), {
  client,
  strict: true,  // Throws if signing fails
});
```

Next Steps

- Express Middleware - Sign HTTP API responses
- MCP Integration - Secure MCP transport
- API Reference - Complete API documentation

Express Middleware

Sign it. Prove it. -- in your Express app.

JACS provides `jacsMiddleware` for Express v4/v5 that verifies incoming signed request bodies and optionally auto-signs JSON responses. No body-parser gymnastics, no monkey-patching.

5-Minute Quickstart

1. Install

```
npm install @hai.ai/jacs express
```

2. Create a JACS client

```
import { JacsClient } from '@hai.ai/jacs/client';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
```

3. Add signing middleware

```
import express from 'express';
import { jacsMiddleware } from '@hai.ai/jacs/express';

const app = express();
app.use(express.text({ type: 'application/json' }));
app.use(jacsMiddleware({ client, verify: true }));

app.post('/api/data', (req, res) => {
  console.log(req.jacsPayload); // verified payload
  res.json({ status: 'ok' });
});

app.listen(3000);
```

Quick Start

```
import express from 'express';
import { JacsClient } from '@hai.ai/jacs/client';
import { jacsMiddleware } from '@hai.ai/jacs/express';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
const app = express();

app.use(express.text({ type: 'application/json' }));
app.use(jacsMiddleware({ client, verify: true }));

app.post('/api/data', (req, res) => {
  console.log(req.jacsPayload); // verified payload
  res.json({ status: 'ok' });
});

app.listen(3000);
```

Options

```
jacsMiddleware({
  client?: JacsClient;      // Pre-initialized client (preferred)
  configPath?: string;       // Path to jacs.config.json (if no client)
  sign?: boolean;            // Auto-sign res.json() responses (default: false)
  verify?: boolean;          // Verify incoming POST/PUT/PATCH bodies (default: true)
  optional?: boolean;        // Allow unsigned requests through (default: false)
  authReplay?: boolean | {   // Replay protection for auth-style endpoints (default:
false)
    enabled?: boolean;
    maxAgeSeconds?: number;    // default: 30
    clockSkewSeconds?: number; // default: 5
    cacheTtlSeconds?: number;  // default: maxAge + skew
  };
})
```

If neither `client` nor `configPath` is provided, the middleware initializes a client with `JacsClient.quickstart({ name: 'jacs-express', domain: 'localhost' })` on first request.

What the Middleware Does

Every request gets `req.jacsClient` -- a `JacsClient` instance you can use for manual signing/verification in route handlers.

POST/PUT/PATCH with `verify: true` (default): The string body is verified as a JACS document. On success, `req.jacsPayload` contains the extracted payload. On failure, a 401 is returned (unless `optional: true`).

With `sign: true`: `res.json()` is intercepted to auto-sign the response body before sending.

Verify Incoming Requests

```
app.use(express.text({ type: 'application/json' }));
app.use(jacsMiddleware({ client }));

app.post('/api/process', (req, res) => {
  if (!req.jacsPayload) {
    return res.status(400).json({ error: 'Missing payload' });
  }

  const { action, data } = req.jacsPayload;
  res.json({ processed: true, action });
});
```

With `optional: true`, unsigned requests pass through with `req.jacsPayload` unset:

```
app.use(jacsMiddleware({ client, optional: true }));

app.post('/api/mixed', (req, res) => {
  if (req.jacsPayload) {
    // Verified JACS request
    res.json({ verified: true, data: req.jacsPayload });
  } else {
    // Unsigned request -- handle accordingly
    res.json({ verified: false });
  }
});
```

Auth Replay Protection (Auth Endpoints)

Enable replay protection when signed JACS bodies are used as authentication artifacts:

```
app.use(
  jacsMiddleware({
    client,
    verify: true,
    authReplay: { enabled: true, maxAgeSeconds: 30, clockSkewSeconds: 5 },
  })
);
```

When enabled, middleware enforces:

- signature timestamp freshness (`maxAgeSeconds + clockSkewSeconds`)
- single-use `(signerId, signature)` dedupe inside a TTL cache

Notes:

- Keep this mode scoped to auth-style endpoints.
- Cache is in-memory per process; use a shared cache for multi-instance deployments.

Auto-Sign Responses

Enable `sign: true` to intercept `res.json()` calls:

```
app.use(jacsMiddleware({ client, sign: true }));

app.post('/api/data', (req, res) => {
  // This response will be JACS-signed automatically
  res.json({ result: 42, timestamp: new Date().toISOString() });
});
```

Manual Signing in Routes

Use `req.jacsClient` for fine-grained control:

```
app.use(jacsMiddleware({ client }));

app.post('/api/custom', async (req, res) => {
  const result = processData(req.jacsPayload);

  // Sign manually
  const signed = await req.jacsClient.signMessage(result);
  res.type('application/json').send(signed.raw);
});
```

Per-Route Middleware

Apply JACS to specific routes only:

```
const app = express();
const jacs = jacsMiddleware({ client });

// Public routes -- no JACS
app.get('/health', (req, res) => res.json({ status: 'ok' }));

// Protected routes
app.use('/api', express.text({ type: 'application/json' }), jacs);

app.post('/api/secure', (req, res) => {
  res.json({ data: req.jacsPayload });
});
```

Multiple Agents

Use different `JacsClient` instances per route group:

```
const adminClient = await JacsClient.quickstart({
  name: 'admin-agent',
  domain: 'admin.example.com',
  algorithm: 'pq2025',
});
const userClient = await JacsClient.quickstart({
  name: 'user-agent',
  domain: 'user.example.com',
  algorithm: 'ring-Ed25519',
});

app.use('/admin', express.text({ type: 'application/json' }));
app.use('/admin', jacsMiddleware({ client: adminClient }));

app.use('/user', express.text({ type: 'application/json' }));
app.use('/user', jacsMiddleware({ client: userClient }));
```

Migration from JACSExpressMiddleware

The legacy `JACSExpressMiddleware` from `@hai.ai/jacs/http` still works but is deprecated. To migrate:

Old	New
`import { JACSExpressMiddleware } from '@hai.ai/jacs/http'`	`import { jacsMiddleware } from '@hai.ai/jacs/express'`
`JACSExpressMiddleware({ configPath: '...' })`	`jacsMiddleware({ configPath: '...' })`
Per-request agent init	Shared client, lazy-loaded once

Old	New
`res.send()` monkey-patch	`res.json()` interception (opt-in)

The new middleware is simpler, faster (no per-request init), and gives you `req.jacsClient` for manual operations.

Next Steps

- Koa Middleware - Same pattern for Koa
- HTTP Server - Core HTTP integration concepts
- Vercel AI SDK - AI model provenance signing
- API Reference - Complete API documentation

Koa Middleware

Sign it. Prove it. -- in your Koa app.

JACS provides `jacsKoaMiddleware` for Koa with the same design as the Express middleware -- verify incoming signed bodies, optionally auto-sign responses.

Quick Start

```
import Koa from 'koa';
import bodyParser from 'koa-bodyparser';
import { JacsClient } from '@hai.ai/jacs/client';
import { jacsKoaMiddleware } from '@hai.ai/jacs/koa';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});
const app = new Koa();

app.use(bodyParser({ enableTypes: ['text'] }));
app.use(jacsKoaMiddleware({ client, verify: true }));

app.use(async (ctx) => {
  console.log(ctx.state.jacsPayload); // verified payload
  ctx.body = { status: 'ok' };
});

app.listen(3000);
```

Options

```
jacsKoaMiddleware({
  client?: JacsClient;      // Pre-initialized client (preferred)
  configPath?: string;       // Path to jacs.config.json (if no client)
  sign?: boolean;            // Auto-sign ctx.body after next() (default: false)
  verify?: boolean;          // Verify incoming POST/PUT/PATCH bodies (default: true)
  optional?: boolean;        // Allow unsigned requests through (default: false)
  authReplay?: boolean | {   // Replay protection for auth-style endpoints (default:
false)
    enabled?: boolean;
    maxAgeSeconds?: number;    // default: 30
    clockSkewSeconds?: number; // default: 5
    cacheTtlSeconds?: number;  // default: maxAge + skew
  };
})
```

How It Works

Every request gets `ctx.state.jacsClient` for manual use.

POST/PUT/PATCH with `verify: true`: The string body is verified. On success, `ctx.state.jacsPayload` is set. On failure, 401 is returned (unless `optional: true`).

With `sign: true`: After downstream middleware runs, if `ctx.body` is a non-Buffer object, it is signed before the response is sent.

Auth Replay Protection (Auth Endpoints)

Enable replay protection when signed JACS bodies are used as authentication artifacts:

```
app.use(
  jacsKoaMiddleware({
    client,
    verify: true,
    authReplay: { enabled: true, maxAgeSeconds: 30, clockSkewSeconds: 5 },
  })
);
```

When enabled, middleware enforces:

- signature timestamp freshness (`maxAgeSeconds + clockSkewSeconds`)
- single-use `(signerId, signature)` dedupe inside a TTL cache

Notes:

- Keep this mode scoped to auth-style endpoints.
- Cache is in-memory per process; use a shared cache for multi-instance deployments.

Auto-Sign Responses

```
app.use(jacsKoaMiddleware({ client, sign: true }));

app.use(async (ctx) => {
  // This will be JACS-signed automatically after next()
  ctx.body = { result: 42, timestamp: new Date().toISOString() };
});
```

Manual Signing

```
app.use(jacsKoaMiddleware({ client }));

app.use(async (ctx) => {
  const result = processData(ctx.state.jacsPayload);
  const signed = await ctx.state.jacsClient.signMessage(result);
  ctx.type = 'application/json';
  ctx.body = signed.raw;
});
```

Comparison with Express

Feature	Express	Koa
Import	`jacsMiddleware` from `@hai.ai/jacs/express`	`jacsKoaMiddleware` from `@hai.ai/jacs/koa`
Client access	`req.jacsClient`	`ctx.state.jacsClient`
Payload	`req.jacsPayload`	`ctx.state.jacsPayload`
Auto-sign target	`res.json()` interception	`ctx.body` after `next()`

Next Steps

- Express Middleware - Express version
- Vercel AI SDK - AI model provenance signing
- API Reference - Complete API documentation

HTTP Server

JACS provides middleware and utilities for building HTTP servers with cryptographic request/response signing. This enables secure communication between JACS agents over HTTP.

Overview

JACS HTTP integration provides:

- **Request signing**: Sign outgoing HTTP requests with your agent's key
- **Request verification**: Verify incoming requests were signed by a valid agent
- **Response signing**: Automatically sign responses before sending
- **Response verification**: Verify server responses on the client side
- **Framework middleware**: Ready-to-use middleware for Express and Koa

Core Concepts

Request/Response Flow

```
Client                            Server
  |                                 |
  |-- signRequest(payload) -----> |
  |                                 |-- verifyResponse() --> payload
  |                                 |-- process payload
  |                                 |-- signResponse(result)
  |<-- verifyResponse(result) ---|
  |
```

All messages are cryptographically signed, ensuring:

- Message integrity (no tampering)
- Agent identity (verified sender)
- Non-repudiation (proof of origin)

HTTP Client

Basic Client Usage

```
import jacs from '@hai.ai/jacs';
import http from 'http';

async function main() {
  // Load JACS agent
  await jacs.load('./jacs.config.json');

  // Prepare payload
  const payload = {
    message: "Hello, secure server!",
    data: { id: 123, value: "some data" },
    timestamp: new Date().toISOString()
  };

  // Sign the request
  const signedRequest = await jacs.signRequest(payload);

  // Send HTTP request
  const response = await sendRequest(signedRequest, 'localhost', 3000, '/api/echo');

  // Verify the response
  const verifiedResponse = await jacs.verifyResponse(response);
  console.log('Verified payload:', verifiedResponse.payload);
}

function sendRequest(body, host, port, path) {
  return new Promise((resolve, reject) => {
    const options = {
      hostname: host,
      port: port,
      path: path,
      method: 'POST',
      headers: {
        'Content-Type': 'text/plain',
        'Content-Length': Buffer.byteLength(body),
      },
    };

    const req = http.request(options, (res) => {
      let data = '';
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        if (res.statusCode >= 200 && res.statusCode < 300) {
          resolve(data);
        } else {
          reject(new Error(`HTTP ${res.statusCode}: ${data}`));
        }
      });
    });

    req.on('error', reject);
```

```
    req.write(body);
    req.end();
  });
}

main();
```

Using Fetch

```
import jacs from '@hai.ai/jacs';

async function sendJacsRequest(url, payload) {
  await jacs.load('./jacs.config.json');

  // Sign the payload
  const signedRequest = await jacs.signRequest(payload);

  // Send request
  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'text/plain' },
    body: signedRequest
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  // Verify response
  const responseText = await response.text();
  const verified = await jacs.verifyResponse(responseText);

  return verified.payload;
}

// Usage
const result = await sendJacsRequest('http://localhost:3000/api/data', {
  action: 'fetch',
  query: { id: 42 }
});
```

Express Server

Using Express Middleware

JACS provides `JACSExpressMiddleware` that automatically:

- Verifies incoming JACS requests

- Attaches the verified payload to `req.jacsPayload`
- Signs outgoing responses when you call `res.send()` with an object

```
import express from 'express';
import { JACSExpressMiddleware } from '@hai.ai/jacs/http';

const app = express();
const PORT = 3000;

// IMPORTANT: Use express.text() BEFORE JACS middleware
// This ensures req.body is a string for JACS verification
app.use('/api', express.text({ type: '*/*' }));

// Apply JACS middleware to API routes
app.use('/api', JACSExpressMiddleware({
  configPath: './jacs.server.config.json'
}));

// Route handler
app.post('/api/echo', (req, res) => {
  // Access verified payload from req.jacsPayload
  const payload = req.jacsPayload;

  if (!payload) {
    return res.status(400).send('JACS payload missing');
  }

  console.log('Received verified payload:', payload);

  // Send response object - middleware will sign it automatically
  res.send({
    echo: "Server says hello!",
    received: payload,
    timestamp: new Date().toISOString()
  });
});

app.listen(PORT, () => {
  console.log(`JACS Express server listening on port ${PORT}`);
});
```

Middleware Configuration

```
JACSExpressMiddleware({
  configPath: './jacs.config.json'  // Required: path to JACS config
})
```

Manual Request/Response Handling

For more control, you can handle signing manually:

```
import express from 'express';
import jacs from '@hai.ai/jacs';

const app = express();

// Initialize JACS once at startup
await jacs.load('./jacs.config.json');

app.use(express.text({ type: '*/*' }));

app.post('/api/process', async (req, res) => {
  try {
    // Manually verify incoming request
    const verified = await jacs.verifyResponse(req.body);
    const payload = verified.payload;

    // Process the request
    const result = {
      success: true,
      data: processData(payload),
      timestamp: new Date().toISOString()
    };

    // Manually sign the response
    const signedResponse = await jacs.signResponse(result);
    res.type('text/plain').send(signedResponse);

  } catch (error) {
    console.error('JACS verification failed:', error);
    res.status(400).send('Invalid JACS request');
  }
});
```

Koa Server

Using Koa Middleware

```
import Koa from 'koa';
import { JACSKoaMiddleware } from '@hai.ai/jacs/http';

const app = new Koa();
const PORT = 3000;

// Apply JACS Koa middleware
// Handles raw body reading, verification, and response signing
app.use(JACSKoaMiddleware({
  configPath: './jacs.server.config.json'
}));

// Route handler
app.use(async (ctx) => {
  if (ctx.path === '/api/echo' && ctx.method === 'POST') {
    // Access verified payload from ctx.state.jacsPayload or ctx.jacsPayload
    const payload = ctx.state.jacsPayload || ctx.jacsPayload;

    if (!payload) {
      ctx.status = 400;
      ctx.body = 'JACS payload missing';
      return;
    }

    console.log('Received verified payload:', payload);

    // Set response object - middleware will sign it automatically
    ctx.body = {
      echo: "Koa server says hello!",
      received: payload,
      timestamp: new Date().toISOString()
    };
  } else {
    ctx.status = 404;
    ctx.body = 'Not Found. Try POST to /api/echo';
  }
});

app.listen(PORT, () => {
  console.log(`JACS Koa server listening on port ${PORT}`);
});
```

API Reference

jacs.signRequest(payload)

Sign an object as a JACS request.

```
const signedRequest = await jacs.signRequest({
  method: 'getData',
  params: { id: 123 }
});
// Returns: JACS-signed JSON string
```

jacs.verifyResponse(responseString)

Verify a JACS-signed response and extract the payload.

```
const result = await jacs.verifyResponse(jacsResponseString);
// Returns: { payload: {...}, jacsId: '...', ... }

const payload = result.payload;
```

jacs.signResponse(payload)

Sign an object as a JACS response.

```
const signedResponse = await jacs.signResponse({
  success: true,
  data: { result: 42 }
});
// Returns: JACS-signed JSON string
```

JACSExpressMiddleware(options)

Express middleware for JACS request/response handling.

```
import { JACSExpressMiddleware } from '@hai.ai/jacs/http';

app.use(JACSExpressMiddleware({
  configPath: './jacs.config.json'  // Required
}));
```

Request Processing:

- Reads `req.body` as a JACS string

- Verifies the signature
- Attaches payload to `req.jacsPayload`
- On failure, sends 400 response

Response Processing:

- Intercepts `res.send()` calls
- If body is an object, signs it as JACS
- Sends signed string to client

JACSKoaMiddleware(options)

Koa middleware for JACS request/response handling.

```
import { JACSKoaMiddleware } from '@hai.ai/jacs/http';

app.use(JACSKoaMiddleware({
  configPath: './jacs.config.json'  // Required
}));
```

Request Processing:

- Reads raw request body
- Verifies JACS signature
- Attaches payload to `ctx.state.jacsPayload` and `ctx.jacsPayload`

Response Processing:

- Signs `ctx.body` if it's an object
- Converts to JACS string before sending

Complete Example

Server (server.js)

```
import express from 'express';
import { JACSExpressMiddleware } from '@hai.ai/jacs/http';

const app = express();

// JACS middleware for /api routes
app.use('/api', express.text({ type: '*/*' }));
app.use('/api', JACSExpressMiddleware({
  configPath: './jacs.server.config.json'
}));

// Echo endpoint
app.post('/api/echo', (req, res) => {
  const payload = req.jacsPayload;
  res.send({
    echo: payload,
    serverTime: new Date().toISOString()
  });
});

// Calculate endpoint
app.post('/api/calculate', (req, res) => {
  const { operation, a, b } = req.jacsPayload;

  let result;
  switch (operation) {
    case 'add': result = a + b; break;
    case 'subtract': result = a - b; break;
    case 'multiply': result = a * b; break;
    case 'divide': result = a / b; break;
    default: return res.status(400).send({ error: 'Unknown operation' });
  }

  res.send({ operation, a, b, result });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Client (client.js)

```
import jacs from '@hai.ai/jacs';

async function main() {
  await jacs.load('./jacs.client.config.json');

  // Call echo endpoint
  const echoResult = await callApi('/api/echo', {
    message: 'Hello, server!'
  });
  console.log('Echo result:', echoResult);

  // Call calculate endpoint
  const calcResult = await callApi('/api/calculate', {
    operation: 'multiply',
    a: 7,
    b: 6
  });
  console.log('Calculate result:', calcResult);
}

async function callApi(path, payload) {
  const signedRequest = await jacs.signRequest(payload);

  const response = await fetch(`http://localhost:3000${path}`, {
    method: 'POST',
    headers: { 'Content-Type': 'text/plain' },
    body: signedRequest
  });

  const responseText = await response.text();
  const verified = await jacs.verifyResponse(responseText);
  return verified.payload;
}

main().catch(console.error);
```

Security Considerations

Content-Type

JACS requests should use `text/plain` content type since they are signed JSON strings, not raw JSON.

Error Handling

Always handle verification failures gracefully:

```
try {
  const verified = await jacs.verifyResponse(responseText);
  return verified.payload;
} catch (error) {
  console.error('JACS verification failed:', error.message);
  // Handle invalid/tampered response
}
```

Agent Keys

Each server and client needs its own JACS agent with:

- Unique agent ID
- Private/public key pair
- Configuration file pointing to the keys

Middleware Order

For Express, ensure `express.text()` comes **before** `JACSExpressMiddleware`:

```
// Correct order
app.use('/api', express.text({ type: '*/*' }));
app.use('/api', JACSExpressMiddleware({ configPath: '...' }));

// Wrong - JACS middleware won't receive string body
app.use('/api', JACSExpressMiddleware({ configPath: '...' }));
app.use('/api', express.text({ type: '*/*' }));
```

Next Steps

- Express Middleware - More Express integration patterns
- MCP Integration - Model Context Protocol support
- API Reference - Complete API documentation

API Reference

Complete API documentation for the `@hai.ai/jacs` Node.js package.

Installation

```
npm install @hai.ai/jacs
```

v0.7.0: Async-First API

All NAPI operations now return Promises by default. Sync variants are available with a `Sync` suffix, following the Node.js convention (like `fs.readFile` vs `fs.readFileSync`).

```
// Async (default, recommended)
await agent.load('./jacs.config.json');
const doc = await agent.createDocument(JSON.stringify(content));

// Sync (blocks event loop)
agent.loadSync('./jacs.config.json');
const doc = agent.createDocumentSync(JSON.stringify(content));
```

Core Module

```
import { JacsAgent, hashString, createConfig } from '@hai.ai/jacs';
```

JacsAgent Class

The `JacsAgent` class is the primary interface for JACS operations. Each instance maintains its own state and can be used independently, allowing multiple agents in the same process.

Constructor

```
new JacsAgent()
```

Creates a new empty JacsAgent instance. Call `load()` or `loadSync()` to initialize with a configuration.

Example:

```
const agent = new JacsAgent();
await agent.load('./jacs.config.json');
```

---

agent.load(configPath) / agent.loadSync(configPath)

Load and initialize the agent from a configuration file.

Parameters:

- `configPath` (string): Path to the JACS configuration file

Returns: `Promise<string>` (async) or `string` (sync) -- The loaded agent's JSON

Example:

```
const agent = new JacsAgent();

// Async (recommended)
const agentJson = await agent.load('./jacs.config.json');

// Sync
const agentJson = agent.loadSync('./jacs.config.json');

console.log('Agent loaded:', JSON.parse(agentJson).jacsId);
```

---

agent.createDocument(...) / agent.createDocumentSync(...)

Create and sign a new JACS document.

Parameters:

- `documentString` (string): JSON string of the document content
- `customSchema` (string, optional): Path to a custom JSON Schema for validation
- `outputFilename` (string, optional): Filename to save the document
- `noSave` (boolean, optional): If true, don't save to storage (default: false)
- `attachments` (string, optional): Path to file attachments
- `embed` (boolean, optional): If true, embed attachments in the document

Returns: `Promise<string>` (async) or `string` (sync) -- The signed document as a JSON string

Example:

```
// Basic document creation (async)
const doc = await agent.createDocument(JSON.stringify({
  title: 'My Document',
  content: 'Hello, World!'
}));

// Without saving (sync)
const tempDoc = agent.createDocumentSync(
  JSON.stringify({ data: 'temporary' }),
  null, null, true
);
```

---

agent.verifyDocument(...) / agent.verifyDocumentSync(...)

Verify a document's signature and hash integrity.

Parameters:

- `documentString` (string): The signed document JSON string

Returns: `Promise<boolean>` (async) or `boolean` (sync) -- True if the document is valid

Example:

```
const isValid = await agent.verifyDocument(signedDocumentJson);
if (isValid) {
  console.log('Document signature verified');
}
```

---

agent.verifySignature(...) / agent.verifySignatureSync(...)

Verify a document's signature with an optional custom signature field.

Parameters:

- `documentString` (string): The signed document JSON string
- `signatureField` (string, optional): Name of the signature field (default: 'jacsSignature')

Returns: `Promise<boolean>` (async) or `boolean` (sync)

---

agent.updateDocument(...) / agent.updateDocumentSync(...)

Update an existing document, creating a new version.

Parameters:

- `documentKey` (string): The document key in format `"id:version"`
- `newDocumentString` (string): The modified document as JSON string
- `attachments` (Array, optional): Array of attachment file paths
- `embed` (boolean, optional): If true, embed attachments

Returns: `Promise<string>` (async) or `string` (sync)

Example:

```
const doc = JSON.parse(signedDoc);
const documentKey = `${doc.jacsId}:${doc.jacsVersion}`;
const updatedDoc = await agent.updateDocument(
  documentKey,
  JSON.stringify({ ...doc, title: 'Updated Title' })
);
```

---

agent.createAgreement(...) / agent.createAgreementSync(...)

Add an agreement requiring multiple agent signatures to a document.

Parameters:

- `documentString` (string): The document JSON string
- `agentIds` (Array): Array of agent IDs required to sign
- `question` (string, optional): The agreement question
- `context` (string, optional): Additional context
- `agreementFieldName` (string, optional): Field name (default: 'jacsAgreement')

Returns: `Promise<string>` (async) or `string` (sync)

---

agent.signAgreement(...) / agent.signAgreementSync(...)

Sign an agreement as the current agent.

Parameters:

- `documentString` (string): The document with agreement JSON string
- `agreementFieldName` (string, optional): Field name (default: 'jacsAgreement')

Returns: `Promise<string>` (async) or `string` (sync)

---

agent.checkAgreement(...) / agent.checkAgreementSync(...)

Check the status of an agreement.

Parameters:

- `documentString` (string): The document with agreement JSON string
- `agreementFieldName` (string, optional): Field name (default: 'jacsAgreement')

Returns: `Promise<string>` (async) or `string` (sync) -- JSON string with agreement status

---

agent.signArtifact(...) / agent.signArtifactSync(...)

Sign an A2A artifact with JACS provenance. This is the canonical method name.

Parameters:

- `artifactJson` (string): JSON string of the artifact to sign
- `artifactType` (string): Type of artifact (e.g., `"task"`, `"message"`)
- `parentSignaturesJson` (string, optional): JSON string of parent signatures for chain of custody

Returns: `Promise<string>` (async) or `string` (sync) -- The signed, wrapped artifact as a JSON string

Example:

```
const signed = await agent.signArtifact(
  JSON.stringify({ action: 'classify', input: 'hello' }),
  'task'
);
```

---

agent.wrapA2aArtifact(...) / agent.wrapA2aArtifactSync(...)

> **Deprecated since 0.9.0.** Use `signArtifact()` / `signArtifactSync()` instead. This method will be removed in 1.0.0.
>
> Set `JACS_SHOW_DEPRECATIONS=1` to see runtime warnings when deprecated methods are called.

Wraps an A2A artifact with JACS provenance signature. Identical behavior to `signArtifact()` / `signArtifactSync()`.

Parameters: Same as `signArtifact()` / `signArtifactSync()`.

---

agent.signString(...) / agent.signStringSync(...)

Sign arbitrary string data with the agent's private key.

Parameters:

- `data` (string): The data to sign

Returns: `Promise<string>` (async) or `string` (sync) -- Base64-encoded signature

---

agent.verifyString(...) / agent.verifyStringSync(...)

Verify a signature on arbitrary string data.

Parameters:

- `data` (string): The original data
- `signatureBase64` (string): The base64-encoded signature
- `publicKey` (Buffer): The public key as a Buffer
- `publicKeyEncType` (string): The key algorithm (e.g., 'ring-Ed25519')

Returns: `Promise<boolean>` (async) or `boolean` (sync)

---

agent.signRequest(params) -- V8-thread-only

Sign a request payload, wrapping it in a JACS document. This method is synchronous (no `Sync` suffix) because it uses V8-thread-only APIs.

Parameters:

- `params` (any): The request payload object

Returns: string -- JACS-signed request as a JSON string

---

agent.verifyResponse(documentString) -- V8-thread-only

Verify a JACS-signed response and extract the payload. Synchronous only.

Parameters:

- `documentString` (string): The JACS-signed response

Returns: object -- Object containing the verified payload

---

agent.verifyResponseWithAgentId(documentString) -- V8-thread-only

Verify a response and return both the payload and signer's agent ID. Synchronous only.

Parameters:

- `documentString` (string): The JACS-signed response

Returns: object -- Object with payload and agent ID

---

agent.verifyAgent(...) / agent.verifyAgentSync(...)

Verify the agent's own signature and hash, or verify another agent file.

Parameters:

- `agentFile` (string, optional): Path to an agent file to verify

Returns: `Promise<boolean>` (async) or `boolean` (sync)

---

agent.updateAgent(...) / agent.updateAgentSync(...)

Update the agent document with new data.

Parameters:

- `newAgentString` (string): The modified agent document as JSON string

Returns: `Promise<string>` (async) or `string` (sync)

---

agent.signAgent(...) / agent.signAgentSync(...)

Sign another agent's document with a registration signature.

Parameters:

- `agentString` (string): The agent document to sign
- `publicKey` (Buffer): The public key as a Buffer

- `publicKeyEncType` (string): The key algorithm

Returns: `Promise<string>` (async) or `string` (sync)

Utility Functions

hashString(data)

Hash a string using SHA-256.

Parameters:

- `data` (string): The string to hash

Returns: string -- Hexadecimal hash string

```
import { hashString } from '@hai.ai/jacs';
const hash = hashString('data to hash');
```

---

createConfig(options)

Create a JACS configuration JSON string programmatically.

Parameters:

- `jacsUseSecurity` (string, optional)
- `jacsDataDirectory` (string, optional)
- `jacsKeyDirectory` (string, optional)
- `jacsAgentPrivateKeyFilename` (string, optional)
- `jacsAgentPublicKeyFilename` (string, optional)
- `jacsAgentKeyAlgorithm` (string, optional)
- `jacsPrivateKeyPassword` (string, optional)
- `jacsAgentIdAndVersion` (string, optional)
- `jacsDefaultStorage` (string, optional)

Returns: string -- Configuration as JSON string

HTTP Module

```
import { JACSExpressMiddleware, JACSKoaMiddleware } from '@hai.ai/jacs/http';
```

JACSExpressMiddleware(options)

Express middleware for JACS request/response handling.

Parameters:

- `options.configPath` (string): Path to JACS configuration file

Returns: Express middleware function

JACSKoaMiddleware(options)

Koa middleware for JACS request/response handling.

Parameters:

- `options.configPath` (string): Path to JACS configuration file

Returns: Koa middleware function

MCP Module

```
import {
  JACSTransportProxy,
  createJACSTransportProxy,
  createJACSTransportProxyAsync
} from '@hai.ai/jacs/mcp';
```

createJACSTransportProxy(transport, configPath, role)

Factory function for creating a transport proxy.

Parameters:

- `transport` : The underlying MCP transport
- `configPath` (string): Path to JACS configuration file
- `role` (string): 'server' or 'client'

Returns: JACSTransportProxy instance

---

createJACSTransportProxyAsync(transport, configPath, role)

Async factory that waits for JACS to be fully loaded.

Returns: Promise

TypeScript Support

The package includes full TypeScript definitions. Import types as needed:

```
import { JacsAgent, hashString, createConfig } from '@hai.ai/jacs';

const agent: JacsAgent = new JacsAgent();
const hash: string = hashString('data');
```

Deprecated Functions

The following module-level functions are deprecated. Use `new JacsAgent()` and instance methods instead:

- `load()` -> Use `agent.load()` / `agent.loadSync()`
- `signAgent()` -> Use `agent.signAgent()` / `agent.signAgentSync()`
- `verifyString()` -> Use `agent.verifyString()` / `agent.verifyStringSync()`
- `signString()` -> Use `agent.signString()` / `agent.signStringSync()`
- `verifyAgent()` -> Use `agent.verifyAgent()` / `agent.verifyAgentSync()`
- `updateAgent()` -> Use `agent.updateAgent()` / `agent.updateAgentSync()`
- `verifyDocument()` -> Use `agent.verifyDocument()` / `agent.verifyDocumentSync()`
- `updateDocument()` -> Use `agent.updateDocument()` / `agent.updateDocumentSync()`
- `verifySignature()` -> Use `agent.verifySignature()` / `agent.verifySignatureSync()`
- `createAgreement()` -> Use `agent.createAgreement()` / `agent.createAgreementSync()`
- `signAgreement()` -> Use `agent.signAgreement()` / `agent.signAgreementSync()`
- `createDocument()` -> Use `agent.createDocument()` / `agent.createDocumentSync()`
- `checkAgreement()` -> Use `agent.checkAgreement()` / `agent.checkAgreementSync()`
- `signRequest()` -> Use `agent.signRequest()` (V8-thread-only, sync)
- `verifyResponse()` -> Use `agent.verifyResponse()` (V8-thread-only, sync)

- `verifyResponseWithAgentId()` -> Use `agent.verifyResponseWithAgentId()` (V8-thread-only, sync)

Migration Example:

```
// Old (deprecated, v0.6.x)
import jacs from '@hai.ai/jacs';
jacs.load('./jacs.config.json');
const doc = jacs.createDocument(JSON.stringify({ data: 'test' }));

// New (v0.7.0, async)
import { JacsAgent } from '@hai.ai/jacs';
const agent = new JacsAgent();
await agent.load('./jacs.config.json');
const doc = await agent.createDocument(JSON.stringify({ data: 'test' }));

// New (v0.7.0, sync)
import { JacsAgent } from '@hai.ai/jacs';
const agent = new JacsAgent();
agent.loadSync('./jacs.config.json');
const doc = agent.createDocumentSync(JSON.stringify({ data: 'test' }));
```

Error Handling

All methods may throw errors. Use try/catch for error handling:

```
try {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');
  const doc = await agent.createDocument(JSON.stringify({ data: 'test' }));
} catch (error) {
  console.error('JACS error:', error.message);
}
```

See Also

- Installation - Getting started
- Basic Usage - Common usage patterns
- MCP Integration - Model Context Protocol
- HTTP Server - HTTP integration
- Express Middleware - Express.js patterns

Python Installation

The JACS Python package (`jacs`) provides Python bindings to the JACS Rust library, making it easy to integrate JACS into AI/ML workflows, data science projects, and Python applications.

Requirements

- **Python**: Version 3.10 or higher
- **pip**: For package management
- **Operating System**: Linux, macOS, or Windows with WSL
- **Architecture**: x86_64 or ARM64

Installation

Using pip

```
pip install jacs
```

For framework adapters (LangChain, FastAPI, CrewAI, Anthropic, etc.) use optional extras, e.g. `pip install jacs[langchain]` , `jacs[fastapi]` , or `jacs[all]` . Optional: `jacs[langgraph]` , `jacs[ws]` . See Framework Adapters and the package `pyproject.toml` .

Using conda

```
conda install -c conda-forge jacs
```

Using poetry

```
poetry add jacs
```

Development Installation

```
# Clone the repository
git clone https://github.com/HumanAssisted/JACS
cd JACS/jacspy

# Create virtual environment
python -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\activate

# Install in development mode
pip install -e .
```

Verify Installation

Create a simple test to verify everything is working:

```
# test.py
import jacs
print('JACS Python bindings loaded successfully!')

# Quick check (no config file; in-memory agent)
from jacs.client import JacsClient
client = JacsClient.ephemeral()
signed = client.sign_message({"hello": "jacs"})
result = client.verify(signed.raw_json)
print('Sign & verify OK:', result.valid)
```

Or with an existing config file:

```
import jacs
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')
print('Agent loaded successfully!')
```

Run the test:

```
python test.py
```

Package Structure

The `jacs` package provides Python bindings to the JACS Rust library:

Core Module

```python
import jacs

# Create and load agent
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Utility functions
hash_value = jacs.hash_string("data to hash")
config_json = jacs.create_config(
    jacs_data_directory="./data",
    jacs_key_directory="./keys",
    jacs_agent_key_algorithm="ring-Ed25519"
)
```

JacsAgent Methods

```python
# Create a new agent instance
agent = jacs.JacsAgent()

# Load configuration
agent.load('./jacs.config.json')

# Document operations
signed_doc = agent.create_document(json_string)
is_valid = agent.verify_document(document_string)

# Signing operations
signature = agent.sign_string("data to sign")
```

Configuration

Configuration File

Create a `jacs.config.json` file:

```json
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_default_storage": "fs",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "jacs_agent_id_and_version": "agent-uuid:version-uuid"
}
```

Load Configuration in Python

```python
import jacs

# Create agent and load configuration
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')
```

Programmatic Configuration

```python
import jacs

# Create a configuration JSON string programmatically
config_json = jacs.create_config(
    jacs_data_directory="./jacs_data",
    jacs_key_directory="./jacs_keys",
    jacs_agent_key_algorithm="ring-Ed25519",
    jacs_default_storage="fs"
)
```

Environment Variables

JACS reads environment variables that override configuration file settings:

```bash
export JACS_DATA_DIRECTORY="./production_data"
export JACS_KEY_DIRECTORY="./production_keys"
export JACS_AGENT_KEY_ALGORITHM="ring-Ed25519"
export JACS_DEFAULT_STORAGE="fs"
```

Storage Backends

Configure storage in `jacs.config.json`:

File System (Default)

```json
{
  "jacs_default_storage": "fs",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys"
}
```

Local Indexed SQLite

```
{
  "jacs_default_storage": "rusqlite"
}
```

Use `rusqlite` when you want local full-text search and the upgraded `DocumentService` behavior in bindings and MCP.

AWS Storage

```
{
  "jacs_default_storage": "aws"
}
```

AWS credentials are read from standard AWS environment variables.

Memory Storage (Testing)

```
{
  "jacs_default_storage": "memory"
}
```

Cryptographic Algorithms

ring-Ed25519 (Recommended)

```
{
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

Pros: Fast, secure, small signatures **Cons**: Requires elliptic curve support

RSA-PSS

```
{
  "jacs_agent_key_algorithm": "RSA-PSS"
}
```

Pros: Widely supported, proven security **Cons**: Larger signatures, slower

pq-dilithium (Post-Quantum)

```
{
  "jacs_agent_key_algorithm": "pq-dilithium"
}
```

Pros: Quantum-resistant **Cons**: Experimental, large signatures

pq2025 (Post-Quantum Hybrid)

```
{
  "jacs_agent_key_algorithm": "pq2025"
}
```

Pros: Combines ML-DSA-87 with hybrid approach **Cons**: Newest algorithm, largest signatures

Development Setup

Project Structure

```
my-jacs-project/
├── requirements.txt
├── jacs.config.json
├── src/
│   ├── agent.py
│   ├── tasks.py
│   └── agreements.py
├── jacs_data/
│   ├── agents/
│   ├── tasks/
│   └── documents/
├── jacs_keys/
│   ├── private.pem
│   └── public.pem
└── tests/
    └── test_jacs.py
```

Requirements.txt Setup

```
jacs>=0.9.0
fastapi>=0.100.0  # For FastMCP integration
uvicorn>=0.23.0   # For ASGI server
pydantic>=2.0.0   # For data validation
```

Basic Application

```python
# src/app.py
import jacs
import json

def main():
    # Create and load agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Create a document
    document_data = {
        "title": "My First Document",
        "content": "Hello from Python!"
    }

    signed_doc = agent.create_document(json.dumps(document_data))
    print('Document created')

    # Verify the document
    is_valid = agent.verify_document(signed_doc)
    print(f'Document valid: {is_valid}')

    print('JACS agent ready!')
    return agent

if __name__ == "__main__":
    agent = main()
```

Virtual Environment Setup

Using venv

```bash
# Create virtual environment
python -m venv jacs-env

# Activate (Linux/macOS)
source jacs-env/bin/activate

# Activate (Windows)
jacs-env\Scripts\activate

# Install JACS
pip install jacs
```

Using conda

```
# Create conda environment
conda create -n jacs-env python=3.11

# Activate environment
conda activate jacs-env

# Install JACS
pip install jacs
```

Using poetry

```
# Initialize poetry project
poetry init

# Add JACS dependency
poetry add jacs

# Install dependencies
poetry install

# Activate shell
poetry shell
```

Jupyter Notebook Setup

```
# Install Jupyter in your JACS environment
pip install jupyter

# Start Jupyter
jupyter notebook
```

```python
# In your notebook
import jacs
import json

# Create and load agent
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create a simple document
doc = agent.create_document(json.dumps({
    "title": "Notebook Analysis",
    "data": [1, 2, 3, 4, 5]
}))

print("Document created!")
print("JACS ready for notebook use!")
```

Common Issues

Module Not Found

If you get `ModuleNotFoundError: No module named 'jacs'`:

```bash
# Check Python version
python --version  # Should be 3.10+

# Check if jacs is installed
pip list | grep jacs

# Reinstall if needed
pip uninstall jacs
pip install jacs
```

Permission Errors

If you get permission errors accessing files:

```bash
# Check directory permissions
ls -la jacs_data/ jacs_keys/

# Fix permissions
chmod 755 jacs_data/ jacs_keys/
chmod 600 jacs_keys/*.pem
```

Binary Compatibility

If you get binary compatibility errors:

```
# Update pip and reinstall
pip install --upgrade pip
pip uninstall jacs
pip install jacs --no-cache-dir
```

Windows Issues

On Windows, you may need Visual C++ Build Tools:

```
# Install Visual C++ Build Tools
# Or use conda-forge
conda install -c conda-forge jacs
```

Type Hints and IDE Support

JACS is built with Rust and PyO3, providing Python bindings:

```python
import jacs
import json

# Create agent instance
agent: jacs.JacsAgent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create and verify documents
signed_doc: str = agent.create_document(json.dumps({"title": "Test"}))
is_valid: bool = agent.verify_document(signed_doc)
```

Testing Setup

```python
# tests/test_jacs.py
import unittest
import jacs
import json

class TestJACS(unittest.TestCase):
    def setUp(self):
        # Requires a valid jacs.config.json file
        self.agent = jacs.JacsAgent()
        self.agent.load('./jacs.config.json')

    def test_document_creation(self):
        doc_data = {"title": "Test Document", "content": "Test content"}
        signed_doc = self.agent.create_document(json.dumps(doc_data))

        # Document should be a valid JSON string
        parsed = json.loads(signed_doc)
        self.assertIn("jacsId", parsed)
        self.assertIn("jacsSignature", parsed)

    def test_document_verification(self):
        doc_data = {"title": "Verify Test"}
        signed_doc = self.agent.create_document(json.dumps(doc_data))

        is_valid = self.agent.verify_document(signed_doc)
        self.assertTrue(is_valid)

    def test_sign_string(self):
        signature = self.agent.sign_string("test data")
        self.assertIsInstance(signature, str)
        self.assertTrue(len(signature) > 0)

if __name__ == "__main__":
    unittest.main()
```

Next Steps

Now that you have JACS installed:

1. **Basic Usage** - Learn core JACS operations
2. **MCP Integration** - Add Model Context Protocol support
3. **FastMCP Integration** - Build advanced MCP servers
4. **API Reference** - Complete API documentation

Examples

Check out the complete examples in the examples directory:

- Basic agent creation and task management
- Jupyter notebook workflows
- FastMCP server implementation
- AI/ML pipeline integration

Simplified API

The simplified API (`jacs.simple`) provides a streamlined, module-level interface for common JACS operations. It's designed to get you signing and verifying in under 2 minutes.

Quick Start

Zero-config -- one call to start signing:

```
import jacs.simple as jacs

info = jacs.quickstart(name="my-agent", domain="my-agent.example.com")
print(info.config_path, info.public_key_path, info.private_key_path)
signed = jacs.sign_message({"action": "approve", "amount": 100})
result = jacs.verify(signed.raw)
print(f"Valid: {result.valid}, Signer: {result.signer_id}")
```

`quickstart(name, domain, ...)` creates a persistent agent with keys on disk (default algorithm: `pq2025`). If `./jacs.config.json` already exists, it loads it; otherwise it creates a new agent. Returned `AgentInfo` includes config/key paths (`config_path` / `public_key_path` / `private_key_path` in Python, `configPath` / `publicKeyPath` / `privateKeyPath` in Node) plus key directory metadata so you can locate key material immediately. Rust/CLI quickstart requires an explicit password source (`JACS_PRIVATE_KEY_PASSWORD` , or `JACS_PASSWORD_FILE` in CLI). Python/Node can auto-generate a password if needed; set `JACS_SAVE_PASSWORD_FILE=true` if you want that generated password persisted to `./jacs_keys/.jacs_password` . Pass `algorithm="ring-Ed25519"` to override the default (`pq2025`).

To load an existing agent explicitly, use `load()` instead:

```
agent = jacs.load("./jacs.config.json")
signed = jacs.sign_message({"action": "approve", "amount": 100})
```

When to Use the Simplified API

Simplified API	JacsAgent Class
Quick prototyping	Multiple agents in one process
Scripts and CLI tools	Complex multi-document workflows
MCP tool implementations	Fine-grained control
Single-agent applications	Custom error handling

API Reference

quickstart(name, domain, description=None, algorithm=None, config_path=None)

Create a persistent agent with keys on disk. If `./jacs.config.json` already exists, loads it. Otherwise creates a new agent, saving keys and config to disk. If `JACS_PRIVATE_KEY_PASSWORD` is unset, Python quickstart auto-generates a secure password in-process (`JACS_SAVE_PASSWORD_FILE=true` persists it to `./jacs_keys/.jacs_password`). Call this once before `sign_message()` or `verify()` .

Parameters:

- `name` (str, required): Agent name used for first-time creation.
- `domain` (str, required): Agent domain used for DNS/public-key verification workflows.
- `description` (str, optional): Human-readable description.
- `algorithm` (str, optional): Signing algorithm. Default: `"pq2025"` . Also: `"ring-Ed25519"` , `"RSA-PSS"` .
- `config_path` (str, optional): Config path (default: `"./jacs.config.json"`).

Returns: `AgentInfo` dataclass

```
info = jacs.quickstart(name="my-agent", domain="my-agent.example.com")
print(f"Agent ID: {info.agent_id}")
print(f"Config path: {info.config_path}")
print(f"Public key: {info.public_key_path}")
print(f"Private key: {info.private_key_path}")

# Or with a specific algorithm
info = jacs.quickstart(
    name="my-agent",
    domain="my-agent.example.com",
    algorithm="ring-Ed25519",
)
```

---

load(config_path=None, strict=None)

Load a persistent agent from a configuration file. Use this instead of `quickstart(name=..., domain=..., ...)` when you want to load a specific config file explicitly.

Parameters:

- `config_path` (str, optional): Path to jacs.config.json (default: "./jacs.config.json")
- `strict` (bool, optional): If True, verification failures raise; if None, uses `JACS_STRICT_MODE` env var

Returns: `AgentInfo` dataclass

Raises: `JacsError` if config not found or invalid

```
info = jacs.load("./jacs.config.json")
print(f"Agent ID: {info.agent_id}")
print(f"Config: {info.config_path}")
```

---

is_loaded()

Check if an agent is currently loaded.

Returns: bool

```
if not jacs.is_loaded():
    jacs.load("./jacs.config.json")
```

---

get_agent_info()

Get information about the currently loaded agent.

Returns: `AgentInfo` or None if no agent is loaded

```
info = jacs.get_agent_info()
if info:
    print(f"Agent: {info.agent_id}")
```

---

verify_self()

Verify the loaded agent's own integrity (signature and hash).

Returns: `VerificationResult`

Raises: `AgentNotLoadedError` if no agent is loaded

```
result = jacs.verify_self()
if result.valid:
    print("Agent integrity verified")
else:
    print(f"Errors: {result.errors}")
```

---

sign_message(data)

Sign arbitrary data as a JACS document.

Parameters:

- `data` (any): Dict, list, string, or any JSON-serializable value

Returns: `SignedDocument`

Raises: `AgentNotLoadedError` if no agent is loaded

```
# Sign a dict
signed = jacs.sign_message({
    "action": "transfer",
    "amount": 500,
    "recipient": "agent-123"
})

print(f"Document ID: {signed.document_id}")
print(f"Signed by: {signed.agent_id}")
print(f"Timestamp: {signed.timestamp}")
print(f"Raw JSON: {signed.raw}")
```

---

sign_file(file_path, embed=False)

Sign a file with optional content embedding.

Parameters:

- `file_path` (str): Path to the file to sign
- `embed` (bool, optional): If True, embed file content in the document (default: False)

Returns: `SignedDocument`

Raises: `JacsError` if file not found or no agent loaded

```
# Reference only (stores hash)
signed = jacs.sign_file("contract.pdf", embed=False)

# Embed content (creates portable document)
embedded = jacs.sign_file("contract.pdf", embed=True)
```

---

verify(signed_document)

Verify a signed document and extract its content.

Parameters:

- `signed_document` (str|dict|SignedDocument): The signed document as JSON string, dict, or SignedDocument

Returns: `VerificationResult`

Raises: `AgentNotLoadedError` if no agent is loaded

```
result = jacs.verify(signed_json)

if result.valid:
    print(f"Signed by: {result.signer_id}")
    print(f"Timestamp: {result.timestamp}")
else:
    print(f"Invalid: {', '.join(result.errors)}")
```

---

verify_standalone(document, key_resolution="local", data_directory=None, key_directory=None)

Verify a signed document **without** loading an agent. Use when you only need to verify (e.g. a lightweight API).

Parameters: `document` (str|dict), `key_resolution` (str), `data_directory` (str, optional), `key_directory` (str, optional)

Returns: `VerificationResult`

---

verify_by_id(document_id)

Verify a document by its storage ID (`uuid:version`) without passing the full JSON. Requires the document to be stored locally (e.g. in the agent's data directory).

Parameters: `document_id` (str): Document ID in format `"uuid:version"`

Returns: `VerificationResult`

---

reencrypt_key(old_password, new_password)

Re-encrypt the loaded agent's private key with a new password.

Parameters: `old_password` (str), `new_password` (str)

Raises: `AgentNotLoadedError` if no agent loaded; `JacsError` if password is wrong

---

audit(config_path=None, recent_n=None)

Run a read-only security audit and health checks. Returns a dict with `risks`, `health_checks`, `summary`, and `overall_status`. Does not require a loaded agent; does not modify state.

See Security Model — Security Audit for full details and options.

```
result = jacs.audit()
print(f"Risks: {len(result['risks'])}, Status: {result['overall_status']}")
```

---

update_agent(new_agent_data)

Update the agent document with new data and re-sign it.

This function expects a **complete agent document** (not partial updates). Use `export_agent()` to get the current document, modify it, then pass it here.

Parameters:

- `new_agent_data` (dict|str): Complete agent document as JSON string or dict

Returns: str - The updated and re-signed agent document

Raises: `AgentNotLoadedError` if no agent loaded, `JacsError` if validation fails

```
import json

# Get current agent document
agent_doc = json.loads(jacs.export_agent())

# Modify fields
agent_doc["jacsAgentType"] = "hybrid"
agent_doc["jacsContacts"] = [{"contactFirstName": "Jane", "contactLastName": "Doe"}]

# Update (creates new version, re-signs, re-hashes)
updated = jacs.update_agent(agent_doc)
new_doc = json.loads(updated)

print(f"New version: {new_doc['jacsVersion']}")
print(f"Previous: {new_doc['jacsPreviousVersion']}")
```

Valid `jacsAgentType` values: `"human"`, `"human-org"`, `"hybrid"`, `"ai"`

---

update_document(document_id, new_document_data, attachments=None, embed=False)

Update an existing document with new data and re-sign it.

Note: The original document must have been saved to disk (created without `no_save=True`).

Parameters:

- `document_id` (str): The document ID (jacsId) to update
- `new_document_data` (dict|str): Updated document as JSON string or dict
- `attachments` (list, optional): List of file paths to attach
- `embed` (bool, optional): If True, embed attachment contents

Returns: `SignedDocument` with the updated document

Raises: `JacsError` if document not found, no agent loaded, or validation fails

```
import json

# Create a document (must be saved to disk)
original = jacs.sign_message({"status": "pending", "amount": 100})

# Later, update it
doc = json.loads(original.raw)
doc["content"]["status"] = "approved"

updated = jacs.update_document(original.document_id, doc)
new_doc = json.loads(updated.raw)

print(f"New version: {new_doc['jacsVersion']}")
print(f"Previous: {new_doc['jacsPreviousVersion']}")
```

---

export_agent()

Export the current agent document for sharing or inspection.

Returns: str - The agent JSON document

Raises: `AgentNotLoadedError` if no agent loaded

```
agent_doc = jacs.export_agent()
print(agent_doc)

# Parse to inspect
agent = json.loads(agent_doc)
print(f"Agent type: {agent['jacsAgentType']}")
```

---

get_dns_record(domain, ttl=3600)

Return the DNS TXT record line for the loaded agent (for DNS-based discovery). Format: `_v1.agent.jacs.{domain}. TTL IN TXT "v=hai.ai; ..."`.

Returns: str

---

get_well_known_json()

Return the well-known JSON object for the loaded agent (e.g. for `/.well-known/jacs-pubkey.json`). Keys: `publicKey`, `publicKeyHash`, `algorithm`, `agentId`.

Returns: dict

---

get_public_key()

Get the loaded agent's public key in PEM format for sharing with others.

Returns: str - PEM-encoded public key

Raises: `AgentNotLoadedError` if no agent loaded

```
pem = jacs.get_public_key()
print(pem)
# -----BEGIN PUBLIC KEY-----
# ...
# -----END PUBLIC KEY-----
```

Type Definitions

All types are Python dataclasses for convenient access:

AgentInfo

```
@dataclass
class AgentInfo:
    agent_id: str        # Agent's UUID
    config_path: str     # Path to loaded config
    public_key_path: Optional[str] = None  # Path to public key file
```

SignedDocument

```python
@dataclass
class SignedDocument:
    raw: str            # Full JSON document with signature
    document_id: str    # Document's UUID (jacsId)
    agent_id: str       # Signing agent's ID
    timestamp: str      # ISO 8601 timestamp
```

VerificationResult

```python
@dataclass
class VerificationResult:
    valid: bool                  # True if signature verified
    signer_id: Optional[str]     # Agent who signed
    timestamp: Optional[str]     # When it was signed
    attachments: List[Attachment]  # File attachments
    errors: List[str]            # Error messages if invalid
```

Attachment

```python
@dataclass
class Attachment:
    filename: str       # Original filename
    mime_type: str      # MIME type
    content_hash: str   # SHA-256 hash of file content
    content: Optional[str] = None  # Base64-encoded content (if embedded)
    size_bytes: int = 0 # Size of the original file
```

Exceptions

```python
class JacsError(Exception):
    """Base exception for JACS errors."""
    pass

class AgentNotLoadedError(JacsError):
    """Raised when an operation requires a loaded agent."""
    pass
```

Complete Example

```python
import json
import jacs.simple as jacs
from jacs.types import JacsError, AgentNotLoadedError

# Load agent
agent = jacs.load("./jacs.config.json")
print(f"Loaded agent: {agent.agent_id}")

# Verify agent integrity
self_check = jacs.verify_self()
if not self_check.valid:
    raise RuntimeError("Agent integrity check failed")

# Sign a transaction
transaction = {
    "type": "payment",
    "from": agent.agent_id,
    "to": "recipient-agent-uuid",
    "amount": 250.00,
    "currency": "USD",
    "memo": "Q1 Service Payment"
}

signed = jacs.sign_message(transaction)
print(f"Transaction signed: {signed.document_id}")

# Verify the transaction (simulating recipient)
verification = jacs.verify(signed.raw)

if verification.valid:
    doc = json.loads(signed.raw)
    print(f"Payment verified from: {verification.signer_id}")
    print(f"Amount: {doc['content']['amount']} {doc['content']['currency']}")
else:
    print(f"Verification failed: {', '.join(verification.errors)}")

# Sign a file
contract_signed = jacs.sign_file("./contract.pdf", embed=True)
print(f"Contract signed: {contract_signed.document_id}")

# Update agent metadata
agent_doc = json.loads(jacs.export_agent())
agent_doc["jacsAgentType"] = "ai"
if not agent_doc.get("jacsContacts") or len(agent_doc["jacsContacts"]) == 0:
    agent_doc["jacsContacts"] = [{"contactFirstName": "AI", "contactLastName":
"Agent"}]
updated_agent = jacs.update_agent(agent_doc)
print("Agent metadata updated")

# Share public key
public_key = jacs.get_public_key()
print("Share this public key for verification:")
print(public_key)
```

MCP Integration

The simplified API works well with FastMCP tool implementations:

```python
from fastmcp import FastMCP
import jacs.simple as jacs

mcp = FastMCP("My Server")

# Load agent once at startup
jacs.load("./jacs.config.json")

@mcp.tool()
def approve_request(request_id: str) -> dict:
    """Approve a request with cryptographic signature."""
    signed = jacs.sign_message({
        "action": "approve",
        "request_id": request_id,
        "approved_by": jacs.get_agent_info().agent_id
    })
    return {"signed_approval": signed.raw}

@mcp.tool()
def verify_approval(signed_json: str) -> dict:
    """Verify a signed approval."""
    result = jacs.verify(signed_json)
    return {
        "valid": result.valid,
        "signer": result.signer_id,
        "errors": result.errors
    }
```

Error Handling

```python
import jacs.simple as jacs
from jacs.types import JacsError, AgentNotLoadedError

try:
    jacs.load("./missing-config.json")
except JacsError as e:
    print(f"Config not found: {e}")

try:
    # Will fail if no agent loaded
    jacs.sign_message({"data": "test"})
except AgentNotLoadedError as e:
    print(f"No agent: {e}")

try:
    jacs.sign_file("/nonexistent/file.pdf")
except JacsError as e:
    print(f"File not found: {e}")

# Verification doesn't throw - check result.valid
result = jacs.verify("invalid json")
if not result.valid:
    print(f"Verification errors: {result.errors}")
```

See Also

- Basic Usage - JacsAgent class usage
- API Reference - Complete JacsAgent API
- MCP Integration - Model Context Protocol

Basic Usage

This chapter covers fundamental JACS operations in Python. For quick signing and verification, start with the **Simplified API** (`jacs.simple`) or **JacsClient**; the sections below use the `JacsAgent` class directly for fine-grained control.

Initializing an Agent

Create and Load Agent

```python
import jacs

# Create a new agent instance
agent = jacs.JacsAgent()

# Load configuration from file
agent.load('./jacs.config.json')
```

Configuration File

Create `jacs.config.json` :

```json
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_default_storage": "fs",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "jacs_agent_id_and_version": "agent-uuid:version-uuid"
}
```

Creating Documents

Basic Document Creation

```
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create a document from JSON
document_data = {
    "title": "Project Proposal",
    "content": "Quarterly development plan",
    "budget": 50000
}

signed_document = agent.create_document(json.dumps(document_data))
print('Signed document:', signed_document)
```

With Custom Schema

Validate against a custom JSON Schema:

```
signed_document = agent.create_document(
    json.dumps(document_data),
    './schemas/proposal.schema.json'  # custom schema path
)
```

With Output File

```
signed_document = agent.create_document(
    json.dumps(document_data),
    None,                     # no custom schema
    './output/proposal.json'  # output filename
)
```

Without Saving

```
signed_document = agent.create_document(
    json.dumps(document_data),
    None,   # no custom schema
    None,   # no output filename
    True    # no_save = True
)
```

With Attachments

```
signed_document = agent.create_document(
    json.dumps(document_data),
    None,                        # no custom schema
    None,                        # no output filename
    False,                       # save the document
    './attachments/report.pdf',  # attachment path
    True                         # embed files
)
```

Verifying Documents

Verify Document Signature

```
# Verify a document's signature and hash
is_valid = agent.verify_document(signed_document_json)
print('Document valid:', is_valid)
```

Verify Specific Signature Field

```
# Verify with a custom signature field
is_valid = agent.verify_signature(
    signed_document_json,
    'jacsSignature'  # signature field name
)
```

Updating Documents

Update Existing Document

```
# Original document key format: "id:version"
document_key = 'doc-uuid:version-uuid'

# Modified document content (must include jacsId and jacsVersion)
updated_data = {
    "jacsId": "doc-uuid",
    "jacsVersion": "version-uuid",
    "title": "Updated Proposal",
    "content": "Revised quarterly plan",
    "budget": 75000
}

updated_document = agent.update_document(
    document_key,
    json.dumps(updated_data)
)

print('Updated document:', updated_document)
```

Update with New Attachments

```
updated_document = agent.update_document(
    document_key,
    json.dumps(updated_data),
    ['./new-report.pdf'],  # new attachments
    True                   # embed files
)
```

Signing and Verification

Sign Arbitrary Data

```
# Sign any string data
signature = agent.sign_string('Important message to sign')
print('Signature:', signature)
```

Verify Arbitrary Data

```
# Verify a signature on string data
is_valid = agent.verify_string(
    'Important message to sign',  # original data
    signature_base64,             # base64 signature
    public_key_bytes,             # public key as bytes
    'ring-Ed25519'                # algorithm
)
```

Working with Agreements

Create an Agreement

```
# Add agreement requiring multiple agent signatures
document_with_agreement = agent.create_agreement(
    signed_document_json,
    ['agent1-uuid', 'agent2-uuid'],           # required signers
    'Do you agree to these terms?',            # question
    'Q1 2024 service contract',                # context
    'jacsAgreement'                            # field name
)
```

Sign an Agreement

```
# Sign the agreement as the current agent
signed_agreement = agent.sign_agreement(
    document_with_agreement_json,
    'jacsAgreement'  # agreement field name
)
```

Check Agreement Status

```
# Check which agents have signed
status = agent.check_agreement(
    document_with_agreement_json,
    'jacsAgreement'
)

print('Agreement status:', json.loads(status))
```

Agent Operations

Verify Agent

```
# Verify the loaded agent's signature
is_valid = agent.verify_agent()
print('Agent valid:', is_valid)

# Verify a specific agent file
is_valid_other = agent.verify_agent('./other-agent.json')
```

Update Agent

```
# Update agent document
updated_agent_json = agent.update_agent(json.dumps({
    "jacsId": "agent-uuid",
    "jacsVersion": "version-uuid",
    "name": "Updated Agent Name",
    "description": "Updated description"
}))
```

Sign External Agent

```
# Sign another agent's document with registration signature
signed_agent_json = agent.sign_agent(
    external_agent_json,
    public_key_bytes,
    'ring-Ed25519'
)
```

Request/Response Signing

Sign a Request

```
# Sign request parameters as a JACS document
signed_request = agent.sign_request({
    "method": "GET",
    "path": "/api/resource",
    "timestamp": datetime.now().isoformat(),
    "body": {"query": "data"}
})
```

Verify a Response

```
# Verify a signed response
result = agent.verify_response(signed_response_json)
print('Response valid:', result)

# Verify and get signer's agent ID
result_with_id = agent.verify_response_with_agent_id(signed_response_json)
print('Signer ID:', result_with_id)
```

Utility Functions

Hash String

```
import jacs

# SHA-256 hash of a string
hash_value = jacs.hash_string('data to hash')
print('Hash:', hash_value)
```

Create Configuration

```
import jacs

# Programmatically create a config JSON string
config_json = jacs.create_config(
    jacs_data_directory='./jacs_data',
    jacs_key_directory='./jacs_keys',
    jacs_agent_key_algorithm='ring-Ed25519',
    jacs_default_storage='fs'
)

print('Config:', config_json)
```

Error Handling

```python
import jacs

agent = jacs.JacsAgent()

try:
    agent.load('./jacs.config.json')
except Exception as error:
    print(f'Failed to load agent: {error}')

try:
    doc = agent.create_document(json.dumps({'data': 'test'}))
    print('Document created')
except Exception as error:
    print(f'Failed to create document: {error}')

try:
    is_valid = agent.verify_document(invalid_json)
except Exception as error:
    print(f'Verification failed: {error}')
```

Complete Example

```python
import jacs
import json

def main():
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Create a task document
    task = {
        "title": "Code Review",
        "description": "Review pull request #123",
        "assignee": "developer-uuid",
        "deadline": "2024-02-01"
    }

    signed_task = agent.create_document(json.dumps(task))
    print('Task created')

    # Verify the task
    if agent.verify_document(signed_task):
        print('Task signature valid')

    # Create agreement for task acceptance
    task_with_agreement = agent.create_agreement(
        signed_task,
        ['manager-uuid', 'developer-uuid'],
        'Do you accept this task assignment?'
    )

    # Sign the agreement
    signed_agreement = agent.sign_agreement(task_with_agreement)
    print('Agreement signed')

    # Check agreement status
    status = agent.check_agreement(signed_agreement)
    print('Status:', status)

    # Hash some data for reference
    task_hash = jacs.hash_string(signed_task)
    print('Task hash:', task_hash)

if __name__ == "__main__":
    main()
```

Working with Document Data

Parse Signed Documents

```
import json

# Create and sign a document
doc_data = {"title": "My Document", "content": "Hello, World!"}
signed_doc = agent.create_document(json.dumps(doc_data))

# Parse the signed document to access JACS fields
parsed = json.loads(signed_doc)
print('Document ID:', parsed.get('jacsId'))
print('Document Version:', parsed.get('jacsVersion'))
print('Signature:', parsed.get('jacsSignature'))
```

Document Key Format

```
# Document keys combine ID and version
doc_id = parsed['jacsId']
doc_version = parsed['jacsVersion']
document_key = f"{doc_id}:{doc_version}"

# Use the key for updates
updated_doc = agent.update_document(document_key, json.dumps({
    **parsed,
    "content": "Updated content"
}))
```

Configuration Management

Load from File

```
import jacs

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')
```

Environment Variables

JACS reads environment variables that override configuration file settings:

```bash
export JACS_DATA_DIRECTORY="./production_data"
export JACS_KEY_DIRECTORY="./production_keys"
export JACS_AGENT_KEY_ALGORITHM="ring-Ed25519"
export JACS_DEFAULT_STORAGE="fs"
```

Programmatic Configuration

```python
import jacs
import json
import os

# Create config programmatically
config_json = jacs.create_config(
    jacs_data_directory='./jacs_data',
    jacs_key_directory='./jacs_keys',
    jacs_agent_key_algorithm='ring-Ed25519',
    jacs_default_storage='fs'
)

# Write to file
with open('jacs.config.json', 'w') as f:
    f.write(config_json)

# Then load it
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')
```

Next Steps

- MCP Integration - Model Context Protocol support
- FastMCP Integration - Build advanced MCP servers
- API Reference - Complete API documentation

MCP Integration (Python)

Python exposes two different MCP stories:

1. **Secure a local FastMCP transport** with `jacs.mcp`
2. **Expose JACS operations as MCP tools** with `jacs.adapters.mcp`

Use the first when you already have an MCP server or client. Use the second when you want the model to call JACS signing, agreement, A2A, or trust helpers as normal MCP tools.

What Is Supported

- Local FastMCP server wrapping with `JACSMCPServer`
- Local FastMCP client wrapping with `JACSMCPClient`
- One-line server creation with `create_jacs_mcp_server()`
- FastMCP tool registration with `register_jacs_tools()`, `register_a2a_tools()`, and `register_trust_tools()`

Important Constraints

- `JACSMCPClient`, `JACSMCPServer`, and `jacs_call()` enforce **loopback-only** URLs
- Unsigned fallback is **disabled by default**
- `strict=True` is about config loading and failure behavior, not an opt-in to security

1. Secure A FastMCP Server

The shortest path is the factory:

```python
from jacs.mcp import create_jacs_mcp_server

mcp = create_jacs_mcp_server("My Server", "./jacs.config.json")

@mcp.tool()
def hello(name: str) -> str:
    return f"Hello, {name}!"
```

If you already have a `FastMCP` instance:

```python
from fastmcp import FastMCP
from jacs.mcp import JACSMCPServer

mcp = JACSMCPServer(FastMCP("Secure Server"), "./jacs.config.json")
```

2. Secure A FastMCP Client

```python
from jacs.mcp import JACSMCPClient

client = JACSMCPClient("http://localhost:8000/sse", "./jacs.config.json")

async with client:
    result = await client.call_tool("hello", {"name": "World"})
```

To allow unsigned fallback explicitly:

```python
client = JACSMCPClient(
    "http://localhost:8000/sse",
    "./jacs.config.json",
    allow_unsigned_fallback=True,
)
```

3. Register JACS As MCP Tools

This is the better fit when the model should be able to ask for signatures, agreements, A2A cards, or trust-store operations directly.

```python
from fastmcp import FastMCP
from jacs.client import JacsClient
from jacs.adapters.mcp import (
    register_jacs_tools,
    register_a2a_tools,
    register_trust_tools,
)

client = JacsClient.quickstart(name="mcp-agent", domain="mcp.local")
mcp = FastMCP("JACS Tools")

register_jacs_tools(mcp, client=client)
register_a2a_tools(mcp, client=client)
register_trust_tools(mcp, client=client)
```

The core tool set includes document signing, verification, agreements, audit, and agent-info helpers. The A2A and trust helpers are opt-in registrations.

Useful Helper APIs

From `jacs.mcp`:

- `jacs_tool` to sign a specific tool's response
- `jacs_middleware()` for explicit Starlette middleware
- `jacs_call()` for one-off authenticated local MCP calls

Example Paths In This Repo

- `jacspy/examples/mcp/server.py`
- `jacspy/examples/mcp/client.py`
- `jacspy/examples/mcp_server.py`
- `jacspy/tests/test_adapters_mcp.py`

When To Use Adapters Instead

Choose Python Framework Adapters instead of MCP when:

- the model and tools already live in the same Python process
- you only need signed LangChain, LangGraph, CrewAI, or FastAPI boundaries
- you do not need MCP clients to connect from outside the app

Framework Adapters

Use adapters when the model already runs inside your Python app and you want provenance at the framework boundary, not a separate MCP server.

Choose The Adapter

If you need...	API	Start here
Signed LangChain tool results	`jacs_signing_middleware`, `signed_tool`	LangChain / LangGraph section below
Signed LangGraph `ToolNode` outputs	`jacs_wrap_tool_call`, `with_jacs_signing`	LangChain / LangGraph section below
Signed FastAPI responses and verified inbound requests	`JacsMiddleware`, `jacs_route`	FastAPI section below
Signed CrewAI task output	`jacs_guardrail`, `signed_task`	CrewAI section below
Signed Anthropic tool return values	`jacs.adapters.anthropic.signed_tool`	Anthropic section below

Install only the extra you need:

```
pip install jacs[langchain]
pip install jacs[fastapi]
pip install jacs[crewai]
pip install jacs[anthropic]
```

Optional: `jacs[langgraph]` (LangGraph ToolNode), `jacs[ws]` (WebSockets). See `pyproject.toml` for the full list.

LangChain / LangGraph

This is the smallest JACS path if your model already lives in LangChain.

LangChain middleware

```
from langchain.agents import create_agent
from jacs.client import JacsClient
from jacs.adapters.langchain import jacs_signing_middleware

client = JacsClient.quickstart(name="langchain-agent", domain="langchain.local")

agent = create_agent(
    model="openai:gpt-4o",
    tools=[search_tool, calc_tool],
    middleware=[jacs_signing_middleware(client=client)],
)
```

LangGraph `ToolNode`

```
from jacs.adapters.langchain import with_jacs_signing

tool_node = with_jacs_signing([search_tool, calc_tool], client=client)
```

Wrap one tool instead of the whole graph

```
from jacs.adapters.langchain import signed_tool

signed_search = signed_tool(search_tool, client=client)
```

The executable example to start from in this repo is `jacspy/examples/langchain/signing_callback.py`.

FastAPI / Starlette

Use this when the trust boundary is an API route instead of an MCP transport.

```
from fastapi import FastAPI
from jacs.client import JacsClient
from jacs.adapters.fastapi import JacsMiddleware

client = JacsClient.quickstart(name="api-agent", domain="api.local")
app = FastAPI()
app.add_middleware(JacsMiddleware, client=client)
```

Useful options:

- `strict=True` to reject verification failures instead of passing through
- `sign_responses=False` or `verify_requests=False` to narrow the behavior

- `a2a=True` to also expose A2A discovery routes from the same FastAPI app

For auth-style endpoints, replay protection is available:

```
app.add_middleware(
    JacsMiddleware,
    client=client,
    strict=True,
    auth_replay_protection=True,
    auth_max_age_seconds=30,
    auth_clock_skew_seconds=5,
)
```

To sign only one route:

```
from jacs.adapters.fastapi import jacs_route

@app.get("/signed")
@jacs_route(client=client)
async def signed_endpoint():
    return {"ok": True}
```

CrewAI

CrewAI support is guardrail-first:

```
from crewai import Task
from jacs.adapters.crewai import jacs_guardrail

task = Task(
    description="Summarize the report",
    agent=my_agent,
    guardrail=jacs_guardrail(client=client),
)
```

If you build tasks with factories, `signed_task()` can pre-attach the guardrail.

Anthropic / Claude SDK

Use the Anthropic adapter when you want signed return values from normal Python tool functions:

```
from jacs.adapters.anthropic import signed_tool

@signed_tool(client=client)
def get_weather(location: str) -> str:
    return f"Weather in {location}: sunny"
```

When To Use MCP Instead

Choose Python MCP Integration instead of adapters when:

- the model is outside your process and talks over MCP
- you want an MCP tool suite for JACS operations
- you need the same server to be usable by external MCP clients

API Reference

Complete API documentation for the `jacs` Python package.

For most use cases, the **Simplified API** (`jacs.simple`) and **JacsClient** (instance-based, multiple agents) are recommended. This page documents the lower-level `JacsAgent` class and module-level functions.

Installation

```
pip install jacs
```

Core Module

```
import jacs
from jacs import JacsAgent
```

JacsAgent Class

The `JacsAgent` class is the primary interface for JACS operations. Each instance maintains its own state and can be used independently, allowing multiple agents in the same process.

Constructor

```
JacsAgent()
```

Creates a new empty JacsAgent instance. Call `load()` to initialize with a configuration.

Example:

```
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')
```

---

agent.load(config_path)

Load and initialize the agent from a configuration file.

Parameters:

- `config_path` (str): Path to the JACS configuration file

Returns: str - The loaded agent's JSON

Example:

```
agent = jacs.JacsAgent()
agent_json = agent.load('./jacs.config.json')
print('Agent loaded:', json.loads(agent_json)['jacsId'])
```

---

agent.create_document(document_string, custom_schema=None, output_filename=None, no_save=False, attachments=None, embed=False)

Create and sign a new JACS document.

Parameters:

- `document_string` (str): JSON string of the document content
- `custom_schema` (str, optional): Path to a custom JSON Schema for validation
- `output_filename` (str, optional): Filename to save the document
- `no_save` (bool, optional): If True, don't save to storage (default: False)
- `attachments` (str, optional): Path to file attachments
- `embed` (bool, optional): If True, embed attachments in the document

Returns: str - The signed document as a JSON string

Example:

```python
# Basic document creation
doc = agent.create_document(json.dumps({
    'title': 'My Document',
    'content': 'Hello, World!'
}))

# With custom schema
validated_doc = agent.create_document(
    json.dumps({'title': 'Validated', 'amount': 100}),
    custom_schema='./schemas/invoice.schema.json'
)

# Without saving
temp_doc = agent.create_document(
    json.dumps({'data': 'temporary'}),
    no_save=True
)

# With attachments
doc_with_file = agent.create_document(
    json.dumps({'report': 'Monthly Report'}),
    attachments='./report.pdf',
    embed=True
)
```

---

agent.verify_document(document_string)

Verify a document's signature and hash integrity.

Parameters:

- `document_string` (str): The signed document JSON string

Returns: bool - True if the document is valid

Example:

```python
is_valid = agent.verify_document(signed_document_json)
if is_valid:
    print('Document signature verified')
else:
    print('Document verification failed')
```

---

agent.verify_signature(document_string, signature_field=None)

Verify a document's signature with an optional custom signature field.

Parameters:

- `document_string` (str): The signed document JSON string
- `signature_field` (str, optional): Name of the signature field (default: 'jacsSignature')

Returns: bool - True if the signature is valid

Example:

```
# Verify default signature field
is_valid = agent.verify_signature(doc_json)

# Verify custom signature field
is_valid_custom = agent.verify_signature(doc_json, 'customSignature')
```

---

agent.update_document(document_key, new_document_string, attachments=None, embed=False)

Update an existing document, creating a new version.

Parameters:

- `document_key` (str): The document key in format `"id:version"`
- `new_document_string` (str): The modified document as JSON string
- `attachments` (list, optional): List of attachment file paths
- `embed` (bool, optional): If True, embed attachments

Returns: str - The updated document as a JSON string

Example:

```
# Parse existing document to get key
doc = json.loads(signed_doc)
document_key = f"{doc['jacsId']}:{doc['jacsVersion']}"

# Update the document
updated_doc = agent.update_document(
    document_key,
    json.dumps({
        **doc,
        'title': 'Updated Title',
        'content': 'Modified content'
    })
)
```

---

agent.create_agreement(document_string, agent_ids, question=None, context=None, agreement_field_name=None)

Add an agreement requiring multiple agent signatures to a document.

Parameters:

- `document_string` (str): The document JSON string
- `agent_ids` (list): List of agent IDs required to sign
- `question` (str, optional): The agreement question
- `context` (str, optional): Additional context for the agreement
- `agreement_field_name` (str, optional): Field name for the agreement (default: 'jacsAgreement')

Returns: str - The document with agreement as a JSON string

Example:

```
doc_with_agreement = agent.create_agreement(
    signed_document_json,
    ['agent-1-uuid', 'agent-2-uuid', 'agent-3-uuid'],
    question='Do you agree to these terms?',
    context='Q1 2024 Service Agreement',
    agreement_field_name='jacsAgreement'
)
```

---

agent.sign_agreement(document_string, agreement_field_name=None)

Sign an agreement as the current agent.

Parameters:

- `document_string` (str): The document with agreement JSON string
- `agreement_field_name` (str, optional): Field name of the agreement (default: 'jacsAgreement')

Returns: str - The document with this agent's signature added

Example:

```
signed_agreement = agent.sign_agreement(
    doc_with_agreement_json,
    'jacsAgreement'
)
```

---

agent.check_agreement(document_string, agreement_field_name=None)

Check the status of an agreement (which agents have signed).

Parameters:

- `document_string` (str): The document with agreement JSON string
- `agreement_field_name` (str, optional): Field name of the agreement (default: 'jacsAgreement')

Returns: str - JSON string with agreement status

Example:

```
status_json = agent.check_agreement(signed_agreement_json)
status = json.loads(status_json)

print('Required signers:', status['required'])
print('Signatures received:', status['signed'])
print('Complete:', status['complete'])
```

---

agent.sign_artifact(artifact_json, artifact_type, parent_signatures_json=None)

Sign an A2A artifact with JACS provenance. This is the canonical method name.

Parameters:

- `artifact_json` (str): JSON string of the artifact to sign
- `artifact_type` (str): Type of artifact (e.g., `"task"`, `"message"`)
- `parent_signatures_json` (str, optional): JSON string of parent signatures for chain of custody

Returns: str - The signed, wrapped artifact as a JSON string

Example:

```
signed = agent.sign_artifact(
    json.dumps({"action": "classify", "input": "hello"}),
    "task"
)
```

---

agent.wrap_a2a_artifact(artifact_json, artifact_type, parent_signatures_json=None)

> **Deprecated since 0.9.0.** Use `sign_artifact()` instead. This method will be removed in 1.0.0.

Set `JACS_SHOW_DEPRECATIONS=1` to see runtime warnings when deprecated methods are called.

Wraps an A2A artifact with JACS provenance signature. Identical behavior to `sign_artifact()`.

Parameters: Same as `sign_artifact()`.

---

agent.sign_string(data)

Sign arbitrary string data with the agent's private key.

Parameters:

- `data` (str): The data to sign

Returns: str - Base64-encoded signature

Example:

```
signature = agent.sign_string('Important message')
print('Signature:', signature)
```

---

agent.verify_string(data, signature_base64, public_key, public_key_enc_type)

Verify a signature on arbitrary string data.

Parameters:

- `data` (str): The original data
- `signature_base64` (str): The base64-encoded signature
- `public_key` (bytes): The public key as bytes
- `public_key_enc_type` (str): The key algorithm (e.g., 'ring-Ed25519')

Returns: bool - True if the signature is valid

Example:

```
is_valid = agent.verify_string(
    'Important message',
    signature_base64,
    public_key_bytes,
    'ring-Ed25519'
)
```

---

agent.sign_request(params)

Sign a request payload, wrapping it in a JACS document.

Parameters:

- `params` (any): The request payload (will be JSON serialized)

Returns: str - JACS-signed request as a JSON string

Example:

```
signed_request = agent.sign_request({
    'method': 'GET',
    'path': '/api/data',
    'timestamp': datetime.now().isoformat(),
    'body': {'query': 'value'}
})
```

---

agent.verify_response(document_string)

Verify a JACS-signed response and extract the payload.

Parameters:

- `document_string` (str): The JACS-signed response

Returns: dict - Dictionary containing the verified payload

Example:

```
result = agent.verify_response(jacs_response_string)
payload = result.get('payload')
print('Verified payload:', payload)
```

---

agent.verify_response_with_agent_id(document_string)

Verify a response and return both the payload and signer's agent ID.

Parameters:

- `document_string` (str): The JACS-signed response

Returns: dict - Dictionary with payload and agent ID

Example:

```
result = agent.verify_response_with_agent_id(jacs_response_string)
print('Payload:', result['payload'])
print('Signed by agent:', result['agentId'])
```

---

agent.verify_agent(agent_file=None)

Verify the agent's own signature and hash, or verify another agent file.

Parameters:

- `agent_file` (str, optional): Path to an agent file to verify

Returns: bool - True if the agent is valid

Example:

```
# Verify the loaded agent
is_valid = agent.verify_agent()

# Verify another agent file
is_other_valid = agent.verify_agent('./other-agent.json')
```

---

agent.update_agent(new_agent_string)

Update the agent document with new data.

Parameters:

- `new_agent_string` (str): The modified agent document as JSON string

Returns: str - The updated agent document

Example:

```
current_agent = json.loads(agent.load('./jacs.config.json'))
updated_agent = agent.update_agent(json.dumps({
    **current_agent,
    'description': 'Updated description'
}))
```

---

agent.sign_agent(agent_string, public_key, public_key_enc_type)

Sign another agent's document with a registration signature.

Parameters:

- `agent_string` (str): The agent document to sign
- `public_key` (bytes): The public key as bytes
- `public_key_enc_type` (str): The key algorithm

Returns: str - The signed agent document

Example:

```
signed_agent = agent.sign_agent(
    external_agent_json,
    public_key_bytes,
    'ring-Ed25519'
)
```

Module-Level Functions

These functions operate on a global agent singleton and are maintained for backwards compatibility. **New code should use the `JacsAgent` class instead.**

jacs.load(config_path)

Load the global agent from a configuration file.

```
import jacs
jacs.load('./jacs.config.json')
```

jacs.sign_request(data)

Sign a request using the global agent.

```
signed = jacs.sign_request({'method': 'tools/call', 'params': {...}})
```

jacs.verify_request(data)

Verify an incoming request using the global agent.

```
payload = jacs.verify_request(incoming_request_string)
```

jacs.sign_response(data)

Sign a response using the global agent.

```
signed = jacs.sign_response({'result': 'success'})
```

jacs.verify_response(data)

Verify an incoming response using the global agent.

```
result = jacs.verify_response(response_string)
payload = result.get('payload')
```

MCP Module

```
from jacs.mcp import JACSMCPServer, JACSMCPClient, create_jacs_mcp_server, jacs_call
```

Canonical MCP documentation lives at Python MCP Integration. This API section lists the MCP entry points only:

- `JACSMCPServer(mcp_server, config_path="./jacs.config.json", strict=False)` - Wrap a FastMCP server with JACS request verification and response signing.
- `JACSMCPClient(url, config_path="./jacs.config.json", strict=False, **kwargs)` - Create a FastMCP client with JACS signing/verification interceptors.
- `create_jacs_mcp_server(name, config_path=None)` - One-line server factory.
- `jacs_call(server_url, method, **params)` - One-shot authenticated MCP call.

For examples, strict-mode behavior, and security guidance, see Python MCP Integration.

Configuration

Configuration File Format

Create a `jacs.config.json` file:

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_agent_id_and_version": "your-agent-id:version",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "jacs_agent_private_key_filename": "private.pem",
  "jacs_agent_public_key_filename": "public.pem",
  "jacs_data_directory": "./jacs_data",
  "jacs_default_storage": "fs",
  "jacs_key_directory": "./jacs_keys"
}
```

Configuration Options

Field	Type	Description
`jacs_agent_id_and_version`	string	Agent ID and version in format `"id:version"`
`jacs_agent_key_algorithm`	string	Signing algorithm: `"ring-Ed25519"`, `"RSA-PSS"`, `"pq-dilithium"`, `"pq2025"`
`jacs_agent_private_key_filename`	string	Private key filename
`jacs_agent_public_key_filename`	string	Public key filename
`jacs_data_directory`	string	Directory for data storage
`jacs_key_directory`	string	Directory for key storage
`jacs_default_storage`	string	Storage backend: `"fs"`, `"s3"`, `"memory"`

Error Handling

All methods may raise exceptions. Use try/except for error handling:

```python
try:
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')
    doc = agent.create_document(json.dumps({'data': 'test'}))
except FileNotFoundError as e:
    print(f'Configuration file not found: {e}')
except ValueError as e:
    print(f'Invalid configuration: {e}')
except Exception as e:
    print(f'JACS error: {e}')
```

Common Exceptions

Exception	Description
`FileNotFoundError`	Configuration file or key file not found
`ValueError`	Invalid configuration or document format
`RuntimeError`	Agent not loaded or cryptographic operation failed

Type Hints

The package supports type hints for better IDE integration:

```
from jacs import JacsAgent
import json

def process_document(agent: JacsAgent, data: dict) -> str:
    """Create and return a signed document."""
    doc_string = json.dumps(data)
    return agent.create_document(doc_string)

def verify_and_extract(agent: JacsAgent, doc: str) -> dict:
    """Verify document and extract content."""
    if agent.verify_document(doc):
        return json.loads(doc)
    raise ValueError("Document verification failed")
```

Thread Safety

`JacsAgent` instances use internal locking and are thread-safe. You can safely use the same agent instance across multiple threads:

```python
import threading
from jacs import JacsAgent

agent = JacsAgent()
agent.load('./jacs.config.json')

def worker(data):
    # Safe to call from multiple threads
    doc = agent.create_document(json.dumps(data))
    return doc

threads = [
    threading.Thread(target=worker, args=({'id': i},))
    for i in range(10)
]
for t in threads:
    t.start()
for t in threads:
    t.join()
```

See Also

- Installation - Getting started
- Basic Usage - Common usage patterns
- MCP Integration - Model Context Protocol
- Examples - More complex examples

Go (`jacsgo`) Installation and Quick Start

`jacsgo` provides Go bindings for signing and verifying JACS documents in services, APIs, and agent runtimes.

> Note: Go bindings are community-maintained. Python and Node.js currently have broader framework adapter coverage. For full MCP surface use the Rust `jacs-mcp` server; the Go MCP examples in the repo are demo code.

Install

```
go get github.com/HumanAssisted/JACS/jacsgo
```

Minimal Sign + Verify

Create an agent first (CLI: `jacs create --name my-agent`, or programmatically with `jacs.Create()` and `JACS_PRIVATE_KEY_PASSWORD`). Then:

```go
package main

import (
    "fmt"
    "log"

    jacs "github.com/HumanAssisted/JACS/jacsgo"
)

func main() {
    // Load agent: nil = default ./jacs.config.json
    if err := jacs.Load(nil); err != nil {
        log.Fatal("create an agent first: jacs create --name my-agent")
    }

    signed, err := jacs.SignMessage(map[string]interface{}{
        "event":  "tool-result",
        "status": "ok",
    })
    if err != nil {
        log.Fatal(err)
    }

    result, err := jacs.Verify(signed.Raw)
    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf("Valid: %t signer=%s\n", result.Valid, result.SignerID)
}
```

Programmatic agent creation

Use `jacs.Create(name, &jacs.CreateAgentOptions{...})` . Password must be set in options or via `JACS_PRIVATE_KEY_PASSWORD` . See the jacsgo README for the full API table and options.

Concurrent use

For multiple agents in one process, use `NewJacsAgent()` , then `agent.Load(path)` and agent methods; call `agent.Close()` when done. Attestation, A2A (agent cards, trust policy), and protocol helpers are available on `JacsAgent` and as package-level wrappers (see godoc or the jacsgo README).

Common Go Use Cases

- Sign outbound API/MCP payloads before crossing trust boundaries
- Verify inbound signed payloads before executing sensitive actions
- Sign files (`SignFile`) for portable chain-of-custody workflows
- Generate DNS TXT fingerprints (`GetDnsRecord`) for public identity verification

MCP and HTTP Patterns

The Go repository includes runnable examples for transport-level signing:

- `jacsgo/examples/mcp/main.go` for MCP-style request/response signing
- `jacsgo/examples/http/` for signed HTTP client/server traffic

Identity and Trust Notes

- JACS agent identity is key-based (`jacsId` + versioned signatures)
- Verification behavior follows the configured key-resolution order in the runtime (for example local and remote resolution modes supported by the underlying JACS core)
- DID interoperability is possible at the integration layer without requiring blockchain infrastructure

See DNS-Based Verification and DID Integration (No Blockchain Required).

JSON Schemas

JACS uses JSON Schema (Draft-07) to define the structure and validation rules for all documents in the system. These schemas ensure consistency, enable validation, and provide a contract for interoperability between agents.

Schema Architecture

JACS schemas follow a hierarchical composition pattern:

```
┌─────────────────────────────────────────────────────────┐
│                    Document Schemas                     │
│  (agent.schema.json, task.schema.json, message.schema.json)  │
└─────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│                    Header Schema                        │
│              (header.schema.json)                       │
│  Base fields: jacsId, jacsVersion, jacsSignature, etc.  │
└─────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│                  Component Schemas                      │
│   signature.schema.json, agreement.schema.json,         │
│   files.schema.json, embedding.schema.json, etc.        │
└─────────────────────────────────────────────────────────┘
```

Schema Categories

Configuration Schema

Schema	Purpose
`jacs.config.schema.json`	Agent configuration file format

Document Schemas

Schema	Purpose
`header/v1/header.schema.json`	Base fields for all JACS documents

Schema	Purpose
`agent/v1/agent.schema.json`	Agent identity and capabilities
`task/v1/task.schema.json`	Task workflow and state management
`message/v1/message.schema.json`	Inter-agent messages
`node/v1/node.schema.json`	Graph node representation
`program/v1/program.schema.json`	Executable program definitions
`eval/v1/eval.schema.json`	Evaluation and assessment records
`agentstate/v1/agentstate.schema.json`	Signed agent state files (memory, skills, plans, configs, hooks, other)
`commitment/v1/commitment.schema.json`	Shared, signed agreements between agents
`todo/v1/todo.schema.json`	Private, signed todo lists with inline items

Component Schemas

Schema	Purpose
`signature/v1/signature.schema.json`	Cryptographic signatures
`agreement/v1/agreement.schema.json`	Multi-party agreements
`files/v1/files.schema.json`	File attachments
`embedding/v1/embedding.schema.json`	Vector embeddings
`contact/v1/contact.schema.json`	Contact information
`service/v1/service.schema.json`	Service definitions
`tool/v1/tool.schema.json`	Tool capabilities
`action/v1/action.schema.json`	Action definitions
`unit/v1/unit.schema.json`	Unit of measurement
`todoitem/v1/todoitem.schema.json`	Inline todo item (goal or task)

Schema Locations

Schemas are available at:

- **HTTPS URLs**: `https://hai.ai/schemas/...`
- **Local files**: `jacs/schemas/...`

Example schema URLs:

```
https://hai.ai/schemas/jacs.config.schema.json
https://hai.ai/schemas/header/v1/header.schema.json
https://hai.ai/schemas/agent/v1/agent.schema.json
https://hai.ai/schemas/components/signature/v1/signature.schema.json
```

Using Schemas

In Documents

Every JACS document must include a `$schema` field:

```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsType": "agent",
  ...
}
```

In Configuration Files

Reference the config schema for IDE support:

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  ...
}
```

Custom Schema Validation

Validate documents against custom schemas:

```python
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create document with custom schema
doc = agent.create_document(
    json.dumps({'invoice_id': 'INV-001', 'amount': 100.00}),
    custom_schema='./schemas/invoice.schema.json'
)
```

```python
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
agent.load('./jacs.config.json');

// Create document with custom schema
const doc = agent.createDocument(
  JSON.stringify({ invoice_id: 'INV-001', amount: 100.00 }),
  './schemas/invoice.schema.json'
);
```

HAI Extensions

JACS schemas include a custom `hai` property that categorizes fields:

Value	Description
`meta`	Metadata fields (IDs, dates, versions)
`base`	Core cryptographic fields (hashes, signatures)
`agent`	Agent-controlled content fields

This categorization helps determine which fields should be included in hash calculations and signature operations.

Versioning

Schemas are versioned using directory paths:

```
schemas/
├── header/
│   └── v1/
│       └── header.schema.json
├── agent/
│   └── v1/
│       └── agent.schema.json
└── components/
    └── signature/
        └── v1/
            └── signature.schema.json
```

Configuration options allow specifying schema versions:

```
{
  "jacs_agent_schema_version": "v1",
  "jacs_header_schema_version": "v1",
  "jacs_signature_schema_version": "v1"
}
```

Schema Composition

Document schemas use JSON Schema's `allOf` to compose the header with type-specific fields:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "jacsAgentType": { ... },
        "jacsServices": { ... }
      }
    }
  ]
}
```

This ensures all documents share common header fields while allowing type-specific extensions.

Creating Custom Schemas

Create custom schemas that extend JACS schemas:

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://example.com/schemas/invoice.schema.json",
  "title": "Invoice",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "invoiceNumber": {
          "type": "string",
          "description": "Unique invoice identifier"
        },
        "amount": {
          "type": "number",
          "minimum": 0,
          "description": "Invoice amount"
        },
        "currency": {
          "type": "string",
          "enum": ["USD", "EUR", "GBP"],
          "default": "USD"
        },
        "lineItems": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "description": { "type": "string" },
              "quantity": { "type": "integer", "minimum": 1 },
              "unitPrice": { "type": "number", "minimum": 0 }
            },
            "required": ["description", "quantity", "unitPrice"]
          }
        }
      },
      "required": ["invoiceNumber", "amount"]
    }
  ]
}
```

Validation Rules

Required Fields

All JACS documents require these header fields:

- `jacsId` - Unique document identifier (UUID v4)
- `jacsType` - Document type identifier
- `jacsVersion` - Version identifier (UUID v4)

- `jacsVersionDate` - Version timestamp (ISO 8601)
- `jacsOriginalVersion` - First version identifier
- `jacsOriginalDate` - Creation timestamp
- `jacsLevel` - Document level (raw, config, artifact, derived)
- `$schema` - Schema reference URL

Format Validation

Fields use JSON Schema format keywords:

- `uuid` - UUID v4 format
- `date-time` - ISO 8601 date-time format
- `uri` - Valid URI format

Enum Constraints

Many fields have enumerated valid values:

```
{
  "jacsLevel": {
    "enum": ["raw", "config", "artifact", "derived"]
  },
  "jacsAgentType": {
    "enum": ["human", "human-org", "hybrid", "ai"]
  },
  "jacs_agent_key_algorithm": {
    "enum": ["RSA-PSS", "ring-Ed25519", "pq-dilithium", "pq2025"]
  }
}
```

Schema Reference

For detailed documentation on specific schemas:

- Agent Schema - Agent identity and capabilities
- Document Schema - Document header and structure
- Task Schema - Task workflow management
- Agent State Schema - Signed agent state documents
- Commitment Schema - Shared agreements between agents
- Todo List Schema - Private task tracking with inline items
- Conversation Schema - Message threading and ordering
- Configuration - Configuration file format

See Also

- Custom Schemas - Creating custom document types
- Security Model - How schemas relate to security
- API Reference - Using schemas in code

Agent Schema

The Agent Schema defines the structure for agent identity documents in JACS. Agents represent entities that can sign documents, participate in agreements, and provide services.

Schema Location

```
https://hai.ai/schemas/agent/v1/agent.schema.json
```

Overview

Agent documents describe:

- **Identity**: Unique identifiers and versioning
- **Type**: Human, organizational, hybrid, or AI classification
- **Services**: Capabilities the agent offers
- **Contacts**: How to reach human or hybrid agents
- **Domain**: Optional DNS-based verification

Schema Structure

The agent schema extends the Header Schema using JSON Schema composition:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "title": "Agent",
  "description": "General schema for human, hybrid, and AI agents",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    { "type": "object", "properties": { ... } }
  ]
}
```

Agent Types

The `jacsAgentType` field classifies the agent:

Type	Description
`human`	A biological entity (individual person)
`human-org`	A group of people (organization, company)
`hybrid`	Combination of human and AI components
`ai`	Fully artificial intelligence

```
{
  "jacsAgentType": {
    "type": "string",
    "enum": ["human", "human-org", "hybrid", "ai"]
  }
}
```

Contact Requirements

Human and hybrid agents must provide contact information:

```
{
  "if": {
    "properties": {
      "jacsAgentType": { "enum": ["human", "human-org", "hybrid"] }
    }
  },
  "then": {
    "required": ["jacsContacts"]
  }
}
```

Agent Properties

Core Fields (from Header)

All agents inherit these header fields:

Field	Type	Required	Description
`jacsId`	string (UUID)	Yes	Unique agent identifier
`jacsVersion`	string (UUID)	Yes	Current version identifier
`jacsVersionDate`	string (date-time)	Yes	Version timestamp
`jacsType`	string	Yes	Set to "agent"
`jacsOriginalVersion`	string (UUID)	Yes	First version identifier
`jacsOriginalDate`	string (date-time)	Yes	Creation timestamp

Field	Type	Required	Description
`jacsLevel`	string	Yes	Document level
`jacsSignature`	object	No	Cryptographic signature
`jacsSha256`	string	No	Content hash

Agent-Specific Fields

Field	Type	Required	Description
`jacsAgentType`	string	Yes	Agent classification
`jacsAgentDomain`	string	No	Domain for DNS verification
`jacsServices`	array	Yes	Services the agent provides
`jacsContacts`	array	Conditional	Contact information (required for human/hybrid)

Services

Services describe capabilities the agent offers:

```
{
  "jacsServices": [{
    "name": "Document Signing Service",
    "serviceDescription": "Sign and verify JACS documents",
    "successDescription": "Documents are signed with valid signatures",
    "failureDescription": "Invalid documents or signing errors",
    "costDescription": "Free for basic usage, paid tiers available",
    "idealCustomerDescription": "Developers building secure agent systems",
    "termsOfService": "https://example.com/tos",
    "privacyPolicy": "https://example.com/privacy",
    "isDev": false,
    "tools": [...]
  }]
}
```

Service Schema Fields

Field	Type	Required	Description
`name`	string	No	Service name
`serviceDescription`	string	Yes	What the service does
`successDescription`	string	Yes	What success looks like
`failureDescription`	string	Yes	What failure looks like

Field	Type	Required	Description
`costDescription`	string	No	Pricing information
`idealCustomerDescription`	string	No	Target customer profile
`termsOfService`	string	No	Legal terms URL or text
`privacyPolicy`	string	No	Privacy policy URL or text
`copyright`	string	No	Usage rights for provided data
`eula`	string	No	End-user license agreement
`isDev`	boolean	No	Whether this is a dev/test service
`tools`	array	No	Tool definitions
`piiDesired`	array	No	Types of sensitive data needed

PII Types

Services can declare what personally identifiable information they need:

```
{
  "piiDesired": ["email", "phone", "address"]
}
```

Valid PII types:

- `signature` - Digital signatures
- `cryptoaddress` - Cryptocurrency addresses
- `creditcard` - Payment card numbers
- `govid` - Government identification
- `social` - Social security numbers
- `email` - Email addresses
- `phone` - Phone numbers
- `address` - Physical addresses
- `zip` - Postal codes
- `PHI` - Protected health information
- `MHI` - Mental health information
- `identity` - Identity documents
- `political` - Political affiliation
- `bankaddress` - Banking information
- `income` - Income data

Contacts

Contact information for human and hybrid agents:

```
{
  "jacsContacts": [{
    "firstName": "Jane",
    "lastName": "Smith",
    "email": "jane@example.com",
    "phone": "+1-555-0123",
    "isPrimary": true,
    "mailAddress": "123 Main St",
    "mailState": "CA",
    "mailZip": "94102",
    "mailCountry": "USA"
  }]
}
```

Contact Schema Fields

Field	Type	Description
`firstName`	string	First name
`lastName`	string	Last name
`addressName`	string	Location name
`phone`	string	Phone number
`email`	string (email)	Email address
`mailName`	string	Name at address
`mailAddress`	string	Street address
`mailAddressTwo`	string	Address line 2
`mailState`	string	State/province
`mailZip`	string	Postal code
`mailCountry`	string	Country
`isPrimary`	boolean	Primary contact flag

DNS Verification

Agents can link to a domain for DNSSEC-validated verification:

```
{
  "jacsAgentDomain": "example.com"
}
```

The domain should have a DNS TXT record at `_v1.agent.jacs.example.com.` containing the agent's public key fingerprint.

See the DNS chapter for complete setup instructions.

Complete Example

AI Agent

```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsType": "agent",
  "jacsOriginalVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsOriginalDate": "2024-01-15T10:30:00Z",
  "jacsLevel": "artifact",
  "jacsAgentType": "ai",
  "jacsServices": [{
    "name": "Code Review Service",
    "serviceDescription": "Automated code review and analysis",
    "successDescription": "Review completed with actionable feedback",
    "failureDescription": "Unable to process or analyze code",
    "isDev": false
  }],
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "agentVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "date": "2024-01-15T10:30:00Z",
    "signature": "base64-encoded-signature...",
    "publicKeyHash": "sha256-hash-of-public-key",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsId", "jacsVersion", "jacsAgentType", "jacsServices"]
  },
  "jacsSha256": "document-hash..."
}
```

Human Agent


```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsId": "660e8400-e29b-41d4-a716-446655440001",
  "jacsVersion": "a47ac10b-58cc-4372-a567-0e02b2c3d480",
  "jacsVersionDate": "2024-01-15T11:00:00Z",
  "jacsType": "agent",
  "jacsOriginalVersion": "a47ac10b-58cc-4372-a567-0e02b2c3d480",
  "jacsOriginalDate": "2024-01-15T11:00:00Z",
  "jacsLevel": "artifact",
  "jacsAgentType": "human",
  "jacsAgentDomain": "smith.example.com",
  "jacsServices": [{
    "name": "Consulting",
    "serviceDescription": "Technical consulting services",
    "successDescription": "Project goals achieved",
    "failureDescription": "Unable to meet requirements",
    "termsOfService": "https://smith.example.com/tos"
  }],
  "jacsContacts": [{
    "firstName": "John",
    "lastName": "Smith",
    "email": "john@smith.example.com",
    "isPrimary": true
  }]
}
```

Organization Agent

```
{
  "$schema": "https://hai.ai/schemas/agent/v1/agent.schema.json",
  "jacsId": "770e8400-e29b-41d4-a716-446655440002",
  "jacsVersion": "b47ac10b-58cc-4372-a567-0e02b2c3d481",
  "jacsVersionDate": "2024-01-15T12:00:00Z",
  "jacsType": "agent",
  "jacsOriginalVersion": "b47ac10b-58cc-4372-a567-0e02b2c3d481",
  "jacsOriginalDate": "2024-01-15T12:00:00Z",
  "jacsLevel": "artifact",
  "jacsAgentType": "human-org",
  "jacsAgentDomain": "acme.com",
  "jacsServices": [{
    "name": "Enterprise Software",
    "serviceDescription": "Enterprise software solutions",
    "successDescription": "Software deployed and operational",
    "failureDescription": "Deployment or integration failure",
    "privacyPolicy": "https://acme.com/privacy",
    "piiDesired": ["email", "phone"]
  }],
  "jacsContacts": [{
    "addressName": "Acme Corporation",
    "email": "contact@acme.com",
    "phone": "+1-800-555-ACME",
    "mailAddress": "1 Corporate Plaza",
    "mailState": "NY",
    "mailZip": "10001",
    "mailCountry": "USA",
    "isPrimary": true
  }]
}
```

Creating Agents

Python

```python
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# The agent is created during configuration setup
# Agent document is available after loading
agent_json = agent.load('./jacs.config.json')
agent_doc = json.loads(agent_json)

print(f"Agent ID: {agent_doc['jacsId']}")
print(f"Agent Type: {agent_doc['jacsAgentType']}")
```

Node.js

```
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
const agentJson = agent.load('./jacs.config.json');
const agentDoc = JSON.parse(agentJson);

console.log(`Agent ID: ${agentDoc.jacsId}`);
console.log(`Agent Type: ${agentDoc.jacsAgentType}`);
```

CLI

```
# Create a new agent
jacs agent create

# View agent details
jacs agent show
```

Verifying Agents

```
# Verify the loaded agent
is_valid = agent.verify_agent()

# Verify another agent file
is_valid = agent.verify_agent('./other-agent.json')
```

```
// Verify the loaded agent
const isValid = agent.verifyAgent();

// Verify another agent file
const isOtherValid = agent.verifyAgent('./other-agent.json');
```

See Also

- Document Schema - Header fields documentation
- Task Schema - Task workflow schema
- DNS Verification - Setting up domain verification
- Creating an Agent - Agent creation guide

Document Schema

The Document Schema (Header Schema) defines the base structure for all JACS documents. Every JACS document type (agents, tasks, messages, etc.) extends this schema.

Schema Location

```
https://hai.ai/schemas/header/v1/header.schema.json
```

Overview

The header schema provides:

- **Unique Identification**: Every document has a unique ID and version
- **Version Tracking**: Full history with previous version references
- **Cryptographic Integrity**: Signatures and hashes for verification
- **File Attachments**: Support for embedded or linked files
- **Vector Embeddings**: Pre-computed embeddings for semantic search
- **Agreements**: Multi-party signature support

Core Fields

Identification

Field	Type	Required	Description
`$schema`	string	Yes	Schema URL for validation
`jacsId`	string (UUID)	Yes	Unique document identifier
`jacsType`	string	Yes	Document type (agent, task, etc.)

```
{
  "$schema": "https://hai.ai/schemas/header/v1/header.schema.json",
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsType": "document"
}
```

Versioning

Field	Type	Required	Description
`jacsVersion`	string (UUID)	Yes	Current version identifier
`jacsVersionDate`	string (date-time)	Yes	Version creation timestamp
`jacsPreviousVersion`	string (UUID)	No	Previous version (if not first)
`jacsOriginalVersion`	string (UUID)	Yes	First version identifier
`jacsOriginalDate`	string (date-time)	Yes	Document creation timestamp
`jacsBranch`	string (UUID)	No	Branch identifier for JACS databases

```
{
  "jacsVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsPreviousVersion": "e36ac10b-58cc-4372-a567-0e02b2c3d478",
  "jacsOriginalVersion": "a47ac10b-58cc-4372-a567-0e02b2c3d476",
  "jacsOriginalDate": "2024-01-01T09:00:00Z"
}
```

Document Level

The `jacsLevel` field indicates the intended use:

Level	Description
`raw`	Raw data that should not change
`config`	Configuration meant to be updated
`artifact`	Generated content that may be updated
`derived`	Computed from other documents

```
{
  "jacsLevel": "artifact"
}
```

Cryptographic Fields

Signature

The `jacsSignature` field contains the creator's cryptographic signature:

```
{
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "agentVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "date": "2024-01-15T10:30:00Z",
    "signature": "base64-encoded-signature-string",
    "publicKeyHash": "sha256-hash-of-public-key",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsId", "jacsVersion", "jacsType", "content"]
  }
}
```

Signature Schema Fields

Field	Type	Required	Description
`agentID`	string (UUID)	Yes	Signing agent's ID
`agentVersion`	string (UUID)	Yes	Signing agent's version
`date`	string (date-time)	Yes	Signing timestamp
`signature`	string	Yes	Base64-encoded signature
`publicKeyHash`	string	Yes	Hash of public key used
`signingAlgorithm`	string	Yes	Algorithm used (`ring-Ed25519` , `RSA-PSS` , `pq2025` ; `pq-dilithium` is legacy/deprecated)
`fields`	array	Yes	Fields included in signature
`response`	string	No	Text response with signature
`responseType`	string	No	agree, disagree, or reject

Registration

The `jacsRegistration` field contains a signature from a registration authority:

```
{
  "jacsRegistration": {
    "agentID": "registrar-agent-id",
    "agentVersion": "registrar-version",
    "date": "2024-01-15T10:35:00Z",
    "signature": "registrar-signature",
    "publicKeyHash": "registrar-key-hash",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsId", "jacsSignature"]
  }
}
```

Hash

The `jacsSha256` field contains a SHA-256 hash of all document content (excluding this field):

```
{
  "jacsSha256": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
}
```

Agreements

Documents can include multi-party agreements using `jacsAgreement`:

```
{
  "jacsAgreement": {
    "agentIDs": [
      "agent-1-uuid",
      "agent-2-uuid",
      "agent-3-uuid"
    ],
    "question": "Do you agree to these terms?",
    "context": "Q1 2024 Service Agreement",
    "signatures": [
      {
        "agentID": "agent-1-uuid",
        "signature": "...",
        "responseType": "agree",
        "date": "2024-01-15T11:00:00Z"
      }
    ]
  },
  "jacsAgreementHash": "hash-of-content-at-agreement-time"
}
```

Agreement Schema Fields

Field	Type	Required	Description
`agentIDs`	array	Yes	Required signers
`question`	string	No	What parties are agreeing to
`context`	string	No	Additional context
`signatures`	array	No	Collected signatures

File Attachments

Documents can include file attachments using `jacsFiles`:

```
{
  "jacsFiles": [
    {
      "mimetype": "application/pdf",
      "path": "./documents/contract.pdf",
      "embed": true,
      "contents": "base64-encoded-file-contents",
      "sha256": "file-content-hash"
    },
    {
      "mimetype": "image/png",
      "path": "./images/diagram.png",
      "embed": false,
      "sha256": "file-content-hash"
    }
  ]
}
```

File Schema Fields

Field	Type	Required	Description
`mimetype`	string	Yes	MIME type of the file
`path`	string	Yes	File location (local path)
`embed`	boolean	Yes	Whether to embed contents
`contents`	string	No	Base64-encoded file contents
`sha256`	string	No	Hash for content verification

Vector Embeddings

Documents can include pre-computed embeddings for semantic search:

```
{
  "jacsEmbedding": [
    {
      "llm": "text-embedding-ada-002",
      "vector": [0.0023, -0.0089, 0.0156, ...]
    }
  ]
}
```

Embedding Schema Fields

Field	Type	Required	Description
`llm`	string	Yes	Model used for embedding
`vector`	array	Yes	Vector of numbers

Complete Example

```
{
  "$schema": "https://hai.ai/schemas/header/v1/header.schema.json",
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsType": "document",
  "jacsVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsOriginalVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsOriginalDate": "2024-01-15T10:30:00Z",
  "jacsLevel": "artifact",

  "title": "Sample Document",
  "content": "This is the document content.",

  "jacsFiles": [
    {
      "mimetype": "application/pdf",
      "path": "./attachment.pdf",
      "embed": false,
      "sha256": "abc123..."
    }
  ],

  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "agentVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "date": "2024-01-15T10:30:00Z",
    "signature": "signature-base64...",
    "publicKeyHash": "key-hash...",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsId", "jacsVersion", "title", "content"]
  },

  "jacsSha256": "document-hash..."
}
```

HAI Field Categories

Fields include a `hai` property indicating their category:

Category	Description	Examples
`meta`	Metadata (IDs, dates)	jacsId, jacsVersion, jacsVersionDate
`base`	Cryptographic data	jacsSha256, signature
`agent`	Agent-controlled content	Custom content fields

This categorization determines which fields are included in hash and signature calculations.

Working with Documents

Creating Documents

```python
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create a basic document
doc = agent.create_document(json.dumps({
    'title': 'My Document',
    'content': 'Document content here'
}))
```

```typescript
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
agent.load('./jacs.config.json');

const doc = agent.createDocument(JSON.stringify({
  title: 'My Document',
  content: 'Document content here'
}));
```

Verifying Documents

```python
is_valid = agent.verify_document(doc_json)
```

```typescript
const isValid = agent.verifyDocument(docJson);
```

Updating Documents

```python
doc = json.loads(signed_doc)
document_key = f"{doc['jacsId']}:{doc['jacsVersion']}"

updated = agent.update_document(
    document_key,
    json.dumps({**doc, 'content': 'Updated content'})
)
```

```
const doc = JSON.parse(signedDoc);
const documentKey = `${doc.jacsId}:${doc.jacsVersion}`;

const updated = agent.updateDocument(
  documentKey,
  JSON.stringify({...doc, content: 'Updated content'})
);
```

Adding Attachments

```
doc = agent.create_document(
    json.dumps({'title': 'Report'}),
    attachments='./report.pdf',
    embed=True
)
```

```
const doc = agent.createDocument(
  JSON.stringify({ title: 'Report' }),
  null,     // custom_schema
  null,     // output_filename
  false,    // no_save
  './report.pdf',  // attachments
  true      // embed
);
```

Version History

Documents maintain a version chain:

```
Original (v1) ← Previous (v2) ← Current (v3)
     │
     └── jacsOriginalVersion points here for all versions
```

Each version:

- Has its own `jacsVersion` UUID
- References `jacsPreviousVersion` (except the first)
- All share the same `jacsId` and `jacsOriginalVersion`

See Also

- Agent Schema - Agent document structure
- Task Schema - Task document structure

- Working with Documents - Document operations guide
- Agreements - Multi-party agreements

Task Schema

The Task Schema defines the structure for task documents in JACS. Tasks represent work items with defined states, assigned agents, and completion criteria.

Schema Location

```
https://hai.ai/schemas/task/v1/task.schema.json
```

Overview

Task documents manage:

- **Workflow States**: From creation through completion
- **Agent Assignment**: Customer and assigned agent tracking
- **Actions**: Desired outcomes and completion criteria
- **Agreements**: Start and end agreements between parties
- **Relationships**: Sub-tasks, copies, and merges

Schema Structure

The task schema extends the Header Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://hai.ai/schemas/task/v1/task-schema.json",
  "title": "Task",
  "description": "General schema for stateful resources.",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    { "type": "object", "properties": { ... } }
  ]
}
```

Task States

Tasks progress through defined workflow states:

State	Description
`creating`	Task is being drafted
`rfp`	Request for proposal - seeking agents
`proposal`	Agent has submitted a proposal
`negotiation`	Terms being negotiated
`started`	Work has begun
`review`	Work submitted for review
`completed`	Task is finished

```
{
  "jacsTaskState": "started"
}
```

State Transitions

```
creating → rfp → proposal → negotiation → started → review → completed
                    ↑_______________|
                (may cycle back for renegotiation)
```

Task Properties

Core Fields (from Header)

Tasks inherit all document header fields plus task-specific fields.

Task-Specific Fields

Field	Type	Required	Description
`jacsTaskName`	string	No	Human-readable task name
`jacsTaskSuccess`	string	No	Description of success criteria
`jacsTaskState`	string	Yes	Current workflow state
`jacsTaskCustomer`	object	Yes	Customer agent signature
`jacsTaskAgent`	object	No	Assigned agent signature
`jacsTaskStartDate`	string (date-time)	No	When work started

Field	Type	Required	Description
`jacsTaskCompleteDate`	string (date-time)	No	When work completed
`jacsTaskActionsDesired`	array	Yes	Required actions
`jacsStartAgreement`	object	No	Agreement to begin work
`jacsEndAgreement`	object	No	Agreement that work is complete

Relationship Fields

Field	Type	Description
`jacsTaskSubTaskOf`	array	Parent task IDs
`jacsTaskCopyOf`	array	Source task IDs (branching)
`jacsTaskMergedTasks`	array	Tasks folded into this one

Actions

Actions define what needs to be accomplished:

```
{
  "jacsTaskActionsDesired": [
    {
      "name": "Create API Endpoint",
      "description": "Build REST endpoint for user registration",
      "cost": {
        "value": 500,
        "unit": "USD"
      },
      "duration": {
        "value": 8,
        "unit": "hours"
      },
      "completionAgreementRequired": true,
      "tools": [...]
    }
  ]
}
```

Action Schema Fields

Field	Type	Required	Description
`name`	string	Yes	Action name

Field	Type	Required	Description
`description`	string	Yes	What needs to be done
`tools`	array	No	Tools that can be used
`cost`	object	No	Cost estimate
`duration`	object	No	Time estimate
`completionAgreementRequired`	boolean	No	Requires sign-off

Unit Schema

Costs and durations use the unit schema:

```
{
  "cost": {
    "value": 100,
    "unit": "USD"
  },
  "duration": {
    "value": 2,
    "unit": "days"
  }
}
```

Agreements

Tasks can include start and end agreements:

Start Agreement

Signed when parties agree to begin work:

```
{
  "jacsStartAgreement": {
    "agentIDs": ["customer-uuid", "agent-uuid"],
    "question": "Do you agree to begin this work?",
    "context": "Project XYZ - Phase 1",
    "signatures": [...]
  }
}
```

End Agreement

Signed when parties agree work is complete:

```
{
  "jacsEndAgreement": {
    "agentIDs": ["customer-uuid", "agent-uuid"],
    "question": "Do you agree this work is complete?",
    "context": "Final deliverables reviewed",
    "signatures": [...]
  }
}
```

Complete Example


```
{
  "$schema": "https://hai.ai/schemas/task/v1/task.schema.json",
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsType": "task",
  "jacsVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsVersionDate": "2024-01-15T10:30:00Z",
  "jacsOriginalVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "jacsOriginalDate": "2024-01-15T10:30:00Z",
  "jacsLevel": "artifact",

  "jacsTaskName": "Build Authentication System",
  "jacsTaskSuccess": "Users can register, login, and manage sessions",
  "jacsTaskState": "started",

  "jacsTaskCustomer": {
    "agentID": "customer-agent-uuid",
    "agentVersion": "customer-version-uuid",
    "date": "2024-01-15T10:30:00Z",
    "signature": "customer-signature...",
    "publicKeyHash": "customer-key-hash",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsTaskName", "jacsTaskActionsDesired"]
  },

  "jacsTaskAgent": {
    "agentID": "assigned-agent-uuid",
    "agentVersion": "agent-version-uuid",
    "date": "2024-01-16T09:00:00Z",
    "signature": "agent-signature...",
    "publicKeyHash": "agent-key-hash",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsTaskName", "jacsTaskActionsDesired"]
  },

  "jacsTaskStartDate": "2024-01-16T09:00:00Z",

  "jacsStartAgreement": {
    "agentIDs": ["customer-agent-uuid", "assigned-agent-uuid"],
    "question": "Do you agree to begin work on this task?",
    "signatures": [
      {
        "agentID": "customer-agent-uuid",
        "signature": "...",
        "responseType": "agree",
        "date": "2024-01-16T09:00:00Z"
      },
      {
        "agentID": "assigned-agent-uuid",
        "signature": "...",
        "responseType": "agree",
        "date": "2024-01-16T09:05:00Z"
      }
    ]
  },
```

```
  "jacsTaskActionsDesired": [
    {
      "name": "User Registration",
      "description": "Implement user registration with email verification",
      "duration": { "value": 4, "unit": "hours" },
      "completionAgreementRequired": true
    },
    {
      "name": "User Login",
      "description": "Implement secure login with password hashing",
      "duration": { "value": 3, "unit": "hours" },
      "completionAgreementRequired": true
    },
    {
      "name": "Session Management",
      "description": "Implement JWT-based session tokens",
      "duration": { "value": 2, "unit": "hours" },
      "completionAgreementRequired": false
    }
  ],

  "jacsSignature": {
    "agentID": "customer-agent-uuid",
    "agentVersion": "customer-version-uuid",
    "date": "2024-01-15T10:30:00Z",
    "signature": "document-signature...",
    "publicKeyHash": "key-hash...",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsId", "jacsTaskName", "jacsTaskActionsDesired"]
  }
}
```

Task Relationships

Sub-Tasks

Break large tasks into smaller units:

```
{
  "jacsTaskSubTaskOf": ["parent-task-uuid"]
}
```

Task Copies (Branching)

Create variations or branches:

```
{
  "jacsTaskCopyOf": ["original-task-uuid"]
}
```

Merged Tasks

Combine completed tasks:

```
{
  "jacsTaskMergedTasks": [
    "subtask-1-uuid",
    "subtask-2-uuid"
  ]
}
```

Task Workflow

1. Creating a Task

```
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

task = agent.create_document(json.dumps({
    'jacsTaskName': 'Build Feature X',
    'jacsTaskSuccess': 'Feature is deployed and tested',
    'jacsTaskState': 'creating',
    'jacsTaskActionsDesired': [
        {
            'name': 'Implementation',
            'description': 'Write the code',
            'completionAgreementRequired': True
        }
    ]
}), custom_schema='https://hai.ai/schemas/task/v1/task.schema.json')
```

2. Assigning an Agent

When an agent accepts the task, add their signature to `jacsTaskAgent` and update state to `started`.

3. Signing Start Agreement

Both parties sign the start agreement to confirm work begins.

4. Completing Work

Update state to `review`, then both parties sign the end agreement.

5. Final Completion

After end agreement is signed by all parties, update state to `completed`.

State Machine Rules

Current State	Valid Next States
`creating`	`rfp`
`rfp`	`proposal`, `creating`
`proposal`	`negotiation`, `rfp`
`negotiation`	`started`, `proposal`
`started`	`review`
`review`	`completed`, `started`
`completed`	(terminal)

See Also

- Document Schema - Base document fields
- Agent Schema - Agent structure
- Agreements - Working with agreements
- JSON Schemas Overview - Schema architecture

Agent State Schema

The Agent State Schema defines the structure for signed agent state documents in JACS. Agent state documents wrap and cryptographically sign any agent configuration file -- memory files, skills, plans, configs, hooks, or any other document an agent wants to verify.

Schema Location

```
https://hai.ai/schemas/agentstate/v1/agentstate.schema.json
```

Overview

Agent state documents provide:

- **Signed state files**: Cryptographically sign MEMORY.md, skill files, plans, configs, hooks, or any file
- **File integrity**: SHA-256 hashes verify file contents haven't been tampered with
- **Origin tracking**: Record whether state was authored, adopted, generated, or imported
- **Framework tagging**: Identify which agent framework (claude-code, langchain, etc.) the state belongs to
- **General-purpose signing**: Use type `other` to sign any document an agent wants to verify

All documents are stored within the JACS data directory for security.

Schema Structure

The agent state schema extends the Header Schema:

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://hai.ai/schemas/agentstate/v1/agentstate.schema.json",
  "title": "Agent State Document",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" }
  ],
  "properties": {
    "jacsAgentStateType": {
      "type": "string",
      "enum": ["memory", "skill", "plan", "config", "hook", "other"]
    },
    "jacsAgentStateName": { "type": "string" }
  },
  "required": ["jacsAgentStateType", "jacsAgentStateName"]
}
```

State Types

Type	Description	Example
`memory`	Agent memory/knowledge files	MEMORY.md, context files
`skill`	Agent skill definitions	Coding patterns, domain knowledge
`plan`	Agent plans and strategies	Implementation plans, workflows
`config`	Agent configuration files	Settings, preferences
`hook`	Agent hooks and triggers (always embedded)	Pre-commit hooks, event handlers
`other`	Any document the agent wants to sign and verify	Reports, artifacts, custom files

Properties

Required Fields

Field	Type	Description
`jacsAgentStateType`	string (enum)	Type of agent state: memory, skill, plan, config, hook, other
`jacsAgentStateName`	string	Human-readable name for this state document

Optional Fields

Field	Type	Description
`jacsAgentStateDescription`	string	Description of what this state contains
`jacsAgentStateFramework`	string	Agent framework (e.g., "claude-code", "langchain")
`jacsAgentStateVersion`	string	Content version (distinct from `jacsVersion`)
`jacsAgentStateContentType`	string	MIME type (text/markdown, application/json, etc.)
`jacsAgentStateContent`	string	Inline content (used when embedding)
`jacsAgentStateTags`	string[]	Tags for categorization and search
`jacsAgentStateOrigin`	string (enum)	How created: authored, adopted, generated, imported
`jacsAgentStateSourceUrl`	string (uri)	Where content was obtained from

Origin Tracking

Every agent state document can track its provenance:

Origin	Meaning
`authored`	Created by the signing agent
`adopted`	Found unsigned, signed by adopting agent
`generated`	Produced by AI/automation
`imported`	Brought from another JACS installation

File References

Agent state documents can reference external files using `jacsFiles`:

```
{
  "jacsFiles": [
    {
      "mimetype": "text/markdown",
      "path": "MEMORY.md",
      "embed": true,
      "sha256": "b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9",
      "contents": "base64-encoded-gzipped-content"
    }
  ]
}
```

When `embed` is `true`, the file content is stored inline in the document. **Hook-type documents always embed content** for security (prevents time-of-check/time-of-use attacks).

Examples

Minimal Agent State

```
{
  "$schema": "https://hai.ai/schemas/agentstate/v1/agentstate.schema.json",
  "jacsAgentStateType": "memory",
  "jacsAgentStateName": "Project Memory",
  "jacsType": "agentstate",
  "jacsLevel": "config"
}
```

Memory File with Embedding

```
{
  "$schema": "https://hai.ai/schemas/agentstate/v1/agentstate.schema.json",
  "jacsAgentStateType": "memory",
  "jacsAgentStateName": "JACS Project Memory",
  "jacsAgentStateDescription": "Agent memory for the JACS project workspace",
  "jacsAgentStateFramework": "claude-code",
  "jacsAgentStateOrigin": "authored",
  "jacsAgentStateContentType": "text/markdown",
  "jacsAgentStateContent": "# MEMORY.md\n\n## Project: JACS\n- Location:
/home/agent/jacs\n- Rust library for cryptographic signing\n",
  "jacsAgentStateTags": ["jacs", "rust", "crypto"],
  "jacsType": "agentstate",
  "jacsLevel": "config"
}
```

Adopted Skill

```
{
  "$schema": "https://hai.ai/schemas/agentstate/v1/agentstate.schema.json",
  "jacsAgentStateType": "skill",
  "jacsAgentStateName": "JSON Schema Validation",
  "jacsAgentStateOrigin": "adopted",
  "jacsAgentStateSourceUrl": "https://agentskills.io/skills/json-schema",
  "jacsAgentStateVersion": "2.1.0",
  "jacsType": "agentstate",
  "jacsLevel": "config"
}
```

General-Purpose Signed Document

Use type `other` to sign any document:

```
{
  "$schema": "https://hai.ai/schemas/agentstate/v1/agentstate.schema.json",
  "jacsAgentStateType": "other",
  "jacsAgentStateName": "Q1 Financial Report",
  "jacsAgentStateDescription": "Quarterly financial summary for verification",
  "jacsAgentStateContentType": "application/json",
  "jacsAgentStateContent": "{\"revenue\": 150000, \"expenses\": 120000}",
  "jacsType": "agentstate",
  "jacsLevel": "config"
}
```

Rust API

Creating Agent State Documents

```rust
use jacs::schema::agentstate_crud::*;

// Minimal state
let doc = create_minimal_agentstate("memory", "Project Memory", Some("Agent memory
file"))?;

// With file reference
let doc = create_agentstate_with_file("skill", "Rust Patterns", "./skills/rust.md",
true)?;

// With inline content
let doc = create_agentstate_with_content(
    "config",
    "Agent Settings",
    "{\"theme\": \"dark\"}",
    "application/json"
)?;

// General-purpose signing
let doc = create_agentstate_with_content(
    "other",
    "Audit Report",
    "Report contents here...",
    "text/plain"
)?;

// Set metadata
let mut doc = create_minimal_agentstate("memory", "My Memory", None)?;
set_agentstate_framework(&mut doc, "claude-code")?;
set_agentstate_origin(&mut doc, "authored", None)?;
set_agentstate_tags(&mut doc, vec!["project", "notes"])?;
set_agentstate_version(&mut doc, "1.0.0")?;
```

Signing and Verification

```rust
// Create, sign, and store
let doc_string = serde_json::to_string(&doc)?;
let signed_doc = agent.create_document_and_load(&doc_string, None, None)?;

// Verify file integrity
let hash_valid = verify_agentstate_file_hash(&doc)?;
```

MCP Tools

Six MCP tools are available for agent state operations:

Tool	Description
`jacs_sign_state`	Create and sign a new agent state document
`jacs_verify_state`	Verify an existing agent state document by JACS document ID (`jacs_id`)
`jacs_load_state`	Load an agent state document by JACS document ID (`jacs_id`)
`jacs_update_state`	Update and re-sign an agent state document by JACS document ID (`jacs_id`)
`jacs_list_state`	List all agent state documents
`jacs_adopt_state`	Adopt an external file as a signed agent state

MCP Example: Sign a Memory File

```
{
  "tool": "jacs_sign_state",
  "arguments": {
    "state_type": "memory",
    "name": "Project Memory",
    "content": "# My Agent Memory\n\nKey facts about the project...",
    "content_type": "text/markdown",
    "framework": "claude-code",
    "tags": ["project", "memory"]
  }
}
```

MCP Example: Sign Any Document

```
{
  "tool": "jacs_sign_state",
  "arguments": {
    "state_type": "other",
    "name": "Verification Report",
    "content": "{\"status\": \"passed\", \"checks\": 42}",
    "content_type": "application/json"
  }
}
```

Security Notes

- All agent state documents are stored within the JACS data directory for security
- MCP verify/load/update flows are `jacs_id`-based; direct path-only access is disabled
- Hook-type documents always embed content to prevent TOCTOU attacks
- File hashes (SHA-256) are verified on load to detect tampering
- Origin tracking provides provenance auditing
- Documents are signed with the agent's private key, providing non-repudiation

See Also

- JSON Schemas - Schema architecture overview
- Working with Documents - General document operations
- MCP Integration - MCP server setup
- Security Model - Cryptographic details

Commitment Schema

Commitments are shared, signed agreements between agents. They represent what an agent commits to doing, optionally within a conversation or linked to a task or todo item.

Key design: Commitments work standalone. They do not require goals, tasks, conversations, or any other document type to be created first.

Schema

- **ID**: `https://hai.ai/schemas/commitment/v1/commitment.schema.json`
- **Type**: `jacsType: "commitment"`
- **Level**: `jacsLevel: "config"` (editable, versioned)
- **Extends**: `header.schema.json` via `allOf`

Required Fields

Field	Type	Description
`jacsCommitmentDescription`	string	Human-readable description of the commitment
`jacsCommitmentStatus`	enum	Lifecycle status

Status Lifecycle

```
pending -> active -> completed
                  -> failed
                  -> renegotiated
         -> disputed
         -> revoked
```

Status	Meaning
`pending`	Created but not yet started
`active`	Work is underway
`completed`	Successfully fulfilled
`failed`	Could not be fulfilled
`renegotiated`	Terms changed, replaced by new commitment
`disputed`	One party contests the commitment

Status	Meaning
`revoked`	Withdrawn by the owner

Optional Fields

Field	Type	Description
`jacsCommitmentTerms`	object	Structured terms (deliverable, deadline, compensation, etc.)
`jacsCommitmentDisputeReason`	string	Reason when status is `disputed` or `revoked`
`jacsCommitmentTaskId`	uuid	Reference to a task document
`jacsCommitmentConversationRef`	uuid	Thread ID of the conversation that produced this commitment
`jacsCommitmentTodoRef`	string	Todo item reference in format `list-uuid:item-uuid`
`jacsCommitmentQuestion`	string	Structured question prompt
`jacsCommitmentAnswer`	string	Answer to the question
`jacsCommitmentCompletionQuestion`	string	Question to verify completion
`jacsCommitmentCompletionAnswer`	string	Answer verifying completion
`jacsCommitmentStartDate`	date-time	When the commitment period begins
`jacsCommitmentEndDate`	date-time	Deadline
`jacsCommitmentRecurrence`	object	Recurrence pattern (`frequency` + `interval`)
`jacsCommitmentOwner`	signature	Single-agent owner signature

Cross-References

Commitments can link to other document types:

- **Conversation**: `jacsCommitmentConversationRef` holds a thread UUID
- **Todo item**: `jacsCommitmentTodoRef` uses format `list-uuid:item-uuid`
- **Task**: `jacsCommitmentTaskId` holds a task document UUID

These references are optional. Commitments work independently.

Multi-Agent Agreements

Commitments use the standard JACS agreement mechanism from the header schema. Two or more agents can co-sign a commitment using `jacsAgreement`.

Example

```
{
  "$schema": "https://hai.ai/schemas/commitment/v1/commitment.schema.json",
  "jacsCommitmentDescription": "Deliver Q1 analytics report by March 15",
  "jacsCommitmentStatus": "active",
  "jacsCommitmentTerms": {
    "deliverable": "PDF report with charts",
    "deadline": "2026-03-15T00:00:00Z"
  },
  "jacsCommitmentStartDate": "2026-01-15T00:00:00Z",
  "jacsCommitmentEndDate": "2026-03-15T00:00:00Z",
  "jacsType": "commitment",
  "jacsLevel": "config"
}
```

Rust API

```rust
use jacs::schema::commitment_crud::*;

// Create
let commitment = create_minimal_commitment("Deliver report").unwrap();

// With structured terms
let commitment = create_commitment_with_terms(
    "Weekly standup",
    serde_json::json!({"frequency": "weekly"}),
).unwrap();

// Update status
update_commitment_status(&mut commitment, "active").unwrap();

// Dispute
dispute_commitment(&mut commitment, "Terms not met").unwrap();

// Cross-references
set_conversation_ref(&mut commitment, &thread_id).unwrap();
set_todo_ref(&mut commitment, "list-uuid:item-uuid").unwrap();
set_task_ref(&mut commitment, &task_id).unwrap();
```

Versioning

Since commitments use `jacsLevel: "config"`, they can be updated. Each update creates a new `jacsVersion` linked to the previous via `jacsPreviousVersion`. This provides a full audit trail of status changes and modifications.

See Also

- Todo List Schema - Private task tracking
- Conversation Schema - Message threading
- Document Schema - Header fields and signing

Todo List Schema

Todo lists are private, signed documents belonging to a single agent. They contain inline items (goals and tasks) and are re-signed as a whole when any item changes.

Schema

- **ID**: `https://hai.ai/schemas/todo/v1/todo.schema.json`
- **Type**: `jacsType: "todo"`
- **Level**: `jacsLevel: "config"` (editable, versioned)
- **Extends**: `header.schema.json` via `allOf`
- **Component**: `todoitem.schema.json` for inline items

Required Fields

Field	Type	Description
`jacsTodoName`	string	Human-readable name for this list
`jacsTodoItems`	array	Inline todo items

Optional Fields

Field	Type	Description
`jacsTodoArchiveRefs`	uuid[]	UUIDs of archived todo lists

Todo Items

Each item in `jacsTodoItems` is an inline object following `todoitem.schema.json`.

Required Item Fields

Field	Type	Description
`itemId`	uuid	Stable UUID that does not change on re-signing
`itemType`	enum	`"goal"` (broad objective) or `"task"` (specific action)

Field	Type	Description
`description`	string	Human-readable description
`status`	enum	`"pending"`, `"in-progress"`, `"completed"`, `"abandoned"`

Optional Item Fields

Field	Type	Description
`priority`	enum	`"low"`, `"medium"`, `"high"`, `"critical"`
`childItemIds`	uuid[]	Sub-goals or tasks under this item
`relatedCommitmentId`	uuid	Commitment that formalizes this item
`relatedConversationThread`	uuid	Conversation thread related to this item
`completedDate`	date-time	When the item was completed
`assignedAgent`	uuid	Agent assigned to this item
`tags`	string[]	Tags for categorization

Cross-References

Todo items can link to other document types:

- **Commitment**: `relatedCommitmentId` links an item to a commitment
- **Conversation**: `relatedConversationThread` links an item to a message thread

References use the `list-uuid:item-uuid` format when referenced FROM other documents (e.g., `jacsCommitmentTodoRef` on a commitment). Use `build_todo_item_ref()` and `parse_todo_item_ref()` from `reference_utils` for this format.

Item Hierarchy

Items support parent-child relationships via `childItemIds`:

```
Goal: "Ship Q1 release"
  ├── Task: "Write documentation"
  ├── Task: "Run integration tests"
  └── Goal: "Performance optimization"
        ├── Task: "Profile database queries"
        └── Task: "Add caching layer"
```

Example


```
{
  "$schema": "https://hai.ai/schemas/todo/v1/todo.schema.json",
  "jacsTodoName": "Q1 Sprint",
  "jacsTodoItems": [
    {
      "itemId": "550e8400-e29b-41d4-a716-446655440001",
      "itemType": "goal",
      "description": "Ship analytics dashboard",
      "status": "in-progress",
      "priority": "high",
      "childItemIds": [
        "550e8400-e29b-41d4-a716-446655440002"
      ]
    },
    {
      "itemId": "550e8400-e29b-41d4-a716-446655440002",
      "itemType": "task",
      "description": "Build chart components",
      "status": "pending",
      "priority": "medium",
      "relatedCommitmentId": "660e8400-e29b-41d4-a716-446655440000",
      "tags": ["frontend", "charts"]
    }
  ],
  "jacsType": "todo",
  "jacsLevel": "config"
}
```

Rust API

```rust
use jacs::schema::todo_crud::*;

// Create a list
let mut list = create_minimal_todo_list("Sprint Work").unwrap();

// Add items
let goal_id = add_todo_item(&mut list, "goal", "Ship Q1", Some("high")).unwrap();
let task_id = add_todo_item(&mut list, "task", "Write tests", None).unwrap();

// Build hierarchy
add_child_to_item(&mut list, &goal_id, &task_id).unwrap();

// Progress tracking
update_todo_item_status(&mut list, &task_id, "in-progress").unwrap();
mark_todo_item_complete(&mut list, &task_id).unwrap();

// Cross-references
set_item_commitment_ref(&mut list, &task_id, &commitment_id).unwrap();
set_item_conversation_ref(&mut list, &task_id, &thread_id).unwrap();

// Archive completed items
let completed = remove_completed_items(&mut list).unwrap();
```

Versioning

Since todo lists use `jacsLevel: "config"`, each modification creates a new signed version. The `itemId` fields remain stable across versions, enabling consistent cross-referencing even as items are added, updated, or removed.

See Also

- Commitment Schema - Shared agreements
- Conversation Schema - Message threading
- Document Schema - Header fields and signing

Conversation Schema

Conversations use the existing message schema enhanced with thread tracking and message ordering. There is no separate "conversation" schema - conversations are sequences of signed messages sharing a thread ID.

Schema

- **ID**: `https://hai.ai/schemas/message/v1/message.schema.json`
- **Type**: `jacsType: "message"`
- **Level**: `jacsLevel: "raw"` (immutable once signed)
- **Extends**: `header.schema.json` via `allOf`

Message Fields

Required

Field	Type	Description
`to`	string[]	Recipient agent identifiers
`from`	string[]	Sender agent identifiers
`content`	object	Message body (free-form object)

Optional

Field	Type	Description
`threadID`	string	UUID of the conversation thread
`jacsMessagePreviousId`	uuid	UUID of the previous message in this thread
`attachments`	array	File attachments

Threading Model

Messages form a thread via two fields:

1. **`threadID`** - All messages in a conversation share the same thread ID

2. **`jacsMessagePreviousId`** - Each message references the previous one, creating an ordered chain

```
Message 1 (threadID: "abc-123", previousId: null)
    └── Message 2 (threadID: "abc-123", previousId: msg1.jacsId)
         └── Message 3 (threadID: "abc-123", previousId: msg2.jacsId)
```

Immutability

Messages use `jacsLevel: "raw"`, making them immutable once signed. To continue a conversation, create a new message referencing the previous one. This ensures the integrity of the conversation history.

Example

```
{
  "$schema": "https://hai.ai/schemas/message/v1/message.schema.json",
  "threadID": "550e8400-e29b-41d4-a716-446655440000",
  "content": {
    "body": "I agree to the proposed terms.",
    "subject": "Re: Q1 Deliverables"
  },
  "to": ["agent-b-uuid"],
  "from": ["agent-a-uuid"],
  "jacsMessagePreviousId": "660e8400-e29b-41d4-a716-446655440001",
  "jacsType": "message",
  "jacsLevel": "raw"
}
```

Rust API

```rust
use jacs::schema::conversation_crud::*;

// Start a new conversation (generates thread ID)
let (first_msg, thread_id) = start_new_conversation(
    serde_json::json!({"body": "Hello, let's discuss terms."}),
    vec!["agent-b".to_string()],
    vec!["agent-a".to_string()],
).unwrap();

// Continue the conversation
let reply = create_conversation_message(
    &thread_id,
    serde_json::json!({"body": "Sounds good. Here are my terms."}),
    vec!["agent-a".to_string()],
    vec!["agent-b".to_string()],
    Some(&previous_message_jacs_id),
).unwrap();

// Extract thread info
let tid = get_thread_id(&message).unwrap();
let prev = get_previous_message_id(&message);
```

Cross-References

Conversations can be referenced by other document types:

- **Commitment**: `jacsCommitmentConversationRef` stores the thread UUID
- **Todo item**: `relatedConversationThread` stores the thread UUID

This allows tracking which conversation led to a commitment or is related to a work item.

See Also

- Commitment Schema - Agreements arising from conversations
- Todo List Schema - Private task tracking
- Document Schema - Header fields and signing

Config File Schema

This page documents the `jacs.config.json` schema fields. For a comprehensive configuration guide including observability setup, storage backends, zero-config quickstart, and production patterns, see the Configuration Reference.

Schema Location

```
https://hai.ai/schemas/jacs.config.schema.json
```

Minimal Configuration

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_agent_id_and_version": "YOUR_AGENT_ID:YOUR_VERSION",
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

All other settings use sensible defaults (`./jacs_data` , `./jacs_keys` , `fs` storage). Override only what you need.

Fields

Field	Type	Description
`jacs_data_directory`	string	Path to store documents and agents
`jacs_key_directory`	string	Path to store cryptographic keys
`jacs_agent_private_key_filename`	string	Private key filename
`jacs_agent_public_key_filename`	string	Public key filename
`jacs_agent_key_algorithm`	string	Signing algorithm
`jacs_default_storage`	string	Storage backend

Configuration Options

Key Configuration

jacs_agent_key_algorithm

Specifies the cryptographic algorithm for signing:

Value	Description
`ring-Ed25519`	Ed25519 signatures (recommended)
`RSA-PSS`	RSA with PSS padding
`pq-dilithium`	Post-quantum Dilithium
`pq2025`	Post-quantum composite

```
{
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

jacs_agent_private_key_filename

Name of the private key file in the key directory:

```
{
  "jacs_agent_private_key_filename": "private.pem"
}
```

If the key is encrypted, it will have `.enc` appended automatically when loading.

jacs_agent_public_key_filename

Name of the public key file:

```
{
  "jacs_agent_public_key_filename": "public.pem"
}
```

jacs_private_key_password

Password for encrypted private keys:

```
{
  "jacs_private_key_password": "your-password"
}
```

Warning: Do not store passwords in config files for production. Use the `JACS_PRIVATE_KEY_PASSWORD` environment variable instead.

Storage Configuration

jacs_default_storage

Specifies where documents are stored:

Value	Description
`fs`	Local filesystem
`aws`	AWS S3 storage
`hai`	HAI cloud storage

```
{
  "jacs_default_storage": "fs"
}
```

jacs_data_directory

Path for storing documents and agents:

```
{
  "jacs_data_directory": "./jacs_data"
}
```

Agent Identity

jacs_agent_id_and_version

Load an existing agent by ID and version:

```
{
  "jacs_agent_id_and_version": "550e8400-e29b-41d4-a716-446655440000:f47ac10b-58cc-4372-a567-0e02b2c3d479"
}
```

Schema Versions

Specify which schema versions to use:

```
{
  "jacs_agent_schema_version": "v1",
  "jacs_header_schema_version": "v1",
  "jacs_signature_schema_version": "v1"
}
```

DNS Configuration

For DNSSEC-based agent verification:

jacs_agent_domain

Domain for DNS-based public key verification:

```
{
  "jacs_agent_domain": "example.com"
}
```

jacs_dns_validate

Enable DNS TXT fingerprint validation:

```
{
  "jacs_dns_validate": true
}
```

jacs_dns_strict

Require DNSSEC validation (no fallback):

```
{
  "jacs_dns_strict": true
}
```

jacs_dns_required

Require domain and DNS validation:

```
{
  "jacs_dns_required": true
}
```

Security

jacs_use_security

Enable strict security features:

```
{
  "jacs_use_security": "1"
}
```

Values: `"0"`, `"1"`, or `"false"`, `"true"`

Observability Fields

The `observability` object supports `logs`, `metrics`, and `tracing` sub-objects. For full details on all destinations, sampling options, and production patterns, see the Configuration Reference.

Environment Variables

Configuration can be overridden with environment variables:

Variable	Config Field
`JACS_PRIVATE_KEY_PASSWORD`	`jacs_private_key_password`
`JACS_DATA_DIRECTORY`	`jacs_data_directory`
`JACS_KEY_DIRECTORY`	`jacs_key_directory`

See Also

- Configuration Reference - Full configuration guide with examples
- JSON Schemas Overview - Schema architecture
- Observability (Rust API) - Rust observability API
- Observability & Monitoring Guide - Structured events, OTEL collector setup

JACS Attestation vs. Other Standards

JACS occupies a unique position as an agent-layer attestation framework. This page compares JACS with related standards and explains when to use them together.

Comparison Table

Feature	JACS	in-toto / SLSA	Sigstore / cosign	SCITT	IETF
Primary domain	AI agent runtime	Build provenance	Artifact signing	Transparency logs	Hardw attesta
Identity model	Decentralized (key pairs)	Build system certs	Keyless (OIDC)	Issuer certs	Platfor
Agent-native	Yes	No	No	No	Partial
Offline verification	Yes	Yes (with keys)	No (requires Rekor)	No (requires log)	Depen
Multi-agent quorum	Yes (M-of-N)	No	No	No	No
Evidence normalization	Yes (A2A, email, JWT, custom)	No	No	No	Partial
Transform receipts	Yes (derivation chains)	Yes (build steps)	No	No	No
Probabilistic claims	Yes (confidence + assurance)	No	No	No	No
Post-quantum	Yes (ML-DSA-87)	No	No	No	Depen
Central infrastructure	Not required	Not required	Required (Fulcio + Rekor)	Required (transparency log)	Depen
Schema format	JSON Schema + JCS	in-toto layout	Sigstore bundle	SCITT receipt	CBOR/

JACS vs. in-toto / SLSA

Domain difference: in-toto and SLSA focus on build provenance -- proving that a software artifact was built by a specific builder from specific source code. JACS focuses on runtime agent actions -- proving that a specific agent performed a specific action with specific evidence.

Interoperability: JACS exports attestations as DSSE (Dead Simple Signing Envelope) documents, the same format used by in-toto v1.0+. This means:

- A JACS attestation can include an in-toto predicate type URI
- SLSA verifiers can validate the DSSE envelope structure
- JACS and in-toto attestations can coexist in the same verification pipeline

When to use both: If your workflow includes both software builds (use SLSA/in-toto for build provenance) and AI agent actions (use JACS for runtime attestation), you can link them via derivation chains.

JACS vs. Sigstore / cosign

Domain difference: Sigstore provides signing infrastructure (Fulcio CA, Rekor transparency log) and cosign is a tool for signing container images and artifacts. JACS provides its own signing with decentralized identity.

Key difference: Sigstore's keyless signing relies on centralized OIDC identity providers and a public transparency log. JACS uses self-managed key pairs and does not require any centralized infrastructure.

When to use both: Sigstore for container image signing in CI/CD pipelines. JACS for AI agent action signing at runtime. A planned Sigstore bundle verification adapter (N+2) would let JACS attestations reference Sigstore signatures as evidence.

JACS vs. SCITT

Most overlap. SCITT (Supply Chain Integrity, Transparency and Trust) defines a centralized transparency service for recording signed statements about artifacts.

Key difference: SCITT requires a transparency log (centralized notary). JACS is fully decentralized and offline-capable. JACS verification works without contacting any server.

Complementary use: JACS signs and attests. SCITT logs. An organization could use JACS to create signed attestations and then submit them to a SCITT transparency log for auditability, getting the benefits of both decentralized creation and centralized discoverability.

JACS vs. IETF RATS / EAT

Layer difference: RATS (Remote ATtestation procedureS) and EAT (Entity Attestation Token) focus on hardware and platform attestation -- proving that a device or execution environment is in a known-good state. JACS fills the software agent layer above hardware.

Alignment opportunity: JACS claim names could align with IANA-registered EAT claim types where they overlap. A JACS attestation could reference a RATS attestation result as evidence, creating a trust chain from hardware to agent.

IETF drafts of interest:

- `draft-huang-rats-agentic-eat-cap-attest-00` -- Capability attestation for agents, directly aligned with JACS claims model
- `draft-messous-eat-ai-00` -- EAT profile for AI agents
- `draft-jiang-seat-dynamic-attestation-00` -- Dynamic attestation for runtime assertions

JACS vs. CSA Agentic Trust Framework

The Cloud Security Alliance's Agentic Trust Framework defines progressive trust levels that map directly to JACS's trust model:

CSA Level	JACS Equivalent	Verification
None	No JACS	No signing
Basic	Open	Valid signature accepted
Standard	Verified	Trust store + DNS verification
Enhanced	Strict	Attestation-level evidence required

When to Use JACS

Use JACS when you need:

- **Agent identity** that works without PKI/CA infrastructure
- **Non-repudiable action logging** for AI agent workflows
- **Multi-agent authorization** with quorum (M-of-N approval)
- **Offline verification** without centralized services
- **Evidence-backed trust** that goes beyond simple signing
- **Post-quantum readiness** for long-lived agent identities

When to Use JACS Alongside Other Tools

Scenario	JACS + ...
CI/CD pipeline with AI agents	JACS (agent actions) + SLSA (build provenance)
Enterprise with compliance requirements	JACS (signing) + SCITT (transparency log)
IoT/edge with hardware attestation	JACS (agent layer) + RATS/EAT (hardware layer)
Container-based agent deployment	JACS (runtime signing) + cosign (image signing)

Security Model

JACS implements a comprehensive security model designed to ensure authenticity, integrity, and non-repudiation for all agent communications and documents.

Security Model (v0.6.0)

- **Passwords**: The private key password must be set only via the `JACS_PRIVATE_KEY_PASSWORD` environment variable. It is never stored in config files.
- **Keys**: Private keys are encrypted at rest (AES-256-GCM with PBKDF2, 600k iterations). Public keys and config may be stored on disk.
- **Path validation**: All paths built from untrusted input (e.g. `publicKeyHash`, filenames) are validated via `require_relative_path_safe()` to prevent directory traversal. This single validation function is used in data and key directory path builders and the trust store. It rejects empty segments, `.`, `..`, null bytes, and Windows drive-prefixed paths.
- **Trust ID canonicalization**: Trust-store operations normalize canonical agent docs (`jacsId` + `jacsVersion`) into a safe `UUID:VERSION_UUID` identifier before filesystem use, preserving path-safety checks while supporting standard agent document layout.
- **Filesystem schema policy**: Local schema loading is disabled by default and requires `JACS_ALLOW_FILESYSTEM_SCHEMAS=true`. When enabled, schema paths must remain within configured roots (`JACS_DATA_DIRECTORY` and/or `JACS_SCHEMA_DIRECTORY`) after normalized/canonical path checks.
- **Network endpoint policy**: Registry verification requires HTTPS for `JACS_REGISTRY_URL` (legacy alias: `HAI_API_URL`). Localhost HTTP is allowed for local testing only.
- **No secrets in config**: Config files must not contain passwords or other secrets. The example config (`jacs.config.example.json`) does not include `jacs_private_key_password`.
- **Dependency auditing**: Run `cargo audit` (Rust), `npm audit` (Node.js), or `pip audit` (Python) to check for known vulnerabilities.

Core Security Principles

1. Cryptographic Identity

Every JACS agent has a unique cryptographic identity:

- **Key Pair**: Each agent possesses a private/public key pair
- **Agent ID**: Unique UUID identifying the agent
- **Public Key Hash**: SHA-256 hash of the public key for verification

```
{
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "publicKeyHash": "sha256-of-public-key",
    "signingAlgorithm": "ring-Ed25519"
  }
}
```

2. Document Integrity

All documents include cryptographic guarantees:

- **Signature**: Cryptographic signature over specified fields
- **Hash**: SHA-256 hash of document contents
- **Version Tracking**: Immutable version history

3. Non-Repudiation

Signatures provide proof of origin:

- Agents cannot deny signing a document
- Timestamps record when signatures were made
- Public keys enable independent verification

Security Audit (`audit()`)

JACS provides a read-only **security audit** that checks configuration, directories, secrets, trust store, storage, quarantine/failed files, and optionally re-verifies recent documents. It does not modify state.

Purpose: Surface misconfiguration, missing keys, unexpected paths, and verification failures in one report.

Options (all optional):

- `config_path` : Path to `jacs.config.json` (default: 12-factor load)
- `data_directory` / `key_directory` : Override paths
- `recent_verify_count` : Number of recent documents to re-verify (default 10, max 100)

Return structure: `AuditResult` with `overall_status` , `risks` (list of `AuditRisk`), `health_checks` (list of `ComponentHealth`), `summary` , `checked_at` , and optional `quarantine_entries` / `failed_entries` .

Rust:

```rust
use jacs::audit::{audit, AuditOptions};

let result = audit(AuditOptions::default())?;
println!("{}", jacs::format_audit_report(&result));
```

Python:

```python
import jacs.simple as jacs

result = jacs.audit()  # dict with risks, health_checks, summary, overall_status
print(f"Risks: {len(result['risks'])}, Status: {result['overall_status']}")
```

Node.js:

```node.js
import * as jacs from '@hai.ai/jacs/simple';

const result = jacs.audit({ recentN: 5 });
console.log(`Risks: ${result.risks.length}, Status: ${result.overall_status}`);
```

Available in all bindings and as an MCP tool (`jacs_audit`) for automation.

Threat Model

Protected Against

Threat	Protection
Tampering	Content hashes detect modifications
Impersonation	Cryptographic signatures verify identity
Replay Attacks	Timestamps and version IDs ensure freshness; future timestamps rejected; optional signature expiration via `JACS_MAX_SIGNATURE_AGE_SECONDS`
Man-in-the-Middle	DNS verification via DNSSEC; TLS certificate validation
Key Compromise	Key rotation through versioning
Weak Passwords	Minimum 28-bit entropy enforcement (35-bit for single class)

Trust Assumptions

1. Private keys are kept secure

2. Cryptographic algorithms are sound
3. DNS infrastructure (when used) is trustworthy

Signature Process

Signing a Document

1. **Field Selection**: Determine which fields to sign
2. **Canonicalization**: Serialize fields deterministically
3. **Signature Generation**: Sign with private key
4. **Hash Computation**: Compute SHA-256 of signed document

```
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create signed document
doc = agent.create_document(json.dumps({
    'title': 'Confidential Report',
    'content': 'Sensitive data here'
}))

# Document now includes jacsSignature and jacsSha256
```

Verifying a Document

1. **Hash Verification**: Recompute hash and compare
2. **Signature Verification**: Verify signature with public key
3. **Agent Verification**: Optionally verify agent identity via DNS

```
is_valid = agent.verify_document(doc_json)
is_signature_valid = agent.verify_signature(doc_json)
```

Key Management

Key Generation

JACS generates cryptographic key pairs during agent creation:

```
# Keys are created in the configured key directory
jacs_keys/
├── private.pem    # Private key (keep secure!)
└── public.pem     # Public key (can be shared)
```

Key Protection

Encryption at Rest:

Private keys are encrypted using AES-256-GCM with a key derived via PBKDF2-HMAC-SHA256 (600,000 iterations). Never store the password in config files.

```
# Set via environment variable only
export JACS_PRIVATE_KEY_PASSWORD="secure-password"
```

> **Important**: The CLI can prompt for the password during `jacs init`, but scripts and servers must set `JACS_PRIVATE_KEY_PASSWORD` as an environment variable.

Password Entropy Requirements:

JACS enforces password entropy minimums for private key encryption. Password validation is performed at encryption time, and weak passwords are rejected with helpful error messages:

- Minimum **28-bit entropy** for passwords with 2+ character classes (mixed case, numbers, symbols)
- Minimum **35-bit entropy** for single-character-class passwords (e.g., all lowercase)
- Entropy is calculated based on character class diversity and length
- Weak passwords result in immediate rejection during key encryption
- Error messages guide users toward stronger password choices

Example of rejected weak passwords:

- `password` - Too common and predictable
- `12345678` - Insufficient character diversity
- `abc` - Too short

File Permissions:

```
chmod 700 ./jacs_keys
chmod 600 ./jacs_keys/private.pem
```

Key Rotation

Update agent version to rotate keys:

1. Generate new key pair
2. Create new agent version
3. Sign new version with old key
4. Update configuration to use new keys

TLS Certificate Validation

JACS includes configurable TLS certificate validation for secure network communication.

Default Behavior (Development)

By default, JACS warns about invalid TLS certificates but accepts them to facilitate development environments with self-signed certificates:

```
WARNING: Invalid TLS certificate detected. Set JACS_STRICT_TLS=true for production.
```

Production Configuration

For production deployments, enable strict TLS validation:

```
export JACS_STRICT_TLS=true
```

When enabled, JACS will:

- Reject connections with invalid, expired, or self-signed certificates
- Enforce proper certificate chain validation
- Fail fast with clear error messages for certificate issues

Implementation: Certificate validation logic is located in `jacs/src/schema/utils.rs`.

Security Implications

Mode	Behavior	Use Case
Default (dev)	Warn on invalid certs, allow connection	Local development, testing

Mode	Behavior	Use Case
Strict (`JACS_STRICT_TLS=true`)	Reject invalid certs	Production, staging

For registry verification endpoints, `JACS_REGISTRY_URL` (legacy `HAI_API_URL`) must use HTTPS. HTTP is only allowed for localhost test endpoints.

Signature Timestamp Validation

JACS signatures include timestamps to prevent replay attacks and ensure temporal integrity.

How It Works

1. **Timestamp Inclusion**: Every signature includes a UTC timestamp recording when it was created
2. **Future Timestamp Rejection**: Signatures with timestamps more than 5 minutes in the future are rejected
3. **Optional Signature Expiration**: Configurable via `JACS_MAX_SIGNATURE_AGE_SECONDS` (disabled by default since JACS documents are designed to be eternal)
4. **Validation**: Timestamp validation occurs during signature verification

Configuring Signature Expiration

By default, signatures do not expire. JACS documents are designed to be idempotent and eternal. For use cases that require expiration:

```
# Enable expiration (e.g., 90 days)
export JACS_MAX_SIGNATURE_AGE_SECONDS=7776000

# Default: no expiration (0)
export JACS_MAX_SIGNATURE_AGE_SECONDS=0
```

Protection Against Replay Attacks

The 5-minute future tolerance window:

- Allows for reasonable clock skew between systems
- Prevents attackers from creating signatures with future timestamps
- Ensures signatures cannot be pre-generated for later fraudulent use

```
{
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "signature": "...",
    "date": "2024-01-15T10:30:00Z"  // Must be within 5 min of verifier's clock
  }
}
```

Clock Synchronization

For reliable timestamp validation across distributed systems:

- Ensure all agents use NTP or similar time synchronization
- Monitor for clock drift in production environments
- Consider the 5-minute tolerance when debugging verification failures

Verification Claims

Agents can claim a verification level that determines security requirements. This follows the principle: **"If you claim it, you must prove it."**

Claim Levels

Claim	Required Conditions	Behavior
`unverified` (default)	None	Relaxed DNS/TLS settings allowed; self-asserted identity
`verified`	Domain with DNSSEC	Strict TLS, strict DNS with DNSSEC validation required
`verified-registry`	Above + registry verification	Must be registered and verified by a JACS registry
`verified-hai.ai` (legacy alias)	Same as `verified-registry`	Backward-compatible alias

Setting a Verification Claim

Add the `jacsVerificationClaim` field to your agent definition:

```
{
  "jacsAgentType": "ai",
  "jacsVerificationClaim": "verified",
  "jacsAgentDomain": "myagent.example.com",
  "jacsServices": [...]
}
```

Claim Enforcement

When an agent claims `verified`, `verified-registry`, or legacy `verified-hai.ai`:

1. **Domain Required**: The `jacsAgentDomain` field must be set
2. **Strict DNS**: DNS lookup uses DNSSEC validation (no insecure fallback)
3. **DNS Required**: Public key fingerprint must match DNS TXT record
4. **Strict TLS**: TLS certificate validation is mandatory (no self-signed certs)

For `verified-registry` (or legacy `verified-hai.ai`) claims, additional enforcement:

5. **Registry Registration**: Agent must be registered with the configured registry (for HAI-hosted registry, hai.ai)
6. **Public Key Match**: Registered public key must match the agent's key
7. **Network Required**: Verification fails if the registry API is unreachable

Backward Compatibility

- Agents without `jacsVerificationClaim` are treated as `unverified`
- Existing agents continue to work with their current DNS settings
- No breaking changes for agents that don't opt into verified status

Error Messages

If verification fails, clear error messages explain what's wrong:

```
Verification claim 'verified' failed: Verified agents must have jacsAgentDomain set.
Agents claiming 'verified' must meet the required security conditions.
```

```
Verification claim 'verified-registry' failed: Agent 'uuid' is not registered with the
configured registry.
Agents claiming 'verified-registry' must be registered with a reachable registry
endpoint.
```

Security Considerations

1. **No Downgrade**: Once an agent claims `verified`, it cannot be verified with relaxed settings
2. **Claim Changes**: Changing the claim requires creating a new agent version
3. **Network Dependency**: `verified-registry` requires network access to the registry endpoint
4. **Audit Trail**: Verification claim and enforcement results are logged

DNS-Based Verification

JACS supports DNSSEC-validated identity verification:

How It Works

1. Agent publishes public key fingerprint in DNS TXT record
2. Verifier queries DNS for `_v1.agent.jacs.<domain>.`
3. DNSSEC validates the response authenticity
4. Fingerprint is compared against agent's public key

Configuration

```
{
  "jacs_agent_domain": "myagent.example.com",
  "jacs_dns_validate": true,
  "jacs_dns_strict": true
}
```

Security Levels

Mode	Description
`jacs_dns_validate: false`	No DNS verification
`jacs_dns_validate: true`	Attempt DNS verification, allow fallback
`jacs_dns_strict: true`	Require DNSSEC validation
`jacs_dns_required: true`	Fail if domain not present

Trust Store Management

JACS maintains a trust store for managing trusted agent relationships.

Trusting Agents

Before trusting an agent, JACS performs public key hash verification:

```
# Trust an agent after verifying their public key hash
agent.trust_agent(agent_id, public_key_hash)
```

Untrusting Agents

The `untrust_agent()` method properly handles the case when an agent is not in the trust store:

```
try:
    agent.untrust_agent(agent_id)
except AgentNotTrusted as e:
    # Agent was not in the trust store
    print(f"Agent {agent_id} was not trusted: {e}")
```

Trust Store Security

Operation	Validation
`trust_agent()`	UUID format validation, path traversal rejection, public key hash verification, self-signature verification before adding
`untrust_agent()`	UUID format validation, path containment check, returns `AgentNotTrusted` error if agent not found
`get_trusted_agent()`	UUID format validation, path containment check
`is_trusted()`	UUID format validation, safe lookup without side effects
Key cache (`load_public_key_from_cache`)	`require_relative_path_safe()` rejects traversal in `publicKeyHash`
Key cache (`save_public_key_to_cache`)	`require_relative_path_safe()` rejects traversal in `publicKeyHash`

Path Traversal Protection (v0.6.0): All trust store operations that construct file paths from agent IDs or key hashes use defense-in-depth:

1. **UUID format validation**: Agent IDs must match `UUID:UUID` format (rejects special characters)
2. **Path character rejection**: Explicit rejection of `..`, `/`, `\`, and null bytes
3. **Path containment check**: For existing files, canonicalized paths are verified to stay within the trust store directory
4. `require_relative_path_safe()` : Key hashes are validated to prevent traversal before constructing cache file paths

Best Practices

1. **Verify Before Trust**: Always verify an agent's public key hash through an out-of-band channel before trusting
2. **Audit Trust Changes**: Log all trust store modifications for security auditing
3. **Periodic Review**: Regularly review and prune the trust store

Agreement Security

Multi-party agreements provide additional security:

Agreement Structure

```
{
  "jacsAgreement": {
    "agentIDs": ["agent-1", "agent-2", "agent-3"],
    "signatures": [
      {
        "agentID": "agent-1",
        "signature": "...",
        "responseType": "agree",
        "date": "2024-01-15T10:00:00Z"
      }
    ]
  },
  "jacsAgreementHash": "hash-at-agreement-time"
}
```

Agreement Guarantees

1. **Content Lock**: `jacsAgreementHash` ensures all parties agreed to same content
2. **Individual Consent**: Each signature records explicit agreement
3. **Response Types**: Support for agree, disagree, or reject
4. **Timestamp**: Records when each party signed

Request/Response Security

For MCP and HTTP communication:

Request Signing

```
signed_request = agent.sign_request({
    'method': 'tools/call',
    'params': {'name': 'echo', 'arguments': {'text': 'hello'}}
})
```

The signed request includes:

- Full JACS document structure
- Agent signature
- Timestamp
- Content hash

Response Verification

```
result = agent.verify_response(response_string)
payload = result.get('payload')
agent_id = result.get('agentId')  # Who signed the response
```

Algorithm Security

Supported Algorithms

Algorithm	Type	Security Level
`ring-Ed25519`	Elliptic Curve	High (recommended)
`RSA-PSS`	RSA	High
`pq-dilithium`	Post-Quantum	Quantum-resistant
`pq2025`	Composite	Transitional

Algorithm Selection

Choose based on requirements:

- **General Use**: `ring-Ed25519` - fast, secure, small signatures
- **Legacy Systems**: `RSA-PSS` - widely supported
- **Future-Proofing**: `pq-dilithium` - quantum-resistant
- **Transition**: `pq2025` - hybrid classical/post-quantum

Security Best Practices

1. Key Storage

```bash
# Never commit keys to version control
echo "jacs_keys/" >> .gitignore

# Secure file permissions
chmod 700 ./jacs_keys
chmod 600 ./jacs_keys/private.pem
```

2. Password Handling

```bash
# Use environment variables
export JACS_PRIVATE_KEY_PASSWORD="$(pass show jacs/key-password)"
```

3. Transport Security

Always use TLS for network communication:

```bash
# HTTPS for web transport
client = JACSMCPClient("https://localhost:8000/sse")  # Good
# client = JACSMCPClient("http://localhost:8000/sse")  # Avoid in production
```

4. Verification Policies

```json
{
  "jacs_dns_strict": true,
  "jacs_dns_required": true,
  "jacs_enable_filesystem_quarantine": "true"
}
```

5. Audit Logging

Enable observability for security auditing:

```
{
  "observability": {
    "logs": {
      "enabled": true,
      "level": "info"
    }
  }
}
```

Security Checklist

Development

- [ ] Generate unique keys for each environment
- [ ] Never commit private keys
- [ ] Use test keys separate from production

Production

- [ ] Encrypt private keys at rest
- [ ] Use environment variables for secrets (never store `jacs_private_key_password` in config)
- [ ] Enable DNS verification
- [ ] Configure strict security mode
- [ ] Enable audit logging
- [ ] Use TLS for all network transport
- [ ] Restrict key file permissions (0600 for keys, 0700 for key directory)
- [ ] Implement key rotation policy
- [ ] Set `JACS_STRICT_TLS=true` for certificate validation
- [ ] Use strong passwords (28+ bit entropy, 35+ for single character class)
- [ ] Enable signature timestamp validation
- [ ] Verify public key hashes before trusting agents
- [ ] Run `cargo audit` / `npm audit` / `pip audit` regularly for dependency vulnerabilities

Verification

- [ ] Always verify documents before trusting
- [ ] Verify agent signatures
- [ ] Check agreement completeness
- [ ] Validate DNS records when required

Security Considerations

Supply Chain

- Verify JACS packages are from official sources
- Use package checksums
- Keep dependencies updated

Side Channels

- Use constant-time comparison for signatures
- Protect against timing attacks
- Secure memory handling for keys

Recovery

- Backup key material securely
- Document key recovery procedures
- Plan for key compromise scenarios

Troubleshooting Verification Claims

Common Issues and Solutions

"Verified agents must have jacsAgentDomain set"

Problem: You set `jacsVerificationClaim` to `verified` but didn't specify a domain.

Solution: Either add a domain or use unverified:

```
// Option 1: Add a domain (recommended for production)
{
  "jacsVerificationClaim": "verified",
  "jacsAgentDomain": "myagent.example.com"
}

// Option 2: Use unverified if DNS verification isn't needed
{
  "jacsVerificationClaim": "unverified"
}
```

"Agent is not registered with the registry"

Problem: You're using `verified-registry` (or legacy `verified-hai.ai`) but the agent isn't registered.

Solution:

1. Register your agent with your configured registry (for HAI-hosted registry, hai.ai)
2. Or use `verified` for DNS-only verification:

```
{
  "jacsVerificationClaim": "verified",
  "jacsAgentDomain": "myagent.example.com"
}
```

"Cannot downgrade from 'verified' to 'unverified'"

Problem: You're trying to change an existing agent's claim to a lower level.

Solution: Verification claims cannot be downgraded for security. Options:

1. Keep the current claim level
2. Create a new agent with the desired claim level
3. If this is a test/development scenario, start fresh

```
# Create a new agent instead
jacs create --type ai --claim unverified
```

"DNS fingerprint mismatch"

Problem: The public key hash in DNS doesn't match your agent's key.

Solution:

1. Regenerate the DNS record with your current keys:

   ```
   jacs dns-record
   ```

2. Update your DNS TXT record with the new value
3. Wait for DNS propagation (can take up to 48 hours)

"Strict DNSSEC validation failed"

Problem: Your domain doesn't have DNSSEC enabled.

Solution:

1. Enable DNSSEC with your domain registrar
2. Publish DS records at the parent zone
3. Or use `verified` with non-strict DNS (development only)

Claim Level Reference

Claim	Security Level	Requirements
`unverified`	0 (lowest)	None - self-asserted identity
`verified`	1	Domain + DNS TXT record + DNSSEC
`verified-registry`	2 (highest)	Above + registry registration
`verified-hai.ai` (legacy alias)	2 (highest)	Alias of `verified-registry`

Upgrade vs Downgrade Rules

- **Upgrades allowed**: `unverified` → `verified` → `verified-registry` (legacy alias `verified-hai.ai` is same level)
- **Downgrades blocked**: Cannot go from higher to lower claim
- **Same level allowed**: Can update agent while keeping same claim

Quick Diagnostic Commands

```
# Check your agent's current claim
jacs info | grep jacsVerificationClaim

# Verify DNS record is correct
jacs dns-check

# Test verification
jacs verify --agent your-agent-id:version
```

See Also

- Cryptographic Algorithms - Algorithm details
- DNS Verification - DNS-based identity
- Configuration - Security configuration
- Agreements - Multi-party agreements

Key Rotation

Key rotation is the process of replacing an agent's cryptographic keys while preserving the ability to verify documents signed with previous keys. JACS implements version-aware key management to support secure key lifecycle operations.

Why Key Rotation Matters

Key Compromise Recovery

When a private key is compromised, the agent must be able to:

- Generate new keys and continue operating
- Revoke trust in the compromised key
- Maintain verifiability of documents signed before the compromise

Cryptographic Agility

Cryptographic algorithms evolve. Key rotation enables:

- Migration from older algorithms to newer ones
- Transition to post-quantum cryptography when needed
- Algorithm upgrades without breaking existing signatures

Compliance Requirements

Many security standards require periodic key rotation:

- PCI-DSS mandates regular key changes
- SOC 2 requires key management policies
- NIST guidelines recommend rotation schedules

Agent Versioning

JACS uses a versioned identity model where each key rotation creates a new agent version.

Version Format

Agent identifiers follow the format: `{agent_id}:{version_uuid}`

- **jacsId**: The stable agent identity (UUID v4) - never changes
- **jacsVersion**: Current version UUID - changes on each update
- **jacsPreviousVersion**: Links to the prior version
- **jacsOriginalVersion**: The first version ever created

```
{
  "jacsId": "550e8400-e29b-41d4-a716-446655440000",
  "jacsVersion": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "jacsPreviousVersion": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
  "jacsOriginalVersion": "6ba7b810-9dad-11d1-80b4-00c04fd430c8"
}
```

Version Chain

Each version forms a linked chain back to the original:

```
Original (v1) <-- Previous (v2) <-- Current (v3)
   |                  |                   |
 key-A              key-B               key-C
```

This chain provides an audit trail of all key changes and allows verification of any version.

Version-Aware Verification

The critical insight enabling key rotation is that signatures contain both the agent ID and the version that created them.

Signature Structure

```
{
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "agentVersion": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
    "publicKeyHash": "sha256-of-public-key-A",
    "signingAlgorithm": "ring-Ed25519",
    "signature": "base64-encoded-signature",
    "date": "2024-01-15T10:00:00Z"
  }
}
```

Key Resolution Process

When verifying a signature:

1. Extract `agentVersion` and `publicKeyHash` from the signature
2. Look up the public key that was active for that version
3. Verify the signature using that historical key

```
// Pseudocode for version-aware verification
fn verify_signature(doc: &Document) -> Result<()> {
    let sig = &doc.jacs_signature;

    // Find the key that was active for this version
    let public_key = resolve_key_for_version(
        &sig.agent_id,
        &sig.agent_version,
        &sig.public_key_hash,
    )?;

    // Verify with the historical key
    verify_with_key(&doc, &sig, &public_key)
}
```

Key Lookup Priority

The verification system tries multiple sources:

1. **Local cache by hash** - Fastest, key already stored locally
2. **Trust store by version** - Most accurate for known agents
3. **Trust store by hash** - Fallback for legacy entries
4. **DNS lookup** - External verification, authoritative
5. **Fail** - Key not found, verification impossible

Key Rotation Process

Step-by-Step Rotation

1. **Generate new key pair** with the desired algorithm
2. **Create new agent version** with updated key information
3. **Sign new version with old key** (transition signature)
4. **Update DNS records** to include new key fingerprint
5. **Store old public key** for future verifications

Transition Signature

The transition signature proves the key rotation was authorized by the holder of the old key:

```
JACS_KEY_ROTATION:{agent_id}:{old_key_hash}:{new_key_hash}:{timestamp}
```

This signed message:

- Proves continuity of ownership
- Provides an audit trail
- Binds old and new keys together cryptographically

CLI Commands (Planned)

> **Note**: These CLI commands are planned for a future release. Currently, key rotation must be performed programmatically using the Rust API.

```
# Rotate keys with default algorithm (Coming Soon)
jacs agent rotate-keys

# Rotate to post-quantum algorithm (Coming Soon)
jacs agent rotate-keys --algorithm pq2025

# List key history (Coming Soon)
jacs agent keys list

# Revoke a compromised key (Coming Soon)
jacs agent keys revoke <key-hash>
```

Example Rotation Flow

```
Time T0: Agent created
  - jacsId: "abc-123"
  - jacsVersion: "v1-uuid"
  - jacsCurrentKeyHash: "hash-A"

Time T1: Agent signs document D1
  - D1.jacsSignature.agentVersion: "v1-uuid"
  - D1.jacsSignature.publicKeyHash: "hash-A"

Time T2: Key rotation
  - New keys generated with hash-B
  - jacsVersion: "v2-uuid"
  - jacsKeyHistory: [{ hash: "hash-A", status: "rotated" }]
  - jacsCurrentKeyHash: "hash-B"

Time T3: Verify D1
  - Extract agentVersion "v1-uuid" and hash "hash-A"
  - Look up key: find "hash-A" with status "rotated"
  - Verification succeeds (old key still valid for old docs)

Time T4: Agent signs document D2
  - D2.jacsSignature.agentVersion: "v2-uuid"
  - D2.jacsSignature.publicKeyHash: "hash-B"
```

Trust Store with Version History

The trust store maintains a history of all public keys for each trusted agent.

TrustedAgent Structure

```
{
  "agent_id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Agent",
  "trusted_at": "2024-01-15T10:00:00Z",
  "current_key_hash": "abc123...",
  "domain": "agent.example.com",
  "key_history": [
    {
      "public_key_hash": "xyz789...",
      "public_key_pem": "-----BEGIN PUBLIC KEY-----\n...",
      "signing_algorithm": "ring-Ed25519",
      "trusted_at": "2024-01-01T00:00:00Z",
      "first_version": "11111111-1111-1111-1111-111111111111",
      "last_version": "22222222-2222-2222-2222-222222222222",
      "status": "rotated"
    },
    {
      "public_key_hash": "abc123...",
      "public_key_pem": "-----BEGIN PUBLIC KEY-----\n...",
      "signing_algorithm": "ring-Ed25519",
      "trusted_at": "2024-01-15T10:00:00Z",
      "first_version": "33333333-3333-3333-3333-333333333333",
      "last_version": null,
      "status": "active"
    }
  ]
}
```

Key Status Values

Status	Description
`active`	Currently in use for signing
`rotated`	Superseded by newer key, still valid for old signatures
`revoked`	Compromised, signatures should not be trusted
`expired`	Past validity period

Looking Up Keys

```rust
impl TrustedAgent {
    /// Get the public key that was active for a specific agent version
    fn get_key_for_version(&self, version: &str) -> Option<&KeyEntry> {
        self.key_history.iter().find(|entry| {
            match (&entry.first_version, &entry.last_version) {
                (Some(first), Some(last)) => {
                    version >= first && version <= last
                }
                (Some(first), None) => {
                    version >= first  // Current key
                }
                _ => false
            }
        })
    }

    /// Get the public key by its hash
    fn get_key_by_hash(&self, hash: &str) -> Option<&KeyEntry> {
        self.key_history.iter().find(|e| e.public_key_hash == hash)
    }
}
```

DNS Support for Key Versions

DNS records can advertise multiple key versions for an agent.

Multi-Version DNS Records

Each key version gets its own TXT record:

```
; Current key
_v1.agent.jacs.example.com. 3600 IN TXT "v=hai.ai; jacs_agent_id={id}; ver=current;
alg=SHA-256; hash={hash1}"

; Previous key (still valid for old signatures)
_v1.agent.jacs.example.com. 3600 IN TXT "v=hai.ai; jacs_agent_id={id}; ver=rotated;
valid_until=2025-01-15; hash={hash2}"
```

DNS Record Generation

```bash
# Generate DNS records for all active keys
jacs agent dns --all-keys
```

Security Considerations

Key Revocation

When a key is compromised:

1. **Mark as revoked** in the agent's key history
2. **Update DNS** to include revocation status
3. **Signatures fail verification** when made with revoked keys
4. **Notify trusted peers** if possible

Overlap Period

During rotation, both old and new keys may be valid:

- New documents should be signed with the new key
- Old documents remain verifiable with the old key
- DNS may advertise both keys during transition

Secure Deletion

After rotation:

- Old private keys should be securely deleted
- Only public keys are retained for verification
- Key metadata must be protected from modification

Best Practices

Rotation Schedule

- **Regular rotation**: Quarterly or annually for compliance
- **Algorithm upgrade**: When transitioning to stronger cryptography
- **Incident response**: Immediately after suspected compromise

Pre-Rotation Checklist

- [ ] Backup current agent state
- [ ] Verify all systems can handle new key format

- [ ] Plan DNS propagation time
- [ ] Notify dependent systems of upcoming change

Post-Rotation Checklist

- [ ] Verify new key is active
- [ ] Confirm old documents still verify
- [ ] Update DNS records
- [ ] Securely delete old private key
- [ ] Test signing with new key

See Also

- Security Model - Overall security architecture
- Cryptographic Algorithms - Algorithm details
- DNS Verification - DNS-based identity verification

Cryptographic Algorithms

JACS supports multiple cryptographic algorithms for digital signatures, providing flexibility for different security requirements and future-proofing against quantum computing threats.

Supported Algorithms

Algorithm	Config Value	Type	Key Size	Signature Size	Recommended Use
Ed25519	`ring-Ed25519`	Elliptic Curve	32 bytes	64 bytes	General purpose (default)
RSA-PSS	`RSA-PSS`	RSA	2048-4096 bits	256-512 bytes	Legacy systems
Dilithium	`pq-dilithium`	Lattice-based	~1.3 KB	~2.4 KB	Post-quantum
PQ2025	`pq2025`	Hybrid	~1.3 KB	~2.5 KB	Transitional

Ed25519 (ring-Ed25519)

The recommended algorithm for most use cases.

Overview

Ed25519 is an elliptic curve signature scheme using Curve25519. JACS uses the `ring` cryptographic library implementation.

Characteristics

- **Speed**: Extremely fast signing and verification
- **Key Size**: 32-byte private key, 32-byte public key
- **Signature Size**: 64 bytes
- **Security Level**: ~128 bits (classical)

Configuration

```
{
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

Use Cases

- General agent communication
- MCP message signing
- HTTP request/response signing
- Document signing

Example

```python
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')  # Using ring-Ed25519

# Sign a message
signature = agent.sign_string("Hello, World!")
print(f"Signature (64 bytes): {len(signature)} characters base64")
```

RSA-PSS

Industry-standard RSA with Probabilistic Signature Scheme padding.

Overview

RSA-PSS provides compatibility with systems that require RSA signatures. JACS uses 2048-bit or larger keys.

Characteristics

- **Speed**: Slower than Ed25519
- **Key Size**: 2048-4096 bits
- **Signature Size**: Same as key size (256-512 bytes)
- **Security Level**: ~112-128 bits (2048-bit key)

Configuration

```
{
  "jacs_agent_key_algorithm": "RSA-PSS"
}
```

Use Cases

- Integration with legacy systems
- Compliance requirements mandating RSA
- Interoperability with enterprise PKI

Considerations

- Larger signatures increase document size
- Slower than Ed25519
- Larger keys needed for equivalent security

Dilithium (pq-dilithium)

NIST-standardized post-quantum digital signature algorithm.

Overview

Dilithium is a lattice-based signature scheme selected by NIST for post-quantum cryptography standardization. It provides security against both classical and quantum computers.

Characteristics

- **Speed**: Moderate (faster than RSA, slower than Ed25519)
- **Key Size**: ~1.3 KB public key, ~2.5 KB private key
- **Signature Size**: ~2.4 KB
- **Security Level**: NIST Level 3 (quantum-resistant)

Configuration

```
{
  "jacs_agent_key_algorithm": "pq-dilithium"
}
```

Use Cases

- Long-term document security
- Protection against future quantum attacks
- High-security applications
- Government/defense requirements

Considerations

- Larger signatures and keys than classical algorithms
- Newer algorithm (less battle-tested)
- May be required for future compliance

PQ2025 (Hybrid)

Transitional hybrid scheme combining classical and post-quantum algorithms.

Overview

PQ2025 combines Ed25519 with Dilithium, providing security even if one algorithm is broken. This approach is recommended by security researchers during the quantum transition period.

Characteristics

- **Speed**: Slower (two signatures computed)
- **Key Size**: Combined Ed25519 + Dilithium
- **Signature Size**: ~2.5 KB (combined)
- **Security Level**: Max of both algorithms

Configuration

```
{
  "jacs_agent_key_algorithm": "pq2025"
}
```

Use Cases

- Transitioning to post-quantum
- Maximum security requirements
- Uncertainty about algorithm security
- Long-lived documents

Considerations

- Largest signatures
- Slowest signing/verification
- Best for paranoid security requirements

Algorithm Selection Guide

Decision Matrix

Requirement	Recommended Algorithm
Best performance	`ring-Ed25519`
Smallest signatures	`ring-Ed25519`
Legacy compatibility	`RSA-PSS`
Quantum resistance	`pq-dilithium`
Maximum security	`pq2025`
General purpose	`ring-Ed25519`

By Use Case

Web APIs and MCP:

```
{
  "jacs_agent_key_algorithm": "ring-Ed25519"
}
```

Fast signing is critical for real-time communication.

Legal/Financial Documents:

```
{
  "jacs_agent_key_algorithm": "pq-dilithium"
}
```

Long-term validity requires quantum resistance.

Enterprise Integration:

```
{
  "jacs_agent_key_algorithm": "RSA-PSS"
}
```

Compatibility with existing PKI infrastructure.

High-Security:

```
{
  "jacs_agent_key_algorithm": "pq2025"
}
```

Belt-and-suspenders approach for maximum protection.

Key Generation

Keys are generated automatically when creating an agent:

```
# Directory structure after agent creation
jacs_keys/
├── private.pem    # Algorithm-specific private key
└── public.pem     # Algorithm-specific public key
```

Key Formats

Algorithm	Private Key Format	Public Key Format
ring-Ed25519	PEM (PKCS#8)	PEM (SPKI)
RSA-PSS	PEM (PKCS#8)	PEM (SPKI)
pq-dilithium	PEM (custom)	PEM (custom)
pq2025	PEM (combined)	PEM (combined)

Signature Structure

Signatures in JACS documents include algorithm metadata:

```
{
  "jacsSignature": {
    "agentID": "550e8400-e29b-41d4-a716-446655440000",
    "agentVersion": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "date": "2024-01-15T10:30:00Z",
    "signature": "base64-encoded-signature",
    "publicKeyHash": "sha256-of-public-key",
    "signingAlgorithm": "ring-Ed25519",
    "fields": ["jacsId", "jacsVersion", "content"]
  }
}
```

The `signingAlgorithm` field enables verifiers to use the correct verification method.

Hashing

JACS uses SHA-256 for all hash operations:

- Document content hashing (`jacsSha256`)
- Public key fingerprints (`publicKeyHash`)
- Agreement content locking (`jacsAgreementHash`)

```
{
  "jacsSha256": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
}
```

Algorithm Migration

To migrate to a new algorithm:

1. **Generate New Keys**

   ```
   {
     "jacs_agent_key_algorithm": "pq-dilithium"
   }
   ```

2. **Create New Agent Version**

```
# Load with old algorithm
agent.load('./old-config.json')

# Update to new algorithm and generate new version
new_agent = agent.update_agent(json.dumps({
    # ... agent data with new keys
}))
```

3. **Update Configuration**

```
{
  "jacs_agent_id_and_version": "agent-id:new-version",
  "jacs_agent_key_algorithm": "pq-dilithium"
}
```

4. **Maintain Backward Compatibility**

   - Keep old agent versions for verifying old documents
   - Old signatures remain valid with old public keys

Performance Comparison

Approximate performance (varies by hardware):

Algorithm	Sign (ops/sec)	Verify (ops/sec)	Key Gen (ms)
ring-Ed25519	~50,000	~20,000	<1
RSA-PSS (2048)	~1,000	~30,000	~100
pq-dilithium	~5,000	~10,000	~1
pq2025	~4,000	~8,000	~2

Security Considerations

Algorithm Agility

JACS documents include the signing algorithm, enabling:

- Verification with correct algorithm
- Graceful algorithm transitions
- Multi-algorithm environments

Forward Secrecy

Signatures don't provide forward secrecy. For confidentiality:

- Use TLS for transport
- Consider additional encryption layers

Key Compromise

If a private key is compromised:

1. Generate new key pair
2. Create new agent version
3. Revoke trust in compromised version
4. Re-sign critical documents

See Also

- Security Model - Overall security architecture
- Configuration - Algorithm configuration
- DNS Verification - Public key fingerprint verification

Algorithm Selection Guide

Choosing the right signing algorithm affects key size, signature size, verification speed, and compliance posture. This guide helps you pick the right one.

Supported Algorithms

Algorithm	Config Value	Public Key	Signature	Best For
Ed25519	`ring-Ed25519`	32 bytes	64 bytes	Speed, small signatures
RSA-PSS	`RSA-PSS`	~550 bytes (4096-bit)	~512 bytes	Broad compatibility
ML-DSA-87	`pq2025`	2,592 bytes	4,627 bytes	Post-quantum compliance (FIPS-204)
Dilithium	`pq-dilithium`	>1,000 bytes	~3,293-4,644 bytes	**Deprecated** -- use `pq2025`

How to Choose

```
Do you need FIPS/NIST post-quantum compliance?
  ├── Yes → pq2025
  └── No
      ├── Need maximum interop with existing PKI/TLS systems? → RSA-PSS
      └── Need speed and small payloads? → ring-Ed25519
```

Default recommendation for new projects: `pq2025`

Ed25519 and RSA-PSS are well-understood and widely deployed, but neither is quantum-resistant. If you don't have a specific reason to choose one of them, start with `pq2025` so you don't have to migrate later.

When to Choose Post-Quantum

Choose `pq2025` (ML-DSA-87, FIPS-204) when:

- Your compliance team asks about quantum readiness
- Government or defense contracts require FIPS-204

- You need long-lived signatures that must remain valid for 10+ years
- You want to avoid a future algorithm migration

JACS supports ML-DSA-87 (FIPS-204) for post-quantum digital signatures. When your compliance team asks about quantum readiness, JACS already has the answer.

The tradeoff is size: ML-DSA-87 public keys are 2,592 bytes and signatures are 4,627 bytes -- roughly 80x larger than Ed25519. For most applications this is negligible, but if you're signing millions of small messages and bandwidth matters, consider Ed25519.

Cross-Algorithm Verification

JACS verification works across algorithms. An agreement can contain signatures from RSA, Ed25519, and ML-DSA agents and all verify correctly. This heterogeneous verification is important for cross-organization scenarios where different parties chose different algorithms.

Each agent uses one algorithm (chosen at creation time), but can **verify** signatures from all supported algorithms.

Configuration

Set the algorithm in your `jacs.config.json`:

```
{
  "jacs_agent_key_algorithm": "pq2025"
}
```

Or via environment variable:

```
export JACS_AGENT_KEY_ALGORITHM=pq2025
```

Valid values: `ring-Ed25519`, `RSA-PSS`, `pq2025`

In Python and Node.js, pass the algorithm to `quickstart(...)`:

```
from jacs.client import JacsClient
client = JacsClient.quickstart(
    name="algo-agent",
    domain="algo.example.com",
    algorithm="pq2025",
)
```

```
import { JacsClient } from "@hai.ai/jacs";
const client = await JacsClient.quickstart({
  name: "algo-agent",
  domain: "algo.example.com",
  algorithm: "pq2025",
});
```

Current Limitations

- Each agent uses one algorithm, chosen at creation time. You cannot change an agent's algorithm after creation.
- Algorithm negotiation between agents is planned but not yet implemented.
- `pq-dilithium` is deprecated in favor of `pq2025` (ML-DSA-87). Use `pq2025` for new agents and verification hints.

Storage Backends

JACS has two storage layers today:

- Low-level file/object storage via `MultiStorage`
- Signed document CRUD/search via `DocumentService`

Those are related, but they are not identical. The most important rule is the signed-document contract:

- Every `DocumentService` read verifies the stored JACS document before returning it.
- Every `create()` and `update()` verifies the signed document before persisting it.
- If an update payload changes an already-signed JACS document without re-signing it, the write fails.
- Visibility changes create a new signed version instead of mutating metadata in place.

Built-in Core Backends

Backend	Config Value	Core Surface	Notes
Filesystem	`fs`	`MultiStorage` + `DocumentService`	Default. Signed JSON files on disk.
Local indexed SQLite	`rusqlite`	`DocumentService` + `SearchProvider`	Stores signed documents in a local SQLite DB with FTS search.
AWS object storage	`aws`	`MultiStorage`	Object-store backend.
Memory	`memory`	`MultiStorage`	Non-persistent, useful for tests and temporary flows.
Browser local storage	`local`	`MultiStorage`	WASM-only.

For local indexed document search in JACS core, use `rusqlite`.

Filesystem (`fs`)

Filesystem is the default signed-document backend.

```
{
  "jacs_default_storage": "fs",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys"
}
```

Typical layout:

```
jacs_data/
├── agent/
│   └── {agent-id}:{agent-version}.json
└── documents/
    ├── {document-id}:{version}.json
    └── archive/
```

Use filesystem when you want the simplest possible deployment, inspectable files, and no local database dependency.

Local Indexed SQLite (`rusqlite`)

`rusqlite` is the built-in indexed document backend used by the upgraded bindings and MCP search path.

```
{
  "jacs_default_storage": "rusqlite",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys"
}
```

With this setting:

- Signed documents are stored in `./jacs_data/jacs_documents.sqlite3`
- Full-text search comes from SQLite FTS
- `DocumentService` reads and writes enforce verification
- Updating visibility creates a new signed successor version

Use `rusqlite` when you want local full-text search, filtered document queries, and a single-machine deployment.

AWS (`aws`)

AWS support is an object-store backend for lower-level storage operations.

```
{
  "jacs_default_storage": "aws"
}
```

Required environment variables:

```
export JACS_ENABLE_AWS_BUCKET_NAME="my-jacs-bucket"
export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_REGION="us-west-2"
```

Use `aws` when you need remote object storage. If you also need a richer signed-document query surface, use one of the database-focused crates below.

Memory (`memory`)

Memory storage is non-persistent:

```
{
  "jacs_default_storage": "memory"
}
```

Use it for tests, temporary operations, and ephemeral agent flows.

Extracted Backend Crates

Several richer database backends now live outside the JACS core crate:

- `jacs-postgresql`
- `jacs-duckdb`
- `jacs-redb`
- `jacs-surrealdb`

These crates implement the same storage/search traits in their own packages. They are not built-in `jacs_default_storage` values for the core crate.

Choosing a Backend

Scenario	Recommendation
Default local usage	`fs`

Scenario	Recommendation
Local search + filtering	`rusqlite`
Ephemeral tests	`memory`
Remote object storage	`aws`
Postgres / vector / multi-model needs	Use an extracted backend crate

Migration Notes

Switching backends does not migrate data automatically.

When you change `jacs_default_storage`:

1. Export the signed documents you need to keep.
2. Update the config value.
3. Create/import the new backend's data store.
4. Re-run verification on imported documents as part of migration validation.

Custom Schemas

JACS allows you to define custom document schemas that extend the base header schema, enabling type-safe, validated documents for your specific use cases.

Overview

Custom schemas:

- Inherit all JACS header fields (jacsId, jacsVersion, jacsSignature, etc.)
- Add domain-specific fields with validation
- Enable IDE autocompletion and type checking
- Ensure document consistency across your application

Creating a Custom Schema

Basic Structure

Custom schemas extend the JACS header using `allOf`:

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://example.com/schemas/invoice.schema.json",
  "title": "Invoice",
  "description": "Invoice document with JACS signing",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "invoiceNumber": {
          "type": "string",
          "description": "Unique invoice identifier"
        },
        "amount": {
          "type": "number",
          "minimum": 0
        },
        "currency": {
          "type": "string",
          "enum": ["USD", "EUR", "GBP"]
        }
      },
      "required": ["invoiceNumber", "amount"]
    }
  ]
}
```

Step-by-Step Guide

1. **Create the schema file**

```
// schemas/order.schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://mycompany.com/schemas/order.schema.json",
  "title": "Order",
  "description": "E-commerce order document",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "orderId": {
          "type": "string",
          "pattern": "^ORD-[0-9]{6}$"
        },
        "customer": {
          "type": "object",
          "properties": {
            "name": { "type": "string" },
            "email": { "type": "string", "format": "email" }
          },
          "required": ["name", "email"]
        },
        "items": {
          "type": "array",
          "minItems": 1,
          "items": {
            "type": "object",
            "properties": {
              "sku": { "type": "string" },
              "quantity": { "type": "integer", "minimum": 1 },
              "price": { "type": "number", "minimum": 0 }
            },
            "required": ["sku", "quantity", "price"]
          }
        },
        "total": {
          "type": "number",
          "minimum": 0
        },
        "status": {
          "type": "string",
          "enum": ["pending", "processing", "shipped", "delivered", "cancelled"]
        }
      },
      "required": ["orderId", "customer", "items", "total", "status"]
    }
  ]
}
```

2. **Use the schema when creating documents**

```python
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

order = agent.create_document(
    json.dumps({
        'orderId': 'ORD-123456',
        'customer': {
            'name': 'Jane Smith',
            'email': 'jane@example.com'
        },
        'items': [
            {'sku': 'WIDGET-001', 'quantity': 2, 'price': 29.99}
        ],
        'total': 59.98,
        'status': 'pending'
    }),
    custom_schema='./schemas/order.schema.json'
)
```

```typescript
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
agent.load('./jacs.config.json');

const order = agent.createDocument(
  JSON.stringify({
    orderId: 'ORD-123456',
    customer: {
      name: 'Jane Smith',
      email: 'jane@example.com'
    },
    items: [
      { sku: 'WIDGET-001', quantity: 2, price: 29.99 }
    ],
    total: 59.98,
    status: 'pending'
  }),
  './schemas/order.schema.json'
);
```

Schema Best Practices

Use Meaningful IDs

```json
{
  "$id": "https://mycompany.com/schemas/v1/order.schema.json"
}
```

Include version in the path for schema evolution.

Document Everything

```
{
  "properties": {
    "status": {
      "type": "string",
      "description": "Current order status in the fulfillment workflow",
      "enum": ["pending", "processing", "shipped", "delivered", "cancelled"]
    }
  }
}
```

Use Appropriate Validation

```
{
  "properties": {
    "email": {
      "type": "string",
      "format": "email"
    },
    "phone": {
      "type": "string",
      "pattern": "^\\+?[1-9]\\d{1,14}$"
    },
    "quantity": {
      "type": "integer",
      "minimum": 1,
      "maximum": 1000
    }
  }
}
```

Group Related Fields

```
{
  "properties": {
    "shipping": {
      "type": "object",
      "properties": {
        "address": { "type": "string" },
        "city": { "type": "string" },
        "country": { "type": "string" },
        "postalCode": { "type": "string" }
      }
    },
    "billing": {
      "type": "object",
      "properties": {
        "address": { "type": "string" },
        "city": { "type": "string" },
        "country": { "type": "string" },
        "postalCode": { "type": "string" }
      }
    }
  }
}
```

Advanced Schema Features

Conditional Validation

Different requirements based on field values:

```
{
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "paymentMethod": {
          "type": "string",
          "enum": ["credit_card", "bank_transfer", "crypto"]
        }
      }
    }
  ],
  "if": {
    "properties": {
      "paymentMethod": { "const": "credit_card" }
    }
  },
  "then": {
    "properties": {
      "cardLastFour": {
        "type": "string",
        "pattern": "^[0-9]{4}$"
      }
    },
    "required": ["cardLastFour"]
  }
}
```

Reusable Definitions

```
{
  "$defs": {
    "address": {
      "type": "object",
      "properties": {
        "street": { "type": "string" },
        "city": { "type": "string" },
        "country": { "type": "string" },
        "postalCode": { "type": "string" }
      },
      "required": ["street", "city", "country"]
    }
  },
  "properties": {
    "shippingAddress": { "$ref": "#/$defs/address" },
    "billingAddress": { "$ref": "#/$defs/address" }
  }
}
```

Array Constraints

```
{
  "properties": {
    "tags": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1,
      "maxItems": 10,
      "uniqueItems": true
    }
  }
}
```

Pattern Properties

For dynamic field names:

```
{
  "properties": {
    "metadata": {
      "type": "object",
      "patternProperties": {
        "^x-": { "type": "string" }
      },
      "additionalProperties": false
    }
  }
}
```

Schema Inheritance

Extending Custom Schemas

Create schema hierarchies:

```
// schemas/base-transaction.schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://example.com/schemas/base-transaction.schema.json",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "transactionId": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" },
        "amount": { "type": "number" }
      },
      "required": ["transactionId", "timestamp", "amount"]
    }
  ]
}


// schemas/payment.schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://example.com/schemas/payment.schema.json",
  "allOf": [
    { "$ref": "https://example.com/schemas/base-transaction.schema.json" },
    {
      "type": "object",
      "properties": {
        "paymentMethod": { "type": "string" },
        "processorId": { "type": "string" }
      },
      "required": ["paymentMethod"]
    }
  ]
}
```

Validation

Python Validation

```python
import jacs
import json

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

try:
    # This will fail validation - missing required field
    doc = agent.create_document(
        json.dumps({
            'orderId': 'ORD-123456'
            # Missing: customer, items, total, status
        }),
        custom_schema='./schemas/order.schema.json'
    )
except Exception as e:
    print(f"Validation failed: {e}")
```

Node.js Validation

```node
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
agent.load('./jacs.config.json');

try {
  // This will fail - invalid enum value
  const doc = agent.createDocument(
    JSON.stringify({
      orderId: 'ORD-123456',
      customer: { name: 'Jane', email: 'jane@example.com' },
      items: [{ sku: 'A', quantity: 1, price: 10 }],
      total: 10,
      status: 'invalid_status'  // Not in enum
    }),
    './schemas/order.schema.json'
  );
} catch (error) {
  console.error('Validation failed:', error.message);
}
```

Example Schemas

Medical Record

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://healthcare.example.com/schemas/medical-record.schema.json",
  "title": "Medical Record",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "patientId": { "type": "string" },
        "recordType": {
          "type": "string",
          "enum": ["visit", "lab_result", "prescription", "diagnosis"]
        },
        "provider": {
          "type": "object",
          "properties": {
            "name": { "type": "string" },
            "npi": { "type": "string", "pattern": "^[0-9]{10}$" }
          }
        },
        "date": { "type": "string", "format": "date" },
        "notes": { "type": "string" },
        "confidential": { "type": "boolean", "default": true }
      },
      "required": ["patientId", "recordType", "provider", "date"]
    }
  ]
}
```

Legal Contract

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://legal.example.com/schemas/contract.schema.json",
  "title": "Legal Contract",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "contractNumber": { "type": "string" },
        "parties": {
          "type": "array",
          "minItems": 2,
          "items": {
            "type": "object",
            "properties": {
              "name": { "type": "string" },
              "role": { "type": "string" },
              "agentId": { "type": "string", "format": "uuid" }
            }
          }
        },
        "effectiveDate": { "type": "string", "format": "date" },
        "expirationDate": { "type": "string", "format": "date" },
        "terms": { "type": "string" },
        "jurisdiction": { "type": "string" }
      },
      "required": ["contractNumber", "parties", "effectiveDate", "terms"]
    }
  ]
}
```

IoT Sensor Reading

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://iot.example.com/schemas/sensor-reading.schema.json",
  "title": "Sensor Reading",
  "allOf": [
    { "$ref": "https://hai.ai/schemas/header/v1/header.schema.json" },
    {
      "type": "object",
      "properties": {
        "deviceId": { "type": "string" },
        "sensorType": {
          "type": "string",
          "enum": ["temperature", "humidity", "pressure", "motion"]
        },
        "value": { "type": "number" },
        "unit": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" },
        "location": {
          "type": "object",
          "properties": {
            "latitude": { "type": "number" },
            "longitude": { "type": "number" }
          }
        }
      },
      "required": ["deviceId", "sensorType", "value", "timestamp"]
    }
  ]
}
```

Schema Versioning

Version in Path

```
{
  "$id": "https://example.com/schemas/v1/order.schema.json"
}
```

Version Field

```
{
  "properties": {
    "schemaVersion": {
      "type": "string",
      "const": "1.0.0"
    }
  }
}
```

Migration Strategy

1. Create new schema version
2. Update application to support both versions
3. Migrate existing documents
4. Deprecate old version

See Also

- JSON Schemas Overview - Built-in schemas
- Document Schema - Header fields
- Configuration - Schema configuration

Trust Store Operations

The JACS trust store is a local directory of agent public keys and metadata that your agent has explicitly chosen to trust. It enables offline signature verification without a central authority -- once you trust an agent, you can verify its signatures without network access.

How it works

When you add an agent to your trust store, JACS:

1. Parses the agent's JSON document
2. Extracts the public key and verifies the agent's self-signature
3. Saves the agent document, public key, and metadata to `~/.jacs/trust_store/`

After that, any document signed by that agent can be verified locally using the cached public key.

API

All bindings expose five trust store functions:

Function	Description
`trust_agent(agent_json)`	Add an agent to the trust store (verifies self-signature first)
`list_trusted_agents()`	List all trusted agent IDs
`is_trusted(agent_id)`	Check if an agent is in the trust store
`get_trusted_agent(agent_id)`	Retrieve the full agent JSON
`untrust_agent(agent_id)`	Remove an agent from the trust store

Python example

```python
import jacs

# Receive an agent document from a partner organization
remote_agent_json = receive_from_partner()

# Add to trust store (self-signature is verified automatically)
agent_id = jacs.trust_agent(remote_agent_json)
print(f"Now trusting: {agent_id}")

# Later, check trust before processing a signed document
if jacs.is_trusted(sender_id):
    # Verify their signature using the cached public key
    result = jacs.verify(signed_document)

# List all trusted agents
for aid in jacs.list_trusted_agents():
    print(aid)

# Remove trust
jacs.untrust_agent(agent_id)
```

Node.js example

```node
import { trustAgent, isTrusted, listTrustedAgents, untrustAgent } from '@hai.ai/jacs';

// Add a partner's agent to the trust store
const agentId = trustAgent(remoteAgentJson);

// Check trust
if (isTrusted(senderId)) {
  const result = verify(signedDocument);
}

// List and remove
const trusted = listTrustedAgents();
untrustAgent(agentId);
```

Cross-organization scenario

A realistic deployment involves two organizations that need to verify each other's agent signatures:

1. **Org B** creates an agent and publishes its public key via DNS TXT records or a HAI key distribution endpoint
2. **Org A** fetches Org B's agent document (via `fetch_remote_key` or direct exchange)

3. **Org A** calls `trust_agent()` with Org B's agent JSON -- JACS verifies the self-signature and caches the public key
4. From this point on, Org A can verify any document signed by Org B's agent **offline**, using only the local trust store

This is the same model as SSH `known_hosts` or PGP key signing: trust is established once through a verified channel, then used repeatedly without network round-trips.

Security notes

- `trust_agent()` cryptographically verifies the agent's self-signature before adding it to the store. A tampered agent document will be rejected.
- Agent IDs are validated against path traversal attacks before any filesystem operations.
- The trust store directory (`~/.jacs/trust_store/`) should be protected with appropriate file permissions.
- Revoking trust with `untrust_agent()` removes both the agent document and cached key material.

Infrastructure vs Tools: JACS as Middleware

Most signing libraries work as **tools**: the developer calls `sign()` and `verify()` manually at each point where integrity matters. JACS can work that way too, but its real value appears when it operates as **infrastructure** -- signing happens automatically as a side effect of normal framework usage.

The difference

Approach	Developer effort	Coverage
Tool	Call `sign()` / `verify()` at every boundary	Only where you remember to add it
Infrastructure	Add 1-3 lines of setup	Every request/response automatically

Transport-level: MCP proxies

JACS MCP transport proxies sit between client and server. Every JSON-RPC message is signed on the way out and verified on the way in. The MCP tools themselves never call a signing function -- it happens at the transport layer.

```
Client --> [JACS Proxy: sign] --> Server
Client <-- [JACS Proxy: verify] <-- Server
```

No application code changes. The proxy handles it.

Framework-level: Express / FastAPI middleware

A single middleware line signs every HTTP response automatically:

```
# FastAPI -- one line of setup
app.add_middleware(JacsMiddleware, client=jacs_client)
# Every response now carries a JACS signature header
```

```
// Express -- one line of setup
app.use(jacsMiddleware({ client }));
// Every response now carries a JACS signature header
```

The route handlers are unchanged. Signing is invisible to the developer writing business logic.

Protocol-level: A2A agent cards

When JACS publishes an A2A agent card, the card includes the agent's public key and supported algorithms. Any other A2A-compatible agent can verify signatures without prior arrangement -- the trust bootstrapping is built into the protocol.

Why this matters

Manual signing has the same problem as manual memory management: developers forget, and the places they forget are the places attackers target. Infrastructure-level signing eliminates that gap.

- **MCP transport**: every tool call and result is signed, not just the ones you thought to protect
- **HTTP middleware**: every API response is signed, including error responses and health checks
- **A2A integration**: every agent interaction is verifiable, including discovery

The developer adds setup code once. After that, signing happens everywhere automatically -- including in code paths the developer never explicitly considered.

When to use each approach

Use JACS as a **tool** when you need fine-grained control: signing specific documents, creating agreements between named parties, or building custom verification workflows.

Use JACS as **infrastructure** when you want blanket coverage: every message signed, every response verifiable, every agent interaction auditable. This is the recommended default for production deployments.

Both approaches use the same keys, the same signatures, and the same verification. The difference is who calls `sign()` -- you, or the framework.

DNS Trust Anchoring

JACS uses DNS TXT records to anchor agent identity to domain names, providing a decentralized trust layer that does not require a central certificate authority. This page explains the trust model, configuration levels, and known limitations.

How It Works

When an agent has `jacsAgentDomain` set, JACS publishes a TXT record at `_v1.agent.jacs.<domain>` containing a fingerprint of the agent's public key. During verification, JACS resolves this record and compares the fingerprint against the agent's actual key material.

The TXT record format:

```
v=hai.ai; id=<agent-uuid>; alg=sha256; enc=base64; fp=<digest>
```

If the digest matches the local public key hash, the agent's identity is confirmed through DNS.

Four Configuration Levels

`dns_validate`	`dns_required`	`dns_strict`	CLI Flag	Behavior
false	false	false	`--ignore-dns`	No DNS checks at all. Verification relies only on embedded fingerprints.
true	false	false	`--no-dns`	Attempt DNS lookup; fall back to embedded fingerprint on failure.
true	true	false	`--require-dns`	DNS TXT record must exist and match. No fallback to embedded fingerprint.
true	true	true	`--require-`	DNS TXT record must exist, match,

dns_validate	dns_required	dns_strict	CLI Flag	Behavior
			`strict-dns`	and be DNSSEC-authenticated.

Default behavior: When no flags are set, `dns_validate` and `dns_required` are derived from whether `jacsAgentDomain` is present in the agent document. If a domain is set, validation and requirement default to `true`. `dns_strict` always defaults to `false`.

Verified claims override: Agents with `jacsVerificationClaim` set to a verified level automatically use `validate=true, strict=true, required=true` regardless of flags.

Security Model Assumptions

1. **Domain ownership implies identity**: The entity controlling DNS for a domain is authorized to speak for agents on that domain.
2. **TXT records are tamper-evident with DNSSEC**: When `--require-strict-dns` is used, the full DNSSEC chain of trust (root -> TLD -> domain -> record) provides cryptographic integrity.
3. **Embedded fingerprints are a weaker fallback**: Without DNS, JACS falls back to the signature fingerprint (`jacsSignature.publicKeyHash`) in the signed artifact/agent material. This proves key consistency but not domain ownership.

Known Attack Vectors

Attack	Risk Level	Mitigated By
DNS cache poisoning	Medium	DNSSEC (`--require-strict-dns`), short TTLs
TXT record manipulation (compromised DNS credentials)	High	DNSSEC, monitoring, key rotation
DNS spoofing (man-in-the-middle)	Medium	DNSSEC validation, DNS-over-HTTPS resolvers
Stale records after key rotation	Low	TTL management, re-publishing records before rotation
Downgrade to embedded-only	Medium	Use `--require-dns` to prevent fallback

What JACS Provides

- **Fingerprint binding**: The TXT record ties a specific public key to a domain, preventing key substitution.
- **Multiple verification levels**: From no-DNS (local development) to strict DNSSEC (production cross-org).
- **Fallback logic**: When DNS is unavailable and not required, verification degrades gracefully to embedded fingerprint comparison.
- **Error specificity**: Distinct error messages for "record missing," "fingerprint mismatch," "DNSSEC failed," and "agent ID mismatch."

What JACS Does Not Yet Provide

- **Active DNSSEC chain validation**: JACS relies on the system resolver (or DoH) for DNSSEC; it does not perform independent DNSKEY/DS chain validation.
- **Certificate Transparency-style monitoring**: No log of historical TXT record changes. Domain owners must monitor independently.
- **Automatic key-to-DNS synchronization**: Publishing and updating TXT records is a manual step (or CI/CD-driven).

Recommendations

Environment	Minimum Setting	Reason
Local development	`--ignore-dns` or `--no-dns`	No real domain needed
Internal org	`--no-dns`	DNS available but not critical
Cross-org production	`--require-dns`	Prevents impersonation across trust boundaries
High-security / regulated	`--require-strict-dns`	Full DNSSEC chain required

For production cross-organization deployments, use `--require-dns` at minimum. Enable DNSSEC on your domain and use `--require-strict-dns` when the infrastructure supports it.

See Also

- DNS-Based Verification -- setup guide with provider-specific instructions
- Security Model -- broader security architecture

- Key Rotation -- coordinating key changes with DNS updates

Failure Modes

This page documents the error messages you will see when multi-agent agreements fail. Each scenario is validated by the chaos agreement tests in the JACS test suite.

Partial Signing (Agent Crash)

What happened: An agreement was created for N agents but one or more agents never signed -- they crashed, timed out, or disconnected before calling `sign_agreement`.

Error message:

```
not all agents have signed: ["<unsigned-agent-id>"] { ... agreement object ... }
```

What to do: Identify the unsigned agent from the error, re-establish contact, and have them call `sign_agreement` on the document. The partially-signed document is still valid and can accept additional signatures -- signing is additive.

Quorum Not Met

What happened: An agreement with an explicit quorum (M-of-N via `AgreementOptions`) received fewer than M signatures.

Error message:

```
Quorum not met: need 2 signatures, have 1 (unsigned: ["<agent-id>"])
```

What to do: Either collect more signatures to meet the quorum threshold, or create a new agreement with a lower quorum if appropriate. The unsigned agent IDs in the error tell you exactly who still needs to sign.

Tampered Signature

What happened: A signature byte was modified after an agent signed the agreement. The cryptographic verification layer detects that the signature does not match the signed content.

Error message:

The exact message comes from the crypto verification layer and varies by algorithm, but it will always fail on the signature check rather than reporting missing signatures. You will not see "not all agents have signed" for this case -- the error is a cryptographic verification failure.

What to do: This indicates data corruption in transit or deliberate tampering. Discard the document and request a fresh copy from the signing agent. Do not attempt to re-sign a document with a corrupted signature.

Tampered Document Body

What happened: The document content was modified after signatures were applied. JACS stores an integrity hash of the agreement-relevant fields at signing time, and any body modification causes a mismatch.

Error message:

```
Agreement verification failed: agreement hashes do not match
```

What to do: The document body no longer matches what the agents originally signed. Discard the modified document and go back to the last known-good version. If the modification was intentional (e.g., an amendment), create a new agreement on the updated document and collect fresh signatures from all parties.

In-Memory Consistency After Signing

What happened: `sign_agreement` succeeded but `save()` was never called -- for example, a storage backend failure or process interruption before persistence.

Error message: None. This is not an error. After `sign_agreement` returns successfully, the signed document is immediately retrievable and verifiable from in-memory storage.

What to do: Retry the `save()` call to persist to disk. The in-memory state is consistent: you can retrieve the document with `get_document`, verify it with `check_agreement`, serialize it, and transfer it to other agents for additional signatures -- all without saving first.

See Also

- Creating and Using Agreements - Agreement creation and signing workflow
- Security Model - Overall security architecture
- Cryptographic Algorithms - Algorithm details and signature verification

Testing

This chapter covers testing strategies for applications that use JACS, including unit testing, integration testing, and mocking approaches.

Testing Fundamentals

Test Agent Setup

Create dedicated test configurations to isolate tests from production:

```
// jacs.test.config.json
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_data_directory": "./test_data",
  "jacs_key_directory": "./test_keys",
  "jacs_agent_private_key_filename": "test_private.pem",
  "jacs_agent_public_key_filename": "test_public.pem",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "jacs_default_storage": "fs"
}
```

Test Fixtures

Set up test fixtures before running tests:

Python (pytest):

```python
import pytest
import jacs
import tempfile
import shutil

@pytest.fixture
def test_agent():
    """Create a test agent with temporary directories."""
    temp_dir = tempfile.mkdtemp()
    data_dir = f"{temp_dir}/data"
    key_dir = f"{temp_dir}/keys"

    # Initialize directories
    import os
    os.makedirs(data_dir)
    os.makedirs(key_dir)

    # Create test config
    config = {
        "jacs_data_directory": data_dir,
        "jacs_key_directory": key_dir,
        "jacs_agent_key_algorithm": "ring-Ed25519",
        "jacs_default_storage": "fs"
    }

    config_path = f"{temp_dir}/jacs.config.json"
    with open(config_path, 'w') as f:
        import json
        json.dump(config, f)

    agent = jacs.JacsAgent()
    agent.load(config_path)

    yield agent

    # Cleanup
    shutil.rmtree(temp_dir)

def test_create_document(test_agent):
    """Test document creation."""
    import json
    doc = test_agent.create_document(json.dumps({
        'title': 'Test Document'
    }))

    assert doc is not None
    parsed = json.loads(doc)
    assert 'jacsId' in parsed
    assert 'jacsSignature' in parsed
```

Node.js (Jest):

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';
import path from 'path';
import os from 'os';

describe('JACS Document Tests', () => {
  let agent;
  let tempDir;

  beforeAll(() => {
    // Create temp directory
    tempDir = fs.mkdtempSync(path.join(os.tmpdir(), 'jacs-test-'));
    const dataDir = path.join(tempDir, 'data');
    const keyDir = path.join(tempDir, 'keys');

    fs.mkdirSync(dataDir);
    fs.mkdirSync(keyDir);

    // Create test config
    const config = {
      jacs_data_directory: dataDir,
      jacs_key_directory: keyDir,
      jacs_agent_key_algorithm: 'ring-Ed25519',
      jacs_default_storage: 'fs'
    };

    const configPath = path.join(tempDir, 'jacs.config.json');
    fs.writeFileSync(configPath, JSON.stringify(config));

    agent = new JacsAgent();
    agent.load(configPath);
  });

  afterAll(() => {
    // Cleanup
    fs.rmSync(tempDir, { recursive: true });
  });

  test('creates a signed document', () => {
    const doc = agent.createDocument(JSON.stringify({
      title: 'Test Document'
    }));

    const parsed = JSON.parse(doc);
    expect(parsed.jacsId).toBeDefined();
    expect(parsed.jacsSignature).toBeDefined();
  });
});
```

Unit Testing

Testing Document Operations

```python
import pytest
import jacs
import json

def test_document_verification(test_agent):
    """Test that created documents verify correctly."""
    doc = test_agent.create_document(json.dumps({
        'content': 'Test content'
    }))

    is_valid = test_agent.verify_document(doc)
    assert is_valid is True

def test_document_tampering_detected(test_agent):
    """Test that tampered documents fail verification."""
    doc = test_agent.create_document(json.dumps({
        'content': 'Original content'
    }))

    # Tamper with the document
    parsed = json.loads(doc)
    parsed['content'] = 'Tampered content'
    tampered = json.dumps(parsed)

    is_valid = test_agent.verify_document(tampered)
    assert is_valid is False

def test_signature_verification(test_agent):
    """Test signature verification."""
    doc = test_agent.create_document(json.dumps({
        'data': 'test'
    }))

    is_valid = test_agent.verify_signature(doc)
    assert is_valid is True
```

Testing Agreements

```python
def test_agreement_creation(test_agent):
    """Test creating a document with agreement."""
    doc = test_agent.create_document(json.dumps({
        'contract': 'Service Agreement'
    }))

    # Add agreement
    doc_with_agreement = test_agent.create_agreement(
        doc,
        ['agent-1-id', 'agent-2-id'],
        question='Do you agree to these terms?',
        context='Test agreement'
    )

    parsed = json.loads(doc_with_agreement)
    assert 'jacsAgreement' in parsed
    assert len(parsed['jacsAgreement']['agentIDs']) == 2

def test_agreement_signing(test_agent):
    """Test signing an agreement."""
    doc = test_agent.create_document(json.dumps({
        'contract': 'Test'
    }))

    agent_json = test_agent.load('./jacs.test.config.json')
    agent_data = json.loads(agent_json)
    agent_id = agent_data['jacsId']

    doc_with_agreement = test_agent.create_agreement(
        doc,
        [agent_id],
        question='Agree?'
    )

    signed = test_agent.sign_agreement(doc_with_agreement)

    status_json = test_agent.check_agreement(signed)
    status = json.loads(status_json)

    assert status['complete'] is True
```

Agreement Completion Semantics (Strict)

`check_agreement()` is intentionally strict: it should fail until **all** required signers have signed. Add explicit tests for each state:

1. Unsigned agreement: must fail.
2. Partially signed agreement: must still fail.
3. Fully signed agreement: must pass with `complete == true`.

This prevents accidental "partial approval" workflows in production.

Two-Agent Agreement Harness (Separate Agents)

For realistic interoperability tests, use **two separate agents** with:

- Separate key directories (`agent1/keys` , `agent2/keys`)
- Shared data directory (`shared-data`) so both public keys are locally resolvable
- Relative paths from a temporary test working directory

Python (pytest):

```python
def test_two_party_agreement_requires_both_signatures(tmp_path):
    import os
    import pytest
    import jacs.simple as simple

    (tmp_path / "shared-data").mkdir()
    (tmp_path / "agent1").mkdir()
    (tmp_path / "agent2").mkdir()

    original_cwd = os.getcwd()
    try:
        os.chdir(tmp_path)
        a1 = simple.create(
            name="agent-1",
            password="TestP@ss123!#",
            algorithm="ring-Ed25519",
            data_directory="shared-data",
            key_directory="agent1/keys",
            config_path="agent1/jacs.config.json",
        )
        a2 = simple.create(
            name="agent-2",
            password="TestP@ss123!#",
            algorithm="ring-Ed25519",
            data_directory="shared-data",
            key_directory="agent2/keys",
            config_path="agent2/jacs.config.json",
        )

        simple.load("agent1/jacs.config.json")
        agreement = simple.create_agreement(
            {"proposal": "two-party-approval"},
            [a1.agent_id, a2.agent_id],
            question="Approve?",
        )

        with pytest.raises(Exception):
            simple.check_agreement(agreement)
        signed_by_a1 = simple.sign_agreement(agreement)
        with pytest.raises(Exception):
            simple.check_agreement(signed_by_a1)

        simple.load("agent2/jacs.config.json")
        signed_by_both = simple.sign_agreement(signed_by_a1)
        status = simple.check_agreement(signed_by_both)
        assert status.complete is True
    finally:
        os.chdir(original_cwd)
```

Node.js (Mocha/Jest style):

```
const root = fs.mkdtempSync(path.join(os.tmpdir(), 'jacs-two-agent-'));
const originalCwd = process.cwd();
try {
  process.chdir(root);
  fs.mkdirSync('agent1', { recursive: true });
  fs.mkdirSync('agent2', { recursive: true });

  simpleA.create({
    name: 'agent-a',
    password: 'TestP@ss123!#',
    algorithm: 'ring-Ed25519',
    dataDirectory: 'shared-data',
    keyDirectory: 'agent1/keys',
    configPath: 'agent1/jacs.config.json'
  });
  simpleB.create({
    name: 'agent-b',
    password: 'TestP@ss123!#',
    algorithm: 'ring-Ed25519',
    dataDirectory: 'shared-data',
    keyDirectory: 'agent2/keys',
    configPath: 'agent2/jacs.config.json'
  });

  simpleA.load('agent1/jacs.config.json');
  simpleB.load('agent2/jacs.config.json');
  const infoA = simpleA.getAgentInfo();
  const infoB = simpleB.getAgentInfo();

  const agreement = simpleA.createAgreement(
    { proposal: 'two-party-approval' },
    [infoA.agentId, infoB.agentId]
  );

  expect(() => simpleA.checkAgreement(agreement)).to.throw();
  const signedByA = simpleA.signAgreement(agreement);
  expect(() => simpleA.checkAgreement(signedByA)).to.throw();

  const signedByBoth = simpleB.signAgreement(signedByA);
  const status = simpleB.checkAgreement(signedByBoth);
  expect(status.complete).to.equal(true);
} finally {
  process.chdir(originalCwd);
  fs.rmSync(root, { recursive: true, force: true });
}
```

Testing Request/Response Signing

```python
def test_request_signing(test_agent):
    """Test signing a request payload."""
    payload = {
        'method': 'tools/call',
        'params': {'name': 'test_tool'}
    }

    signed = test_agent.sign_request(payload)
    assert signed is not None

    # Verify the signed request
    result = test_agent.verify_response(signed)
    assert result is not None
    assert 'payload' in result
```

Integration Testing

Testing MCP Integration

Python:

```python
import pytest
import asyncio
from jacs.mcp import JACSMCPServer, JACSMCPClient
from fastmcp import FastMCP

@pytest.fixture
def mcp_server(test_agent):
    """Create a test MCP server."""
    mcp = FastMCP("Test Server")

    @mcp.tool()
    def echo(text: str) -> str:
        return f"Echo: {text}"

    return JACSMCPServer(mcp)

@pytest.mark.asyncio
async def test_mcp_tool_call(mcp_server, test_agent):
    """Test calling an MCP tool with JACS authentication."""
    # This would require setting up actual server/client connection
    # For unit testing, test the signing/verification separately
    pass
```

Node.js:

```
import { JACSExpressMiddleware } from '@hai.ai/jacs/http';
import express from 'express';
import request from 'supertest';

describe('JACS Express Middleware', () => {
  let app;
  let agent;

  beforeAll(() => {
    app = express();
    app.use('/api', express.text({ type: '*/*' }));
    app.use('/api', JACSExpressMiddleware({
      configPath: './jacs.test.config.json'
    }));

    app.post('/api/echo', (req, res) => {
      res.send({ echo: req.jacsPayload });
    });

    agent = new JacsAgent();
    agent.load('./jacs.test.config.json');
  });

  test('accepts valid JACS requests', async () => {
    const signedRequest = agent.signRequest({
      message: 'Hello'
    });

    const response = await request(app)
      .post('/api/echo')
      .set('Content-Type', 'text/plain')
      .send(signedRequest);

    expect(response.status).toBe(200);
  });

  test('rejects invalid requests', async () => {
    const response = await request(app)
      .post('/api/echo')
      .set('Content-Type', 'text/plain')
      .send('{"invalid": "request"}');

    expect(response.status).toBe(400);
  });
});
```

Testing HTTP Endpoints

```python
import pytest
from fastapi import FastAPI
from fastapi.testclient import TestClient
import jacs
import json

app = FastAPI()

@app.post("/api/document")
async def create_doc(request_body: str):
    agent = jacs.JacsAgent()
    agent.load('./jacs.test.config.json')

    result = agent.verify_response(request_body)
    if result:
        # Process the verified payload
        return {"status": "success", "payload": result.get("payload")}
    return {"status": "error"}

@pytest.fixture
def client():
    return TestClient(app)

def test_endpoint_accepts_signed_request(client, test_agent):
    """Test that endpoint accepts properly signed requests."""
    signed = test_agent.sign_request({
        'action': 'create',
        'data': {'title': 'Test'}
    })

    response = client.post("/api/document", content=signed)
    assert response.status_code == 200
```

Mocking

Mocking JACS Agent

Python:

```python
from unittest.mock import Mock, patch
import json

def test_with_mocked_agent():
    """Test with a mocked JACS agent."""
    mock_agent = Mock()

    # Mock create_document to return a fake signed document
    mock_agent.create_document.return_value = json.dumps({
        'jacsId': 'mock-id',
        'jacsVersion': 'mock-version',
        'content': 'test',
        'jacsSignature': {'signature': 'mock-sig'}
    })

    # Mock verify_document to always return True
    mock_agent.verify_document.return_value = True

    # Use the mock in your tests
    doc = mock_agent.create_document(json.dumps({'content': 'test'}))
    assert mock_agent.verify_document(doc) is True
```

Node.js:

```typescript
// Mock for testing
const mockAgent = {
  createDocument: jest.fn().mockReturnValue(JSON.stringify({
    jacsId: 'mock-id',
    jacsVersion: 'mock-version',
    content: 'test',
    jacsSignature: { signature: 'mock-sig' }
  })),
  verifyDocument: jest.fn().mockReturnValue(true),
  signRequest: jest.fn().mockImplementation((payload) =>
    JSON.stringify({ payload, jacsSignature: { signature: 'mock' } })
  ),
  verifyResponse: jest.fn().mockImplementation((response) =>
    ({ payload: JSON.parse(response).payload })
  )
};

test('uses mocked agent', () => {
  const doc = mockAgent.createDocument(JSON.stringify({ test: true }));
  expect(mockAgent.createDocument).toHaveBeenCalled();
  expect(mockAgent.verifyDocument(doc)).toBe(true);
});
```

Mocking MCP Transport

```
// Mock transport for MCP testing
class MockTransport {
  constructor() {
    this.messages = [];
  }

  send(message) {
    this.messages.push(message);
  }

  async receive() {
    return this.messages.shift();
  }
}

test('MCP client with mock transport', async () => {
  const mockTransport = new MockTransport();
  // Use mock transport in tests
});
```

Test Coverage

Rust Coverage

For Rust coverage, we recommend **cargo-llvm-cov** for its cross-platform support and accuracy with cryptographic code.

Installation:

```
cargo install cargo-llvm-cov
```

Running coverage:

```
# Print coverage summary to stdout
cargo llvm-cov

# Generate and open HTML report in browser
cargo llvm-cov --open

# With specific features enabled
cargo llvm-cov --features cli

# Export LCOV format for CI integration
cargo llvm-cov --lcov --output-path lcov.info
```

Why cargo-llvm-cov?

Factor	cargo-llvm-cov	tarpaulin
Platform support	Linux, macOS, Windows	Linux primarily
Accuracy	LLVM source-based (highly accurate)	Ptrace-based (some inaccuracies)
Coverage types	Line, region, branch	Line primarily

For CI integration, export to LCOV format and upload to Codecov or similar services.

Python Coverage

```
# Run tests with coverage
pytest --cov=myapp --cov-report=html tests/

# View coverage report
open htmlcov/index.html
```

Node.js Coverage

```
# Run tests with coverage
npm test -- --coverage

# Or with Jest directly
jest --coverage
```

CI/CD Integration

GitHub Actions

```yaml
# .github/workflows/test.yml
name: Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm install

      - name: Generate test keys
        run: |
          mkdir -p test_keys test_data
          # Generate test keys (implementation depends on your setup)

      - name: Run tests
        run: npm test

      - name: Upload coverage
        uses: codecov/codecov-action@v3
```

Test Environment Variables

```bash
# Set test environment
export JACS_TEST_MODE=1
export JACS_TEST_CONFIG=./jacs.test.config.json
```

RAII Test Fixtures (Rust)

For Rust tests that modify global state (environment variables, file system, etc.), use RAII guards to ensure cleanup even on panic. This pattern is essential for test isolation and reliability.

TrustTestGuard Pattern

The JACS codebase uses a `TrustTestGuard` pattern for tests that modify the HOME environment variable:

```rust
use std::env;
use tempfile::TempDir;

/// RAII guard for test isolation that ensures HOME is restored even on panic.
struct TrustTestGuard {
    _temp_dir: TempDir,
    original_home: Option<String>,
}

impl TrustTestGuard {
    fn new() -> Self {
        // Save original HOME before modifying
        let original_home = env::var("HOME").ok();

        let temp_dir = TempDir::new().expect("Failed to create temp directory");

        // SAFETY: Only used in #[serial] tests - no concurrent access
        unsafe {
            env::set_var("HOME", temp_dir.path().to_str().unwrap());
        }

        Self {
            _temp_dir: temp_dir,
            original_home,
        }
    }
}

impl Drop for TrustTestGuard {
    fn drop(&mut self) {
        // Restore original HOME even during panic unwinding
        unsafe {
            match &self.original_home {
                Some(home) => env::set_var("HOME", home),
                None => env::remove_var("HOME"),
            }
        }
    }
}

// Usage in tests:
#[test]
#[serial]  // Use serial_test crate to prevent parallel execution
fn test_with_isolated_home() {
    let _guard = TrustTestGuard::new();  // Setup

    // Test code here - HOME points to temp directory

    // Guard automatically restores HOME on drop, even if test panics
}
```

Key benefits:

- **Panic safety**: Cleanup runs even if the test panics
- **No manual cleanup**: Drop trait handles restoration automatically
- **Environment isolation**: Each test gets a fresh temporary directory
- **Composable**: Multiple guards can be combined for complex setups

Property-Based Testing

For cryptographic code, property-based testing helps verify invariants that hold across many random inputs. We recommend proptest for Rust.

Key Properties to Test

1. **Round-trip**: Sign then verify should always succeed
2. **Tamper detection**: Modified content should fail verification
3. **Key independence**: Different keys produce different signatures

```rust
use proptest::prelude::*;

proptest! {
    #[test]
    fn signature_roundtrip(content in ".*") {
        let signed = sign_content(&content)?;
        prop_assert!(verify_signature(&signed).is_ok());
    }

    #[test]
    fn tamper_detection(content in ".*", tamper_pos in 0usize..1000) {
        let signed = sign_content(&content)?;
        let tampered = tamper_at_position(&signed, tamper_pos);
        prop_assert!(verify_signature(&tampered).is_err());
    }
}
```

Fuzzing

Fuzz testing is recommended for parsing and decoding functions to discover edge cases and potential security issues.

Recommended Tool: cargo-fuzz

```
# Install
cargo install cargo-fuzz

# Create a fuzz target
cargo fuzz init
cargo fuzz add base64_decode

# Run fuzzing
cargo +nightly fuzz run base64_decode
```

Priority Fuzz Targets for JACS

1. **Base64 decoding** - Handles untrusted input from signatures
2. **Agent JSON parsing** - Complex nested structures
3. **Document validation** - Schema compliance checking
4. **Timestamp parsing** - Date/time format handling

Fuzzing documentation will be expanded as fuzz targets are added to the JACS test suite.

Best Practices

1. Isolate Tests

- Use separate test configurations
- Create temporary directories for each test run
- Clean up after tests (use RAII guards in Rust)

2. Test Edge Cases

```python
def test_empty_document():
    """Test handling of empty documents."""
    with pytest.raises(Exception):
        test_agent.create_document('')

def test_invalid_json():
    """Test handling of invalid JSON."""
    with pytest.raises(Exception):
        test_agent.create_document('not json')

def test_large_document():
    """Test handling of large documents."""
    large_content = 'x' * 1000000
    doc = test_agent.create_document(json.dumps({
        'content': large_content
    }))
    assert doc is not None
```

3. Test Security Properties

```python
def test_signature_changes_with_content():
    """Verify different content produces different signatures."""
    doc1 = test_agent.create_document(json.dumps({'a': 1}))
    doc2 = test_agent.create_document(json.dumps({'a': 2}))

    sig1 = json.loads(doc1)['jacsSignature']['signature']
    sig2 = json.loads(doc2)['jacsSignature']['signature']

    assert sig1 != sig2
```

4. Test Error Handling

```python
def test_verify_invalid_signature():
    """Test that invalid signatures are rejected."""
    doc = test_agent.create_document(json.dumps({'data': 'test'}))
    parsed = json.loads(doc)

    # Corrupt the signature
    parsed['jacsSignature']['signature'] = 'invalid'
    corrupted = json.dumps(parsed)

    assert test_agent.verify_document(corrupted) is False
```

See Also

- Python API Reference - API documentation
- Node.js API Reference - API documentation
- Security Model - Security testing considerations

MCP Overview

Use MCP when the boundary is model-to-tool inside an application or local workstation. Use A2A when the boundary is agent-to-agent across organizations or services.

Choose The MCP Path

There are three supported ways to use JACS with MCP today:

1. **Run `jacs mcp`** when you want a ready-made MCP server with the broadest tool surface.
2. **Wrap an existing MCP transport** when you already have an MCP server or client and want signed JSON-RPC.
3. **Register JACS as MCP tools** when you want the model to call signing, verification, agreement, A2A, or trust operations directly.

Best Fit By Runtime

Runtime	Best starting point	What it gives you
Rust	`jacs-mcp`	Full MCP server with document, agreement, trust, A2A, and audit tools
Python	`jacs.mcp` or `jacs.adapters.mcp`	Local SSE transport security or FastMCP tool registration
Node.js	`@hai.ai/jacs/mcp`	Transport proxy or MCP tool registration for existing SDK-based servers

Important Constraints

- **Python MCP wrappers are local-only.** `JACSMCPClient`, `JACSMCPServer`, and `jacs_call()` enforce loopback URLs.
- **Unsigned fallback is off by default.** Both Python and Node fail closed unless you explicitly allow unsigned fallback.
- **Node has two factories.** `createJACSTransportProxy()` takes a loaded `JacsClient` or `JacsAgent`; `createJACSTransportProxyAsync()` is the config-path variant.

1. Ready-Made Server: `jacs mcp`

Install the unified binary and start the MCP server:

```
cargo install jacs-cli
jacs mcp
```

The MCP server is built into the `jacs` binary (stdio transport only, no HTTP). It includes document signing, agreements, trust store operations, A2A tools, and security audit tools. See `jacs-mcp/README.md` in the repo for the full tool list and client configuration examples.

2. Transport Security Around Your Existing MCP Code

Python

Use `jacs.mcp` when you already have a FastMCP server or client and want transparent signing around the SSE transport:

```
from fastmcp import FastMCP
from jacs.mcp import JACSMCPServer

mcp = JACSMCPServer(FastMCP("Secure Server"), "./jacs.config.json")
```

For clients:

```
from jacs.mcp import JACSMCPClient

client = JACSMCPClient("http://localhost:8000/sse", "./jacs.config.json")
```

Helpful utilities in the same module:

- `create_jacs_mcp_server()` for a one-line FastMCP server
- `jacs_middleware()` for explicit Starlette middleware wiring
- `jacs_call()` for one-off authenticated local calls

See Python MCP Integration for the detailed patterns.

Node.js

Use the transport proxy when you already have an MCP transport:

```
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { JacsClient } from '@hai.ai/jacs/client';
import { createJACSTransportProxy } from '@hai.ai/jacs/mcp';

const client = await JacsClient.quickstart({
  name: 'mcp-agent',
  domain: 'mcp.local',
});

const transport = new StdioServerTransport();
const secureTransport = createJACSTransportProxy(transport, client, 'server');
```

If you only have a config path:

```
import { createJACSTransportProxyAsync } from '@hai.ai/jacs/mcp';

const secureTransport = await createJACSTransportProxyAsync(
  transport,
  './jacs.config.json',
  'server',
);
```

See Node.js MCP Integration for examples and tool registration.

3. Register JACS Operations As MCP Tools

This is different from transport security. Here the model gets explicit MCP tools such as `jacs_sign_document`, `jacs_verify_document`, agreement helpers, and trust helpers.

Python

```
from fastmcp import FastMCP
from jacs.client import JacsClient
from jacs.adapters.mcp import (
    register_jacs_tools,
    register_a2a_tools,
    register_trust_tools,
)

client = JacsClient.quickstart(name="mcp-agent", domain="mcp.local")
mcp = FastMCP("JACS Tools")
register_jacs_tools(mcp, client=client)
register_a2a_tools(mcp, client=client)
register_trust_tools(mcp, client=client)
```

Node.js

```
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { JacsClient } from '@hai.ai/jacs/client';
import { registerJacsTools } from '@hai.ai/jacs/mcp';

const server = new Server(
  { name: 'jacs-tools', version: '1.0.0' },
  { capabilities: { tools: {} } },
);

const client = await JacsClient.quickstart({
  name: 'mcp-agent',
  domain: 'mcp.local',
});

registerJacsTools(server, client);
```

The Node tool set is intentionally smaller than the Rust MCP server. Use `jacs mcp` when you need the largest supported MCP surface.

Example Paths In This Repo

- `jacs-mcp/README.md`
- `jacspy/examples/mcp/server.py`
- `jacspy/examples/mcp/client.py`
- `jacsnpm/examples/mcp.stdio.server.js`
- `jacsnpm/examples/mcp.stdio.client.js`

Related Guides

- Python MCP Integration
- Node.js MCP Integration
- A2A Interoperability
- Python Framework Adapters

A2A Interoperability

Use A2A when your agent needs to be discoverable and verifiable by another service, team, or organization. This is the cross-boundary story; MCP is the inside-the-app story.

What JACS Adds To A2A

- **Agent Cards** with JACS provenance metadata
- **Signed artifacts** such as `a2a-task` or `a2a-message`
- **Trust policy** for deciding whether another agent is acceptable
- **Chain of custody** via parent signatures

The Core Flow

1. Export An Agent Card

Python:

```python
from jacs.client import JacsClient

client = JacsClient.quickstart(name="my-agent", domain="my-agent.example.com")
card = client.export_agent_card(url="http://localhost:8080")
```

Node.js:

```node
import { JacsClient } from '@hai.ai/jacs/client';

const client = await JacsClient.quickstart({
  name: 'my-agent',
  domain: 'my-agent.example.com',
});

const card = client.exportAgentCard();
```

2. Serve Discovery Documents

Python has the strongest first-class server helpers today.

Quick demo server:

```python
from jacs.a2a import JACSA2AIntegration

JACSA2AIntegration.quickstart(
    name="my-agent",
    domain="my-agent.example.com",
    url="http://localhost:8080",
).serve(port=8080)
```

Production FastAPI mounting:

```python
from jacs.a2a_server import create_a2a_app, jacs_a2a_routes

app = create_a2a_app(client, title="My A2A Agent")
# or:
# app.include_router(jacs_a2a_routes(client))
```

Node.js has two discovery helpers:

- `client.getA2A().listen(port)` for a minimal demo server
- `jacsA2AMiddleware(client, options)` for mounting discovery routes in an existing Express app

```node
import express from 'express';
import { jacsA2AMiddleware } from '@hai.ai/jacs/a2a-server';

const app = express();
app.use(jacsA2AMiddleware(client, { url: 'http://localhost:3000' }));
app.listen(3000);
```

3. Sign And Verify Artifacts

Python:

```python
signed = client.sign_artifact({"taskId": "t-1", "operation": "classify"}, "task")
result = client.get_a2a().verify_wrapped_artifact(signed)
assert result["valid"]
```

Node.js:

```node
const signed = await client.signArtifact(
  { taskId: 't-1', operation: 'classify' },
  'task',
);

const result = await client.verifyArtifact(signed);
console.log(result.valid);
```

Trust Policies

Trust policy answers a different question from cryptographic verification.

- **Trust policy**: should this remote agent be admitted?
- **Artifact verification**: is this specific signed payload valid?

The current policy meanings are:

Policy	Behavior
`open`	Accept all agents without verification
`verified`	Require the JACS provenance extension (`urn:jacs:provenance-v1`) in the agent card (**default**)
`strict`	Require the signer to be in the local trust store

That means `verified` is about **JACS provenance on the Agent Card**, not about a promise that every foreign key has already been resolved.

Python

```python
a2a = client.get_a2a()
assessment = a2a.assess_remote_agent(remote_card_json, policy="strict")

if assessment["allowed"]:
    result = a2a.verify_wrapped_artifact(artifact, assess_trust=True)
```

Node.js

```node.js
const a2a = client.getA2A();
const assessment = a2a.assessRemoteAgent(remoteCardJson);

if (assessment.allowed) {
  const result = await a2a.verifyWrappedArtifact(signedArtifact);
}
```

Bootstrap Patterns

Use the trust store when you want explicit admission:

- Export the agent document with `share_agent()` / `shareAgent()`
- Exchange the public key with `share_public_key()` / `getPublicKey()`
- Add the remote agent with `trust_agent_with_key()` / `trustAgentWithKey()`

This is the cleanest path into `strict` policy.

Current Runtime Differences

- **Python**: `jacs.a2a_server` is the clearest full discovery story.
- **Node.js**: `jacsA2AMiddleware()` serves five `.well-known` routes from Express, but the generated `jwks.json` and `jacs-pubkey.json` payloads are still placeholder metadata. `listen()` is intentionally smaller and only suitable for demos.

Example Paths In This Repo

- `jacs-mcp/README.md`
- `jacspy/tests/test_a2a_server.py`
- `jacsnpm/src/a2a-server.js`
- `jacsnpm/examples/a2a-agent-example.js`
- `jacs/tests/a2a_cross_language_tests.rs`

A2A Quickstart

Three focused mini-guides to get your JACS agent working with A2A.

Guide	What You'll Do	Time
1. Serve	Publish your Agent Card so other agents can find you	2 min
2. Discover & Trust	Find remote agents and assess their trustworthiness	2 min
3. Exchange	Sign and verify A2A artifacts with chain of custody	3 min

> **Single-page version:** See the A2A Quick Start at the repo root for a 10-line journey.

JACS for A2A Developers

Already using the A2A protocol? Here's what JACS adds -- and what stays the same.

What Stays the Same

- **Agent Cards** follow the v0.4.0 shape. Your existing Agent Card fields (`name` , `description` , `skills` , `url`) are preserved.
- **Discovery** uses `/.well-known/agent-card.json` . No new endpoints are required for basic interop.
- **JSON-RPC** transport is untouched. JACS works alongside A2A, not instead of it.

What JACS Adds

A2A Alone	With JACS
Agent Card has no signature	Agent Card is JWS-signed + JWKS published
Artifacts are unsigned payloads	Artifacts carry `jacsSignature` with signer ID, algorithm, and timestamp
Trust is transport-level (TLS)	Trust is data-level -- signatures persist offline
No chain of custody	`parent_signatures` link artifacts into a verifiable chain
No standard trust policy	`open` / `verified` / `strict` policies built in

Minimal Integration (Add JACS to Existing A2A Code)

If you already serve an Agent Card, adding JACS provenance takes two steps:

Step 1: Add the JACS extension to your Agent Card's `capabilities`:

```
{
  "capabilities": {
    "extensions": [{
      "uri": "urn:jacs:provenance-v1",
      "description": "JACS cryptographic document signing",
      "required": false
    }]
  }
}
```

Step 2: Sign artifacts before sending them:

```
from jacs.client import JacsClient

client = JacsClient.quickstart(name="my-agent", domain="my-agent.example.com")
# Wrap your existing artifact payload
signed = client.sign_artifact(your_existing_artifact, "task")
# Send `signed` instead of the raw artifact
```

Receiving agents that don't understand JACS will ignore the extra fields. Receiving agents that do understand JACS can verify the signature and assess trust.

Dual Key Architecture

JACS generates two key pairs per agent:

- **Post-quantum (ML-DSA-87)** for JACS document signatures -- future-proof
- **Traditional (RSA/ECDSA)** for JWS Agent Card signatures -- A2A ecosystem compatibility

This means your agent is compatible with both the current A2A ecosystem and quantum-resistant verification.

Troubleshooting FAQ

Q: `pip install jacs[a2a-server]` fails. A: The `a2a-server` extra requires Python 3.10+ and adds FastAPI + uvicorn. If you only need signing (not serving), use `pip install jacs` with no extras.

Q: `discover_and_assess` returns `jacs_registered: false`. A: The remote agent's Agent Card does not include the `urn:jacs:provenance-v1` extension. This is normal for non-JACS A2A agents. With

the `open` trust policy, they are still allowed; with `verified`, they are rejected.

Q: Verification returns `valid: true` but `trust.allowed: false`. A: The signature is cryptographically correct, but the trust policy rejected the signer. With `strict` policy, the signer must be in your local trust store. Add them with `a2a.trust_a2a_agent(card_json)`.

Q: `sign_artifact` raises "no agent loaded". A: Call `JacsClient.quickstart(name="my-agent", domain="my-agent.example.com")` or `JacsClient(config_path=...)` before signing. The client must have a loaded agent with keys.

Q: Agent Card export returns empty skills. A: Skills are derived from `jacsServices` in the agent definition. Pass `skills=[...]` to `export_agent_card()` to override, or define services when creating the agent.

Q: My existing A2A client doesn't understand the JACS fields. A: This is expected. JACS fields (`jacsId`, `jacsSignature`, `jacsSha256`) are additive. Non-JACS clients should ignore unknown fields per JSON convention. If a client rejects them, strip JACS fields before sending by extracting `signed["payload"]`.

Q: How do I verify artifacts from agents I've never seen before? A: Use `JACS_KEY_RESOLUTION` to configure key lookup. Set `JACS_KEY_RESOLUTION=local,hai` to check your local cache first, then the HAI key service. For offline-only verification, set `JACS_KEY_RESOLUTION=local`.

Next Steps

- **A2A Interoperability Reference** -- Full API reference, well-known documents, MCP integration
- **Trust Store** -- Managing trusted agents
- **Express Middleware** -- Add A2A to existing Express apps
- **Framework Adapters** -- Auto-sign with LangChain, FastAPI, CrewAI
- **Observability & Monitoring Guide** -- Monitor signing and verification events
- **Hero Demo (Python)** -- 3-agent trust verification example
- **Hero Demo (Node.js)** -- Same demo in TypeScript

Serve Your Agent Card

Make your JACS agent discoverable by other A2A agents.

Prerequisites: `pip install jacs[a2a-server]` (Python) or `npm install @hai.ai/jacs express` (Node.js).

Python

```python
from jacs.a2a import JACSA2AIntegration

JACSA2AIntegration.quickstart(url="http://localhost:8080").serve(port=8080)
```

Your agent is now discoverable at `http://localhost:8080/.well-known/agent-card.json`.

Production: Mount into Your Own FastAPI App

```python
from fastapi import FastAPI
from jacs.client import JacsClient
from jacs.a2a_server import jacs_a2a_routes

app = FastAPI()
client = JacsClient.quickstart(name="my-agent", domain="my-agent.example.com")
router = jacs_a2a_routes(client)
app.include_router(router)
```

Node.js (Express)

What Gets Served

All five `.well-known` endpoints are served automatically:

Endpoint	Purpose
`/.well-known/agent-card.json`	A2A Agent Card with JWS signature
`/.well-known/jwks.json`	JWK set for A2A verifiers
`/.well-known/jacs-agent.json`	JACS agent descriptor

Endpoint	Purpose
`/.well-known/jacs-pubkey.json`	JACS public key
`/.well-known/jacs-extension.json`	JACS provenance extension descriptor

The Agent Card includes the `urn:jacs:provenance-v1` extension in `capabilities.extensions`, signaling to other JACS agents that your agent supports cryptographic provenance.

Next Steps

- **Discover & Trust Remote Agents** -- Find other agents and assess their trustworthiness
- **Exchange Signed Artifacts** -- Sign and verify A2A artifacts
- **A2A Interoperability Reference** -- Full API reference

Discover & Trust Remote Agents

Find other A2A agents and decide whether to trust them.

Python

```python
from jacs.a2a_discovery import discover_and_assess_sync

result = discover_and_assess_sync("https://agent.example.com")
if result["allowed"]:
    print(f"Trusted: {result['card']['name']} ({result['trust_level']})")
```

Add to Your Trust Store

For `strict` policy, agents must be in your local trust store:

```python
from jacs.client import JacsClient
from jacs.a2a import JACSA2AIntegration

client = JacsClient.quickstart(name="my-agent", domain="my-agent.example.com")
a2a = JACSA2AIntegration(client, trust_policy="strict")

# Assess a remote agent's trustworthiness
assessment = a2a.assess_remote_agent(remote_card_json)
print(f"JACS registered: {assessment['jacs_registered']}")
print(f"Allowed: {assessment['allowed']}")

# Add to trust store (verifies agent's self-signature first)
a2a.trust_a2a_agent(remote_card_json)
```

Async API

```python
from jacs.a2a_discovery import discover_agent, discover_and_assess

card = await discover_agent("https://agent.example.com")
result = await discover_and_assess("https://agent.example.com", policy="verified",
client=client)
```

Node.js

Trust Policies

Policy	Behavior
`open`	Accept all agents without verification
`verified`	Require the JACS provenance extension (`urn:jacs:provenance-v1`) in the agent card (**default**)
`strict`	Require the signer to be in the local trust store

How Trust Flows

```
1. Discover  -- Fetch /.well-known/agent-card.json from a remote URL
2. Assess    -- Check for JACS extension, verify signatures
3. Decide    -- Trust policy determines if the agent is allowed
4. Trust     -- Optionally add the agent to your local trust store
```

With `open` policy, all agents pass step 3. With `verified` , agents must have the JACS extension. With `strict` , agents must be explicitly added to the trust store in step 4 before they pass.

Next Steps

- **Exchange Signed Artifacts** -- Sign and verify artifacts with trusted agents
- **Serve Your Agent Card** -- Make your agent discoverable
- **Trust Store** -- Managing the local trust store

Exchange Signed Artifacts

Sign artifacts with cryptographic provenance and verify artifacts from other agents.

Python

Sign and Verify

```python
from jacs.client import JacsClient

client = JacsClient.quickstart(name="my-agent", domain="my-agent.example.com")

# Sign an artifact
signed = client.sign_artifact({"action": "classify", "input": "data"}, "task")

# Verify it (with trust assessment)
a2a = client.get_a2a()
result = a2a.verify_wrapped_artifact(signed, assess_trust=True)
print(f"Valid: {result['valid']}, Allowed: {result['trust']['allowed']}")
```

Chain of Custody

When multiple agents process data in sequence, link artifacts into a verifiable chain:

```python
# Agent A signs step 1
step1 = client_a.sign_artifact({"step": 1, "data": "raw"}, "message")

# Agent B signs step 2, referencing step 1 as parent
step2 = client_b.sign_artifact(
    {"step": 2, "data": "processed"},
    "message",
    parent_signatures=[step1],
)

# Verify the full chain
result = a2a.verify_wrapped_artifact(step2)
assert result["valid"]
assert result["parent_signatures_valid"]
```

Build an Audit Trail

```
chain = a2a.create_chain_of_custody([step1, step2])
# chain contains: steps (ordered), signers, timestamps, validity
```

Node.js

Artifact Types

The `artifact_type` parameter labels the payload for downstream processing:

Type	Use Case
`task`	Task assignments, work requests
`message`	Inter-agent messages
`result`	Task results, responses

You can use any string -- these are conventions, not enforced types.

What Gets Signed

Every signed artifact includes:

Field	Description
`jacsId`	Unique document ID
`jacsSignature`	Signer ID, algorithm, timestamp, and base64 signature
`jacsSha256`	Content hash for integrity verification
`jacsType`	The artifact type label
`jacsParentSignatures`	Parent artifacts for chain of custody (if any)
`payload`	The original artifact data

Non-JACS receivers can safely ignore the `jacs*` fields and extract `payload` directly.

Next Steps

- **Serve Your Agent Card** -- Make your agent discoverable
- **Discover & Trust Remote Agents** -- Find and assess other agents

- **A2A Interoperability Reference** -- Full API reference
- **Hero Demo (Python)** -- 3-agent trust verification example

Sign vs. Attest: When to Use Which

This guide helps you choose the right JACS API for your use case.

Decision Tree

Start here: What do you need to prove?

1. **"This data hasn't been tampered with"**
   - Use `sign_message()` / `signMessage()`
   - This gives you a cryptographic signature and integrity hash.
2. **"This data hasn't been tampered with AND here's why it should be trusted"**
   - Use `create_attestation()` / `createAttestation()`
   - This gives you signature + integrity + claims + evidence + derivation chain.
3. **"I have an existing signed document and want to add trust context"**
   - Use `lift_to_attestation()` / `liftToAttestation()`
   - This wraps an existing JACS-signed document into a new attestation.
4. **"I need to export a trust proof for external systems"**
   - Use `export_attestation_dsse()` / `exportAttestationDsse()`
   - This creates an in-toto DSSE envelope compatible with SLSA and Sigstore.
5. **"I need to send signed data to another agent or service"**
   - Use `sign_artifact()` / `signArtifact()` via the A2A integration.
   - This wraps your data in a JACS provenance envelope for cross-boundary exchange.
   - See A2A Interoperability and A2A + Attestation Composition.

Quick Reference

Scenario	API	Output
Log an AI action	`sign_message()`	Signed document
Record a human review decision	`create_attestation()`	Attestation with claims
Attach evidence from another system	`create_attestation()` with `evidence`	Attestation with evidence refs

Scenario	API	Output
Wrap an existing signed doc with trust context	`lift_to_attestation()`	New attestation referencing original
Export for SLSA/Sigstore	`export_attestation_dsse()`	DSSE envelope
Verify signature only	`verify()`	Valid/invalid + signer
Verify signature + claims + evidence	`verify_attestation(full=True)`	Full verification result
Exchange artifact with another agent	`sign_artifact()` / A2A	Signed wrapped artifact

Examples

Just need integrity? Use signing.

```
signed = client.sign_message({"action": "approve"})
result = client.verify(signed.raw_json)
# result["valid"] == True
```

Need trust context? Use attestation.

```
att = client.create_attestation(
    subject={"type": "artifact", "id": "doc-001", "digests": {"sha256": "..."}},
    claims=[{"name": "reviewed", "value": True, "confidence": 0.95}],
)
result = client.verify_attestation(att.raw_json, full=True)
# result["valid"] == True, result["evidence"] == [...]
```

Already signed? Lift to attestation.

```
signed = client.sign_message({"content": "original"})
att = client.lift_to_attestation(signed, [{"name": "approved", "value": True}])
# att now has attestation metadata referencing the original document
```

Common Patterns

AI Agent Action Logging

Use `sign_message()` for each tool call or action. The signature proves the agent took the action and the data hasn't been modified.

Human Review Attestation

Use `create_attestation()` with claims like `reviewed_by: human` and `confidence: 0.95`. This creates an auditable record that a human reviewed and approved the output.

Multi-step Pipeline

Use `create_attestation()` with a `derivation` field to capture input/output relationships. Each step attests to its own transformation with references to upstream attestations.

Cross-system Verification

Use `export_attestation_dsse()` to generate an in-toto DSSE envelope that external systems (SLSA verifiers, Sigstore) can validate independently.

Tutorial: Add Attestations to Your Workflow

This step-by-step tutorial walks you through adding attestation support to an existing JACS workflow. You'll go from basic signing to full attestation creation and verification in under 5 minutes.

Prerequisites

- JACS installed (Python, Node.js, or CLI)
- Attestation feature enabled (built with `--features attestation`)

Step 1: Create an Agent

Use an ephemeral agent for testing (no files on disk):

{{#tabs }} {{#tab name="Python" }}

```python
from jacs.client import JacsClient

client = JacsClient.ephemeral(algorithm="ed25519")
print(f"Agent ID: {client.agent_id}")
```

{{#endtab }} {{#tab name="Node.js" }}

```node
const { JacsClient } = require('@hai.ai/jacs/client');

const client = await JacsClient.ephemeral('ring-Ed25519');
console.log(`Agent ID: ${client.agentId}`);
```

{{#endtab }} {{#tab name="CLI" }}

```cli
export JACS_PRIVATE_KEY_PASSWORD="YourP@ssw0rd"
jacs quickstart --algorithm ed25519
```

{{#endtab }} {{#endtabs }}

Step 2: Sign a Document

Sign some data to establish the base document:

{{#tabs }} {{#tab name="Python" }}

```python
signed = client.sign_message({"action": "approve", "amount": 100})
print(f"Document ID: {signed.document_id}")
```

{{#endtab }} {{#tab name="Node.js" }}

```node
const signed = await client.signMessage({ action: 'approve', amount: 100 });
console.log(`Document ID: ${signed.documentId}`);
```

{{#endtab }} {{#endtabs }}

Step 3: Create an Attestation

Now add trust context -- *why* this document should be trusted:

{{#tabs }} {{#tab name="Python" }}

```python
import hashlib
content_hash = hashlib.sha256(signed.raw_json.encode()).hexdigest()
attestation = client.create_attestation(
    subject={
        "type": "artifact",
        "id": signed.document_id,
        "digests": {"sha256": content_hash},
    },
    claims=[
        {
            "name": "reviewed_by",
            "value": "human",
            "confidence": 0.95,
            "assuranceLevel": "verified",
        }
    ],
)
print(f"Attestation ID: {attestation.document_id}")
```

{{#endtab }} {{#tab name="Node.js" }}

```
const { createHash } = require('crypto');
const contentHash = createHash('sha256').update(signed.raw).digest('hex');
const attestation = await client.createAttestation({
  subject: {
    type: 'artifact',
    id: signed.documentId,
    digests: { sha256: contentHash },
  },
  claims: [{
    name: 'reviewed_by',
    value: 'human',
    confidence: 0.95,
    assuranceLevel: 'verified',
  }],
});
console.log(`Attestation ID: ${attestation.documentId}`);
```

{{#endtab }} {{#endtabs }}

Step 4: Verify the Attestation

Local Verification (fast -- signature + hash only)

{{#tabs }} {{#tab name="Python" }}

```
result = client.verify_attestation(attestation.raw_json)
print(f"Valid: {result['valid']}")
print(f"Signature OK: {result['crypto']['signature_valid']}")
print(f"Hash OK: {result['crypto']['hash_valid']}")
```

{{#endtab }} {{#tab name="Node.js" }}

```
const result = await client.verifyAttestation(attestation.raw);
console.log(`Valid: ${result.valid}`);
console.log(`Signature OK: ${result.crypto.signature_valid}`);
console.log(`Hash OK: ${result.crypto.hash_valid}`);
```

{{#endtab }} {{#endtabs }}

Full Verification (thorough -- includes evidence + derivation chain)

{{#tabs }} {{#tab name="Python" }}

```python
full = client.verify_attestation(attestation.raw_json, full=True)
print(f"Valid: {full['valid']}")
print(f"Evidence: {full.get('evidence', [])}")
print(f"Chain: {full.get('chain')}")
```

{{#endtab }} {{#tab name="Node.js" }}

```node
const full = await client.verifyAttestation(attestation.raw, { full: true });
console.log(`Valid: ${full.valid}`);
console.log(`Evidence: ${JSON.stringify(full.evidence)}`);
console.log(`Chain: ${JSON.stringify(full.chain)}`);
```

{{#endtab }} {{#endtabs }}

Step 5: Add Evidence (Optional)

Evidence references link to external proofs that support your claims:

{{#tabs }} {{#tab name="Python" }}

```python
attestation_with_evidence = client.create_attestation(
    subject={
        "type": "artifact",
        "id": "doc-001",
        "digests": {"sha256": "abc123..."},
    },
    claims=[{"name": "scanned", "value": True, "confidence": 1.0}],
    evidence=[
        {
            "kind": "custom",
            "digests": {"sha256": "evidence-hash..."},
            "uri": "https://scanner.example.com/results/123",
            "collectedAt": "2026-03-04T00:00:00Z",
            "verifier": {"name": "security-scanner", "version": "2.0"},
        }
    ],
)
```

{{#endtab }} {{#tab name="Node.js" }}

```
const attWithEvidence = await client.createAttestation({
  subject: {
    type: 'artifact',
    id: 'doc-001',
    digests: { sha256: 'abc123...' },
  },
  claims: [{ name: 'scanned', value: true, confidence: 1.0 }],
  evidence: [{
    kind: 'custom',
    digests: { sha256: 'evidence-hash...' },
    uri: 'https://scanner.example.com/results/123',
    collectedAt: '2026-03-04T00:00:00Z',
    verifier: { name: 'security-scanner', version: '2.0' },
  }],
});
```

{{#endtab }} {{#endtabs }}

Step 6: Export as DSSE (Optional)

Export your attestation as a DSSE (Dead Simple Signing Envelope) for compatibility with in-toto, SLSA, and Sigstore:

{{#tabs }} {{#tab name="Python" }}

```
envelope = client.export_attestation_dsse(attestation.raw_json)
print(f"Payload type: {envelope['payloadType']}")
print(f"Signatures: {len(envelope['signatures'])}")
```

{{#endtab }} {{#tab name="Node.js" }}

```
const envelope = await client.exportAttestationDsse(attestation.raw);
console.log(`Payload type: ${envelope.payloadType}`);
console.log(`Signatures: ${envelope.signatures.length}`);
```

{{#endtab }} {{#endtabs }}

What's Next?

- Sign vs. Attest decision guide -- when to use which API
- Attestation error catalog -- understand verification results
- What is an attestation? -- concept deep dive

Writing a Custom Evidence Adapter

Evidence adapters normalize external proof sources into JACS attestation claims and evidence references. JACS ships with A2A and Email adapters; you can add your own for JWT tokens, TLSNotary proofs, or any custom evidence source.

What Is an EvidenceAdapter?

An `EvidenceAdapter` is a Rust trait with three methods:

```rust
pub trait EvidenceAdapter: Send + Sync + std::fmt::Debug {
    /// Returns the kind string (e.g., "jwt", "tlsnotary", "custom").
    fn kind(&self) -> &str;

    /// Normalize raw evidence bytes + metadata into claims + evidence reference.
    fn normalize(
        &self,
        raw: &[u8],
        metadata: &serde_json::Value,
    ) -> Result<(Vec<Claim>, EvidenceRef), Box<dyn Error>>;

    /// Verify a previously created evidence reference.
    fn verify_evidence(
        &self,
        evidence: &EvidenceRef,
    ) -> Result<EvidenceVerificationResult, Box<dyn Error>>;
}
```

The adapter lifecycle:

1. **At attestation creation time:** `normalize()` is called with raw evidence bytes and optional metadata. It returns structured claims and an `EvidenceRef` that will be embedded in the attestation document.
2. **At verification time (full tier):** `verify_evidence()` is called with the stored `EvidenceRef` to re-validate the evidence.

The normalize() Contract

`normalize()` must:

- Compute content-addressable digests of the raw evidence using `compute_digest_set_bytes()`
- Decide whether to embed the evidence (recommended for data under 64KB)
- Extract meaningful claims from the evidence

- Set appropriate `confidence` and `assuranceLevel` values
- Include a `collectedAt` timestamp
- Return a `VerifierInfo` identifying your adapter and version

`normalize()` must NOT:

- Make network calls (normalization should be deterministic and fast)
- Modify the raw evidence
- Set confidence to 1.0 unless the evidence is self-verifying (e.g., a valid cryptographic proof)

The verify_evidence() Contract

`verify_evidence()` must:

- Verify the digest integrity (re-hash and compare)
- Check freshness (is the `collectedAt` timestamp within acceptable bounds?)
- Return a detailed `EvidenceVerificationResult` with `digest_valid`, `freshness_valid`, and human-readable `detail`

`verify_evidence()` may:

- Make network calls (for remote evidence resolution)
- Access the file system (for local evidence files)
- Return partial results (e.g., digest valid but freshness expired)

Step-by-Step: Building a JWT Adapter

Here is a complete example of a JWT evidence adapter:

```rust
use crate::attestation::adapters::EvidenceAdapter;
use crate::attestation::digest::compute_digest_set_bytes;
use crate::attestation::types::*;
use serde_json::Value;
use std::error::Error;

#[derive(Debug)]
pub struct JwtAdapter;

impl EvidenceAdapter for JwtAdapter {
    fn kind(&self) -> &str {
        "jwt"
    }

    fn normalize(
        &self,
        raw: &[u8],
        metadata: &Value,
    ) -> Result<(Vec<Claim>, EvidenceRef), Box<dyn Error>> {
        // 1. Parse the JWT (header.payload.signature)
        let jwt_str = std::str::from_utf8(raw)?;
        let parts: Vec<&str> = jwt_str.split('.').collect();
        if parts.len() != 3 {
            return Err("Invalid JWT: expected 3 dot-separated parts".into());
        }

        // 2. Decode the payload (base64url)
        let payload_bytes = base64::Engine::decode(
            &base64::engine::general_purpose::URL_SAFE_NO_PAD,
            parts[1],
        )?;
        let payload: Value = serde_json::from_slice(&payload_bytes)?;

        // 3. Compute content-addressable digests
        let digests = compute_digest_set_bytes(raw);

        // 4. Extract claims (only non-PII fields per TRD Decision 14)
        let mut claims = vec![];
        if let Some(iss) = payload.get("iss") {
            claims.push(Claim {
                name: "jwt-issuer".into(),
                value: iss.clone(),
                confidence: Some(0.8),
                assurance_level: Some(AssuranceLevel::Verified),
                issuer: iss.as_str().map(String::from),
                issued_at: Some(crate::time_utils::now_rfc3339()),
            });
        }
        if let Some(sub) = payload.get("sub") {
            claims.push(Claim {
                name: "jwt-subject".into(),
                value: sub.clone(),
                confidence: Some(0.8),
                assurance_level: Some(AssuranceLevel::Verified),
                issuer: None,
                issued_at: None,
            });
        }
```

```rust
        // 5. Build the evidence reference
        let evidence = EvidenceRef {
            kind: EvidenceKind::Jwt,
            digests,
            uri: metadata.get("uri").and_then(|v| v.as_str()).map(String::from),
            embedded: raw.len() < 65536,
            embedded_data: if raw.len() < 65536 {
                Some(Value::String(jwt_str.to_string()))
            } else {
                None
            },
            collected_at: crate::time_utils::now_rfc3339(),
            resolved_at: None,
            sensitivity: EvidenceSensitivity::Restricted, // JWTs may contain PII
            verifier: VerifierInfo {
                name: "jacs-jwt-adapter".into(),
                version: env!("CARGO_PKG_VERSION").into(),
            },
        };

        Ok((claims, evidence))
    }

    fn verify_evidence(
        &self,
        evidence: &EvidenceRef,
    ) -> Result<EvidenceVerificationResult, Box<dyn Error>> {
        // Re-verify the digest
        let digest_valid = if let Some(ref data) = evidence.embedded_data {
            let raw = data.as_str().unwrap_or("").as_bytes();
            let recomputed = compute_digest_set_bytes(raw);
            recomputed.sha256 == evidence.digests.sha256
        } else {
            // Cannot verify without embedded data or fetchable URI
            false
        };

        // Check freshness (example: 5 minute max age)
        let freshness_valid = true; // Implement actual time check

        Ok(EvidenceVerificationResult {
            kind: "jwt".into(),
            digest_valid,
            freshness_valid,
            detail: if digest_valid {
                "JWT digest verified".into()
            } else {
                "JWT digest mismatch or data unavailable".into()
            },
        })
    }
}
```

Testing Your Adapter

Write tests that cover:

1. **Normal case:** Valid evidence normalizes to expected claims
2. **Invalid input:** Malformed evidence returns a clear error
3. **Digest verification:** Round-trip through normalize + verify_evidence
4. **Empty evidence:** Edge case handling

```rust
#[cfg(test)]
mod tests {
    use super::*;
    use serde_json::json;

    #[test]
    fn jwt_normalize_extracts_issuer() {
        let adapter = JwtAdapter;
        // Build a minimal JWT (header.payload.signature)
        let header = base64::Engine::encode(
            &base64::engine::general_purpose::URL_SAFE_NO_PAD,
            b"{\"alg\":\"RS256\"}",
        );
        let payload = base64::Engine::encode(
            &base64::engine::general_purpose::URL_SAFE_NO_PAD,
            b"{\"iss\":\"auth.example.com\",\"sub\":\"user-123\"}",
        );
        let jwt = format!("{}.{}.fake-sig", header, payload);

        let (claims, evidence) = adapter
            .normalize(jwt.as_bytes(), &json!({}))
            .expect("normalize should succeed");

        assert!(claims.iter().any(|c| c.name == "jwt-issuer"));
        assert_eq!(evidence.kind, EvidenceKind::Jwt);
    }
}
```

Registering Your Adapter with the Agent

Adapters are registered on the `Agent` struct via the evidence adapter list. To add your adapter to the default set, modify `adapters/mod.rs`:

```rust
pub fn default_adapters() -> Vec<Box<dyn EvidenceAdapter>> {
    vec![
        Box::new(a2a::A2aAdapter),
        Box::new(email::EmailAdapter),
        Box::new(jwt::JwtAdapter),  // Add your adapter here
    ]
}
```

For runtime registration (without modifying JACS source), use the agent's adapter API (when available in a future release).

Privacy Considerations

The `EvidenceSensitivity` enum controls how evidence is handled:

- **Public:** Evidence can be freely shared and embedded
- **Restricted:** Evidence should be handled with care; consider redacting PII
- **Confidential:** Evidence should not be embedded; use content-addressable URI references only

For JWTs and other credential-based evidence, default to `Restricted` and only include non-PII fields (`iss` , `sub` , `aud` , `iat` , `exp`) in claims.

Framework Adapter Attestation Guide

JACS provides Python framework adapters for LangChain, FastAPI, CrewAI, and Anthropic. Each adapter can be configured to produce attestations (not just signatures) for tool calls, API requests, and agent actions.

Common Patterns

All framework adapters share these attestation patterns:

Default Claims

When `attest=True` is enabled on any adapter, it automatically includes these default claims:

```
[
    {"name": "framework", "value": "langchain", "confidence": 1.0},
    {"name": "tool_name", "value": "my_tool", "confidence": 1.0},
    {"name": "timestamp", "value": "2026-03-04T...", "confidence": 1.0},
]
```

Custom Claims

Add your own claims to any adapter call:

```
extra_claims = [
    {"name": "reviewed_by", "value": "human", "confidence": 0.95},
    {"name": "approved", "value": True, "assuranceLevel": "verified"},
]
```

Evidence Attachment

Attach evidence references from external systems:

```
evidence = [
    {
        "kind": "custom",
        "digests": {"sha256": "abc123..."},
        "uri": "https://scanner.example.com/report/456",
        "collectedAt": "2026-03-04T00:00:00Z",
        "verifier": {"name": "security-scanner", "version": "2.0"},
    }
]
```

LangChain

Enabling Attestation on Tool Calls

Use `jacs_wrap_tool_call` with `attest=True`:

```python
from jacs.adapters.langchain import jacs_wrap_tool_call
from jacs.client import JacsClient

client = JacsClient.quickstart()

# Wrap a tool call with attestation
@jacs_wrap_tool_call(client, attest=True)
def my_tool(query: str) -> str:
    return f"Result for: {query}"

# The tool call now produces a signed attestation
result = my_tool("test query")
# result.attestation contains the signed attestation document
```

Using the signed_tool Decorator

```python
from jacs.adapters.langchain import signed_tool

@signed_tool(client, attest=True, claims=[
    {"name": "data_source", "value": "internal_db", "confidence": 1.0}
])
def lookup_customer(customer_id: str) -> dict:
    return {"name": "Alice", "status": "active"}
```

With LangChain Chains

```python
from jacs.adapters.langchain import with_jacs_signing

# Wrap an entire chain with attestation
signed_chain = with_jacs_signing(
    chain=my_chain,
    client=client,
    attest=True,
)
```

FastAPI

Attestation Middleware

The `JacsMiddleware` can be configured to produce attestations for all responses:

```python
from fastapi import FastAPI
from jacs.adapters.fastapi import JacsMiddleware
from jacs.client import JacsClient

app = FastAPI()
client = JacsClient.quickstart()

app.add_middleware(
    JacsMiddleware,
    client=client,
    attest=True,  # Produce attestations, not just signatures
    default_claims=[
        {"name": "service", "value": "my-api", "confidence": 1.0},
    ],
)
```

Per-Route Attestation

Use `jacs_route` for route-level attestation control:

```python
from jacs.adapters.fastapi import jacs_route

@app.post("/approve")
@jacs_route(client, attest=True, claims=[
    {"name": "action", "value": "approve", "confidence": 1.0},
    {"name": "requires_review", "value": True},
])
async def approve_request(request_id: str):
    return {"approved": True, "request_id": request_id}
```

The response headers will include `X-JACS-Attestation-Id` with the attestation document ID.

CrewAI

Attestation Guardrails

Use `jacs_guardrail` with attestation mode to create trust-verified task execution:

```
from jacs.adapters.crewai import jacs_guardrail, JacsSignedTool
from jacs.client import JacsClient

client = JacsClient.quickstart()

@jacs_guardrail(client, attest=True)
def verified_analysis(task_result):
    """Guardrail that attests to analysis quality."""
    return task_result
```

Signed Tasks

```
from jacs.adapters.crewai import signed_task

@signed_task(client, attest=True, claims=[
    {"name": "analysis_type", "value": "financial", "confidence": 0.9},
])
def analyze_portfolio(data):
    return {"risk_score": 0.3, "recommendation": "hold"}
```

JacsSignedTool

```
class MyTool(JacsSignedTool):
    """A CrewAI tool with built-in attestation."""
    name = "market_data"
    description = "Fetch market data"
    attest = True
    default_claims = [
        {"name": "data_source", "value": "bloomberg"},
    ]

    def _run(self, ticker: str) -> dict:
        return {"ticker": ticker, "price": 150.0}
```

Anthropic

Tool Hook Attestation

The Anthropic adapter hooks into Claude tool calls to produce attestations:

```
from jacs.adapters.anthropic import signed_tool, JacsToolHook
from jacs.client import JacsClient

client = JacsClient.quickstart()

@signed_tool(client, attest=True)
def search_database(query: str) -> str:
    return "Found 3 results"

# Or use the hook class for more control
hook = JacsToolHook(
    client=client,
    attest=True,
    default_claims=[
        {"name": "model", "value": "claude-4.6"},
        {"name": "tool_use_id", "value": "auto"},  # Auto-filled from tool call
    ],
)
```

With the Anthropic SDK

```
import anthropic
from jacs.adapters.anthropic import JacsToolHook

client = anthropic.Anthropic()
jacs_client = JacsClient.quickstart()
hook = JacsToolHook(jacs_client, attest=True)

# Register tools with JACS attestation
tools = hook.wrap_tools([
    {
        "name": "get_weather",
        "description": "Get weather for a location",
        "input_schema": {"type": "object", "properties": {"location": {"type":
"string"}}},
    }
])
```

Verifying Framework Attestations

All framework attestations use the same JACS verification API:

```
# Verify any attestation (from any framework adapter)
result = client.verify_attestation(attestation_json, full=True)
print(f"Valid: {result['valid']}")
print(f"Framework: {result['claims'][0]['value']}")
print(f"Evidence: {result.get('evidence', [])}")
```

Strict vs. Permissive Mode

All adapters respect the `strict` flag on `JacsClient`:

- **Permissive (default):** Signing/attestation failures log warnings but do not block the operation
- **Strict:** Signing/attestation failures raise exceptions and block the operation

```
# Strict mode: attestation failure = operation failure
client = JacsClient.quickstart(strict=True)

# Permissive mode: attestation failure = warning + continue
client = JacsClient.quickstart(strict=False)
```

A2A + Attestation: Using Both Together

A2A provenance and attestation serve different purposes. This guide explains when and how to combine them.

When You Need Both

Use A2A alone when you need to prove *who sent what* across agent boundaries. Use attestation alone when you need to record *why data should be trusted* within a single agent's workflow.

Use both when:

- You send data to another agent AND need to explain why it's trustworthy
- You receive data from another agent AND want to attest that you reviewed it
- You're building a multi-agent pipeline where each step adds trust evidence

The Composition Rule

> **A2A chain-of-custody provides movement lineage. Attestation derivation provides claim lineage.**

A2A tracks *where* an artifact has been (Agent A → Agent B → Agent C). Attestation tracks *what trust claims* have been made about it (scanned → reviewed → approved).

They compose naturally: an agent receives a signed artifact via A2A, then creates an attestation recording its analysis of that artifact.

Example Workflow

```
Agent A: Signs artifact with A2A provenance
    ↓ (cross-boundary exchange)
Agent B: Verifies A2A signature, attests review with evidence
    ↓ (cross-boundary exchange)
Agent C: Verifies both the A2A chain and the attestation
```

Python

```python
from jacs.client import JacsClient

# --- Agent A: Sign and send ---
agent_a = JacsClient.quickstart(name="scanner", domain="scanner.example.com")
a2a_a = agent_a.get_a2a()
signed = a2a_a.sign_artifact(
    {"scan_result": "clean", "target": "file.bin"},
    "message",
)

# --- Agent B: Receive, verify, attest ---
agent_b = JacsClient.quickstart(name="reviewer", domain="reviewer.example.com")
a2a_b = agent_b.get_a2a()

# Verify the A2A artifact from Agent A
verify_result = a2a_b.verify_wrapped_artifact(signed)
assert verify_result["valid"]

# Now attest WHY the review is trustworthy
import hashlib, json
content_hash = hashlib.sha256(json.dumps(signed, sort_keys=True).encode()).hexdigest()
attestation = agent_b.create_attestation(
    subject={"type": "artifact", "id": signed["jacsId"], "digests": {"sha256":
content_hash}},
    claims=[{"name": "reviewed", "value": True, "confidence": 0.9}],
)

# Send the attestation onward via A2A
attested_artifact = a2a_b.sign_artifact(
    {"attestation_id": attestation.document_id, "original_artifact":
signed["jacsId"]},
    "message",
    parent_signatures=[signed],
)
```

Node.js

```
import { JacsClient } from '@hai.ai/jacs/client';

// --- Agent A: Sign and send ---
const agentA = await JacsClient.quickstart({ name: 'scanner', domain:
'scanner.example.com' });
const a2aA = agentA.getA2A();
const signed = await a2aA.signArtifact(
  { scanResult: 'clean', target: 'file.bin' },
  'message',
);

// --- Agent B: Receive, verify, attest ---
const agentB = await JacsClient.quickstart({ name: 'reviewer', domain:
'reviewer.example.com' });
const a2aB = agentB.getA2A();

const verifyResult = await a2aB.verifyWrappedArtifact(signed);
console.assert(verifyResult.valid);

// Attest the review
const attestation = agentB.createAttestation({
  subject: { type: 'artifact', id: signed.jacsId, digests: { sha256: '...' } },
  claims: [{ name: 'reviewed', value: true, confidence: 0.9 }],
});
```

What NOT to Do

- **Don't use A2A trust policy to validate attestation evidence.** A2A policy (`open` / `verified` / `strict`) controls agent admission, not evidence quality. An `allowed` agent can still produce bad evidence.
- **Don't use attestation to determine transport trust.** Attestation claims don't tell you whether an agent should be allowed to communicate. Use `assess_remote_agent()` for that.
- **Don't conflate chain-of-custody with derivation chain.** A2A parent signatures track artifact movement. Attestation derivation tracks how one claim was produced from another. They are complementary, not interchangeable.

Further Reading

- Trust Layers — the three-layer model and terminology
- A2A Interoperability — full A2A reference
- Attestation Tutorial — creating and verifying attestations
- Sign vs. Attest — choosing the right API

Observability & Monitoring Guide

JACS emits structured events at every signing, verification, and agreement lifecycle step. This guide shows you how to capture those events and route them to your monitoring stack.

For Rust-specific API details (ObservabilityConfig, LogDestination, MetricsConfig, etc.), see the Observability (Rust API).

Structured Event Reference

Every event includes an `event` field for filtering. The table below is derived directly from the source code.

Signing Events

Event	Level	Fields	Source
`document_signed`	`info`	`algorithm`, `duration_ms`	`crypt/mod.rs`
`batch_signed`	`info`	`algorithm`, `batch_size`, `duration_ms`	`crypt/mod.rs`
`signing_procedure_complete`	`info`	`agent_id`, `algorithm`, `timestamp`, `placement_key`	`agent/mod.rs`

Verification Events

Event	Level	Fields	Source
`signature_verified`	`info`	`algorithm`, `valid`, `duration_ms`	`crypt/mod.rs`
`verification_complete`	`info` / `error`	`document_id`, `signer_id`, `algorithm`, `timestamp`, `valid`, `duration_ms`	`agent/mod.rs`

`verification_complete` emits at `info` when `valid=true` and at `error` when `valid=false`.

Agreement Events

Event	Level	Fields	Source
`agreement_created`	`info`	`document_id`, `agent_count`, `quorum`, `has_timeout`	`agent/agreement.rs`
`signature_added`	`info`	`document_id`, `signer_id`, `current`, `total`, `required`	`agent/agreement.rs`
`quorum_reached`	`info`	`document_id`, `signatures`, `required`, `total`	`agent/agreement.rs`
`agreement_expired`	`warn`	`document_id`, `deadline`	`agent/agreement.rs`

Enabling OTEL Export

JACS ships with three optional feature flags for OpenTelemetry backends. By default, only stderr and file logging are available.

```
# Enable all three OTEL pipelines
cargo build --features otlp-logs,otlp-metrics,otlp-tracing

# Or enable just tracing
cargo build --features otlp-tracing
```

Feature	What it adds
`otlp-logs`	OTLP log export (opentelemetry, opentelemetry-otlp, opentelemetry-appender-tracing, tokio)
`otlp-metrics`	OTLP metrics export (opentelemetry, opentelemetry-otlp, opentelemetry_sdk, tokio)
`otlp-tracing`	Distributed tracing (opentelemetry, opentelemetry-otlp, tracing-opentelemetry, tokio)

Convenience helpers for automatic counter/gauge recording for sign and verify operations are always available without any feature flag.

OTEL Collector Configuration

Route JACS events through an OpenTelemetry Collector. This configuration receives OTLP over HTTP, batches events, and exports to common backends.

```yaml
# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 5s
    send_batch_size: 512
  filter/jacs:
    logs:
      include:
        match_type: regexp
        record_attributes:
          - key: event
            value:
"document_signed|signature_verified|verification_complete|agreement_.*|batch_signed|si
gning_procedure_complete|quorum_reached|signature_added"

exporters:
  # Debug: print to collector stdout
  debug:
    verbosity: detailed

  # Datadog
  datadog:
    api:
      key: "${DD_API_KEY}"
      site: datadoghq.com

  # Splunk HEC
  splunkhec:
    token: "${SPLUNK_HEC_TOKEN}"
    endpoint: "https://splunk-hec:8088/services/collector"
    source: "jacs"
    sourcetype: "jacs:events"

  # Generic OTLP (Grafana Cloud, Honeycomb, etc.)
  otlphttp:
    endpoint: "${OTLP_ENDPOINT}"
    headers:
      Authorization: "Bearer ${OTLP_API_KEY}"

service:
  pipelines:
    logs:
      receivers: [otlp]
      processors: [batch, filter/jacs]
      exporters: [debug]          # Replace with your exporter
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [debug]
    traces:
      receivers: [otlp]
```

```
      processors: [batch]
      exporters: [debug]
```

Pointing JACS at the Collector

In `jacs.config.json`:

```
{
  "observability": {
    "logs": {
      "enabled": true,
      "level": "info",
      "destination": {
        "otlp": { "endpoint": "http://localhost:4318" }
      }
    },
    "metrics": {
      "enabled": true,
      "destination": {
        "otlp": { "endpoint": "http://localhost:4318" }
      },
      "export_interval_seconds": 30
    },
    "tracing": {
      "enabled": true,
      "sampling": { "ratio": 1.0, "parent_based": true },
      "resource": {
        "service_name": "my-jacs-service",
        "environment": "production"
      },
      "destination": {
        "otlp": { "endpoint": "http://localhost:4318" }
      }
    }
  }
}
```

Or via environment variables (useful in containers):

```
export OTEL_EXPORTER_OTLP_ENDPOINT="http://collector:4318"
export OTEL_SERVICE_NAME="jacs-production"
export OTEL_RESOURCE_ATTRIBUTES="deployment.environment=production"
```

Feeding Events to Datadog

1. Deploy the OTEL Collector with the `datadog` exporter (see config above).
2. Set `DD_API_KEY` in the collector's environment.
3. In Datadog, JACS events appear under **Logs > Search** with `source:opentelemetry`.

4. Create a monitor on `event:verification_complete AND valid:false` to alert on verification failures.

Alternatively, use the Datadog Agent's built-in OTLP receiver:

```
# datadog.yaml
otlp_config:
  receiver:
    protocols:
      http:
        endpoint: 0.0.0.0:4318
```

Feeding Events to Splunk

1. Deploy the OTEL Collector with the `splunkhec` exporter.
2. Set `SPLUNK_HEC_TOKEN` in the collector's environment.
3. Events arrive in Splunk with `sourcetype=jacs:events`.
4. Search: `sourcetype="jacs:events" event="verification_complete" valid=false`

Agreement Monitoring

Agreement events give you a complete lifecycle view: creation, each signature, quorum, and expiry. Here are practical queries.

Agreements Approaching Timeout

Filter for `agreement_created` events where `has_timeout=true`, then correlate with `quorum_reached`. Any `agreement_created` without a matching `quorum_reached` within the timeout window is at risk.

Failed Quorum Detection

```
event="signature_added" | stats max(current) as sigs, max(required) as needed by
document_id
| where sigs < needed
```

Signature Velocity

Track `signature_added` events over time to see how quickly agents sign after agreement creation:

```
event="signature_added" | timechart count by document_id
```

Expiry Alerts

The `agreement_expired` event (level `warn`) fires when an agent attempts to sign or verify an expired agreement. Alert on this directly:

```
event="agreement_expired" | alert
```

Latency Tracking

Both `document_signed` and `signature_verified` include `duration_ms`. Use these to track signing and verification performance:

```
event="document_signed" | stats avg(duration_ms) as avg_sign_ms, p99(duration_ms) as
p99_sign_ms by algorithm
event="signature_verified" | stats avg(duration_ms) as avg_verify_ms, p99(duration_ms)
as p99_verify_ms by algorithm
```

Post-quantum algorithms (`pq2025`, `pq-dilithium`) will show higher latency than `ring-Ed25519`. Use these metrics to decide whether the security/performance tradeoff is acceptable for your workload.

Next Steps

- Observability (Rust API) -- Full API: ObservabilityConfig, LogDestination, MetricsConfig, TracingConfig
- Algorithm Selection Guide -- Latency implications of algorithm choice
- Failure Modes -- What events to expect when things go wrong

Email Signing and Verification

JACS provides a detached-signature model for email. Your agent signs a raw RFC 5322 `.eml` file and the result is the same email with a `jacs-signature.json` MIME attachment. The recipient extracts that attachment, verifies the cryptographic signature, and compares content hashes to detect tampering.

There are only two functions you need:

Action	Function	What you supply	What you get back
Sign	`jacs::email::sign_email()`	raw `.eml` bytes + your `EmailSigner`	`.eml` bytes with `jacs-signature.json`
Verify	`jacs::email::verify_email()`	signed `.eml` bytes + sender's public key + verifier	`ContentVerificationResult` (pass/fail per field)

Signing an email

```
use jacs::email::{sign_email, EmailSigner};

// 1. Load raw email bytes (RFC 5322 format)
let raw_eml = std::fs::read("outgoing.eml")?;

// 2. Sign — your agent implements EmailSigner (see below)
let signed_eml = sign_email(&raw_eml, &my_agent)?;

// 3. Send signed_eml — it is a valid .eml with the JACS attachment
std::fs::write("outgoing_signed.eml", &signed_eml)?;
```

The `EmailSigner` trait

Your agent must implement four methods:

```rust
pub trait EmailSigner {
    /// Sign raw bytes. Return the signature bytes.
    fn sign_bytes(&self, data: &[u8]) -> Result<Vec<u8>, Box<dyn std::error::Error>>;

    /// Your agent's JACS ID (e.g. "abc123:v1").
    fn jacs_id(&self) -> &str;

    /// The key identifier used for signing.
    fn key_id(&self) -> &str;

    /// The signing algorithm name. This comes from your JACS agent's
    /// key configuration — never hardcode it.
    fn algorithm(&self) -> &str;
}
```

The algorithm value (e.g. `"ed25519"`, `"rsa-pss"`, `"pq2025"`) is read from your JACS agent's key metadata at runtime. `sign_email` records it in the `jacs-signature.json` document so the verifier knows which algorithm to use.

What `sign_email` does internally

1. Parses and canonicalizes the email headers and body
2. Computes SHA-256 hashes for each header, body part, and attachment
3. Builds the JACS email signature payload
4. Canonicalizes the payload via RFC 8785 (JCS)
5. Calls your `sign_bytes()` to produce the cryptographic signature
6. Attaches the result as `jacs-signature.json`

You do not need to know any of this to use it — it is a single function call.

Forwarding (re-signing)

If the email already has a `jacs-signature.json` (it was previously signed by another agent), `sign_email` automatically:

1. Renames the existing signature to `jacs-signature-0.json` (or `-1`, `-2`, ...)
2. Computes a `parent_signature_hash` linking to the previous signature
3. Signs the email with a new `jacs-signature.json`

This builds a verifiable forwarding chain. No extra code needed.

Verifying an email

One-call API (recommended)

```
use jacs::email::verify_email;
use jacs::simple::SimpleAgent;

let signed_eml = std::fs::read("incoming_signed.eml")?;
let sender_public_key: Vec<u8> = /* fetch from HAI registry or local store */;

// Any agent can verify — the sender's public key is passed explicitly
let (agent, _) = SimpleAgent::ephemeral(Some("ed25519"))?;
let result = verify_email(&signed_eml, &agent, &sender_public_key)?;

if result.valid {
    println!("Email is authentic and unmodified");
} else {
    // Inspect which fields failed
    for field in &result.field_results {
        println!("{}: {:?}", field.field, field.status);
    }
}
```

`verify_email` does everything in one call:

1. Extracts `jacs-signature.json` from the email
2. Removes it (the signature covers the email *without* itself)
3. Verifies the JACS document signature against the sender's public key
4. Compares every hash in the JACS document against the actual email content
5. Returns per-field results

Two-step API (when you need the JACS document)

If you need to inspect the JACS document metadata (issuer, timestamps) before doing the content comparison:

```rust
use jacs::email::{verify_email_document, verify_email_content};
use jacs::simple::SimpleAgent;

let (agent, _) = SimpleAgent::ephemeral(Some("ed25519"))?;

// Step 1: Verify the cryptographic signature — returns the trusted JACS document
let (doc, parts) = verify_email_document(&signed_eml, &agent, &sender_public_key)?;

// Inspect the document
println!("Signed by: {}", doc.metadata.issuer);
println!("Created at: {}", doc.metadata.created_at);

// Step 2: Compare content hashes
let result = verify_email_content(&doc, &parts);
assert!(result.valid);
```

All cryptographic operations are handled by the JACS agent via `SimpleAgent::verify_with_key()`. The agent's own key is not used -- the sender's public key is passed explicitly.

Field-level results

The `ContentVerificationResult` contains a `field_results` vector with one entry per field:

Status	Meaning
`Pass`	Hash matches — field is authentic
`Modified`	Hash mismatch but case-insensitive email address match (address headers only)
`Fail`	Content does not match the signed hash
`Unverifiable`	Field absent or not verifiable (e.g. Message-ID may change in transit)

Fields checked: `from`, `to`, `cc`, `subject`, `date`, `message_id`, `in_reply_to`, `references`, `body_plain`, `body_html`, and all attachments.

The JACS signature document

The `jacs-signature.json` attachment has this structure:

```
{
  "version": "1.0",
  "document_type": "email_signature",
  "payload": {
    "headers": {
      "from":       { "value": "agent@example.com", "hash": "sha256:..." },
      "to":         { "value": "recipient@example.com", "hash": "sha256:..." },
      "subject":    { "value": "Hello", "hash": "sha256:..." },
      "date":       { "value": "Fri, 28 Feb 2026 12:00:00 +0000", "hash": "sha256:..."
},
      "message_id": { "value": "<msg@example.com>", "hash": "sha256:..." }
    },
    "body_plain": { "content_hash": "sha256:..." },
    "body_html":  null,
    "attachments": [
      { "filename": "report.pdf", "content_hash": "sha256:..." }
    ],
    "parent_signature_hash": null
  },
  "metadata": {
    "issuer": "agent-jacs-id:v1",
    "document_id": "uuid",
    "created_at": "2026-02-28T12:00:00Z",
    "hash": "sha256:..."
  },
  "signature": {
    "key_id": "agent-key-id",
    "algorithm": "ed25519",
    "signature": "base64...",
    "signed_at": "2026-02-28T12:00:00Z"
  }
}
```

`metadata.hash` is the SHA-256 of the RFC 8785 canonical JSON of `payload`. `signature.signature` is the cryptographic signature over that same canonical JSON. The algorithm is always read from the agent — never hardcoded.

Public API summary

All items are re-exported from `jacs::email`:

```
// Signing
jacs::email::sign_email(raw_email: &[u8], signer: &dyn EmailSigner) -> Result<Vec<u8>,
EmailError>
jacs::email::EmailSigner                  // trait your agent implements

// Verification
jacs::email::verify_email(raw, &agent, pubkey)       // one-call: crypto + content
check
jacs::email::verify_email_document(raw, &agent, pk)  // step 1: crypto only
jacs::email::verify_email_content(&doc, &parts)      // step 2: content hash
comparison
jacs::email::normalize_algorithm(...)                 // algorithm name normalization

// Types
jacs::email::ContentVerificationResult    // overall result with field_results
jacs::email::FieldResult                  // per-field status
jacs::email::FieldStatus                  // Pass | Modified | Fail | Unverifiable
jacs::email::JacsEmailSignatureDocument   // the full signature document
jacs::email::EmailError                   // error type

// Attachment helpers (for advanced use)
jacs::email::get_jacs_attachment(...)     // extract jacs-signature.json bytes
jacs::email::remove_jacs_attachment(...)  // strip jacs-signature.json from email
jacs::email::add_jacs_attachment(...)     // inject jacs-signature.json into email
```

Streaming Signing

JACS uses a **buffer-then-sign** pattern for streaming outputs. Token streams from LLMs are accumulated in memory and signed once the stream completes. This is the correct approach for LLM outputs because:

1. **LLM responses are small.** A typical response is under 100KB of text. Buffering this costs nothing.
2. **Signatures cover the complete output.** A partial signature over incomplete text is useless for verification.
3. **Framework adapters handle this automatically.** If you use a JACS adapter, streaming signing just works.

How It Works by Framework

Vercel AI SDK (`streamText`)

The `wrapStream` middleware accumulates `text-delta` chunks via a `TransformStream`. When the stream flushes, it signs the complete text and emits a `provider-metadata` chunk containing the provenance record.

```
import { withProvenance } from '@hai.ai/jacs/vercel-ai';
import { streamText } from 'ai';

const model = withProvenance(openai('gpt-4o'), { client });
const result = await streamText({ model, prompt: 'Explain trust.' });

for await (const chunk of result.textStream) {
  process.stdout.write(chunk); // stream to user in real time
}
// provenance is available after stream completes
```

LangChain / LangGraph

LangChain tools execute synchronously (or await async results) before returning to the model. JACS signs each tool result individually via `wrap_tool_call` or `signed_tool`. No special streaming handling is needed because the signing happens at the tool output boundary, not the token stream.

```python
from jacs.adapters.langchain import jacs_signing_middleware

agent = create_agent(
    model="openai:gpt-4o",
    tools=tools,
    middleware=[jacs_signing_middleware(client=jacs_client)],
)
# Tool results are auto-signed before the model sees them
```

Express / Koa / FastAPI

HTTP middleware signs the response body before it is sent. For streaming HTTP responses (SSE, chunked encoding), sign the complete message content before streaming, or sign each event individually.

```python
# FastAPI: middleware signs JSON responses automatically
from jacs.adapters.fastapi import JacsMiddleware
app.add_middleware(JacsMiddleware)
```

Raw LLM APIs (No Framework Adapter)

If you're calling an LLM API directly without a framework adapter, accumulate the response yourself and sign it when complete:

```python
import jacs.simple as jacs

jacs.quickstart(name="my-agent", domain="my-agent.example.com")

# Accumulate streamed response
chunks = []
async for chunk in llm_stream("What is trust?"):
    chunks.append(chunk)
    print(chunk, end="")  # stream to user

# Sign the complete response
complete_text = "".join(chunks)
signed = jacs.sign_message({"response": complete_text, "model": "gpt-4o"})
```

```
const jacs = require('@hai.ai/jacs/simple');
await jacs.quickstart({ name: 'my-agent', domain: 'my-agent.example.com' });

const chunks = [];
for await (const chunk of llmStream('What is trust?')) {
  chunks.push(chunk);
  process.stdout.write(chunk);
}

const signed = await jacs.signMessage({
  response: chunks.join(''),
  model: 'gpt-4o',
});
```

When NOT to Buffer

The buffer-then-sign pattern assumes the full content fits in memory. This is always true for LLM text responses. If you need to sign very large data (multi-GB files, video streams), use `sign_file` instead, which hashes the file on disk without loading it into memory.

See Also

- Vercel AI SDK Adapter
- LangChain Adapters
- Framework Adapters (Node.js)

CLI Examples

This chapter provides practical examples of using the JACS CLI for common workflows.

Quick Reference

```
jacs init                   # Initialize JACS (config + agent + keys)
jacs agent create           # Create a new agent
jacs document create        # Create a signed document
jacs document verify        # Verify a document signature
jacs document sign-agreement  # Sign an agreement
```

Getting Started

First-Time Setup

Initialize JACS in a new project:

```
# Create a new directory
mkdir my-jacs-project
cd my-jacs-project

# Initialize JACS
jacs init

# This creates:
# - jacs.config.json (configuration)
# - jacs_keys/ (private and public keys)
# - jacs_data/ (document storage)
# - An initial agent document
```

Verify Your Setup

```
# Check the configuration
jacs config read

# Verify your agent
jacs agent verify

# Expected output:
# Agent verification successful
# Agent ID: 550e8400-e29b-41d4-a716-446655440000
# Agent Version: f47ac10b-58cc-4372-a567-0e02b2c3d479
```

Document Operations

Creating Documents

Create from a JSON file:

```
# Create input file
cat > invoice.json << 'EOF'
{
  "type": "invoice",
  "invoiceNumber": "INV-001",
  "customer": "Acme Corp",
  "amount": 1500.00,
  "items": [
    {"description": "Consulting", "quantity": 10, "price": 150}
  ]
}
EOF

# Create signed document
jacs document create -f invoice.json

# Output shows the saved document path
# Document saved to: jacs_data/documents/[uuid]/[version].json
```

Create with custom output:

```
# Specify output filename
jacs document create -f invoice.json -o signed-invoice.json

# Print to stdout (don't save)
jacs document create -f invoice.json --no-save
```

Create with file attachments:

```
# Create document with PDF attachment
jacs document create -f contract.json --attach ./contract.pdf

# Embed attachment content in document
jacs document create -f contract.json --attach ./contract.pdf --embed true

# Attach entire directory
jacs document create -f report.json --attach ./attachments/
```

Create with custom schema:

```
# Use a custom schema for validation
jacs document create -f order.json -s ./schemas/order.schema.json
```

Verifying Documents

Basic verification:

```
# Verify a document
jacs document verify -f ./signed-invoice.json

# Expected output:
# Document verified successfully
# Document ID: 550e8400-e29b-41d4-a716-446655440000
# Signer: Agent Name (agent-uuid)
```

Verbose verification:

```
# Get detailed verification info
jacs document verify -f ./signed-invoice.json -v

# Output includes:
# - Document ID and version
# - Signature algorithm used
# - Signing agent details
# - Timestamp
# - Schema validation results
```

Batch verification:

```
# Verify all documents in a directory
jacs document verify -d ./documents/

# With custom schema
jacs document verify -d ./invoices/ -s ./schemas/invoice.schema.json
```

Updating Documents

Create a new version of an existing document:

```
# Original document
cat > original.json << 'EOF'
{
  "title": "Project Plan",
  "status": "draft",
  "content": "Initial version"
}
EOF

jacs document create -f original.json -o project-v1.json

# Updated content
cat > updated.json << 'EOF'
{
  "title": "Project Plan",
  "status": "approved",
  "content": "Final version with updates"
}
EOF

# Create new version (maintains version history)
jacs document update -f project-v1.json -n updated.json -o project-v2.json

# Verify the updated document
jacs document verify -f project-v2.json -v
```

Extracting Embedded Content

```
# Extract embedded files from a document
jacs document extract -f ./document-with-attachments.json

# Extracts to: jacs_data/extracted/[document-id]/

# Extract from multiple documents
jacs document extract -d ./documents/
```

Agreement Workflows

Creating an Agreement

An agreement requires multiple agents to sign a document:

```
# First, create the document to be agreed upon
cat > service-agreement.json << 'EOF'
{
  "type": "service_agreement",
  "title": "Professional Services Agreement",
  "parties": ["Company A", "Company B"],
  "terms": "...",
  "effectiveDate": "2024-02-01"
}
EOF

jacs document create -f service-agreement.json -o agreement.json

# Create agreement requiring signatures from two agents
# (Use actual agent UUIDs)
jacs document create-agreement \
  -f agreement.json \
  -i "agent1-uuid-here,agent2-uuid-here" \
  -o agreement-pending.json

# Output:
# Agreement created
# Required signatures: 2
# Current signatures: 0
```

Signing an Agreement

```
# First agent signs
jacs document sign-agreement -f agreement-pending.json -o agreement-signed-1.json

# Check status
jacs document check-agreement -f agreement-signed-1.json
# Output:
# Agreement status: pending
# Signatures: 1/2
# Missing: agent2-uuid

# Second agent signs (using their configuration)
JACS_CONFIG_PATH=./agent2.config.json \
  jacs document sign-agreement -f agreement-signed-1.json -o agreement-complete.json

# Verify completion
jacs document check-agreement -f agreement-complete.json
# Output:
# Agreement status: complete
# Signatures: 2/2
```

Complete Agreement Workflow

```bash
#!/bin/bash
# agreement-workflow.sh

# Step 1: Create the contract document
cat > contract.json << 'EOF'
{
  "type": "contract",
  "parties": {
    "seller": "Widget Corp",
    "buyer": "Acme Inc"
  },
  "terms": "Sale of 1000 widgets at $10 each",
  "totalValue": 10000
}
EOF

echo "Creating contract document..."
jacs document create -f contract.json -o contract-signed.json

# Step 2: Get agent IDs
SELLER_AGENT=$(jacs config read | grep agent_id | cut -d: -f2 | tr -d ' ')
BUYER_AGENT="buyer-agent-uuid-here"  # Replace with actual ID

# Step 3: Create agreement
echo "Creating agreement..."
jacs document create-agreement \
  -f contract-signed.json \
  -i "$SELLER_AGENT,$BUYER_AGENT" \
  -o contract-agreement.json

# Step 4: Seller signs
echo "Seller signing..."
jacs document sign-agreement \
  -f contract-agreement.json \
  -o contract-seller-signed.json

# Step 5: Check intermediate status
echo "Checking status..."
jacs document check-agreement -f contract-seller-signed.json

# Step 6: Buyer signs
echo "Buyer signing..."
JACS_CONFIG_PATH=./buyer.config.json \
  jacs document sign-agreement \
  -f contract-seller-signed.json \
  -o contract-complete.json

# Step 7: Verify complete agreement
echo "Final verification..."
jacs document verify -f contract-complete.json -v
jacs document check-agreement -f contract-complete.json

echo "Agreement workflow complete!"
```

Agent Operations

Creating a Custom Agent

```
# Create agent definition file
cat > my-agent.json << 'EOF'
{
  "jacsAgentType": "ai",
  "name": "My Custom Agent",
  "description": "An AI agent for document processing",
  "contact": {
    "email": "agent@example.com"
  },
  "services": [
    {
      "name": "document-processing",
      "description": "Process and sign documents"
    }
  ]
}
EOF

# Create agent with new keys
jacs agent create --create-keys true -f my-agent.json

# Create agent using existing keys
jacs agent create --create-keys false -f my-agent.json
```

DNS-Based Identity

Generate DNS record commands:

```
# Generate TXT record for your domain
jacs agent dns --domain myagent.example.com

# Output (example):
# Add the following DNS TXT record:
# _v1.agent.jacs.myagent.example.com TXT "pk=<base64-public-key-hash>"

# Different providers
jacs agent dns --domain myagent.example.com --provider aws
jacs agent dns --domain myagent.example.com --provider cloudflare
jacs agent dns --domain myagent.example.com --provider azure

# Custom TTL
jacs agent dns --domain myagent.example.com --ttl 7200
```

Verify DNS-published agent:

```
# Look up agent by domain
jacs agent lookup partner.example.com

# Require strict DNSSEC validation
jacs agent lookup partner.example.com --strict

# Verify local agent file against DNS
jacs agent verify -a ./partner-agent.json --require-strict-dns
```

Agent Verification

```
# Basic verification
jacs agent verify

# Verify another agent's file
jacs agent verify -a ./other-agent.json

# With DNS requirements
jacs agent verify --require-dns          # Require DNS (not strict)
jacs agent verify --require-strict-dns   # Require DNSSEC
jacs agent verify --no-dns              # Skip DNS entirely
jacs agent verify --ignore-dns          # Ignore DNS validation failures
```

Task Management

Creating Tasks

```
# Simple task
jacs task create \
  -n "Review Contract" \
  -d "Review the service contract and provide feedback"

# Task with additional data from file
cat > task-details.json << 'EOF'
{
  "priority": "high",
  "dueDate": "2024-02-15",
  "assignee": "legal-team"
}
EOF

jacs task create \
  -n "Contract Review" \
  -d "Detailed review required" \
  -f task-details.json
```

Scripting Examples

Batch Document Processing

```bash
#!/bin/bash
# batch-sign.sh - Sign all JSON files in a directory

INPUT_DIR=$1
OUTPUT_DIR=${2:-"./signed"}

mkdir -p "$OUTPUT_DIR"

for file in "$INPUT_DIR"/*.json; do
  filename=$(basename "$file")
  echo "Signing: $filename"

  jacs document create -f "$file" -o "$OUTPUT_DIR/$filename"

  if [ $? -eq 0 ]; then
    echo "  ✓ Signed successfully"
  else
    echo "  ✗ Signing failed"
  fi
done

echo "Batch signing complete. Output in $OUTPUT_DIR"
```

Verification Report

```bash
#!/bin/bash
# verify-report.sh - Generate verification report

DOC_DIR=$1
REPORT="verification-report.txt"

echo "JACS Document Verification Report" > $REPORT
echo "Generated: $(date)" >> $REPORT
echo "================================" >> $REPORT
echo "" >> $REPORT

passed=0
failed=0

for file in "$DOC_DIR"/*.json; do
  filename=$(basename "$file")

  if jacs document verify -f "$file" > /dev/null 2>&1; then
    echo "✓ PASS: $filename" >> $REPORT
    ((passed++))
  else
    echo "✗ FAIL: $filename" >> $REPORT
    ((failed++))
  fi
done

echo "" >> $REPORT
echo "Summary: $passed passed, $failed failed" >> $REPORT

cat $REPORT
```

Watch for New Documents

```bash
#!/bin/bash
# watch-and-verify.sh - Monitor directory and verify new documents

WATCH_DIR=${1:-"./incoming"}

echo "Watching $WATCH_DIR for new documents..."

inotifywait -m "$WATCH_DIR" -e create -e moved_to |
  while read dir action file; do
    if [[ "$file" == *.json ]]; then
      echo "New document: $file"

      if jacs document verify -f "$WATCH_DIR/$file"; then
        mv "$WATCH_DIR/$file" "./verified/"
        echo "  Moved to verified/"
      else
        mv "$WATCH_DIR/$file" "./rejected/"
        echo "  Moved to rejected/"
      fi
    fi
  done
```

Environment Configuration

Using Environment Variables

```bash
# Use a specific config file
export JACS_CONFIG_PATH=./production.config.json
jacs document create -f invoice.json

# Override specific settings
export JACS_DATA_DIRECTORY=./custom-data
export JACS_KEY_DIRECTORY=./secure-keys
jacs agent create --create-keys true

# One-time override
JACS_CONFIG_PATH=./test.config.json jacs document verify -f test-doc.json
```

Multiple Configurations

```
# Development
alias jacs-dev='JACS_CONFIG_PATH=./dev.config.json jacs'
jacs-dev document create -f test.json

# Production
alias jacs-prod='JACS_CONFIG_PATH=./prod.config.json jacs'
jacs-prod document verify -f important.json

# Different agents
alias jacs-alice='JACS_CONFIG_PATH=./alice.config.json jacs'
alias jacs-bob='JACS_CONFIG_PATH=./bob.config.json jacs'
```

Error Handling

Understanding Exit Codes

```
jacs document verify -f document.json
exit_code=$?

case $exit_code in
  0) echo "Success" ;;
  1) echo "General error" ;;
  2) echo "Invalid arguments" ;;
  3) echo "File not found" ;;
  4) echo "Verification failed" ;;
  5) echo "Signature invalid" ;;
  *) echo "Unknown error: $exit_code" ;;
esac
```

Handling Failures

```bash
#!/bin/bash
# robust-signing.sh

sign_document() {
  local input=$1
  local output=$2

  if ! jacs document create -f "$input" -o "$output" 2>/dev/null; then
    echo "Error: Failed to sign $input" >&2
    return 1
  fi

  if ! jacs document verify -f "$output" 2>/dev/null; then
    echo "Error: Verification failed for $output" >&2
    rm -f "$output"
    return 1
  fi

  echo "Successfully signed: $output"
  return 0
}

# Usage
sign_document "invoice.json" "signed-invoice.json" || exit 1
```

See Also

- CLI Command Reference - Complete command reference
- Configuration Reference - Configuration options
- Rust CLI Usage - Detailed CLI documentation

Node.js Examples

This chapter provides practical Node.js examples using the `@hai.ai/jacs` package.

Setup

```
# Install dependencies
npm install @hai.ai/jacs express @modelcontextprotocol/sdk zod
```

v0.7.0 uses an async-first API. All NAPI operations return Promises by default; sync variants use a `Sync` suffix.

```
// Initialize JACS (ES Modules, async)
import { JacsAgent } from '@hai.ai/jacs';

const agent = new JacsAgent();
await agent.load('./jacs.config.json');
```

Basic Document Operations

Creating and Signing Documents

```
import { JacsAgent } from '@hai.ai/jacs';

async function createSignedDocument() {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  // Create a simple document
  const content = {
    title: 'Invoice',
    invoiceNumber: 'INV-001',
    amount: 1500.00,
    customer: 'Acme Corp',
    items: [
      { description: 'Consulting', quantity: 10, price: 150 }
    ]
  };

  // Create and sign the document
  const signedDoc = await agent.createDocument(JSON.stringify(content));

  // Parse the result
  const doc = JSON.parse(signedDoc);
  console.log('Document ID:', doc.jacsId);
  console.log('Version:', doc.jacsVersion);
  console.log('Signature:', doc.jacsSignature ? 'Present' : 'Missing');

  return doc;
}

createSignedDocument();
```

Verifying Documents

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';

async function verifyDocument(filePath) {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  // Read the document
  const docString = fs.readFileSync(filePath, 'utf-8');

  // Verify signature
  const isValid = await agent.verifyDocument(docString);

  if (isValid) {
    console.log('✓ Document signature is valid');
    const doc = JSON.parse(docString);
    console.log('  Signed by:', doc.jacsSignature?.agentID);
    console.log('  Signed at:', doc.jacsSignature?.date);
  } else {
    console.log('✗ Document signature is INVALID');
  }

  return isValid;
}

verifyDocument('./invoice.json');
```

Updating Documents

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';

async function updateDocument(originalPath, newContent) {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  // Read original document
  const originalDoc = fs.readFileSync(originalPath, 'utf-8');

  // Update with new content (preserves version chain)
  const updatedDoc = await agent.updateDocument(
    originalDoc,
    JSON.stringify(newContent)
  );

  const doc = JSON.parse(updatedDoc);
  console.log('Updated Document ID:', doc.jacsId);
  console.log('New Version:', doc.jacsVersion);

  return doc;
}

// Usage
const updated = await updateDocument('./invoice-v1.json', {
  title: 'Invoice',
  invoiceNumber: 'INV-001',
  amount: 1500.00,
  customer: 'Acme Corp',
  status: 'paid'  // New field
});
```

HTTP Server with Express

Complete Express Server

```
import express from 'express';
import { JACSExpressMiddleware } from '@hai.ai/jacs/http';
import { JacsAgent } from '@hai.ai/jacs';

const app = express();
const PORT = 3000;

// Initialize JACS
const agent = new JacsAgent();
await agent.load('./jacs.config.json');

// Health check (no JACS)
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

// JACS-protected API routes
app.use('/api', express.text({ type: '*/*' }));
app.use('/api', JACSExpressMiddleware({
  configPath: './jacs.config.json'
}));

// Validation middleware
function requirePayload(req, res, next) {
  if (!req.jacsPayload) {
    return res.status(400).json({
      error: 'Invalid JACS request',
      message: 'Request must be signed with valid JACS credentials'
    });
  }
  next();
}

// Echo endpoint
app.post('/api/echo', requirePayload, (req, res) => {
  res.send({
    echo: req.jacsPayload,
    serverTime: new Date().toISOString()
  });
});

// Calculate endpoint
app.post('/api/calculate', requirePayload, (req, res) => {
  const { operation, a, b } = req.jacsPayload;

  let result;
  switch (operation) {
    case 'add': result = a + b; break;
    case 'subtract': result = a - b; break;
    case 'multiply': result = a * b; break;
    case 'divide': result = b !== 0 ? a / b : null; break;
```

```
    default:
      return res.status(400).send({ error: 'Unknown operation' });
  }

  res.send({ operation, a, b, result });
});

// Create document endpoint
app.post('/api/documents', requirePayload, async (req, res) => {
  try {
    const signedDoc = await agent.createDocument(
      JSON.stringify(req.jacsPayload)
    );
    const doc = JSON.parse(signedDoc);

    res.send({
      success: true,
      documentId: doc.jacsId,
      version: doc.jacsVersion
    });
  } catch (error) {
    res.status(500).send({ error: error.message });
  }
});

// Error handler
app.use((err, req, res, next) => {
  console.error('Error:', err);
  res.status(500).send({ error: 'Internal server error' });
});

app.listen(PORT, () => {
  console.log(`JACS Express server running on port ${PORT}`);
});
```

HTTP Client

```
import { JacsAgent } from '@hai.ai/jacs';

async function callJacsApi(url, payload) {
  const agent = new JacsAgent();
  await agent.load('./jacs.client.config.json');

  // Sign the request
  const signedRequest = await agent.signRequest(payload);

  // Send HTTP request
  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'text/plain' },
    body: signedRequest
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  // Verify and extract response
  const responseText = await response.text();
  const verified = await agent.verifyResponse(responseText);

  return verified.payload;
}

// Usage
async function main() {
  // Call echo endpoint
  const echoResult = await callJacsApi(
    'http://localhost:3000/api/echo',
    { message: 'Hello, server!' }
  );
  console.log('Echo:', echoResult);

  // Call calculate endpoint
  const calcResult = await callJacsApi(
    'http://localhost:3000/api/calculate',
    { operation: 'multiply', a: 7, b: 6 }
  );
  console.log('Calculate:', calcResult);
}

main().catch(console.error);
```

MCP Integration

MCP Server

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { JacsClient } from '@hai.ai/jacs/client';
import { createJACSTransportProxy } from '@hai.ai/jacs/mcp';
import { z } from 'zod';

async function main() {
  console.error("JACS MCP Server starting...");

  const client = await JacsClient.quickstart({
    name: 'jacs-demo-server',
    domain: 'mcp.local',
  });

  const baseTransport = new StdioServerTransport();
  const secureTransport = createJACSTransportProxy(
    baseTransport,
    client,
    "server"
  );

  const server = new McpServer({
    name: "jacs-demo-server",
    version: "1.0.0"
  });

  // Register tools
  server.tool("echo", {
    message: z.string().describe("Message to echo")
  }, async ({ message }) => {
    console.error(`Echo called: ${message}`);
    return { content: [{ type: "text", text: `Echo: ${message}` }] };
  });

  server.tool("calculate", {
    operation: z.enum(["add", "subtract", "multiply", "divide"]),
    a: z.number(),
    b: z.number()
  }, async ({ operation, a, b }) => {
    let result;
    switch (operation) {
      case 'add': result = a + b; break;
      case 'subtract': result = a - b; break;
      case 'multiply': result = a * b; break;
      case 'divide': result = b !== 0 ? a / b : 'undefined'; break;
    }
    return { content: [{ type: "text", text: `${a} ${operation} ${b} = ${result}` }]
};
  });

  // Register resource
```

```
  server.resource(
    "server-info",
    "info://server",
    async (uri) => ({
      contents: [{
        uri: uri.href,
        text: JSON.stringify({
          name: "JACS Demo Server",
          version: "1.0.0",
          capabilities: ["echo", "calculate"]
        }),
        mimeType: "application/json"
      }]
    })
  );

  // Connect
  await server.connect(secureTransport);
  console.error("Server running with JACS encryption");
}

main().catch(err => {
  console.error("Fatal error:", err);
  process.exit(1);
});
```

MCP Client

```
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";
import { JacsClient } from '@hai.ai/jacs/client';
import { createJACSTransportProxy } from '@hai.ai/jacs/mcp';

async function main() {
  console.log("JACS MCP Client starting...");

  const client = await JacsClient.quickstart({
    name: 'jacs-demo-client',
    domain: 'mcp.local',
  });

  const baseTransport = new StdioClientTransport({
    command: 'node',
    args: ['mcp-server.js']
  });

  const secureTransport = createJACSTransportProxy(
    baseTransport,
    client,
    "client"
  );

  const mcpClient = new Client({
    name: "jacs-demo-client",
    version: "1.0.0"
  }, {
    capabilities: { tools: {} }
  });

  await mcpClient.connect(secureTransport);
  console.log("Connected to JACS MCP Server");

  // List tools
  const tools = await mcpClient.listTools();
  console.log("Available tools:", tools.tools.map(t => t.name));

  // Call echo
  const echoResult = await mcpClient.callTool({
    name: "echo",
    arguments: { message: "Hello, JACS!" }
  });
  console.log("Echo:", echoResult.content[0].text);

  // Call calculate
  const calcResult = await mcpClient.callTool({
    name: "calculate",
    arguments: { operation: "multiply", a: 6, b: 7 }
  });
  console.log("Calculate:", calcResult.content[0].text);

  await mcpClient.close();
  console.log("Done!");
}
```

```
main().catch(console.error);
```

Agreements

Creating Multi-Party Agreements

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';

async function createAgreement() {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  // Create the contract document
  const contract = {
    type: 'service_agreement',
    title: 'Professional Services Agreement',
    parties: ['Company A', 'Company B'],
    terms: 'Terms and conditions here...',
    value: 50000,
    effectiveDate: '2024-02-01'
  };

  const signedContract = await agent.createDocument(JSON.stringify(contract));

  // Get agent IDs (replace with actual UUIDs)
  const agentIds = [
    'agent1-uuid-here',
    'agent2-uuid-here'
  ];

  // Create agreement
  const agreementDoc = await agent.createAgreement(
    signedContract,
    agentIds,
    'Do you agree to the terms of this service agreement?',
    'This is a legally binding agreement'
  );

  console.log('Agreement created');
  const doc = JSON.parse(agreementDoc);
  console.log('Document ID:', doc.jacsId);
  console.log('Required signatures:', doc.jacsAgreement?.agentIDs?.length);

  // Save for signing
  fs.writeFileSync('agreement-pending.json', agreementDoc);

  return doc;
}

createAgreement();
```

Signing Agreements

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';

async function signAgreement(agreementPath, outputPath) {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  // Read agreement
  const agreementDoc = fs.readFileSync(agreementPath, 'utf-8');

  // Sign agreement
  const signedAgreement = await agent.signAgreement(agreementDoc);

  // Check status
  const statusJson = await agent.checkAgreement(signedAgreement);
  const status = JSON.parse(statusJson);

  console.log('Agreement signed');
  console.log('Status:', status.complete ? 'Complete' : 'Pending');
  console.log('Signatures:', status.signatures?.length || 0);

  // Save
  fs.writeFileSync(outputPath, signedAgreement);

  return status;
}

signAgreement('./agreement-pending.json', './agreement-signed.json');
```

Checking Agreement Status

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';

async function checkAgreementStatus(agreementPath) {
  const agent = new JacsAgent();
  await agent.load('./jacs.config.json');

  const agreementDoc = fs.readFileSync(agreementPath, 'utf-8');
  const statusJson = await agent.checkAgreement(agreementDoc);
  const status = JSON.parse(statusJson);

  console.log('Agreement Status:');
  console.log('  Complete:', status.complete);
  console.log('  Required agents:', status.requiredAgents);
  console.log('  Signed by:', status.signedBy || []);
  console.log('  Missing:', status.missing || []);

  return status;
}

checkAgreementStatus('./agreement.json');
```

Document Store

Simple File-Based Store

```
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';
import path from 'path';

class JacsDocumentStore {
  constructor(configPath, dataDir = './documents') {
    this.configPath = configPath;
    this.dataDir = dataDir;
    this.agent = null;
  }

  async initialize() {
    this.agent = new JacsAgent();
    await this.agent.load(this.configPath);

    if (!fs.existsSync(this.dataDir)) {
      fs.mkdirSync(this.dataDir, { recursive: true });
    }
  }

  async create(content) {
    const signedDoc = await this.agent.createDocument(JSON.stringify(content));
    const doc = JSON.parse(signedDoc);

    const filename = `${doc.jacsId}.json`;
    const filepath = path.join(this.dataDir, filename);

    fs.writeFileSync(filepath, signedDoc);

    return { id: doc.jacsId, version: doc.jacsVersion, path: filepath };
  }

  async get(documentId) {
    const filepath = path.join(this.dataDir, `${documentId}.json`);

    if (!fs.existsSync(filepath)) {
      return null;
    }

    const docString = fs.readFileSync(filepath, 'utf-8');
    return JSON.parse(docString);
  }

  async verify(documentId) {
    const filepath = path.join(this.dataDir, `${documentId}.json`);

    if (!fs.existsSync(filepath)) {
      return { valid: false, error: 'Document not found' };
    }

    const docString = fs.readFileSync(filepath, 'utf-8');
```

```
    const isValid = await this.agent.verifyDocument(docString);

    return { valid: isValid, document: JSON.parse(docString) };
  }

  list() {
    const files = fs.readdirSync(this.dataDir);
    return files
      .filter(f => f.endsWith('.json'))
      .map(f => f.replace('.json', ''));
  }
}

// Usage
async function main() {
  const store = new JacsDocumentStore('./jacs.config.json');
  await store.initialize();

  // Create document
  const result = await store.create({
    type: 'note',
    title: 'Meeting Notes',
    content: 'Discussed project timeline...'
  });
  console.log('Created:', result.id);

  // Verify document
  const verification = await store.verify(result.id);
  console.log('Valid:', verification.valid);

  // List all documents
  const docs = store.list();
  console.log('Documents:', docs);
}

main();
```

Error Handling

Robust Error Handling Pattern

```
import { JacsAgent } from '@hai.ai/jacs';

class JacsError extends Error {
  constructor(message, code, details = {}) {
    super(message);
    this.name = 'JacsError';
    this.code = code;
    this.details = details;
  }
}

async function robustDocumentCreate(configPath, content) {
  let agent;

  try {
    agent = new JacsAgent();
    await agent.load(configPath);
  } catch (error) {
    throw new JacsError(
      'Failed to initialize JACS agent',
      'INIT_ERROR',
      { originalError: error.message }
    );
  }

  try {
    const signedDoc = await agent.createDocument(JSON.stringify(content));
    return JSON.parse(signedDoc);
  } catch (error) {
    throw new JacsError(
      'Failed to create document',
      'CREATE_ERROR',
      { originalError: error.message, content }
    );
  }
}

async function robustDocumentVerify(configPath, docString) {
  let agent;

  try {
    agent = new JacsAgent();
    await agent.load(configPath);
  } catch (error) {
    throw new JacsError(
      'Failed to initialize JACS agent',
      'INIT_ERROR',
      { originalError: error.message }
    );
  }
```

```
  try {
    const isValid = await agent.verifyDocument(docString);
    return { valid: isValid };
  } catch (error) {
    throw new JacsError(
      'Verification error',
      'VERIFY_ERROR',
      { originalError: error.message }
    );
  }
}

// Usage with error handling
async function main() {
  try {
    const doc = await robustDocumentCreate('./jacs.config.json', {
      title: 'Test'
    });
    console.log('Created:', doc.jacsId);
  } catch (error) {
    if (error instanceof JacsError) {
      console.error(`JACS Error [${error.code}]:`, error.message);
      console.error('Details:', error.details);
    } else {
      console.error('Unexpected error:', error);
    }
  }
}

main();
```

Testing

Jest Test Setup

```
// tests/jacs.test.js
import { JacsAgent } from '@hai.ai/jacs';
import fs from 'fs';
import path from 'path';
import os from 'os';

describe('JACS Document Operations', () => {
  let agent;
  let tempDir;
  let configPath;

  beforeAll(async () => {
    // Create temp directory
    tempDir = fs.mkdtempSync(path.join(os.tmpdir(), 'jacs-test-'));
    const dataDir = path.join(tempDir, 'data');
    const keyDir = path.join(tempDir, 'keys');

    fs.mkdirSync(dataDir);
    fs.mkdirSync(keyDir);

    // Create test config
    const config = {
      jacs_data_directory: dataDir,
      jacs_key_directory: keyDir,
      jacs_agent_key_algorithm: 'ring-Ed25519',
      jacs_default_storage: 'fs'
    };

    configPath = path.join(tempDir, 'jacs.config.json');
    fs.writeFileSync(configPath, JSON.stringify(config));

    // Initialize agent
    agent = new JacsAgent();
    await agent.load(configPath);
  });

  afterAll(() => {
    fs.rmSync(tempDir, { recursive: true });
  });

  test('creates a signed document', async () => {
    const content = { title: 'Test Document', value: 42 };
    const signedDoc = await agent.createDocument(JSON.stringify(content));
    const doc = JSON.parse(signedDoc);

    expect(doc.jacsId).toBeDefined();
    expect(doc.jacsVersion).toBeDefined();
    expect(doc.jacsSignature).toBeDefined();
    expect(doc.title).toBe('Test Document');
  });
```

```
  test('verifies a valid document', async () => {
    const content = { title: 'Verify Test' };
    const signedDoc = await agent.createDocument(JSON.stringify(content));

    const isValid = await agent.verifyDocument(signedDoc);
    expect(isValid).toBe(true);
  });

  test('detects tampered document', async () => {
    const content = { title: 'Tamper Test' };
    const signedDoc = await agent.createDocument(JSON.stringify(content));

    // Tamper with document
    const doc = JSON.parse(signedDoc);
    doc.title = 'Modified Title';
    const tamperedDoc = JSON.stringify(doc);

    const isValid = await agent.verifyDocument(tamperedDoc);
    expect(isValid).toBe(false);
  });
});
```

See Also

- Node.js Installation - Setup guide
- Node.js API Reference - Complete API documentation
- MCP Integration - MCP details
- HTTP Server - HTTP integration

Python Examples

This chapter provides practical Python examples using the `jacs` (jacspy) package.

Setup

```
# Install dependencies
pip install jacs fastmcp fastapi uvicorn
```

```
# Initialize JACS
import jacs

agent = jacs.JacsAgent()
agent.load('./jacs.config.json')
```

Basic Document Operations

Creating and Signing Documents

```python
import jacs
import json

def create_signed_document():
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Create document content
    content = {
        "title": "Invoice",
        "invoiceNumber": "INV-001",
        "amount": 1500.00,
        "customer": "Acme Corp",
        "items": [
            {"description": "Consulting", "quantity": 10, "price": 150}
        ]
    }

    # Create and sign the document
    signed_doc = agent.create_document(json.dumps(content))

    # Parse the result
    doc = json.loads(signed_doc)
    print(f"Document ID: {doc['jacsId']}")
    print(f"Version: {doc['jacsVersion']}")
    print(f"Signature: {'Present' if 'jacsSignature' in doc else 'Missing'}")

    return doc

if __name__ == "__main__":
    create_signed_document()
```

Verifying Documents

```python
import jacs
import json

def verify_document(file_path: str) -> bool:
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Read the document
    with open(file_path, 'r') as f:
        doc_string = f.read()

    # Verify signature
    is_valid = agent.verify_document(doc_string)

    if is_valid:
        doc = json.loads(doc_string)
        print("✓ Document signature is valid")
        print(f"  Signed by: {doc.get('jacsSignature', {}).get('agentID')}")
        print(f"  Signed at: {doc.get('jacsSignature', {}).get('date')}")
    else:
        print("✗ Document signature is INVALID")

    return is_valid

if __name__ == "__main__":
    verify_document('./invoice.json')
```

Updating Documents

```python
import jacs
import json

def update_document(original_path: str, new_content: dict) -> dict:
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Read original document
    with open(original_path, 'r') as f:
        original_doc = f.read()

    # Update with new content (preserves version chain)
    updated_doc = agent.update_document(
        original_doc,
        json.dumps(new_content)
    )

    doc = json.loads(updated_doc)
    print(f"Updated Document ID: {doc['jacsId']}")
    print(f"New Version: {doc['jacsVersion']}")

    return doc

if __name__ == "__main__":
    updated = update_document('./invoice-v1.json', {
        "title": "Invoice",
        "invoiceNumber": "INV-001",
        "amount": 1500.00,
        "customer": "Acme Corp",
        "status": "paid"  # New field
    })
```

HTTP Server with FastAPI

Complete FastAPI Server

```python
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import PlainTextResponse
import jacs
import json

app = FastAPI(title="JACS API")

# Initialize JACS agent at startup
agent = None

@app.on_event("startup")
async def startup():
    global agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

# Health check (no JACS)
@app.get("/health")
async def health():
    return {"status": "ok"}

# JACS-protected endpoint
@app.post("/api/echo")
async def echo(request: Request):
    # Read raw body
    body = await request.body()
    body_str = body.decode('utf-8')

    # Verify JACS request
    try:
        verified = jacs.verify_request(body_str)
        payload = json.loads(verified).get('payload')
    except Exception as e:
        raise HTTPException(status_code=400, detail="Invalid JACS request")

    # Process and respond
    result = {
        "echo": payload,
        "serverTime": str(datetime.now())
    }

    # Sign response
    signed_response = jacs.sign_response(result)
    return PlainTextResponse(content=signed_response)

# Create document endpoint
@app.post("/api/documents")
async def create_document(request: Request):
    body = await request.body()
    body_str = body.decode('utf-8')
```

```python
    try:
        verified = jacs.verify_request(body_str)
        payload = json.loads(verified).get('payload')
    except Exception as e:
        raise HTTPException(status_code=400, detail="Invalid JACS request")

    # Create signed document
    signed_doc = agent.create_document(json.dumps(payload))
    doc = json.loads(signed_doc)

    result = {
        "success": True,
        "documentId": doc['jacsId'],
        "version": doc['jacsVersion']
    }

    signed_response = jacs.sign_response(result)
    return PlainTextResponse(content=signed_response)

# Calculate endpoint
@app.post("/api/calculate")
async def calculate(request: Request):
    body = await request.body()
    body_str = body.decode('utf-8')

    try:
        verified = jacs.verify_request(body_str)
        payload = json.loads(verified).get('payload')
    except Exception as e:
        raise HTTPException(status_code=400, detail="Invalid JACS request")

    operation = payload.get('operation')
    a = payload.get('a', 0)
    b = payload.get('b', 0)

    if operation == 'add':
        result = a + b
    elif operation == 'subtract':
        result = a - b
    elif operation == 'multiply':
        result = a * b
    elif operation == 'divide':
        result = a / b if b != 0 else None
    else:
        raise HTTPException(status_code=400, detail="Unknown operation")

    response = {"operation": operation, "a": a, "b": b, "result": result}
    signed_response = jacs.sign_response(response)
    return PlainTextResponse(content=signed_response)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="localhost", port=8000)
```

HTTP Client

```python
import jacs
import requests
import json

def call_jacs_api(url: str, payload: dict) -> dict:
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.client.config.json')

    # Sign the request
    signed_request = jacs.sign_request(payload)

    # Send HTTP request
    response = requests.post(
        url,
        data=signed_request,
        headers={"Content-Type": "text/plain"}
    )

    if response.status_code != 200:
        raise Exception(f"HTTP {response.status_code}")

    # Verify and extract response
    verified = jacs.verify_response(response.text)
    return json.loads(verified).get('payload')

if __name__ == "__main__":
    # Call echo endpoint
    echo_result = call_jacs_api(
        'http://localhost:8000/api/echo',
        {"message": "Hello, server!"}
    )
    print("Echo:", echo_result)

    # Call calculate endpoint
    calc_result = call_jacs_api(
        'http://localhost:8000/api/calculate',
        {"operation": "multiply", "a": 7, "b": 6}
    )
    print("Calculate:", calc_result)
```

MCP Integration

FastMCP Server with JACS

```python
import jacs
from jacs.mcp import JACSMCPServer
from fastmcp import FastMCP
import uvicorn

# Initialize JACS
agent = jacs.JacsAgent()
agent.load('./jacs.config.json')

# Create FastMCP server with JACS
mcp = JACSMCPServer(FastMCP("JACS Demo Server"))

@mcp.tool()
def echo(message: str) -> str:
    """Echo the input message"""
    return f"Echo: {message}"

@mcp.tool()
def calculate(operation: str, a: float, b: float) -> str:
    """Perform basic arithmetic"""
    if operation == 'add':
        result = a + b
    elif operation == 'subtract':
        result = a - b
    elif operation == 'multiply':
        result = a * b
    elif operation == 'divide':
        result = a / b if b != 0 else "undefined"
    else:
        return f"Unknown operation: {operation}"

    return f"{a} {operation} {b} = {result}"

@mcp.resource("info://server")
def server_info() -> str:
    """Get server information"""
    return json.dumps({
        "name": "JACS Demo Server",
        "version": "1.0.0",
        "tools": ["echo", "calculate"]
    })

# Get ASGI app with JACS middleware
app = mcp.sse_app()

if __name__ == "__main__":
    print("Starting JACS MCP Server...")
    uvicorn.run(app, host="localhost", port=8000)
```

MCP Client with JACS

```python
import asyncio
import jacs
from jacs.mcp import JACSMCPClient

async def main():
    # Initialize JACS
    agent = jacs.JacsAgent()
    agent.load('./jacs.client.config.json')

    # Create authenticated client
    client = JACSMCPClient("http://localhost:8000/sse")

    async with client:
        # Call echo tool
        echo_result = await client.call_tool("echo", {
            "message": "Hello from JACS client!"
        })
        print(f"Echo: {echo_result}")

        # Call calculate tool
        calc_result = await client.call_tool("calculate", {
            "operation": "multiply",
            "a": 6,
            "b": 7
        })
        print(f"Calculate: {calc_result}")

        # Read resource
        info = await client.read_resource("info://server")
        print(f"Server info: {info}")

if __name__ == "__main__":
    asyncio.run(main())
```

Agreements

Creating Multi-Party Agreements

```python
import jacs
import json

def create_agreement():
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Create contract document
    contract = {
        "type": "service_agreement",
        "title": "Professional Services Agreement",
        "parties": ["Company A", "Company B"],
        "terms": "Terms and conditions here...",
        "value": 50000,
        "effectiveDate": "2024-02-01"
    }

    signed_contract = agent.create_document(json.dumps(contract))

    # Define required signers (replace with actual UUIDs)
    agent_ids = [
        "agent1-uuid-here",
        "agent2-uuid-here"
    ]

    # Create agreement
    agreement_doc = agent.create_agreement(
        signed_contract,
        agent_ids,
        question="Do you agree to the terms of this service agreement?",
        context="This is a legally binding agreement"
    )

    doc = json.loads(agreement_doc)
    print("Agreement created")
    print(f"Document ID: {doc['jacsId']}")
    print(f"Required signatures: {len(doc.get('jacsAgreement', {}).get('agentIDs',
[]))}")

    # Save for signing
    with open('agreement-pending.json', 'w') as f:
        f.write(agreement_doc)

    return doc

if __name__ == "__main__":
    create_agreement()
```

Signing Agreements

```python
import jacs
import json

def sign_agreement(agreement_path: str, output_path: str) -> dict:
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    # Read agreement
    with open(agreement_path, 'r') as f:
        agreement_doc = f.read()

    # Sign agreement
    signed_agreement = agent.sign_agreement(agreement_doc)

    # Check status
    status_json = agent.check_agreement(signed_agreement)
    status = json.loads(status_json)

    print("Agreement signed")
    print(f"Status: {'Complete' if status.get('complete') else 'Pending'}")
    print(f"Signatures: {len(status.get('signatures', []))}")

    # Save
    with open(output_path, 'w') as f:
        f.write(signed_agreement)

    return status

if __name__ == "__main__":
    sign_agreement('./agreement-pending.json', './agreement-signed.json')
```

Checking Agreement Status

```python
import jacs
import json

def check_agreement_status(agreement_path: str) -> dict:
    # Initialize agent
    agent = jacs.JacsAgent()
    agent.load('./jacs.config.json')

    with open(agreement_path, 'r') as f:
        agreement_doc = f.read()

    status_json = agent.check_agreement(agreement_doc)
    status = json.loads(status_json)

    print("Agreement Status:")
    print(f"  Complete: {status.get('complete')}")
    print(f"  Required agents: {status.get('requiredAgents', [])}")
    print(f"  Signed by: {status.get('signedBy', [])}")
    print(f"  Missing: {status.get('missing', [])}")

    return status

if __name__ == "__main__":
    check_agreement_status('./agreement.json')
```

Document Store

Simple File-Based Store

```python
import jacs
import json
import os
from pathlib import Path
from typing import Optional, Dict, List

class JacsDocumentStore:
    def __init__(self, config_path: str, data_dir: str = './documents'):
        self.config_path = config_path
        self.data_dir = Path(data_dir)
        self.agent = None

    def initialize(self):
        self.agent = jacs.JacsAgent()
        self.agent.load(self.config_path)
        self.data_dir.mkdir(parents=True, exist_ok=True)

    def create(self, content: dict) -> dict:
        signed_doc = self.agent.create_document(json.dumps(content))
        doc = json.loads(signed_doc)

        filename = f"{doc['jacsId']}.json"
        filepath = self.data_dir / filename

        with open(filepath, 'w') as f:
            f.write(signed_doc)

        return {
            'id': doc['jacsId'],
            'version': doc['jacsVersion'],
            'path': str(filepath)
        }

    def get(self, document_id: str) -> Optional[dict]:
        filepath = self.data_dir / f"{document_id}.json"

        if not filepath.exists():
            return None

        with open(filepath, 'r') as f:
            return json.load(f)

    def verify(self, document_id: str) -> dict:
        filepath = self.data_dir / f"{document_id}.json"

        if not filepath.exists():
            return {'valid': False, 'error': 'Document not found'}

        with open(filepath, 'r') as f:
            doc_string = f.read()
```

```python
        is_valid = self.agent.verify_document(doc_string)
        return {'valid': is_valid, 'document': json.loads(doc_string)}

    def list(self) -> List[str]:
        return [
            f.stem for f in self.data_dir.glob('*.json')
        ]

if __name__ == "__main__":
    store = JacsDocumentStore('./jacs.config.json')
    store.initialize()

    # Create document
    result = store.create({
        'type': 'note',
        'title': 'Meeting Notes',
        'content': 'Discussed project timeline...'
    })
    print(f"Created: {result['id']}")

    # Verify document
    verification = store.verify(result['id'])
    print(f"Valid: {verification['valid']}")

    # List all documents
    docs = store.list()
    print(f"Documents: {docs}")
```

Batch Processing

Batch Document Creator

```python
import jacs
import json
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor

class BatchDocumentProcessor:
    def __init__(self, config_path: str):
        self.config_path = config_path

    def create_documents(self, documents: list, output_dir: str) -> list:
        """Create multiple signed documents"""
        output_path = Path(output_dir)
        output_path.mkdir(parents=True, exist_ok=True)

        results = []

        # Initialize agent
        agent = jacs.JacsAgent()
        agent.load(self.config_path)

        for i, content in enumerate(documents):
            try:
                signed_doc = agent.create_document(json.dumps(content))
                doc = json.loads(signed_doc)

                filename = f"{doc['jacsId']}.json"
                filepath = output_path / filename

                with open(filepath, 'w') as f:
                    f.write(signed_doc)

                results.append({
                    'success': True,
                    'index': i,
                    'id': doc['jacsId'],
                    'path': str(filepath)
                })
            except Exception as e:
                results.append({
                    'success': False,
                    'index': i,
                    'error': str(e)
                })

        return results

    def verify_documents(self, input_dir: str) -> list:
        """Verify all documents in a directory"""
        input_path = Path(input_dir)

        # Initialize agent
```

```python
        agent = jacs.JacsAgent()
        agent.load(self.config_path)

        results = []

        for filepath in input_path.glob('*.json'):
            try:
                with open(filepath, 'r') as f:
                    doc_string = f.read()

                is_valid = agent.verify_document(doc_string)
                doc = json.loads(doc_string)

                results.append({
                    'file': filepath.name,
                    'valid': is_valid,
                    'id': doc.get('jacsId')
                })
            except Exception as e:
                results.append({
                    'file': filepath.name,
                    'valid': False,
                    'error': str(e)
                })

        return results

if __name__ == "__main__":
    processor = BatchDocumentProcessor('./jacs.config.json')

    # Create batch of documents
    documents = [
        {'type': 'invoice', 'number': f'INV-{i:03d}', 'amount': i * 100}
        for i in range(1, 11)
    ]

    results = processor.create_documents(documents, './batch-output')

    success_count = sum(1 for r in results if r['success'])
    print(f"Created {success_count}/{len(documents)} documents")

    # Verify all documents
    verification_results = processor.verify_documents('./batch-output')

    valid_count = sum(1 for r in verification_results if r['valid'])
    print(f"Valid: {valid_count}/{len(verification_results)} documents")
```

Testing

Pytest Setup

```python
# tests/test_jacs.py
import pytest
import jacs
import json
import tempfile
import shutil
from pathlib import Path

@pytest.fixture
def jacs_agent():
    """Create a test JACS agent with temporary directories"""
    temp_dir = tempfile.mkdtemp()
    data_dir = Path(temp_dir) / 'data'
    key_dir = Path(temp_dir) / 'keys'

    data_dir.mkdir()
    key_dir.mkdir()

    config = {
        'jacs_data_directory': str(data_dir),
        'jacs_key_directory': str(key_dir),
        'jacs_agent_key_algorithm': 'ring-Ed25519',
        'jacs_default_storage': 'fs'
    }

    config_path = Path(temp_dir) / 'jacs.config.json'
    with open(config_path, 'w') as f:
        json.dump(config, f)

    agent = jacs.JacsAgent()
    agent.load(str(config_path))

    yield agent

    shutil.rmtree(temp_dir)

class TestDocumentOperations:
    def test_create_document(self, jacs_agent):
        content = {'title': 'Test Document', 'value': 42}
        signed_doc = jacs_agent.create_document(json.dumps(content))
        doc = json.loads(signed_doc)

        assert 'jacsId' in doc
        assert 'jacsVersion' in doc
        assert 'jacsSignature' in doc
        assert doc['title'] == 'Test Document'

    def test_verify_valid_document(self, jacs_agent):
        content = {'title': 'Verify Test'}
        signed_doc = jacs_agent.create_document(json.dumps(content))
```

```python
    is_valid = jacs_agent.verify_document(signed_doc)
    assert is_valid is True

def test_detect_tampered_document(self, jacs_agent):
    content = {'title': 'Tamper Test'}
    signed_doc = jacs_agent.create_document(json.dumps(content))

    # Tamper with document
    doc = json.loads(signed_doc)
    doc['title'] = 'Modified Title'
    tampered_doc = json.dumps(doc)

    is_valid = jacs_agent.verify_document(tampered_doc)
    assert is_valid is False

def test_different_content_different_signatures(self, jacs_agent):
    doc1 = jacs_agent.create_document(json.dumps({'a': 1}))
    doc2 = jacs_agent.create_document(json.dumps({'a': 2}))

    parsed1 = json.loads(doc1)
    parsed2 = json.loads(doc2)

    sig1 = parsed1['jacsSignature']['signature']
    sig2 = parsed2['jacsSignature']['signature']

    assert sig1 != sig2
```

Error Handling

Robust Error Handling Pattern

```python
import jacs
import json
from typing import Optional

class JacsError(Exception):
    def __init__(self, message: str, code: str, details: dict = None):
        super().__init__(message)
        self.code = code
        self.details = details or {}

def robust_create_document(config_path: str, content: dict) -> dict:
    """Create a document with comprehensive error handling"""
    try:
        agent = jacs.JacsAgent()
        agent.load(config_path)
    except FileNotFoundError:
        raise JacsError(
            "Configuration file not found",
            "CONFIG_NOT_FOUND",
            {"path": config_path}
        )
    except Exception as e:
        raise JacsError(
            "Failed to initialize JACS agent",
            "INIT_ERROR",
            {"original_error": str(e)}
        )

    try:
        signed_doc = agent.create_document(json.dumps(content))
        return json.loads(signed_doc)
    except Exception as e:
        raise JacsError(
            "Failed to create document",
            "CREATE_ERROR",
            {"original_error": str(e), "content": content}
        )

def robust_verify_document(config_path: str, doc_string: str) -> dict:
    """Verify a document with comprehensive error handling"""
    try:
        agent = jacs.JacsAgent()
        agent.load(config_path)
    except Exception as e:
        raise JacsError(
            "Failed to initialize JACS agent",
            "INIT_ERROR",
            {"original_error": str(e)}
        )

    try:
```

```python
        is_valid = agent.verify_document(doc_string)
        return {"valid": is_valid}
    except Exception as e:
        raise JacsError(
            "Verification error",
            "VERIFY_ERROR",
            {"original_error": str(e)}
        )

if __name__ == "__main__":
    try:
        doc = robust_create_document('./jacs.config.json', {'title': 'Test'})
        print(f"Created: {doc['jacsId']}")
    except JacsError as e:
        print(f"JACS Error [{e.code}]: {e}")
        print(f"Details: {e.details}")
    except Exception as e:
        print(f"Unexpected error: {e}")
```

See Also

- Python Installation - Setup guide
- Python API Reference - Complete API documentation
- Python MCP Integration - MCP details

Integration Examples

This page is now a curated index of examples that still line up with the current APIs. The old monolithic example chapter mixed outdated agent APIs with supported workflows.

MCP

- `jacs-mcp/README.md`
  - Best starting point for the full Rust MCP server
- `jacspy/examples/mcp/server.py`
  - Python FastMCP server wrapped with `JACSMCPServer`
- `jacspy/examples/mcp/client.py`
  - Python FastMCP client wrapped with `JACSMCPClient`
- `jacsnpm/examples/mcp.stdio.server.js`
  - Node stdio server with `createJACSTransportProxy()`
- `jacsnpm/examples/mcp.stdio.client.js`
  - Node stdio client with signed transport

LangChain / LangGraph

- `jacspy/examples/langchain/signing_callback.py`
  - Best current Python example for signed LangGraph tool execution
- `jacsnpm/examples/langchain/basic-agent.ts`
  - Node LangChain.js agent using JACS tools
- `jacsnpm/examples/langchain/signing-callback.ts`
  - Node auto-signing pattern for LangGraph-style flows

A2A

- `jacspy/tests/test_a2a_server.py`
  - Best current Python reference for generated `.well-known` routes
- `jacsnpm/src/a2a-server.js`
  - Node Express A2A discovery middleware
- `jacsnpm/examples/a2a-agent-example.js`
  - Node A2A card and artifact demo
- `jacs/tests/a2a_cross_language_tests.rs`
  - Cross-language behavior reference for signing and verification

HTTP / App Middleware

- `jacspy/examples/http/server.py`
  - FastAPI app with `JacsMiddleware`
- `jacspy/examples/http/client.py`
  - Python client consuming signed responses
- `jacsnpm/examples/expressmiddleware.js`
  - Express middleware example

Rule Of Thumb

If an example and a higher-level prose page disagree, trust:

1. the current binding README
2. the current tests
3. the example that imports the API you intend to use today

CLI Command Reference

This page provides a comprehensive reference for all JACS command-line interface commands. For a workflow-oriented tutorial, see CLI Tutorial. For practical scripting examples, see CLI Examples.

Global Commands

`jacs version`

Prints version and build information for the JACS installation.

```
jacs version
```

`jacs quickstart`

Create a persistent agent with keys on disk and optionally sign data -- no manual setup needed. If `./jacs.config.json` already exists, loads it; otherwise creates a new agent. Agent, keys, and config are saved to `./jacs_data`, `./jacs_keys`, and `./jacs.config.json`. Password is required: set `JACS_PRIVATE_KEY_PASSWORD` (recommended) or `JACS_PASSWORD_FILE` (CLI file bootstrap). Set exactly one explicit source; if both are set, CLI exits with an error. This is the fastest way to start using JACS.

```
# Print agent info (ID, algorithm)
jacs quickstart --name my-agent --domain my-agent.example.com

# Sign JSON from stdin
echo '{"action":"approve"}' | jacs quickstart --name my-agent --domain my-
agent.example.com --sign

# Sign a file
jacs quickstart --name my-agent --domain my-agent.example.com --sign --file
mydata.json

# Use a specific algorithm
jacs quickstart --name my-agent --domain my-agent.example.com --algorithm ring-Ed25519
```

Options:

- `--name <name>` - Agent name used for first-time quickstart creation (required)
- `--domain <domain>` - Agent domain used for DNS/public-key verification workflows (required)
- `--algorithm <algo>` - Signing algorithm (default: `pq2025`). Also: `ring-Ed25519`, `RSA-PSS`
- `--sign` - Sign input (from stdin or `--file`) instead of printing info

- `--file <path>` - Read JSON input from file instead of stdin (requires `--sign`)

`jacs verify`

Verify a signed JACS document. No agent or config file required -- the CLI creates an ephemeral verifier if needed.

```
# Verify a local file
jacs verify signed-document.json

# JSON output (for scripting)
jacs verify signed-document.json --json

# Verify a remote document
jacs verify --remote https://example.com/signed-doc.json

# Specify a directory of public keys
jacs verify signed-document.json --key-dir ./trusted-keys/
```

Options:

- `<file>` - Path to the signed JACS JSON file (positional, required unless `--remote` is used)
- `--remote <url>` - Fetch document from URL before verifying
- `--json` - Output result as JSON (`{"valid": true, "signerId": "...", "timestamp": "..."}`)
- `--key-dir <dir>` - Directory containing public keys for verification

Exit codes: `0` for valid, `1` for invalid or error.

Output (text):

```
Status:    VALID
Signer:    550e8400-e29b-41d4-a716-446655440000
Signed at: 2026-02-10T12:00:00Z
```

Output (JSON):

```
{
  "valid": true,
  "signerId": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-02-10T12:00:00Z"
}
```

If `./jacs.config.json` and agent keys exist in the current directory, the CLI uses them automatically. Otherwise it creates a temporary ephemeral verifier internally.

See the Verification Guide for Python, Node.js, and DNS verification workflows.

`jacs init`

Initialize JACS by creating both configuration and agent (with cryptographic keys). Use this for persistent agent setup.

```
jacs init
```

`jacs help`

Print help information for JACS commands.

```
jacs help [COMMAND]
```

Configuration Commands

`jacs config`

Work with JACS configuration settings.

```
jacs config [SUBCOMMAND]
```

Note: Specific subcommands for config are not detailed in the current help output.

Agent Commands

`jacs agent`

Work with JACS agents - the cryptographic identities that sign and verify documents.

```
jacs agent [SUBCOMMAND]
```

Note: Specific subcommands for agent management are not detailed in the current help output.

Task Commands

`jacs task`

Work with JACS agent tasks - structured workflows between agents.

```
jacs task [SUBCOMMAND]
```

Note: Specific subcommands for task management are not detailed in the current help output.

Document Commands

The `jacs document` command provides comprehensive document management capabilities.

`jacs document create`

Create a new JACS document, either by embedding or parsing a document with optional file attachments.

Usage:

```
jacs document create [OPTIONS]
```

Options:

- `-a <agent-file>` - Path to the agent file. If not specified, uses config `jacs_agent_id_and_version`
- `-f <filename>` - Path to input file. Must be JSON format
- `-o <output>` - Output filename for the created document
- `-d <directory>` - Path to directory of files. Files should end with `.json`
- `-v, --verbose` - Enable verbose output
- `-n, --no-save` - Instead of saving files, print to stdout
- `-s, --schema <schema>` - Path to JSON schema file to use for validation
- `--attach <attach>` - Path to file or directory for file attachments
- `-e, --embed <embed>` - Embed documents or keep them external [possible values: true, false]
- `-h, --help` - Print help information

Examples:

```
# Create document from JSON file
jacs document create -f my-document.json

# Create document with embedded attachment
jacs document create -f document.json --attach ./image.jpg --embed true

# Create document with referenced attachment
jacs document create -f document.json --attach ./data.csv --embed false

# Create from directory of JSON files
jacs document create -d ./documents/

# Create with custom schema validation
jacs document create -f document.json -s custom-schema.json

# Print to stdout instead of saving
jacs document create -f document.json --no-save
```

`jacs document update`

Create a new version of an existing document. Requires both the original JACS file and the modified JACS metadata.

Usage:

```
jacs document update [OPTIONS]
```

Options:

- `-a <agent-file>` - Path to the agent file
- `-f <filename>` - Path to original document file
- `-n <new-file>` - Path to new/modified document file
- `-o <output>` - Output filename for updated document
- `-v, --verbose` - Enable verbose output
- `-n, --no-save` - Print to stdout instead of saving
- `-s, --schema <schema>` - Path to JSON schema file for validation
- `--attach <attach>` - Path to file or directory for additional attachments
- `-e, --embed <embed>` - Embed new attachments or keep them external
- `-h, --help` - Print help information

Example:

```
# Update document with new version
jacs document update -f original.json -n modified.json -o updated.json

# Update and add new attachments
jacs document update -f original.json -n modified.json --attach ./new-file.pdf --embed
false
```

`jacs document verify`

Verify a document's hash, signatures, and schema compliance.

Usage:

```
jacs document verify [OPTIONS]
```

Options:

- `-a <agent-file>` - Path to the agent file
- `-f <filename>` - Path to input file. Must be JSON format
- `-d <directory>` - Path to directory of files. Files should end with `.json`
- `-v, --verbose` - Enable verbose output
- `-s, --schema <schema>` - Path to JSON schema file to use for validation
- `-h, --help` - Print help information

Examples:

```
# Verify single document
jacs document verify -f signed-document.json

# Verify all documents in directory
jacs document verify -d ./documents/

# Verify with custom schema
jacs document verify -f document.json -s custom-schema.json
```

Verification Process:

1. **Hash verification** - Confirms document integrity
2. **Signature verification** - Validates cryptographic signatures
3. **Schema validation** - Ensures document structure compliance
4. **File integrity** - Checks SHA256 checksums of attached files

`jacs document extract`

Extract embedded file contents from documents back to the filesystem.

Usage:

```
jacs document extract [OPTIONS]
```

Options:

- `-a <agent-file>` - Path to the agent file
- `-f <filename>` - Path to input file containing embedded files

- `-d <directory>` - Path to directory of files to process
- `-s, --schema <schema>` - Path to JSON schema file for validation
- `-h, --help` - Print help information

Examples:

```
# Extract embedded files from single document
jacs document extract -f document-with-embedded-files.json

# Extract from all documents in directory
jacs document extract -d ./documents/
```

Extract Process:

1. Reads embedded file contents from document
2. Decodes base64-encoded data
3. Writes files to their original paths
4. Creates backup of existing files (with timestamp)

Agreement Commands

JACS provides specialized commands for managing multi-agent agreements.

`jacs document check-agreement`

Given a document, provide a list of agents that should sign the document.

Usage:

```
jacs document check-agreement [OPTIONS]
```

`jacs document create-agreement`

Create an agreement structure for a document that requires multiple agent signatures.

Usage:

```
jacs document create-agreement [OPTIONS]
```

`jacs document sign-agreement`

Sign the agreement section of a document with the current agent's cryptographic signature.

Usage:

```
jacs document sign-agreement [OPTIONS]
```

Common Patterns

Basic Document Lifecycle

```
# 1. Initialize JACS
jacs init

# 2. Create document with attachments
jacs document create -f document.json --attach ./files/ --embed true

# 3. Verify document integrity
jacs document verify -f created-document.json

# 4. Update document if needed
jacs document update -f original.json -n modified.json

# 5. Extract embedded files when needed
jacs document extract -f document.json
```

Working with Attachments

```
# Embed small files for portability
jacs document create -f doc.json --attach ./small-image.png --embed true

# Reference large files to save space
jacs document create -f doc.json --attach ./large-video.mp4 --embed false

# Attach multiple files from directory
jacs document create -f doc.json --attach ./attachments/ --embed false
```

Schema Validation Workflow

```
# Create with schema validation
jacs document create -f document.json -s schema.json

# Verify against specific schema
jacs document verify -f document.json -s schema.json
```

Global Options

Most commands support these common options:

- `-h, --help` - Show help information
- `-v, --verbose` - Enable verbose output for debugging
- `-a <agent-file>` - Specify custom agent file (overrides config default)

Exit Codes

- `0` - Success
- `1` - General error (invalid arguments, file not found, etc.)
- `2` - Verification failure (hash mismatch, invalid signature, etc.)
- `3` - Schema validation failure

Environment Variables

- `JACS_CONFIG_PATH` - Override default configuration file location
- `JACS_DATA_DIR` - Override default data directory location
- `JACS_AGENT_FILE` - Default agent file to use (if not specified with `-a`)

File Formats

Input Files

- **JSON documents** - Must be valid JSON format
- **Schema files** - JSON Schema format (draft-07 compatible)
- **Agent files** - JACS agent format with cryptographic keys
- **Attachments** - Any file type (automatically detected MIME type)

Output Files

- **JACS documents** - JSON format with JACS metadata, signatures, and checksums
- **Extracted files** - Original format of embedded attachments

Configuration Reference

This is the comprehensive configuration guide covering zero-config quickstart, storage backends, observability, and environment variables. For the raw schema field list, see Config File Schema.

Overview

Key resolution for verifiers

When verifying signed documents, JACS resolves the signer's public key using a configurable order of sources. Set `JACS_KEY_RESOLUTION` (environment variable or in config) to a comma-separated list of sources: `local` (local key cache by `publicKeyHash`), `dns` (DNS TXT fingerprint validation), `hai` (HAI key service). Example: `JACS_KEY_RESOLUTION=local,hai` or `local,dns,hai`. The first source that yields verifiable key material is used. Use `verify_standalone()` with explicit `keyResolution` for one-off verification without loading a full config.

Zero-Config Path

If you just want to sign and verify without manual config setup, use `quickstart(name, domain, ...)`:

```python
import jacs.simple as jacs
info = jacs.quickstart(name="config-agent", domain="config.example.com")
print(info.config_path, info.public_key_path, info.private_key_path)
```

```node
const jacs = require('@hai.ai/jacs/simple');
const info = await jacs.quickstart({
  name: 'config-agent',
  domain: 'config.example.com',
});
console.log(info.configPath, info.publicKeyPath, info.privateKeyPath);
```

```cli
jacs quickstart --name config-agent --domain config.example.com
```

`quickstart(name, domain, ...)` creates a persistent agent with keys on disk (default algorithm: `pq2025`). If `./jacs.config.json` already exists, it loads it; otherwise it creates a new agent. Returned `AgentInfo` includes config/key paths (`config_path` / `public_key_path` / `private_key_path` in Python, `configPath` / `publicKeyPath` / `privateKeyPath` in Node) plus key directory metadata so you can locate key material immediately. Rust/CLI quickstart requires an explicit password source

(`JACS_PRIVATE_KEY_PASSWORD` , or `JACS_PASSWORD_FILE` in CLI). Python/Node can auto-generate a password if needed; set `JACS_SAVE_PASSWORD_FILE=true` if you want that generated password persisted to `./jacs_keys/.jacs_password` .

Minimal Configuration

For persistent agents, a config file needs only two fields (plus `$schema`):

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_agent_id_and_version": "YOUR_AGENT_ID:YOUR_VERSION",
  "jacs_agent_key_algorithm": "pq2025"
}
```

All other settings use sensible defaults (`./jacs_data` , `./jacs_keys` , `fs` storage). Override only what you need.

Complete Example Configuration

```
{
  "$schema": "https://hai.ai/schemas/jacs.config.schema.json",
  "jacs_use_security": "false",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_agent_private_key_filename": "jacs.private.pem.enc",
  "jacs_agent_public_key_filename": "jacs.public.pem",
  "jacs_agent_key_algorithm": "pq2025",
  "jacs_default_storage": "fs",
  "observability": {
    "logs": {
      "enabled": true,
      "level": "info",
      "destination": {
        "type": "file",
        "path": "./logs"
      },
      "headers": {
        "Authorization": "Bearer token",
        "X-API-Key": "secret"
      }
    },
    "metrics": {
      "enabled": true,
      "destination": {
        "type": "prometheus",
        "endpoint": "http://localhost:9090/api/v1/write",
        "headers": {
          "Authorization": "Basic dXNlcjpwYXNz"
        }
      },
      "export_interval_seconds": 60,
      "headers": {
        "X-Service": "jacs"
      }
    },
    "tracing": {
      "enabled": true,
      "sampling": {
        "ratio": 0.1,
        "parent_based": true,
        "rate_limit": 100
      },
      "resource": {
        "service_name": "jacs",
        "service_version": "0.4.0",
        "environment": "production",
        "attributes": {
          "team": "platform",
          "region": "us-west-2"
        }
      }
    }
```

```
  }
}
```

Observability Configuration

JACS supports comprehensive observability through configurable logging, metrics, and tracing. All observability features are optional and can be configured in the `jacs.config.json` file.

Logs Configuration

Controls how JACS generates and outputs log messages.

Field	Type	Required	Description
`enabled`	boolean	Yes	Whether logging is enabled
`level`	string	Yes	Minimum log level: `trace`, `debug`, `info`, `warn`, `error`
`destination`	object	Yes	Where logs are sent (see destinations below)
`headers`	object	No	Additional headers for remote destinations

Log Destinations

File Logging

```
{
  "type": "file",
  "path": "./logs"
}
```

Writes logs to rotating files in the specified directory.

Console Logging (stderr)

```
{
  "type": "stderr"
}
```

Outputs logs to standard error stream.

OpenTelemetry Protocol (OTLP)

```
{
  "type": "otlp",
  "endpoint": "http://localhost:4317",
  "headers": {
    "Authorization": "Bearer token"
  }
}
```

Sends logs to an OTLP-compatible endpoint (like Jaeger, Grafana Cloud).

Null (disabled)

```
{
  "type": "null"
}
```

Discards all log output.

Metrics Configuration

Controls collection and export of application metrics.

Field	Type	Required	Description
`enabled`	boolean	Yes	Whether metrics collection is enabled
`destination`	object	Yes	Where metrics are exported (see destinations below)
`export_interval_seconds`	integer	No	How often to export metrics (default: 60)
`headers`	object	No	Additional headers for remote destinations

Metrics Destinations

Prometheus Remote Write

```
{
  "type": "prometheus",
  "endpoint": "http://localhost:9090/api/v1/write",
  "headers": {
    "Authorization": "Basic dXNlcjpwYXNz"
  }
}
```

Exports metrics in Prometheus format to a remote write endpoint.

OpenTelemetry Protocol (OTLP)

```
{
  "type": "otlp",
  "endpoint": "http://localhost:4317",
  "headers": {
    "Authorization": "Bearer token"
  }
}
```

Exports metrics to an OTLP-compatible endpoint.

File Export

```
{
  "type": "file",
  "path": "./metrics.txt"
}
```

Writes metrics to a local file.

Console Output (stdout)

```
{
  "type": "stdout"
}
```

Prints metrics to standard output.

Tracing Configuration

Controls distributed tracing for request flows.

Field	Type	Required	Description
`enabled`	boolean	Yes	Whether tracing is enabled
`sampling`	object	No	Sampling configuration (see below)
`resource`	object	No	Service identification (see below)

Sampling Configuration

Controls which traces are collected to manage overhead.

Field	Type	Default	Description
`ratio`	number	1.0	Fraction of traces to sample (0.0-1.0)
`parent_based`	boolean	true	Whether to respect parent trace sampling decisions

Field	Type	Default	Description
`rate_limit`	integer	none	Maximum traces per second

Examples:

- `"ratio": 1.0` - Sample all traces (100%)
- `"ratio": 0.1` - Sample 10% of traces
- `"ratio": 0.01` - Sample 1% of traces
- `"rate_limit": 10` - Maximum 10 traces per second

Resource Configuration

Identifies the service in distributed tracing systems.

Field	Type	Required	Description
`service_name`	string	Yes	Name of the service
`service_version`	string	No	Version of the service
`environment`	string	No	Environment (dev, staging, prod)
`attributes`	object	No	Custom key-value attributes

Authentication & Headers

For remote destinations (OTLP, Prometheus), you can specify authentication headers:

Bearer Token Authentication:

```
"headers": {
  "Authorization": "Bearer your-token-here"
}
```

Basic Authentication:

```
"headers": {
  "Authorization": "Basic dXNlcjpwYXNz"
}
```

API Key Authentication:

```
"headers": {
  "X-API-Key": "your-api-key",
  "X-Auth-Token": "your-auth-token"
}
```

Common Patterns

Development Configuration

```
"observability": {
  "logs": {
    "enabled": true,
    "level": "debug",
    "destination": { "type": "stderr" }
  },
  "metrics": {
    "enabled": true,
    "destination": { "type": "stdout" }
  }
}
```

Production Configuration

```
"observability": {
  "logs": {
    "enabled": true,
    "level": "info",
    "destination": {
      "type": "otlp",
      "endpoint": "https://logs.example.com:4317",
      "headers": {
        "Authorization": "Bearer prod-token"
      }
    }
  },
  "metrics": {
    "enabled": true,
    "destination": {
      "type": "prometheus",
      "endpoint": "https://metrics.example.com/api/v1/write"
    },
    "export_interval_seconds": 30
  },
  "tracing": {
    "enabled": true,
    "sampling": {
      "ratio": 0.05,
      "rate_limit": 100
    },
    "resource": {
      "service_name": "jacs",
      "service_version": "0.4.0",
      "environment": "production"
    }
  }
}
```

File-based Configuration

```
"observability": {
  "logs": {
    "enabled": true,
    "level": "info",
    "destination": {
      "type": "file",
      "path": "/var/log/jacs"
    }
  },
  "metrics": {
    "enabled": true,
    "destination": {
      "type": "file",
      "path": "/var/log/jacs/metrics.txt"
    },
    "export_interval_seconds": 60
  }
}
```

Environment Variable Integration

The observability configuration works alongside JACS's core configuration system.

Required Environment Variable

Only **one** environment variable is truly required:

- `JACS_PRIVATE_KEY_PASSWORD` - Password for encrypting/decrypting private keys (required for cryptographic operations)

Configuration-Based Settings

All other JACS settings are **configuration file fields** that have sensible defaults:

- `jacs_data_directory` - Where agent/document data is stored (default: `./jacs_data`)
- `jacs_key_directory` - Where cryptographic keys are stored (default: `./jacs_keys`)
- `jacs_agent_key_algorithm` - Cryptographic algorithm to use (default: `pq2025`)
- `jacs_default_storage` - Storage backend (default: `fs`)
- `jacs_use_security` / `JACS_ENABLE_FILESYSTEM_QUARANTINE` - Enable filesystem quarantine of executable files (default: `false`). The env var `JACS_USE_SECURITY` is deprecated; use `JACS_ENABLE_FILESYSTEM_QUARANTINE` instead.

These can be overridden by environment variables if needed, but they are primarily configured through the `jacs.config.json` file.

The observability configuration is completely optional - JACS will work without any observability configuration.

Storage Configuration

The `jacs_default_storage` field determines where JACS stores agent data, documents, and keys. This is a critical configuration that affects how your data is persisted and accessed.

Available Storage Backends

Backend	Value	Description	Use Case
Filesystem	`"fs"`	Signed JSON documents on local disk	Default, development, single-node deployments
Local Indexed SQLite	`"rusqlite"`	Signed documents in SQLite with FTS search	Local search, bindings, MCP
AWS S3	`"aws"`	Amazon S3 object storage	Remote object storage
Memory	`"memory"`	In-memory object storage (non-persistent)	Testing, temporary data
Web Local	`"local"`	Browser local storage (WASM only)	Web applications

For local indexed document search in JACS core, use `"rusqlite"`. Additional database backends such as PostgreSQL, DuckDB, Redb, and SurrealDB live in separate crates and are **not** core `jacs_default_storage` values.

Backend-Specific Configuration

Filesystem Storage (`"fs"`)

```
{
  "jacs_default_storage": "fs",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys"
}
```

Requirements: None - works out of the box **Data location:** Local directories as specified in config **Best for:** Development, local testing, single-machine deployments

Local Indexed SQLite (`"rusqlite"`)

```
{
  "jacs_default_storage": "rusqlite",
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys"
}
```

Requirements: Built with the default `sqlite` Cargo feature **Database path:** `<jacs_data_directory>/jacs_documents.sqlite3` **Best for:** Local full-text search, MCP/binding document operations, single-machine deployments that want indexed reads

AWS S3 Storage (`"aws"`)

```
{
  "jacs_default_storage": "aws"
}
```

Required Environment Variables:

- `JACS_ENABLE_AWS_BUCKET_NAME` - S3 bucket name
- `AWS_ACCESS_KEY_ID` - AWS access key
- `AWS_SECRET_ACCESS_KEY` - AWS secret key
- `AWS_REGION` - AWS region (optional, defaults to us-east-1)

Best for: Production deployments, distributed systems, cloud-native applications

Memory Storage (`"memory"`)

```
{
  "jacs_default_storage": "memory"
}
```

Requirements: None **Data persistence:** None - data is lost when application stops **Best for:** Unit testing, temporary operations, development scenarios

Storage Behavior

- **Filesystem (`fs`)** stores signed documents as JSON files under `jacs_data/documents/` .
- **Rusqlite (`rusqlite`)** stores signed documents in `jacs_data/jacs_documents.sqlite3` and is the indexed `DocumentService` path used by bindings and MCP.

- **Agent files and keys** remain path-based assets under `jacs_data/` and `jacs_keys/`.
- **Observability data** (logs, metrics) can use separate storage via observability configuration.

DocumentService Guarantees

- Every `DocumentService` read verifies the stored JACS document before returning it.
- Every `create()` and `update()` verifies the signed document before persisting it.
- If an update payload changes a signed JACS document without re-signing it, the write fails.
- Visibility changes create a new signed version instead of mutating metadata in place.

Configuration Examples

Development Setup (Filesystem)

```
{
  "jacs_default_storage": "fs",
  "jacs_data_directory": "./dev_data",
  "jacs_key_directory": "./dev_keys"
}
```

Production Setup (AWS S3)

```
{
  "jacs_default_storage": "aws"
}
```

With environment variables:

```
export JACS_ENABLE_AWS_BUCKET_NAME="my-jacs-production-bucket"
export AWS_ACCESS_KEY_ID="AKIA..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_REGION="us-west-2"
```

Security Considerations

- **AWS S3**: Ensure proper IAM permissions for bucket access
- **Rusqlite**: Protect the local database file with the same filesystem permissions you use for other signed artifacts
- **Filesystem**: Ensure proper file system permissions for data and key directories
- **Keys**: Regardless of storage backend, always set `JACS_PRIVATE_KEY_PASSWORD` for key encryption

Migration Between Storage Backends

When changing storage backends, you'll need to:

1. Export existing data from the current backend
2. Update the `jacs_default_storage` configuration
3. Set any required environment variables for the new backend
4. Import data into the new backend

JACS doesn't automatically migrate data between storage backends - this must be done manually or via custom scripts.

Error Codes

This reference documents error codes and messages you may encounter when using JACS.

CLI Exit Codes

Code	Name	Description
0	Success	Operation completed successfully
1	General Error	Unspecified error occurred
2	Invalid Arguments	Command line arguments invalid
3	File Not Found	Specified file does not exist
4	Verification Failed	Document or signature verification failed
5	Signature Invalid	Cryptographic signature is invalid

Configuration Errors

Missing Configuration

Error: `Configuration file not found: jacs.config.json`

Cause: JACS cannot find the configuration file.

Solution:

```
# Initialize JACS to create configuration
jacs init

# Or specify a custom config path
JACS_CONFIG_PATH=./custom.config.json jacs agent verify
```

Invalid Configuration

Error: `Invalid configuration: missing required field 'jacs_key_directory'`

Cause: Configuration file is missing required fields.

Solution: Ensure your `jacs.config.json` contains all required fields:

```
{
  "jacs_data_directory": "./jacs_data",
  "jacs_key_directory": "./jacs_keys",
  "jacs_agent_key_algorithm": "ring-Ed25519",
  "jacs_default_storage": "fs"
}
```

Key Directory Not Found

Error: `Key directory not found: ./jacs_keys`

Cause: The specified key directory does not exist.

Solution:

```
# Create the directory
mkdir -p ./jacs_keys

# Or run init to create everything
jacs init
```

Cryptographic Errors

Private Key Not Found

Error: `Private key file not found: private.pem`

Cause: The private key file is missing from the key directory.

Solution:

```
# Generate new keys
jacs agent create --create-keys true
```

Invalid Key Format

Error: `Failed to parse private key: invalid PEM format`

Cause: The key file is corrupted or in wrong format.

Solution:

- Regenerate keys with `jacs agent create --create-keys true`

- Ensure the key file is not corrupted

Key Password Required

Error: `Private key is encrypted but no password provided`

Cause: Encrypted private key requires password.

Solution:

```
export JACS_PRIVATE_KEY_PASSWORD="your-password"
jacs document create -f doc.json
```

Algorithm Mismatch

Error: `Key algorithm 'ring-Ed25519' does not match configured algorithm 'RSA-PSS'`

Cause: The key file was created with a different algorithm than configured.

Solution:

- Update config to match key algorithm, or
- Regenerate keys with the correct algorithm

Signature Errors

Verification Failed

Error: `Document verification failed: signature does not match content`

Cause: Document content has been modified after signing.

Solution:

- The document may have been tampered with
- Re-sign the document if you have the original content

Missing Signature

Error: `Document missing jacsSignature field`

Cause: Document was not signed or signature was removed.

Solution:

```
# Create a signed document
jacs document create -f unsigned-doc.json
```

Invalid Signature Format

Error: `Invalid signature format: expected base64 encoded string`

Cause: The signature field is malformed.

Solution:

- Re-sign the document
- Verify the document hasn't been corrupted

Unknown Signing Algorithm

Error: `Unknown signing algorithm: unknown-algo`

Cause: Document was signed with an unsupported algorithm.

Solution:

- Use a supported algorithm: `ring-Ed25519`, `RSA-PSS`, `pq-dilithium`, `pq2025`

DNS Verification Errors

DNSSEC Validation Failed

Error: `strict DNSSEC validation failed for <owner> (TXT not authenticated). Enable DNSSEC and publish DS at registrar`

Cause: DNSSEC mode was requested but the TXT response wasn't authenticated.

Solution:

1. Enable DNSSEC for your domain zone
2. Publish the DS record at your registrar
3. Wait for propagation (up to 48 hours)

DNS Record Not Found

Error: `DNS TXT lookup failed for <owner> (record missing or not yet propagated)`

Cause: The JACS TXT record doesn't exist or hasn't propagated.

Solution:

1. Verify the TXT record was created:

   ```
   dig _v1.agent.jacs.yourdomain.com TXT
   ```

2. Wait for DNS propagation (can take up to 48 hours)
3. Confirm record name and value are correct

DNS Required

Error: `DNS TXT lookup required (domain configured) or provide embedded fingerprint`

Cause: Strict DNS mode is active because a domain is configured.

Solution:

- Publish the TXT record, or
- Run with `--no-dns` during initial setup:

  ```
  jacs agent verify --no-dns
  ```

DNS Lookup Timeout

Error: `DNS lookup timed out for <domain>`

Cause: DNS server did not respond in time.

Solution:

- Check network connectivity
- Try again later
- Verify DNS server is accessible

Document Errors

Invalid JSON

Error: `Failed to parse document: invalid JSON at line 5`

Cause: Document file contains invalid JSON.

Solution:

- Validate JSON with a linter
- Check for syntax errors (missing commas, quotes)

Schema Validation Failed

Error: `Schema validation failed: missing required field 'amount'`

Cause: Document doesn't conform to the specified schema.

Solution:

```
# Check which fields are required by the schema
cat schema.json | jq '.required'

# Add missing fields to your document
```

Document Not Found

Error: `Document not found: 550e8400-e29b-41d4-a716-446655440000`

Cause: The specified document ID doesn't exist in storage.

Solution:

- Verify the document ID is correct
- Check the storage directory

Version Mismatch

Error: `Document version mismatch: expected v2, got v1`

Cause: Attempting to update with incorrect base version.

Solution:

- Get the latest version of the document
- Apply updates to the correct version

Agreement Errors

Agreement Not Found

Error: `Document has no jacsAgreement field`

Cause: Attempting agreement operations on a document without an agreement.

Solution:

```
# Create an agreement first
jacs document create-agreement -f doc.json -i agent1-id,agent2-id
```

Already Signed

Error: `Agent has already signed this agreement`

Cause: Attempting to sign an agreement that was already signed by this agent.

Solution:

- No action needed, the signature is already present

Not Authorized

Error: `Agent is not in the agreement's agentIDs list`

Cause: Attempting to sign with an agent not listed in the agreement.

Solution:

- Only agents listed in `jacsAgreement.agentIDs` can sign

Agreement Locked

Error: `Cannot modify document: agreement is complete`

Cause: Attempting to modify a document with a completed agreement.

Solution:

- Create a new version/agreement if changes are needed

Storage Errors

Storage Backend Error

Error: `Storage error: failed to write to filesystem`

Cause: Unable to write to the configured storage backend.

Solution:

- Check filesystem permissions
- Verify storage directory exists
- Check disk space

AWS S3 Error

Error: `S3 error: AccessDenied`

Cause: AWS credentials don't have required permissions.

Solution:

- Verify IAM permissions include s3:GetObject, s3:PutObject
- Check bucket policy
- Verify credentials are correct

Connection Error

Error: `Failed to connect to storage: connection refused`

Cause: Cannot connect to remote storage backend.

Solution:

- Check network connectivity
- Verify endpoint URL is correct
- Check firewall rules

HTTP/MCP Errors

Request Verification Failed

Error: `JACS request verification failed`

Cause: Incoming HTTP request has invalid JACS signature.

Solution:

- Ensure client is signing requests correctly
- Verify client and server are using compatible keys

Response Verification Failed

Error: `JACS response verification failed`

Cause: Server response has invalid signature.

Solution:

- Check server JACS configuration
- Verify server is signing responses

Middleware Configuration Error

Error: `JACSExpressMiddleware: config file not found`

Cause: Middleware cannot find JACS configuration.

Solution:

```
app.use('/api', JACSExpressMiddleware({
  configPath: './jacs.config.json'  // Verify path is correct
}));
```

Debugging Tips

Enable Verbose Output

```
# CLI verbose mode
jacs document verify -f doc.json -v

# Environment variable
export JACS_DEBUG=true
```

Check Configuration

```
# Display current configuration
jacs config read
```

Verify Agent

```
# Verify agent is properly configured
jacs agent verify -v
```

Test Signing

```
# Create a test document
echo '{"test": true}' > test.json
jacs document create -f test.json -v
```

See Also

- Configuration Reference - Configuration options
- CLI Command Reference - CLI usage
- Security Model - Security details

Attestation Verification Results

This reference explains every field in the `AttestationVerificationResult` returned by `verify_attestation()` and `verify_attestation_full()`.

Result Structure

```
{
  "valid": true,
  "crypto": {
    "signature_valid": true,
    "hash_valid": true
  },
  "evidence": [
    {
      "kind": "custom",
      "digest_valid": true,
      "freshness_valid": true,
      "errors": []
    }
  ],
  "chain": {
    "depth": 1,
    "all_links_valid": true,
    "links": []
  },
  "errors": []
}
```

Top-Level Fields

Field	Type	Description
`valid`	`boolean`	Overall result. `true` only if all sub-checks pass.
`crypto`	`object`	Cryptographic verification results.
`evidence`	`array`	Per-evidence-ref verification results (full tier only).
`chain`	`object|null`	Derivation chain verification (full tier only, if derivation exists).
`errors`	`array`	Human-readable error messages for any failures.

`crypto` Object

Field	Type	Description
`signature_valid`	`boolean`	The cryptographic signature matches the document content and the signer's public key.
`hash_valid`	`boolean`	The `jacsSha256` hash matches the canonicalized document content.

Common failure scenarios:

- `signature_valid: false` -- The document was tampered with after signing, or the wrong public key was used.
- `hash_valid: false` -- The document body was modified after the hash was computed.

`evidence` Array (Full Tier Only)

Each entry corresponds to one evidence reference in the attestation's `evidence` array.

Field	Type	Description
`kind`	`string`	Evidence type (`a2a` , `email` , `jwt` , `tlsnotary` , `custom`).
`digest_valid`	`boolean`	The evidence digest matches the expected value.
`freshness_valid`	`boolean`	The `collectedAt` timestamp is within acceptable bounds.
`errors`	`array`	Error messages specific to this evidence item.

Common failure scenarios:

- `digest_valid: false` -- The evidence content has changed since the attestation was created.
- `freshness_valid: false` -- The evidence is too old. Check `collectedAt` and your freshness policy.

`chain` Object (Full Tier Only)

Present only when the attestation has a `derivation` field.

Field	Type	Description
`depth`	`number`	Number of links in the derivation chain.
`all_links_valid`	`boolean`	Every derivation link verified successfully.
`links`	`array`	Per-link verification details.

Each link in `links`:

Field	Type	Description
`input_digests_valid`	`boolean`	Input digests match the referenced documents.
`output_digests_valid`	`boolean`	Output digests match the transformation result.
`transform`	`object`	Transform metadata (name, hash, reproducible).

Verification Tiers

Local Tier (`verify_attestation()`)

- Checks: `crypto.signature_valid + crypto.hash_valid`
- Speed: < 1ms typical
- Network: None
- Use for: Real-time validation, hot path

Full Tier (`verify_attestation(full=True)`)

- Checks: Everything in local + evidence digests + freshness + derivation chain
- Speed: < 10ms typical (no network), varies with evidence count
- Network: Optional (for remote evidence resolution)
- Use for: Audit trails, compliance, trust decisions

Troubleshooting

"valid is false but crypto shows all true"

The `valid` field aggregates all sub-checks. If `crypto` passes but evidence or chain checks fail, `valid` will be `false`. Check the `evidence` and `chain` fields for details.

"evidence is empty"

If you created the attestation without evidence references, the `evidence` array will be empty. This is not an error -- it means there are no external proofs to verify.

"chain is null"

If the attestation has no `derivation` field, `chain` will be `null`. This is normal for standalone attestations that don't reference prior attestations.

"signature_valid is false after serialization"

JACS uses JSON Canonicalization Scheme (JCS) for hashing. If you serialize and re-parse the document, ensure the serializer preserves field order and does not add/remove whitespace in a way that changes the canonical form.

CLI Reference: jacs attest

The `jacs attest` command creates and verifies attestation documents from the command line. Attestation extends basic signing with structured claims, evidence references, and derivation chains.

jacs attest create

Create a signed attestation document.

Synopsis

```
jacs attest create --claims '<JSON>' [options]
```

Options

Flag	Required	Description
`--claims '<JSON>'`	Yes	JSON array of claims. Each claim must have `name` and `value` fields.
`--subject-type <TYPE>`	No	Type of subject: `agent`, `artifact`, `workflow`, `identity`. Default: derived from context.
`--subject-id <ID>`	No	Identifier of the subject being attested.
`--subject-digest <SHA256>`	No	SHA-256 digest of the subject content.
`--evidence '<JSON>'`	No	JSON array of evidence references.
`--from-document <FILE>`	No	Lift an existing signed JACS document into an attestation. Overrides subject flags.
`-o, --output <FILE>`	No	Write attestation to file instead of stdout.

Examples

Create a basic attestation:

```
jacs attest create \
  --subject-type artifact \
  --subject-id "doc-001" \
  --subject-digest "abc123def456..." \
  --claims '[{"name": "reviewed_by", "value": "human", "confidence": 0.95}]'
```

Attestation with multiple claims:

```
jacs attest create \
  --subject-type agent \
  --subject-id "agent-abc" \
  --subject-digest "sha256hash..." \
  --claims '[
    {"name": "reviewed", "value": true, "confidence": 0.95},
    {"name": "source", "value": "internal_db", "assuranceLevel": "verified"}
  ]'
```

Lift an existing signed document to attestation:

```
jacs attest create \
  --from-document mydata.signed.json \
  --claims '[{"name": "approved", "value": true}]'
```

With evidence references:

```
jacs attest create \
  --subject-type artifact \
  --subject-id "report-456" \
  --subject-digest "def789..." \
  --claims '[{"name": "scanned", "value": true}]' \
  --evidence '[{
    "kind": "custom",
    "digests": {"sha256": "evidence-hash..."},
    "uri": "https://scanner.example.com/results/123",
    "collectedAt": "2026-03-04T00:00:00Z",
    "verifier": {"name": "security-scanner", "version": "2.0"}
  }]'
```

Write to file:

```
jacs attest create \
  --subject-type artifact \
  --subject-id "doc-001" \
  --subject-digest "abc123..." \
  --claims '[{"name": "ok", "value": true}]' \
  -o attestation.json
```

jacs attest verify

Verify an attestation document.

Synopsis

```
jacs attest verify <FILE> [options]
```

Arguments

Argument	Required	Description
`<FILE>`	Yes	Path to the attestation JSON file to verify.

Options

Flag	Required	Description
`--full`	No	Use full verification (evidence + derivation chain). Default: local verification (crypto + hash only).
`--json`	No	Output the verification result as JSON.
`--key-dir <DIR>`	No	Directory containing public keys for verification.
`--max-depth <N>`	No	Maximum derivation chain depth. Default: 10.

Examples

Basic verification (local tier):

```
jacs attest verify attestation.json
```

Output:

```
Attestation verification: VALID
  Signature: OK
  Hash: OK
  Signer: agent-id-abc123
  Algorithm: ring-Ed25519
```

Full verification:

```
jacs attest verify attestation.json --full
```

Output:

```
Attestation verification: VALID
  Signature: OK
  Hash: OK
  Signer: agent-id-abc123
  Algorithm: ring-Ed25519
  Evidence: 1 item(s) verified
    [0] custom: digest OK, freshness OK
  Chain: not present
```

JSON output (for scripting):

```
jacs attest verify attestation.json --json
```

Output:

```
{
  "valid": true,
  "crypto": {
    "signature_valid": true,
    "hash_valid": true,
    "signer_id": "agent-id-abc123",
    "algorithm": "ring-Ed25519"
  },
  "evidence": [],
  "chain": null,
  "errors": []
}
```

Verify with external keys:

```
jacs attest verify attestation.json --key-dir ./trusted_keys/
```

Pipe through jq:

```
jacs attest verify attestation.json --json | jq '.crypto'
```

Piping and Scripting Patterns

Create and verify in one pipeline

```
jacs attest create \
  --subject-type artifact \
  --subject-id "doc-001" \
  --subject-digest "abc..." \
  --claims '[{"name": "ok", "value": true}]' \
  -o att.json && \
jacs attest verify att.json --json | jq '.valid'
```

Check validity in a script

```
#!/bin/bash
set -e

RESULT=$(jacs attest verify "$1" --json 2>/dev/null)
VALID=$(echo "$RESULT" | jq -r '.valid')

if [ "$VALID" = "true" ]; then
    echo "Attestation is valid"
    exit 0
else
    echo "Attestation is INVALID"
    echo "$RESULT" | jq '.errors'
    exit 1
fi
```

Batch verify multiple attestations

```
for file in attestations/*.json; do
    echo -n "$file: "
    jacs attest verify "$file" --json | jq -r '.valid'
done
```

Exit Codes

Code	Meaning
0	Success (create: attestation created; verify: attestation valid)
1	Failure (create: error creating attestation; verify: attestation invalid or error)

Environment Variables

Variable	Description
`JACS_PRIVATE_KEY_PASSWORD`	Password for the agent's private key
`JACS_MAX_DERIVATION_DEPTH`	Override maximum derivation chain depth (default: 10)
`JACS_DATA_DIRECTORY`	Directory for JACS data files
`JACS_KEY_DIRECTORY`	Directory containing keys
`JACS_AGENT_ID_AND_VERSION`	Agent identity for signing

See Also

- Sign vs. Attest Decision Guide
- Attestation Tutorial
- Attestation Verification Results
- CLI Command Reference

Migration Guide

This guide covers migrating between JACS versions and common migration scenarios.

Version Compatibility

JACS maintains backward compatibility for document verification:

- Documents signed with older versions can be verified with newer versions
- Older JACS versions cannot verify documents using newer cryptographic algorithms

Migrating Node.js from 0.6.x to 0.7.0

Breaking Change: Async-First API

In v0.7.0, all NAPI operations return Promises by default. Sync variants are available with a `Sync` suffix, following the Node.js convention (like `fs.readFile` vs `fs.readFileSync`).

Before (v0.6.x):

```
const agent = new JacsAgent();
agent.load('./jacs.config.json');
const doc = agent.createDocument(JSON.stringify(content));
const isValid = agent.verifyDocument(doc);
```

After (v0.7.0, async -- recommended):

```
const agent = new JacsAgent();
await agent.load('./jacs.config.json');
const doc = await agent.createDocument(JSON.stringify(content));
const isValid = await agent.verifyDocument(doc);
```

After (v0.7.0, sync -- for scripts/CLI):

```
const agent = new JacsAgent();
agent.loadSync('./jacs.config.json');
const doc = agent.createDocumentSync(JSON.stringify(content));
const isValid = agent.verifyDocumentSync(doc);
```

Method Renaming Summary

v0.6.x	v0.7.0 Async (default)	v0.7.0 Sync
`agent.load(path)`	`await agent.load(path)`	`agent.loadSync(path)`
`agent.createDocument(...)`	`await agent.createDocument(...)`	`agent.createDocumentSyn`
`agent.verifyDocument(doc)`	`await agent.verifyDocument(doc)`	`agent.verifyDocumentSyn`
`agent.verifyAgent()`	`await agent.verifyAgent()`	`agent.verifyAgentSync()`
`agent.updateAgent(json)`	`await agent.updateAgent(json)`	`agent.updateAgentSync(j`
`agent.updateDocument(...)`	`await agent.updateDocument(...)`	`agent.updateDocumentSyn`
`agent.signString(data)`	`await agent.signString(data)`	`agent.signStringSync(da`
`agent.createAgreement(...)`	`await agent.createAgreement(...)`	`agent.createAgreementSy`
`agent.signAgreement(...)`	`await agent.signAgreement(...)`	`agent.signAgreementSync`
`agent.checkAgreement(...)`	`await agent.checkAgreement(...)`	`agent.checkAgreementSyn`

V8-Thread-Only Methods (No Change)

These methods remain synchronous without a `Sync` suffix because they use V8-thread-only APIs (`Env` , `JsObject`):

- `agent.signRequest(params)` -- unchanged
- `agent.verifyResponse(doc)` -- unchanged
- `agent.verifyResponseWithAgentId(doc)` -- unchanged

Simplified API (Module-Level)

The `@hai.ai/jacs/simple` module follows the same pattern:

```
// v0.6.x
await jacs.quickstart({ name: 'my-agent', domain: 'agent.example.com' });
const signed = jacs.signMessage({ action: 'approve' });

// v0.7.0 async (recommended)
await jacs.quickstart({ name: 'my-agent', domain: 'agent.example.com' });
const signed = await jacs.signMessage({ action: 'approve' });

// v0.7.0 sync
jacs.quickstartSync({ name: 'my-agent', domain: 'agent.example.com' });
const signed = jacs.signMessageSync({ action: 'approve' });
```

Pure Sync Functions (No Change)

These functions do not call NAPI and remain unchanged (no suffix needed):

- `hashString()`, `createConfig()`, `getPublicKey()`, `isLoaded()`, `exportAgent()`, `getAgentInfo()`
- `getDnsRecord()`, `getWellKnownJson()`, `verifyStandalone()`
- Trust store: `trustAgent()`, `listTrustedAgents()`, `untrustAgent()`, `isTrusted()`, `getTrustedAgent()`

Migration Steps

1. **Update dependency:** `npm install @hai.ai/jacs@0.7.0`
2. **Add `await`** to all NAPI method calls, or append `Sync` to method names
3. **Update test assertions** to handle Promises (use `async/await` in test functions)
4. **V8-thread-only methods** (`signRequest`, `verifyResponse`, `verifyResponseWithAgentId`) need no changes

Migrating from 0.5.1 to 0.5.2

Migration Notes

PBKDF2 Iteration Count: New key encryptions use 600,000 iterations (up from 100,000). Existing encrypted keys are decrypted automatically via fallback. To upgrade existing keys, re-encrypt them:

```
# Re-generate keys to use the new iteration count
jacs keygen
```

Deprecated Environment Variables

- `JACS_USE_SECURITY` is now `JACS_ENABLE_FILESYSTEM_QUARANTINE`. The old name still works with a deprecation warning.

New Environment Variables

Variable	Default	Description
`JACS_MAX_SIGNATURE_AGE_SECONDS`	`0` (no expiration)	Maximum age of valid signatures. Set to a positive value to enable (e.g., `7776000` for 90 days).
`JACS_REQUIRE_EXPLICIT_ALGORITHM`	`false`	When `true`, reject verification if `signingAlgorithm` is missing.
`JACS_ENABLE_FILESYSTEM_QUARANTINE`	`false`	Enable filesystem quarantine (replaces `JACS_USE_SECURITY`).

Deprecated Method Aliases (0.9.0)

In v0.9.0, several method names were standardized. The old names remain as aliases for backward compatibility but are deprecated and will be removed in 1.0.0 (minimum 2 minor releases after deprecation).

Runtime Deprecation Warnings

Set the `JACS_SHOW_DEPRECATIONS=1` environment variable to emit runtime warnings when deprecated methods are called:

```
export JACS_SHOW_DEPRECATIONS=1
```

This is recommended during development and CI to identify code that needs updating.

Deprecated Alias Table

SDK	Deprecated Method	Canonical Replacement	Sinc
Python (binding)	`agent.wrap_a2a_artifact()`	`agent.sign_artifact()`	0.9.
Python (A2A)	`a2a.wrap_artifact_with_provenance()`	`a2a.sign_artifact()`	0.9.

SDK	Deprecated Method	Canonical Replacement	Sinc
Node.js (binding)	`agent.wrapA2aArtifact()`	`agent.signArtifact()`	0.9.
Node.js (binding)	`agent.wrapA2aArtifactSync()`	`agent.signArtifactSync()`	0.9.
Node.js (A2A)	`a2a.wrapArtifactWithProvenance()`	`a2a.signArtifact()`	0.9.
Rust (core)	`agent.wrap_a2a_artifact()`	`agent.sign_artifact()`	0.9.
Go	`SignA2AArtifact()` (simple API)	Uses `sign_artifact` internally	--

All aliases behave identically to their canonical replacements. No behavioral changes are needed when migrating -- only rename the method call.

Migration Examples

Python:

```python
# Before (deprecated)
wrapped = agent.wrap_a2a_artifact(artifact_json, "task")

# After (canonical)
signed = agent.sign_artifact(artifact_json, "task")
```

Node.js:

```node
// Before (deprecated)
const wrapped = await agent.wrapA2aArtifact(artifactJson, 'task');

// After (canonical)
const signed = await agent.signArtifact(artifactJson, 'task');
```

Python A2A integration:

```python
# Before (deprecated)
wrapped = a2a.wrap_artifact_with_provenance(artifact, "task")

# After (canonical)
signed = a2a.sign_artifact(artifact, "task")
```

Node.js A2A integration:

```
// Before (deprecated)
const wrapped = await a2a.wrapArtifactWithProvenance(artifact, 'task');

// After (canonical)
const signed = await a2a.signArtifact(artifact, 'task');
```

Module-Level Function Deprecation (Reminder)

Module-level functions (e.g., `jacs.load()`, `jacs.sign_request()` in Python; `load()`, `signRequest()` in Node.js) were deprecated in earlier releases. Use `JacsAgent` instance methods instead. See the Python and Node.js API references for the full list.

Migrating from 0.2.x to 0.3.x

Configuration Changes

New Configuration Fields:

```
{
  "observability": {
    "logs": { "enabled": true, "level": "info" },
    "metrics": { "enabled": false },
    "tracing": { "enabled": false }
  }
}
```

Deprecated Fields:

- `jacs_log_level` → Use `observability.logs.level`
- `jacs_log_file` → Use `observability.logs.destination`

Migration Steps

1. **Update Configuration:**

   ```
   # Backup current config
   cp jacs.config.json jacs.config.json.backup

   # Update to new format
   # Add observability section if needed
   ```

2. **Update Dependencies:**

```
# Node.js
npm install @hai.ai/jacs@latest

# Python
pip install --upgrade jacs
```

3. **Verify Existing Documents:**

```
jacs document verify -d ./jacs_data/documents/
```

Migrating Storage Backends

Filesystem to AWS S3

1. **Create S3 Bucket:**

```
aws s3 mb s3://my-jacs-bucket
```

2. **Update Configuration:**

```
{
  "jacs_default_storage": "aws",
  "jacs_data_directory": "s3://my-jacs-bucket/data"
}
```

3. **Set Environment Variables:**

```
export AWS_ACCESS_KEY_ID="your-key"
export AWS_SECRET_ACCESS_KEY="your-secret"
export AWS_REGION="us-east-1"
```

4. **Migrate Documents:**

```
# Upload existing documents
aws s3 sync ./jacs_data/ s3://my-jacs-bucket/data/
```

5. **Verify Migration:**

```
jacs document verify -d s3://my-jacs-bucket/data/documents/
```

AWS S3 to Filesystem

1. **Download Documents:**

```
aws s3 sync s3://my-jacs-bucket/data/ ./jacs_data/
```

2. **Update Configuration:**

```
{
  "jacs_default_storage": "fs",
  "jacs_data_directory": "./jacs_data"
}
```

3. **Verify Documents:**

```
jacs document verify -d ./jacs_data/documents/
```

Migrating Cryptographic Algorithms

Ed25519 to Post-Quantum

For increased security, you may want to migrate to post-quantum algorithms.

1. **Create New Agent with New Algorithm:**

```
{
  "jacs_agent_key_algorithm": "pq-dilithium"
}
```

```
jacs agent create --create-keys true -f new-agent.json
```

2. **Update Configuration:**

```
{
  "jacs_agent_key_algorithm": "pq-dilithium",
  "jacs_agent_id_and_version": "new-agent-id:new-version"
}
```

3. **Re-sign Critical Documents (Optional):**

```
// Re-sign documents with new algorithm
const oldDoc = JSON.parse(fs.readFileSync('./old-doc.json'));

// Remove old signature fields
delete oldDoc.jacsSignature;
delete oldDoc.jacsSha256;

// Create new signed version
const newDoc = await agent.createDocument(JSON.stringify(oldDoc));
```

Note: Old documents remain valid with old signatures. Re-signing is only needed for documents that require the new algorithm.

Migrating Between Platforms

Node.js to Python

Both platforms use the same document format:

```
// Node.js - create document
const signedDoc = await agent.createDocument(JSON.stringify(content));
fs.writeFileSync('doc.json', signedDoc);
```

```
# Python - verify the same document
with open('doc.json', 'r') as f:
    doc_string = f.read()

is_valid = agent.verify_document(doc_string)
```

Sharing Agents Between Platforms

Agents can be used across platforms by sharing configuration:

1. **Export Agent Files:**

```
jacs_keys/
├── private.pem
└── public.pem
jacs.config.json
```

2. **Use Same Config in Both:**

```
// Node.js
await agent.load('./jacs.config.json');


# Python
agent.load('./jacs.config.json')
```

Migrating Key Formats

Unencrypted to Encrypted Keys

1. **Encrypt Existing Key:**

```
# Backup original
cp jacs_keys/private.pem jacs_keys/private.pem.backup

# Encrypt with password
openssl pkcs8 -topk8 -in jacs_keys/private.pem \
  -out jacs_keys/private.pem.enc -v2 aes-256-cbc

# Remove unencrypted key
rm jacs_keys/private.pem
mv jacs_keys/private.pem.enc jacs_keys/private.pem
```

2. **Update Configuration:**

```
{
  "jacs_agent_private_key_filename": "private.pem"
}
```

3. **Set Password:**

```
export JACS_PRIVATE_KEY_PASSWORD="your-secure-password"
```

Database Migration

Adding Database Storage

If migrating from filesystem to include database storage:

1. **Create Database Schema:**

```sql
CREATE TABLE jacs_documents (
  id UUID PRIMARY KEY,
  version_id UUID NOT NULL,
  document JSONB NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  UNIQUE(id, version_id)
);
```

2. **Import Existing Documents:**

```
const fs = require('fs');
const path = require('path');
const { Pool } = require('pg');

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
const docsDir = './jacs_data/documents';

async function importDocuments() {
  const docDirs = fs.readdirSync(docsDir);

  for (const docId of docDirs) {
    const docPath = path.join(docsDir, docId);
    const versions = fs.readdirSync(docPath);

    for (const versionFile of versions) {
      const docString = fs.readFileSync(
        path.join(docPath, versionFile),
        'utf-8'
      );
      const doc = JSON.parse(docString);

      await pool.query(`
        INSERT INTO jacs_documents (id, version_id, document)
        VALUES ($1, $2, $3)
        ON CONFLICT (id, version_id) DO NOTHING
      `, [doc.jacsId, doc.jacsVersion, doc]);
    }
  }
}

importDocuments();
```

MCP Integration Migration

Adding JACS to Existing MCP Server

1. **Install JACS:**

```
npm install @hai.ai/jacs
```

2. **Wrap Existing Transport:**

```
// Before
const transport = new StdioServerTransport();
await server.connect(transport);

// After
import { createJACSTransportProxy } from '@hai.ai/jacs/mcp';

const baseTransport = new StdioServerTransport();
const secureTransport = createJACSTransportProxy(
  baseTransport,
  './jacs.config.json',
  'server'
);
await server.connect(secureTransport);
```

3. **Update Client:**

```
// Client also needs JACS
const baseTransport = new StdioClientTransport({ command: 'node', args:
['server.js'] });
const secureTransport = createJACSTransportProxy(
  baseTransport,
  './jacs.client.config.json',
  'client'
);
await client.connect(secureTransport);
```

HTTP API Migration

Adding JACS to Existing Express API

1. **Install Middleware:**

```
npm install @hai.ai/jacs
```

2. **Add Middleware to Routes:**

```
import { JACSExpressMiddleware } from '@hai.ai/jacs/http';

// Before
app.use('/api', express.json());

// After - for JACS-protected routes
app.use('/api/secure', express.text({ type: '*/*' }));
app.use('/api/secure', JACSExpressMiddleware({
  configPath: './jacs.config.json'
}));

// Keep non-JACS routes unchanged
app.use('/api/public', express.json());
```

3. **Update Route Handlers:**

```
// Before
app.post('/api/data', (req, res) => {
  const payload = req.body;
  // ...
});

// After
app.post('/api/secure/data', (req, res) => {
  const payload = req.jacsPayload;  // Verified payload
  // ...
});
```

Troubleshooting Migration

Common Issues

Documents Not Verifying After Migration:

- Check algorithm compatibility
- Verify keys were copied correctly
- Ensure configuration paths are correct

Key File Errors:

- Verify file permissions (600 for private key)
- Check key format matches algorithm

- Ensure password is set for encrypted keys

Storage Errors After Migration:

- Verify storage backend is accessible
- Check credentials/permissions
- Ensure directory structure is correct

Verification Checklist

After any migration:

1. **Verify Configuration:**

   ```
   jacs config read
   ```

2. **Verify Agent:**

   ```
   jacs agent verify
   ```

3. **Verify Sample Document:**

   ```
   jacs document verify -f ./sample-doc.json
   ```

4. **Test Document Creation:**

   ```
   echo '{"test": true}' > test.json
   jacs document create -f test.json
   ```

5. **Verify Version:**

   ```
   jacs version
   ```

Rollback Procedures

If migration fails:

1. **Restore Configuration:**

   ```
   cp jacs.config.json.backup jacs.config.json
   ```

2. **Restore Keys:**

```
cp -r jacs_keys.backup/* jacs_keys/
```

3. **Restore Dependencies:**

```
# Node.js
npm install @hai.ai/jacs@previous-version

# Python
pip install jacs==previous-version
```

4. **Verify Rollback:**

```
jacs agent verify
jacs document verify -d ./jacs_data/documents/
```

See Also

- Configuration Reference - Configuration options
- Cryptographic Algorithms - Algorithm details
- Storage Backends - Storage options